

Advanced Statistics and Machine Learning for Health Research

A Practical Course for Health Researchers

June 6, 2026

Table of contents

- 1. Advanced Statistics and Machine Learning for Health Research 1**
 - Welcome 3**
 - 1.1. What you will learn 3
 - 1.2. Structure 4
 - 1.3. Am I ready? 4
 - 1.4. Datasets 5
 - 1.5. Key sources 6
 - 1.6. Licence 6
 - 1. Pre-Course: Foundations 7**
 - 2. Setting Up Your Computing Environment 9**
 - 2.1. Why Two Languages? 9
 - 2.2. Installing R and RStudio 10
 - 2.3. Installing Python and an Editor 12
 - 2.4. Installing Key R Packages 15
 - 2.5. Installing Key Python Packages 19
 - 2.6. Hello World: Your First Analysis 22
 - 2.7. How to Use the Course Materials 24
 - 2.8. Troubleshooting Common Setup Issues 26
 - 2.9. R 29
 - 2.10. Python 29
 - 2.11. R 29
 - 2.12. Python 30

Table of contents

2.13. R	31
2.14. Python	31
2.15. R	32
2.16. Python	33
2.17. References and Further Reading	34
3. Probability, Distributions, and the Language of Uncertainty	37
3.1. Why Probability Matters in Health Sciences	37
3.2. What Is Probability?	37
3.3. Random Variables	39
3.4. The Normal (Gaussian) Distribution	41
3.5. The Binomial Distribution	45
3.6. The Poisson Distribution	48
3.7. The Central Limit Theorem	51
3.8. Bayes' Theorem: Updating Beliefs with Evidence	55
3.9. Exercises	61
3.10. R	61
3.11. Python	62
3.12. R	63
3.13. Python	65
3.14. R	67
3.15. Python	68
3.16. R	68
3.17. Python	70
3.18. R	72
3.19. Python	73
3.20. R	73
3.21. Python	75
3.22. R	77
3.23. Python	78
3.24. R	79
3.25. Python	81
3.26. R	83
3.27. Python	84

Table of contents

3.28. R	84
3.29. Python	86
3.30. Chapter Summary	88
3.31. References and Further Reading	89
4. Statistical Inference: What We Can and Cannot Conclude	91
4.1. Introduction	91
4.2. Point Estimates and Sampling Variability	92
4.3. Confidence Intervals: What They Actually Mean	94
4.4. P-values: What They Are, What They Are Not	95
4.5. Effect Sizes That Matter Clinically	98
4.6. Type I and Type II Errors	101
4.7. Multiple Testing	102
4.8. Exercises	104
4.9. R	105
4.10. Python	107
4.11. R	109
4.12. Python	110
4.13. R	111
4.14. Python	113
4.15. R	114
4.16. Python	115
4.17. Key Takeaways	117
4.18. References and Further Reading	117
5. Regression: The Workhorse of Clinical Research	119
5.1. Introduction	119
5.2. Why Regression?	119
5.3. Simple Linear Regression	120
5.4. Multiple Linear Regression	122
5.5. Assumptions of Linear Regression	124
5.6. Logistic Regression for Binary Outcomes	126
5.7. Model Fit	130
5.8. Categorical Predictors and Dummy Variables	131

Table of contents

5.9. Interaction Terms	133
5.10. Exercises	135
5.11. R	135
5.12. Python	137
5.13. R	139
5.14. Python	140
5.15. R	142
5.16. Python	144
5.17. R	146
5.18. Python	148
5.19. R	150
5.20. Python	152
5.21. Key Takeaways	154
5.22. References and Further Reading	155

II. Advanced Statistical Methods 157

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation Fails	159
6.1. Introduction	160
6.2. The Problem with Categorising Continuous Variables	161
6.3. When Linear Terms Are Not Enough	164
6.4. Fractional Polynomials	165
6.5. Splines: The Recommended Approach	167
6.6. Clinical Example: CSF Glucose and Bacterial Meningitis	172
6.7. Practical Recommendations	177
6.8. Implementation	178
6.9. Comparison with Other Approaches	181
6.10. Exercises	182
6.11. Summary	189
References	189

7. Penalised Regression: Ridge, LASSO, and Elastic Net	191
7.1. Introduction: Why Standard Regression Is Not Enough . .	192
7.2. The Problem: Overfitting and Instability	193
7.3. The Bias-Variance Tradeoff	195
7.4. The Penalised Regression Framework	197
7.5. Ridge Regression (L2 Penalty)	198
7.6. LASSO Regression (L1 Penalty)	199
7.7. Elastic Net: The Best of Both Worlds	203
7.8. Choosing Lambda: Cross-Validation	204
7.9. Regularisation Paths	206
7.10. Clinical Example: Breast Cancer Diagnosis from Image Features	207
7.11. When to Use Which Method	210
7.12. Implementation	211
7.13. Common Pitfalls	216
7.14. Penalised Regression in Prediction Modelling	217
7.15. Exercises	218
7.16. Summary	226
References	227
 8. Survival Analysis: Time-to-Event Data in Medicine	 229
8.1. Why Ordinary Regression Fails for Time-to-Event Data . .	229
8.2. The Kaplan-Meier Estimator	231
8.3. The Log-Rank Test	235
8.4. Cox Proportional Hazards Model	238
8.5. Checking the Proportional Hazards Assumption	241
8.6. Time-Varying Covariates	242
8.7. Competing Risks	243
8.8. Full Clinical Example: PBC Prognostic Model	244
8.9. Exercises	247
8.10. Summary	249
8.11. References and Further Reading	250

III. Bayesian Methods 253

9. Bayesian Inference: A Different Way to Think About Evidence 255

9.1. Introduction	255
9.2. Two Philosophies of Probability	256
9.3. Bayes' Theorem: You Already Know This	258
9.4. Prior Distributions	259
9.5. The Likelihood	264
9.6. The Posterior Distribution	264
9.7. Markov Chain Monte Carlo (MCMC)	266
9.8. R	270
9.9. Python	270
9.10. When Bayesian Methods Add Value	271
9.11. Practical Example: Estimating a Surgical Complication Rate	272
9.12. Exercises	275
9.13. Summary	279
9.14. References and Further Reading	280

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications 281

10.1. Introduction	281
10.2. Bayesian Linear Regression	282
10.3. Bayesian Logistic Regression	285
10.4. Prior Predictive Checks	289
10.5. Posterior Predictive Checks	291
10.6. Bayesian Hierarchical (Multilevel) Models	293
10.7. The Practical Bayesian Workflow	299
10.8. Implementation: Software Choices	301
10.9. Exercises	304
10.10. Summary	307
10.11. References and Further Reading	308

IV. Dimensionality Reduction and Unsupervised Learning 311

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data 313

11.1. Introduction	313
11.2. The Curse of Dimensionality	313
11.3. PCA: Principal Component Analysis	314
11.4. t-SNE: Non-Linear Dimensionality Reduction	320
11.5. UMAP: Uniform Manifold Approximation and Projection	324
11.6. Practical Workflow: Scale, Reduce, Visualise, Interpret	328
11.7. Exercises	331
11.8. Summary	335
11.9. References and Further Reading	335

12. Clustering: Discovering Patient Subgroups 337

12.1. Introduction	337
12.2. K-Means Clustering	338
12.3. Hierarchical Clustering	344
12.4. DBSCAN: Density-Based Clustering	347
12.5. Cluster Validation	351
12.6. Clinical Applications	354
12.7. Integrating Clustering with PCA/UMAP	355
12.8. Cautionary Notes	358
12.9. Exercises	359
12.10 Summary	364
12.11 References and Further Reading	364

V. Introduction to Machine Learning 367

13. What Is Machine Learning? Core Concepts for Clinical Researchers 369

13.1. What Machine Learning Is (and Is Not)	369
13.2. Supervised vs Unsupervised vs Reinforcement Learning	371

Table of contents

13.3. The Bias-Variance Tradeoff	373
13.4. Training, Validation, and Test Sets	374
13.5. Cross-Validation	375
13.6. Feature Engineering in Clinical Data	379
13.7. Feature Selection	380
13.8. Support Vector Machines	383
13.9. ML vs Traditional Statistics: What Is Actually Different?	387
13.10 Exercises	388
13.11 Summary	391
13.12 References and Further Reading	392
14. Decision Trees, Random Forests, and Gradient Boosting	395
14.1. Decision Trees (CART)	395
14.2. Why Single Trees Are Unstable	401
14.3. Bagging: Bootstrap Aggregating	402
14.4. Random Forests	402
14.5. Gradient Boosting	407
14.6. Hyperparameter Tuning	412
14.7. Comparing Methods: When to Use What	415
14.8. Clinical Example: Readmission Risk Stratification	416
14.9. Exercises	419
14.10 Summary	421
14.11 References and Further Reading	422
15. Introduction to Deep Learning for Health Research	425
15.1. When Does Deep Learning Make Sense?	425
15.2. How Neural Networks Learn	426
15.3. Clinical Example: Neural Network for Readmission Prediction	429
15.4. Key Architectures for Clinical Research	434
15.5. Transfer Learning and Foundation Models	439
15.6. Practical Considerations	444
15.7. Challenges and Limitations	446
15.8. Exercises	447
15.9. Summary	450

15.10	References and Further Reading	451
16.	Evaluating Models: Beyond Accuracy	453
16.1.	Introduction	453
16.2.	The Accuracy Trap	453
16.3.	Sensitivity and Specificity	454
16.4.	Predictive Values: When Prevalence Matters	456
16.5.	The Confusion Matrix	459
16.6.	ROC Curves	461
16.7.	Precision-Recall Curves	465
16.8.	Choosing the Threshold: A Clinical Decision	467
16.9.	Fairness: Does the Model Work for Everyone?	470
16.10	Putting It All Together: A Complete Evaluation	473
16.11	Summary	474
16.12	References and Further Reading	475
VI.	Clinical Prediction Models	477
17.	Developing Clinical Prediction Models: A Complete Workflow	479
17.1.	Introduction	479
17.2.	Step 1: Define the Clinical Question	479
17.3.	Step 2: Study Design and Sample Size	481
17.4.	Step 3: Handle Missing Data	483
17.5.	Step 4: Variable Selection	488
17.6.	Step 5: Understand and Combat Overfitting	489
17.7.	Step 6: Build the Model — A Framingham Example	494
17.8.	Step 7: Assess the Final Model	498
17.9.	The Complete Workflow: A Summary	500
17.10	Common Mistakes to Avoid	501
17.11	Exercises	502
17.12	Summary	502
17.13	References and Further Reading	503

Table of contents

18. Performance Assessment and Model Validation	505
18.1. Introduction	505
18.2. The Five Performance Domains	506
18.3. Domain 1: Discrimination	506
18.4. Domain 2: Calibration	510
18.5. Domain 3: Overall Performance	514
18.6. Domain 4: Classification	515
18.7. Domain 5: Clinical Utility	515
18.8. Internal Validation	518
18.9. External Validation	522
18.10 Model Updating	523
18.11 Recommended Reporting: The Minimum Set	527
18.12 Exercises	531
18.13 References and Further Reading	533
19. Reporting Standards, TRIPOD+AI, and Clinical Impact	535
20. Reporting Standards, TRIPOD+AI, and Clinical Impact	537
20.1. Introduction	537
20.2. Why reporting standards matter	537
20.3. The TRIPOD+AI statement	538
20.4. From model to bedside: the implementation gap	540
20.5. Risk communication	542
20.6. Ethical considerations	543
20.7. Writing a statistical methods section	544
20.8. The future of clinical prediction	546
20.9. References and further reading	547
VII. Advanced Research Toolkit	549
21. Causal Inference for Observational Health Data	551
21.1. Introduction	551
21.2. Correlation vs Causation in Medicine	552

Table of contents

21.3. Directed Acyclic Graphs (DAGs)	553
21.4. Propensity Score Methods	556
21.5. Target Trial Emulation	560
21.6. Regression Discontinuity Design	562
21.7. Implementation in Python	563
21.8. Sensitivity Analysis for Unmeasured Confounding	567
21.9. Practical Guidance: Choosing Your Method	567
21.10Exercises	568
21.11Summary	571
21.12References and Further Reading	572
22. Meta-Analysis Methods for Evidence Synthesis	575
22.1. Introduction	575
22.2. What Does a Meta-Analysis Combine?	576
22.3. Effect Measures	576
22.4. Fixed-Effect vs Random-Effects Models	578
22.5. Heterogeneity	579
22.6. Forest Plots	581
22.7. Funnel Plots and Publication Bias	582
22.8. Advanced Meta-Analysis using metafor	584
22.9. Network Meta-Analysis (Brief Overview)	586
22.10Individual Participant Data (IPD) Meta-Analysis	588
22.11Implementation in Python	589
22.12Reporting a Meta-Analysis: PRISMA 2020	593
22.13Exercises	594
22.14Summary	596
22.15References and Further Reading	597
23. Producing Journal-Ready Statistical Analyses	599
23.1. Introduction	599
23.2. What Do Top Journals Expect?	599
23.3. Table 1: Baseline Characteristics	601
23.4. Regression Tables	604
23.5. Publication-Quality Figures	607

Table of contents

23.6. Writing a Statistical Methods Section	612
23.7. TRIPOD+AI Compliance	614
23.8. Reproducible Research Workflow	617
23.9. Case Study: Replicating a Lancet Digital Health Analytical Approach	620
23.10Capstone Projects	623
23.11Exercises	626
23.12Summary	628
23.13References and Further Reading	629
VIII Appendices	631
24. Dataset Codebook	633
Dataset Codebook	635
24.1. Framingham Heart Study (Teaching Dataset)	635
24.2. Wisconsin Diagnostic Breast Cancer (WDBC)	636
24.3. Primary Biliary Cholangitis (PBC)	637
24.4. NHANES Subset	638
24.5. Simulated Meningitis Data	640
24.6. Downloading the data	640
24.7. R	641
24.8. Python	641
25. Mathematical Notation Reference	643
Mathematical Notation Reference	645
25.1. Basic notation	645
25.2. Probability	646
25.3. Distributions	646
25.4. Regression	646
25.5. Penalised regression	647
25.6. Survival analysis	647

Table of contents

25.7. Bayesian statistics	648
25.8. Model performance	648
25.9. Subscripts and superscripts	648
25.10 Greek letters commonly used in statistics	649
26. Further Reading and Resources	651
Further Reading and Resources	653
26.1. Textbooks	653
26.2. Reporting Guidelines	654
26.3. Statistical Methods in Medicine (Key Papers 2024–2026) . .	655
26.4. R Packages (Key Packages, Recently Updated)	658
26.5. Python Packages (Key Packages, Recently Updated)	659
26.6. JAMA Guide to Statistics and Methods	661
26.7. BMJ Statistics Notes	661
26.8. Online Courses and Tutorials	661
Appendices	663
A. References	663

1. Advanced Statistics and Machine Learning for Health Research

Welcome

This course teaches advanced statistics and machine learning to people who work in health research. It assumes you know the basics — means, standard deviations, maybe a t-test — and takes you from there to the methods you actually see in current medical journals: splines, penalised regression, prediction models, Bayesian analysis, and dimensionality reduction.

Everything uses **clinical data**. Every exercise works in **R and Python**. Pick one or try both.

1.1. What you will learn

By the end of this course, you will be able to:

- Model non-linear relationships properly instead of categorising continuous variables
- Build, validate, and report clinical prediction models to current standards (TRIPOD+AI)
- Use machine learning methods (random forests, gradient boosting, deep learning) and know when they help and when they do not
- Apply Bayesian methods including hierarchical models for multi-site studies
- Reduce high-dimensional data with PCA, t-SNE, and UMAP
- Produce analyses that meet the expectations of BMJ, JAMA, and Lancet Digital Health

Welcome

1.2. Structure

Part	What it covers	Time
Pre-Course	Setup, probability, inference, regression refresher	~12 hrs self-study
I	Splines, penalised regression, survival analysis	1.5 days
II	Bayesian inference and hierarchical models	1 day
III	PCA, UMAP, clustering	0.5 days
IV	ML foundations, trees, ensembles, deep learning, model evaluation	2 days
V	Clinical prediction models, validation, reporting	1.5 days
Advanced Toolkit	Causal inference, meta-analysis, journal-ready analysis	Reference material

The **Pre-Course** material should be completed before Day 1 if you are taking this in person. If you are self-studying, work through everything in order.

The **Advanced Research Toolkit** is for after the course — use it when you are doing your own research and need guidance on causal inference, meta-analysis, or producing a journal-ready manuscript.

1.3. Am I ready?

Skim Chapters 1-3. If the material feels familiar, skip ahead to Part I. If it feels new, work through those chapters carefully first — they are the foundation everything else builds on.

Rough time estimates per chapter:

Chapter	Topic	Estimated time
0	Environment setup	1-2 hrs
1	Probability and distributions	2-3 hrs
2	Inference and estimation	2-3 hrs
3	Regression foundations	3-4 hrs
4	Splines and non-linear modelling	3-4 hrs
5	Penalised regression	2-3 hrs
6	Survival analysis	3-4 hrs
13	Bayesian inference	3-4 hrs
14	Applied Bayesian modelling	3-4 hrs
15	Dimensionality reduction	2-3 hrs
16	Clustering	2-3 hrs
7	ML foundations	2-3 hrs
8	Trees and ensembles	2-3 hrs
8b	Introduction to deep learning	2-3 hrs
9	Model evaluation	2-3 hrs
10	Building prediction models	3-4 hrs
11	Performance and validation	3-4 hrs
12	Reporting and TRIPOD+AI	2-3 hrs

1.4. Datasets

All exercises use real or realistically simulated clinical data:

- **Framingham Heart Study** — cardiovascular risk
- **Wisconsin Breast Cancer** — diagnostic classification
- **PBC (Mayo Clinic)** — liver disease survival
- **NHANES** — national health survey
- **Simulated meningitis** — based on Lopez-Ayala et al. (*BMJ* 2025)

Welcome

Details in the [Dataset Codebook](#).

1.5. Key sources

This course draws heavily on:

- Smits, van Kuijk & Wynants, *Improving Health Care with Clinical Prediction Models* (2026, CC BY 4.0)
- Van Calster et al., *Lancet Digital Health* (2025) — performance measures for prediction models
- Lopez-Ayala et al., *BMJ* (2025) — handling continuous variables and splines
- Collins et al., *BMJ* (2024) — TRIPOD+AI reporting guidelines
- Harrell, *Regression Modeling Strategies* (2015)
- McElreath, *Statistical Rethinking* (2020)

Full references appear at the bottom of each chapter.

1.6. Licence

[Creative Commons Attribution 4.0](#). Use it, adapt it, share it — just give credit.

Part I.

Pre-Course: Foundations

2. Setting Up Your Computing Environment

2.1. Why Two Languages?

Throughout this course, you will work with both **R** and **Python**. This is not an accident or an attempt to double your workload. In modern biostatistics, epidemiology, and health data science, both languages appear regularly in published research, collaborative projects, and industry applications. R has deep roots in classical statistics and has an extraordinary ecosystem of packages for survival analysis, Bayesian modeling, and clinical reporting. Python dominates in machine learning, deep learning, and large-scale data engineering. By learning both, you will be able to read and contribute to a wider range of projects, collaborate with more colleagues, and choose the best tool for each task.

You do not need to be an expert programmer to succeed in this course. We will introduce code gradually and explain every line. Think of R and Python as lab instruments: you will learn to use them by doing, and we will always prioritize understanding the statistical ideas over memorizing syntax.

💡 Which language should I pick?

If you have no programming experience, **start with R and RStudio**. RStudio is designed for data analysis and is the standard in

2. Setting Up Your Computing Environment

biostatistics departments. You can always add Python later. If you already know some Python, stick with Python. Both languages can do everything in this course.

2.2. Installing R and RStudio

2.2.1. What Are R and RStudio?

R is a programming language designed for statistical computing and graphics. It runs in a terminal or console, but working with raw R in a terminal is not the most pleasant experience. **RStudio** is an integrated development environment (IDE) that wraps around R, providing a code editor, a console, a file browser, a plot viewer, and much more in a single window. Think of R as the engine and RStudio as the dashboard and steering wheel.

2.2.2. Step 1: Download and Install R

1. Open your web browser and navigate to the Comprehensive R Archive Network (CRAN): <https://cran.r-project.org/>
2. You will see links for your operating system near the top of the page:
 - **Windows:** Click “Download R for Windows,” then click “base,” then click the link that says something like “Download R-4.x.x for Windows.” Run the downloaded **.exe** installer and accept all default settings.
 - **macOS:** Click “Download R for macOS.” Choose the appropriate version for your Mac. If you have an Apple Silicon Mac (M1, M2, M3, or M4 chip), download the **arm64** version. If you have an older Intel Mac, download the **x86_64** version. If you are unsure, click the Apple icon in the top-left corner of your screen,

2.2. Installing R and RStudio

choose “About This Mac,” and look at the “Chip” or “Processor” field. Open the downloaded `.pkg` file and follow the installation prompts.

- **Linux:** Click “Download R for Linux,” select your distribution (Ubuntu, Fedora, etc.), and follow the instructions. On Ubuntu, you can also install R from the terminal:

```
# Ubuntu/Debian
sudo apt update
sudo apt install r-base r-base-dev
```

3. Verify the installation by opening a terminal (or Command Prompt on Windows) and typing:

```
R --version
```

You should see output that includes the R version number (4.4 or later is recommended).

2.2.3. Step 2: Download and Install RStudio

1. Navigate to the Posit (formerly RStudio) download page: <https://posit.co/download/rstudio-desktop/>
2. The page will detect your operating system and suggest the correct installer. Click the download button.
3. Run the installer:
 - **Windows:** Run the `.exe` file and follow the prompts.
 - **macOS:** Open the `.dmg` file and drag RStudio to your Applications folder.
 - **Linux:** Install the `.deb` or `.rpm` package using your package manager.

2. Setting Up Your Computing Environment

4. Open RStudio. You should see four panels: the Source editor (top left), the Console (bottom left), the Environment/History pane (top right), and the Files/Plots/Packages/Help pane (bottom right). If R is installed correctly, you will see a welcome message in the Console that includes the R version number.

2.2.4. Step 3: A Quick Tour of RStudio

- **Console** (bottom left): Type R commands here and press Enter to run them immediately. Good for quick exploration.
- **Source Editor** (top left): Write and save R scripts (`.R` files) or Quarto documents (`.qmd` files). You can run lines or selections by pressing Ctrl+Enter (Cmd+Enter on Mac).
- **Environment** (top right): Shows all variables and data objects currently in memory.
- **Files/Plots/Packages/Help** (bottom right): Browse files on your computer, view plots, manage installed packages, and read documentation.

2.3. Installing Python and an Editor

2.3.1. What Are Python, JupyterLab, and VS Code?

Python is a general-purpose programming language widely used in data science and machine learning. Unlike R, Python was not designed exclusively for statistics, but its ecosystem of scientific libraries (NumPy, pandas, scikit-learn, etc.) makes it extremely powerful for data analysis.

You have two main choices for an editing environment:

- **JupyterLab**: A browser-based interactive environment where you write code in “notebooks” — documents that mix code, output, and

2.3. Installing Python and an Editor

narrative text. Jupyter notebooks (`.ipynb` files) are very popular in data science.

- **VS Code (Visual Studio Code):** A general-purpose code editor from Microsoft that supports Python (and R) through extensions. It can also run Jupyter notebooks directly inside the editor.

We recommend **VS Code** for this course because it handles both R and Python well and is widely used in industry. However, JupyterLab is perfectly fine if you prefer it.

2.3.2. Step 1: Install Python via Miniforge (Recommended)

We recommend installing Python through **Miniforge**, a lightweight distribution that includes the `conda` package manager. Conda makes it easy to install scientific packages and manage separate environments for different projects.

1. Download Miniforge from: <https://conda-forge.org/miniforge/>
2. Choose the installer for your operating system:
 - **Windows:** Download and run the `.exe` installer. Check the box that says “Add Miniforge to my PATH environment variable” during installation.
 - **macOS:** Download the `.sh` script. Open Terminal and run:

```
# Replace the filename with the one you downloaded
bash Miniforge3-MacOSX-arm64.sh
```

- **Linux:** Same approach as macOS — download the `.sh` script and run it in your terminal.
3. Close and reopen your terminal, then verify:

2. Setting Up Your Computing Environment

```
python --version  
conda --version
```

You should see Python 3.11 or later and a conda version number.

2.3.3. Alternative: Install Python from python.org

If you prefer not to use conda, you can download Python directly from <https://www.python.org/downloads/>. Make sure to check the box “Add Python to PATH” during installation on Windows. You will then use `pip` instead of `conda` to install packages.

2.3.4. Step 2: Install VS Code

1. Download VS Code from <https://code.visualstudio.com/>
2. Install it using the standard process for your operating system.
3. Open VS Code, click the Extensions icon in the left sidebar (it looks like four small squares), and install the following extensions:
 - **Python** (by Microsoft) — provides Python language support, debugging, and Jupyter notebook integration.
 - **Jupyter** (by Microsoft) — adds full Jupyter notebook support inside VS Code.
 - **Quarto** (by Quarto) — optional, but useful if you want to edit `.qmd` files in VS Code.
 - **R** (by REditorSupport) — optional, provides R language support in VS Code.

2.3.5. Alternative: Install JupyterLab

If you prefer JupyterLab, install it after setting up Python:

2.4. Installing Key R Packages

```
conda install jupyterlab
# or, if using pip:
pip install jupyterlab
```

Launch it with:

```
jupyter lab
```

This will open JupyterLab in your web browser.

2.4. Installing Key R Packages

R packages extend the language with specialized tools. Think of them as apps you install on your phone — the base R system is the phone itself, and packages add new capabilities.

Open RStudio and run the following command in the Console. This will take several minutes the first time because it downloads and compiles many packages:

```
# Install all packages needed for this course
install.packages(c(
  "tidyverse",      # Data manipulation (dplyr, ggplot2, tidyr, readr, etc.)
  "rms",            # Regression Modeling Strategies (Frank Harrell's toolkit)
  "glmnet",         # Regularized regression (lasso, ridge, elastic net)
  "survival",       # Survival analysis (Cox models, Kaplan-Meier)
  "brms",           # Bayesian regression models via Stan
  "rstanarm",       # Bayesian applied regression modeling via Stan
  "ranger",         # Fast random forests
  "xgboost",        # Gradient boosted trees
  "mice",           # Multiple imputation by chained equations
```

2. Setting Up Your Computing Environment

```
"dcurves",      # Decision curve analysis
"pROC",         # ROC curve analysis
"uwot",         # UMAP dimensionality reduction
"cluster",      # Cluster analysis
"gtsummary",    # Publication-ready summary tables
"MatchIt",      # Propensity score matching
"tidymodels"    # Unified machine learning framework
))
```

2.4.1. What Each Package Does (Brief Overview)

Table 2.1.: Key R packages for this course

Package	Purpose
<code>tidyverse</code>	A collection of packages for data wrangling and visualization. Includes <code>ggplot2</code> (plotting), <code>dplyr</code> (data manipulation), <code>tidyr</code> (reshaping), and more.
<code>rms</code>	Frank Harrell's Regression Modeling Strategies package. Provides tools for fitting and validating regression models with a focus on best practices.
<code>glmnet</code>	Fits penalized (regularized) generalized linear models, including lasso and ridge regression.
<code>survival</code>	The foundational package for survival (time-to-event) analysis in R.

2.4. Installing Key R Packages

Package	Purpose
<code>brms</code>	An interface to Stan for fitting Bayesian generalized linear mixed models using familiar R formula syntax.
<code>rstanarm</code>	Similar to <code>brms</code> but with pre-compiled Stan models for faster startup. Great for common Bayesian regression tasks.
<code>ranger</code>	A fast implementation of random forests, useful for both classification and regression.
<code>xgboost</code>	Extreme gradient boosting — one of the most powerful and popular machine learning algorithms.
<code>mice</code>	Handles missing data through multiple imputation, a principled approach to dealing with incomplete datasets.
<code>dcurves</code>	Implements decision curve analysis for evaluating clinical prediction models.
<code>pROC</code>	Computes and displays ROC curves and calculates the area under the curve (AUC).
<code>uwot</code>	Implements UMAP (Uniform Manifold Approximation and Projection) for dimensionality reduction.
<code>cluster</code>	Provides methods for cluster analysis including k-medoids (PAM), hierarchical clustering, and more.

2. Setting Up Your Computing Environment

Package	Purpose
<code>gtsummary</code>	Creates publication-quality summary and regression tables.
<code>MatchIt</code>	Implements propensity score matching and other matching methods for causal inference.
<code>tidymodels</code>	A unified framework for building, tuning, and evaluating machine learning models in R.

2.4.2. Verifying R Package Installation

After installation, verify that the key packages load without errors:

```
library(tidyverse)
library(rms)
library(glmnet)
library(survival)
library(ranger)
library(xgboost)
```

If you get an error like `there is no package called 'xyz'`, try reinstalling that specific package. If compilation errors occur (common on Linux), you may need to install system-level dependencies. RStudio will usually tell you what is missing.

2.5. Installing Key Python Packages

2.5.1. Using conda (Recommended)

Create a dedicated environment for this course. An environment is an isolated collection of packages that will not interfere with other Python projects on your machine:

```
# Create a new environment called 'mlstats' with Python 3.12
conda create -n mlstats python=3.12

# Activate the environment
conda activate mlstats

# Install all packages
conda install pandas numpy scikit-learn statsmodels matplotlib seaborn

# Some packages are best installed via pip even when using conda
pip install lifelines xgboost pymc bambi arviz umap-learn plotnine miceforest
```

2.5.2. Using pip Only

If you are not using conda:

```
# Create a virtual environment
python -m venv mlstats_env

# Activate it
# On macOS/Linux:
source mlstats_env/bin/activate
# On Windows:
mlstats_env\Scripts\activate
```

2. Setting Up Your Computing Environment

```
# Install packages
pip install pandas numpy scikit-learn statsmodels lifelines xgboost \
    pymc bambi arviz umap-learn matplotlib seaborn plotnine miceforest
```

2.5.3. What Each Python Package Does

Table 2.2.: Key Python packages for this course

Package	Purpose
pandas	The primary data manipulation library. DataFrames (tables) are the central data structure.
numpy	Numerical computing — arrays, linear algebra, random number generation.
scikit-learn	The standard machine learning library. Classification, regression, clustering, preprocessing, model evaluation.
statsmodels	Classical statistical models: linear regression, logistic regression, time series, and more. Provides p-values and confidence intervals that scikit-learn does not.
lifelines	Survival analysis in Python: Kaplan-Meier curves, Cox proportional hazards models, and more.
xgboost	Gradient boosted trees (same algorithm as the R version).

2.5. Installing Key Python Packages

Package	Purpose
pymc	Bayesian statistical modeling and probabilistic programming.
bambi	BAyesian Model-Building Interface — a high-level interface to PyMC using R-style formulas.
arviz	Visualization and diagnostics for Bayesian models.
umap-learn	UMAP dimensionality reduction for Python.
matplotlib	The foundational plotting library for Python.
seaborn	Statistical data visualization built on top of matplotlib. Easier to use for common statistical plots.
plotnine	A Python implementation of R's ggplot2 grammar of graphics. If you like ggplot2, you will like plotnine.
miceforest	Multiple imputation using random forests in Python.

2.5.4. Verifying Python Package Installation

```
import pandas as pd
import numpy as np
import sklearn
import statsmodels
import matplotlib.pyplot as plt

print(f"pandas:      {pd.__version__}")
print(f"numpy:         {np.__version__}")
```

2. Setting Up Your Computing Environment

```
print(f"scikit-learn: {sklearn.__version__}")
print("All key packages loaded successfully!")
```

2.6. Hello World: Your First Analysis

Let us make sure everything works by loading a built-in dataset and creating a simple plot in both languages. We will use the classic `iris` dataset — measurements of petal and sepal dimensions for three species of iris flowers. While not a clinical dataset, it is available in both R and Python without any downloads, making it perfect for a quick test.

2.6.1. R

```
# Load the tidyverse (includes ggplot2 and dplyr)
library(tidyverse)

# The iris dataset is built into R
data(iris)

# Take a quick look at the first few rows
head(iris)

# Summary statistics
summary(iris)

# Create a scatter plot of Sepal Length vs. Petal Length, colored by Species
ggplot(iris, aes(x = Sepal.Length, y = Petal.Length, color = Species)) +
  geom_point(size = 2, alpha = 0.7) +
  labs(
    title = "Iris Dataset: Sepal Length vs. Petal Length",
```

2.6. Hello World: Your First Analysis

```
x = "Sepal Length (cm)",
y = "Petal Length (cm)"
) +
theme_minimal()
```

2.6.2. Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the iris dataset
iris_data = load_iris()
iris = pd.DataFrame(
    iris_data.data,
    columns=iris_data.feature_names
)
iris["species"] = pd.Categorical.from_codes(
    iris_data.target, iris_data.target_names
)

# Take a quick look
print(iris.head())
print(iris.describe())

# Create a scatter plot
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=iris,
    x="sepal length (cm)",
    y="petal length (cm)",
```

2. Setting Up Your Computing Environment

```
    hue="species",  
    alpha=0.7,  
    s=60  
  )  
plt.title("Iris Dataset: Sepal Length vs. Petal Length")  
plt.tight_layout()  
plt.show()
```

If you see a colorful scatter plot with three clusters of points, congratulations — your setup is working.

2.7. How to Use the Course Materials

2.7.1. Navigating This Book

This course is delivered as a **Quarto book** — a collection of chapters that you can read in your web browser. Each chapter builds on previous ones, so we recommend reading them in order the first time through. However, you can always jump to a specific chapter using the table of contents in the left sidebar.

2.7.2. Code Blocks

Throughout the book, you will encounter code blocks like this:

```
# This is an R code block  
x <- c(1, 2, 3, 4, 5)  
mean(x)
```


2.7. How to Use the Course Materials

Many chapters present code in **tabbed panels** labeled “R” and “Python.” Click the tab for your preferred language. We encourage you to try both, but if you are short on time, pick the one you are more comfortable with and come back to the other later.

2.7.3. Exercises

Each chapter ends with exercises. They follow this pattern:

1. A description of the task inside a colored callout box.
2. **Starter code** that sets up the problem and leaves blanks for you to fill in.
3. A **Solution** hidden inside a collapsible box — try the exercise yourself before peeking!

2.7.4. Downloading Notebooks

For hands-on practice, you can download the exercises as standalone notebooks:

- **R users:** Look for `.qmd` or `.Rmd` files in the course repository that you can open in RStudio.
- **Python users:** Look for `.ipynb` (Jupyter notebook) files in the course repository that you can open in JupyterLab or VS Code.

The course repository is available on GitHub. Your instructor will provide the link.

2.7.5. Rendering Quarto Documents

If you want to render `.qmd` files yourself (to produce HTML or PDF output), you need to install Quarto:

2. Setting Up Your Computing Environment

1. Download Quarto from <https://quarto.org/docs/get-started/>
2. Install it following the instructions for your operating system.
3. In RStudio, you can render a `.qmd` file by clicking the “Render” button. In VS Code, use the Quarto extension’s render command. From the terminal:

```
quarto render my_document.qmd
```

2.8. Troubleshooting Common Setup Issues

2.8.1. R and RStudio Issues

Problem: RStudio cannot find R. **Solution:** Make sure you installed R *before* installing RStudio. If you installed them in the wrong order, try reinstalling RStudio. On Windows, RStudio looks for R in standard installation locations. If you installed R to a non-standard path, go to Tools > Global Options > General and set the R version manually.

Problem: Package installation fails with a compilation error. **Solution:** Some R packages need to compile C++ or Fortran code. On Windows, install [Rtools](#). On macOS, install the Xcode Command Line Tools by running `xcode-select --install` in Terminal. On Linux, install `build-essential` and `r-base-dev`.

Problem: `brms` or `rstanarm` fails to install. **Solution:** These packages depend on Stan, a probabilistic programming language that requires a C++ compiler. Follow the instructions above for installing compilation tools. On Windows, make sure Rtools is on your PATH. Installation can take 10–15 minutes — be patient.

Problem: Package loads but you get warnings about versions. **Solution:** Warnings (yellow text) are usually harmless — they often say things like “package was built under R version X.Y.Z.” Errors (red text) are the ones

2.8. Troubleshooting Common Setup Issues

that prevent code from running. If you get errors, try updating the package with `install.packages("package_name")`.

2.8.2. Python Issues

Problem: `python` command not found. **Solution:** On some systems, Python 3 is accessed via `python3` instead of `python`. Try `python3 --version`. If using conda, make sure you have activated your environment with `conda activate mlstats`.

Problem: `ModuleNotFoundError: No module named 'xyz'`. **Solution:** The package is not installed in your current environment. Make sure you have activated the correct conda environment (`conda activate mlstats`) or virtual environment before installing and importing packages.

Problem: `pip install pymc` fails with obscure errors. **Solution:** PyMC can be tricky to install because it depends on compiled numerical libraries. Using conda often resolves these issues: `conda install -c conda-forge pymc`. On Apple Silicon Macs, make sure you are using the ARM64 version of Miniforge.

Problem: Jupyter notebook does not show the correct Python kernel. **Solution:** Install the IPython kernel in your environment: `conda install ipykernel` or `pip install ipykernel`. Then register it: `python -m ipykernel install --user --name mlstats --display-name "ML Stats Course"`.

2.8.3. Common Error Messages

Error: `Error: object 'x' not found` **Solution:** You haven't run the earlier code blocks that create `x`. Go back and run all preceding code chunks in order. In RStudio, use "Run All Chunks Above" from the Run menu. In Jupyter, use "Run All Above" from the Cell menu.

2. Setting Up Your Computing Environment

Error: Error in library(xxx): there is no package called 'xxx' **Solution:** Install it first with `install.packages("xxx")` in R, or `pip install xxx` / `conda install xxx` in Python. Then try loading it again.

2.8.4. General Tips

- **Keep your software updated.** At the start of each semester, update R, RStudio, Python, and your packages.
- **Use separate environments.** Conda environments (Python) and `renv` (R) prevent package version conflicts between projects.
- **Read error messages carefully.** They usually tell you exactly what went wrong, even if the language is technical. Copy the last line of an error message into a search engine — someone else has almost certainly had the same problem.
- **Ask for help.** Post on the course discussion board with the full error message, your operating system, and what you were trying to do. Screenshots are helpful.

Exercise 0.1: Verify Your Setup

Confirm that both R and Python are working by completing the following tasks:

1. In R, load the `tidyverse` package and use `ggplot2` to create a histogram of the `Sepal.Width` column from the built-in `iris` dataset.
2. In Python, use `seaborn` to create a histogram of the `sepal width (cm)` column from scikit-learn's `iris` dataset.

2.9. R

```
# Load tidyverse
library(tidyverse)

# Create a histogram of Sepal.Width from the iris dataset
# YOUR CODE HERE
```

2.10. Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load iris and create a DataFrame
iris_data = load_iris()
iris = pd.DataFrame(iris_data.data, columns=iris_data.feature_names)

# Create a histogram of sepal width
# YOUR CODE HERE
```

Solution

2.11. R

2. Setting Up Your Computing Environment

```
library(tidyverse)

ggplot(iris, aes(x = Sepal.Width)) +
  geom_histogram(binwidth = 0.2, fill = "steelblue", color = "white")
labs(
  title = "Distribution of Sepal Width",
  x = "Sepal Width (cm)",
  y = "Count"
) +
  theme_minimal()
```

2.12. Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris_data = load_iris()
iris = pd.DataFrame(iris_data.data, columns=iris_data.feature_names)

plt.figure(figsize=(8, 5))
sns.histplot(iris["sepal width (cm)"], bins=15, kde=True, color="steelblue")
plt.title("Distribution of Sepal Width")
plt.xlabel("Sepal Width (cm)")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

💡 Exercise 0.2: Explore a Clinical Dataset

Now let us work with something more relevant to health sciences. Both R and Python include the `mtcars` dataset (or we can simulate clinical-like data). In this exercise, create a scatter plot relating two variables and add a trend line.

2.13. R

```
library(tidyverse)

# Let's simulate a small clinical dataset
set.seed(42)
n <- 200
clinical <- tibble(
  age = round(rnorm(n, mean = 55, sd = 12)),
  systolic_bp = round(100 + 0.8 * age + rnorm(n, sd = 10)),
  bmi = round(rnorm(n, mean = 27, sd = 5), 1)
)

# Create a scatter plot of age vs. systolic blood pressure with a trend line
# Hint: use geom_point() and geom_smooth(method = "lm")
# YOUR CODE HERE
```

2.14. Python

2. Setting Up Your Computing Environment

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Simulate a small clinical dataset
np.random.seed(42)
n = 200
clinical = pd.DataFrame({
    "age": np.round(np.random.normal(55, 12, n)).astype(int),
})
clinical["systolic_bp"] = np.round(100 + 0.8 * clinical["age"] + np.random.normal(0, 10, n), 1)
clinical["bmi"] = np.round(np.random.normal(27, 5, n), 1)

# Create a scatter plot of age vs. systolic blood pressure with a trend line
# Hint: use sns.regplot() or sns.lmplot()
# YOUR CODE HERE
```

Solution

2.15. R

2.8. Troubleshooting Common Setup Issues

```
library(tidyverse)

set.seed(42)
n <- 200
clinical <- tibble(
  age = round(rnorm(n, mean = 55, sd = 12)),
  systolic_bp = round(100 + 0.8 * age + rnorm(n, sd = 10)),
  bmi = round(rnorm(n, mean = 27, sd = 5), 1)
)

ggplot(clinical, aes(x = age, y = systolic_bp)) +
  geom_point(alpha = 0.5, color = "darkblue") +
  geom_smooth(method = "lm", color = "firebrick", se = TRUE) +
  labs(
    title = "Age vs. Systolic Blood Pressure",
    subtitle = "Simulated clinical data (n = 200)",
    x = "Age (years)",
    y = "Systolic Blood Pressure (mmHg)"
  ) +
  theme_minimal()
```

2.16. Python

2. Setting Up Your Computing Environment

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)
n = 200
clinical = pd.DataFrame({
    "age": np.round(np.random.normal(55, 12, n)).astype(int),
})
clinical["systolic_bp"] = np.round(100 + 0.8 * clinical["age"] + np.random.normal(0, 10, n))
clinical["bmi"] = np.round(np.random.normal(27, 5, n), 1)

plt.figure(figsize=(8, 5))
sns.regplot(
    data=clinical, x="age", y="systolic_bp",
    scatter_kws={"alpha": 0.5, "color": "darkblue"},
    line_kws={"color": "firebrick"}
)
plt.title("Age vs. Systolic Blood Pressure\nSimulated clinical data (n=200)")
plt.xlabel("Age (years)")
plt.ylabel("Systolic Blood Pressure (mmHg)")
plt.tight_layout()
plt.show()
```

2.17. References and Further Reading

- Wickham, H., & Grolemund, G. (2017). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media. Freely available at <https://r4ds.had.co.nz/>. The definitive introduction to the tidyverse ecosystem in R.

2.17. References and Further Reading

- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media. Freely available at <https://jakevdp.github.io/PythonDataScienceHandbook/>. Covers NumPy, pandas, matplotlib, and scikit-learn in depth.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media. The authoritative guide to pandas, written by the creator of the library.
- Posit. (2024). *Quarto Documentation*. <https://quarto.org/docs/guide/>. The official guide to Quarto, the publishing system used for this course.
- Cetinkaya-Rundel, M., & Hardin, J. (2024). *Introduction to Modern Statistics* (2nd ed.). OpenIntro. Freely available at <https://openintro-ims2.netlify.app/>. An excellent, free statistics textbook that uses R.
- Harrell, F. E. (2015). *Regression Modeling Strategies: With Applications to Linear Models, Logistic and Ordinal Regression, and Survival Analysis* (2nd ed.). Springer. The companion text for the `rms` R package and a masterclass in regression modeling for health sciences.

3. Probability, Distributions, and the Language of Uncertainty

3.1. Why Probability Matters in Health Sciences

Medicine is a discipline of uncertainty. A patient presents with chest pain — is it a heart attack, acid reflux, or a pulled muscle? A new drug shows a 15% reduction in tumor size — is this a real effect or a fluke of sampling? A screening test comes back positive — what is the chance the patient actually has the disease?

Probability gives us a formal language for reasoning about uncertainty. It allows us to quantify how likely different outcomes are, how confident we should be in our conclusions, and how we should update our beliefs when new evidence arrives. Every statistical method you will learn in this course — from logistic regression to Bayesian modeling to machine learning — is built on the foundation of probability theory.

This chapter introduces the key ideas you need. We will move slowly, use concrete clinical examples, and build intuition before introducing formulas.

3.2. What Is Probability?

At its simplest, a **probability** is a number between 0 and 1 that represents how likely an event is to occur. A probability of 0 means the event is

3. Probability, Distributions, and the Language of Uncertainty

impossible; a probability of 1 means it is certain. A probability of 0.5 means the event is equally likely to happen or not happen.

But where do probabilities come from? There are two major perspectives.

3.2.1. The Frequentist Interpretation

The **frequentist** interpretation says that the probability of an event is the long-run relative frequency with which it occurs if we repeat the same process many times. For example:

- If we flip a fair coin many times, it will land heads about 50% of the time. We say $P(\text{heads}) = 0.5$.
- If a particular surgery has a 3% mortality rate, this means that out of every 100 patients who undergo the procedure (under similar conditions), roughly 3 will die.

This interpretation works well when we can imagine repeating an experiment. Coin flips, blood draws from a population, random assignment in a clinical trial — all of these fit naturally into the frequentist framework.

3.2.2. The Subjective (Bayesian) Interpretation

The **subjective** or **Bayesian** interpretation says that probability represents a *degree of belief*. It is a measure of how confident you are that something is true, given what you currently know.

- A cardiologist examining a 65-year-old male smoker with crushing chest pain might say: “I believe there is an 85% probability this is a myocardial infarction.” This is not based on repeating the exact same patient encounter thousands of times — it is an informed judgment.
- Before seeing any data from a clinical trial, a researcher might believe there is a 30% chance a new drug is effective. After seeing positive results from the trial, they update that belief to 80%.

3.3. Random Variables

The Bayesian interpretation is powerful because it applies to one-time events and unique situations. You cannot repeat a patient's life, but you can still reason probabilistically about their prognosis.

3.2.3. Which Interpretation Is Correct?

Both are useful. In this course, we will use frequentist methods (hypothesis tests, confidence intervals, maximum likelihood) and Bayesian methods (posterior distributions, credible intervals, prior updating). Understanding both perspectives will make you a more versatile analyst. The key insight is that probability is a *tool for reasoning under uncertainty*, regardless of your philosophical starting point.

3.3. Random Variables

A **random variable** is a quantity whose value is determined by a random process. In clinical research, random variables are everywhere:

- The number of patients who respond to treatment (a count)
- A patient's systolic blood pressure (a measurement)
- Whether a tumor is malignant or benign (a category)
- Time from diagnosis to death (a duration)

We use capital letters like X , Y , or Z to denote random variables, and lowercase letters like x , y , z for specific observed values.

3.3.1. Discrete Random Variables

A **discrete** random variable takes on a countable number of values, typically whole numbers. Examples:

- The number of adverse events in a clinical trial: 0, 1, 2, 3, ...

3. Probability, Distributions, and the Language of Uncertainty

- The number of positive test results in a batch of 20 specimens: 0, 1, 2, ..., 20
- A patient's pain score on a 0–10 scale: 0, 1, 2, ..., 10

For a discrete random variable, we describe its behavior using a **probability mass function (PMF)**, which gives the probability of each possible value:

$$P(X = x)$$

All probabilities must be non-negative, and they must sum to 1:

$$\sum_{\text{all } x} P(X = x) = 1$$

3.3.2. Continuous Random Variables

A **continuous** random variable can take on any value within a range (including decimals). Examples:

- Body temperature: 36.2, 37.45, 38.1, ...
- Serum creatinine level: any positive real number
- Time to event: any positive duration

For a continuous random variable, the probability of any *exact* value is technically zero (because there are infinitely many possible values). Instead, we talk about the probability of falling within a *range*, and we describe the distribution using a **probability density function (PDF)**, denoted $f(x)$. The probability that X falls between a and b is:

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

3.4. The Normal (Gaussian) Distribution

This is the area under the density curve between a and b . The total area under the entire curve equals 1.

Do not worry if integrals make you nervous. The key idea is: for continuous variables, probability corresponds to *area under a curve*.

3.4. The Normal (Gaussian) Distribution

The **normal distribution** — also called the Gaussian distribution or the “bell curve” — is the most important probability distribution in statistics. It appears everywhere in nature and medicine.

3.4.1. Why Does It Matter?

1. **Many biological measurements are approximately normal.** Heights, weights, blood pressure readings, cholesterol levels, and many lab values follow roughly bell-shaped distributions in large populations.
2. **The Central Limit Theorem** (discussed later in this chapter) tells us that averages of measurements tend to follow a normal distribution, even when individual measurements do not.
3. **Many statistical methods assume normality** (or at least approximate normality) for valid inference.

3.4.2. Properties of the Normal Distribution

A normal distribution is completely described by two parameters:

- **Mean (μ):** The center of the distribution. The peak of the bell curve.

3. Probability, Distributions, and the Language of Uncertainty

- **Standard deviation (σ):** A measure of spread. About 68% of values fall within one standard deviation of the mean, about 95% within two, and about 99.7% within three. This is sometimes called the **68-95-99.7 rule**.

The probability density function is:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

You do not need to memorize this formula. What matters is understanding the shape and the role of μ and σ .

We write $X \sim \text{Normal}(\mu, \sigma^2)$ or $X \sim N(\mu, \sigma^2)$ to say that X follows a normal distribution with mean μ and variance σ^2 .

3.4.3. Clinical Example: Blood Pressure

Systolic blood pressure in healthy adults is approximately normally distributed with a mean of about 120 mmHg and a standard deviation of about 15 mmHg. That is, $\text{SBP} \sim N(120, 15^2)$.

Using the 68-95-99.7 rule:

- About 68% of healthy adults have SBP between 105 and 135 mmHg (120 plus or minus 15).
- About 95% have SBP between 90 and 150 mmHg (120 plus or minus 30).
- About 99.7% have SBP between 75 and 165 mmHg (120 plus or minus 45).

If a patient has a systolic blood pressure of 160 mmHg, how unusual is that? We can answer this using **Z-scores**.

3.4. The Normal (Gaussian) Distribution

3.4.4. Z-Scores: Standardizing Values

A **Z-score** tells you how many standard deviations a value is from the mean:

$$Z = \frac{X - \mu}{\sigma}$$

For our patient with SBP = 160:

$$Z = \frac{160 - 120}{15} = \frac{40}{15} \approx 2.67$$

This patient's blood pressure is about 2.67 standard deviations above the mean. Using a standard normal table or a computer, we find that only about 0.4% of the healthy population would have a blood pressure this high or higher. This is strong evidence that this patient may have hypertension.

The **standard normal distribution** is a normal distribution with $\mu = 0$ and $\sigma = 1$. Any normal distribution can be converted to the standard normal by computing Z-scores.

Let us visualize the normal distribution and Z-scores:

3.4.5. R

```
library(tidyverse)

# Parameters for systolic blood pressure
mu <- 120
sigma <- 15

# Create a sequence of values for the x-axis
```

3. Probability, Distributions, and the Language of Uncertainty

```
x <- seq(mu - 4 * sigma, mu + 4 * sigma, length.out = 300)
y <- dnorm(x, mean = mu, sd = sigma)

bp_data <- tibble(x = x, y = y)

ggplot(bp_data, aes(x = x, y = y)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_area(data = bp_data |> filter(x >= 160),
            aes(x = x, y = y), fill = "firebrick", alpha = 0.4) +
  geom_vline(xintercept = 160, linetype = "dashed", color = "firebrick") +
  annotate("text", x = 168, y = 0.015,
           label = "Patient: 160 mmHg\n(Z = 2.67)", color = "firebrick",
           hjust = 0, size = 3.5) +
  labs(
    title = "Distribution of Systolic Blood Pressure in Healthy Adults",
    subtitle = paste0("Normal(", mu, ", ", sigma, "^2)"),
    x = "Systolic Blood Pressure (mmHg)",
    y = "Density"
  ) +
  theme_minimal()
```

3.4.6. Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

mu = 120
sigma = 15

x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, 300)
y = stats.norm.pdf(x, loc=mu, scale=sigma)
```

3.5. The Binomial Distribution

```
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(x, y, color="steelblue", linewidth=1.5)

# Shade the area above 160
x_fill = x[x >= 160]
y_fill = stats.norm.pdf(x_fill, loc=mu, scale=sigma)
ax.fill_between(x_fill, y_fill, color="firebrick", alpha=0.4)

ax.axvline(160, linestyle="--", color="firebrick")
ax.text(162, 0.015, "Patient: 160 mmHg\n(Z = 2.67)",
        color="firebrick", fontsize=9)

ax.set_title("Distribution of Systolic Blood Pressure in Healthy Adults")
ax.set_xlabel("Systolic Blood Pressure (mmHg)")
ax.set_ylabel("Density")
plt.tight_layout()
plt.show()
```

3.5. The Binomial Distribution

The **binomial distribution** models the number of “successes” in a fixed number of independent trials, where each trial has the same probability of success. Despite the name, a “success” does not have to be a good thing — it is simply the outcome we are counting.

3.5.1. Parameters

- n : the number of trials (a fixed, known integer)
- p : the probability of success on each trial (between 0 and 1)

3. Probability, Distributions, and the Language of Uncertainty

We write $X \sim \text{Binomial}(n, p)$, and the PMF is:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, 2, \dots, n$$

where $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ is the binomial coefficient (“n choose k”), which counts the number of ways to arrange k successes among n trials.

Mean: $E[X] = np$

Variance: $\text{Var}(X) = np(1 - p)$

3.5.2. Clinical Examples

Disease prevalence. In a city, the prevalence of Type 2 diabetes among adults is 12%. If you randomly sample 50 adults, the number with diabetes follows a $\text{Binomial}(50, 0.12)$ distribution. The expected number is $50 \times 0.12 = 6$.

Treatment response. A chemotherapy regimen achieves complete response in 35% of patients. If 20 patients are treated, the number who achieve complete response follows a $\text{Binomial}(20, 0.35)$ distribution.

Key assumptions: The binomial distribution assumes that (1) each trial is independent (one patient’s outcome does not affect another’s), (2) the probability of success is the same for every trial, and (3) the number of trials is fixed in advance. In practice, these assumptions are sometimes approximate (e.g., disease probability may vary with age), but the binomial is still a useful starting model.

3.5.3. R

3.5. The Binomial Distribution

```
library(tidyverse)

# Treatment response: 20 patients, 35% response rate
n <- 20
p <- 0.35

# Calculate probabilities for each possible outcome
outcomes <- tibble(
  k = 0:n,
  probability = dbinom(k, size = n, prob = p)
)

ggplot(outcomes, aes(x = k, y = probability)) +
  geom_col(fill = "steelblue", alpha = 0.8) +
  geom_vline(xintercept = n * p, linetype = "dashed", color = "firebrick") +
  annotate("text", x = n * p + 0.5, y = max(outcomes$probability),
    label = paste0("Expected value = ", n * p),
    hjust = 0, color = "firebrick") +
  labs(
    title = "Binomial Distribution: Treatment Response",
    subtitle = "n = 20 patients, p = 0.35 response rate",
    x = "Number of Responders",
    y = "Probability"
  ) +
  scale_x_continuous(breaks = 0:n) +
  theme_minimal()
```

3.5.4. Python

```
import numpy as np
import matplotlib.pyplot as plt
```

3. Probability, Distributions, and the Language of Uncertainty

```
from scipy import stats

n = 20
p = 0.35

k = np.arange(0, n + 1)
probabilities = stats.binom.pmf(k, n, p)

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(k, probabilities, color="steelblue", alpha=0.8)
ax.axvline(n * p, linestyle="--", color="firebrick")
ax.text(n * p + 0.3, max(probabilities), f"Expected value = {n * p}",
        color="firebrick", fontsize=9)

ax.set_title("Binomial Distribution: Treatment Response")
ax.set_xlabel("Number of Responders")
ax.set_ylabel("Probability")
ax.set_xticks(k)
plt.tight_layout()
plt.show()
```

3.6. The Poisson Distribution

The **Poisson distribution** models the number of events occurring in a fixed interval of time or space, when events happen independently at a constant average rate. It is the go-to distribution for modeling rare events.

3.6. The Poisson Distribution

3.6.1. Parameters

- λ (lambda): the average number of events per interval. This is both the mean and the variance.

We write $X \sim \text{Poisson}(\lambda)$, and the PMF is:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

Mean: $E[X] = \lambda$

Variance: $\text{Var}(X) = \lambda$

The fact that the mean equals the variance is a distinctive property of the Poisson distribution. When you see count data where the variance is much larger than the mean, the Poisson may not be a good fit (a condition called **overdispersion**).

3.6.2. Clinical Examples

Adverse drug reactions. A hospital reports an average of 2.5 serious adverse drug reactions per month. If these events are independent, the number in any given month follows approximately a $\text{Poisson}(2.5)$ distribution. The probability of zero serious reactions in a month is $P(X = 0) = e^{-2.5} \approx 0.082$, or about 8.2%.

Hospital admissions. An emergency department sees an average of 40 patients per shift. The number of arrivals per shift can be modeled with a $\text{Poisson}(40)$ distribution (though in practice, arrivals may cluster, violating independence).

Rare disease incidence. If a rare genetic disorder affects 1 in 100,000 people per year and a city has 500,000 people, we expect about 5 cases per year. The annual count follows approximately a $\text{Poisson}(5)$ distribution.

3. Probability, Distributions, and the Language of Uncertainty

3.6.3. R

```
library(tidyverse)

# Adverse drug reactions: average 2.5 per month
lambda <- 2.5
k <- 0:12

adr_data <- tibble(
  k = k,
  probability = dpois(k, lambda = lambda)
)

ggplot(adr_data, aes(x = k, y = probability)) +
  geom_col(fill = "darkorange", alpha = 0.8) +
  geom_vline(xintercept = lambda, linetype = "dashed", color = "firebrick") +
  labs(
    title = "Poisson Distribution: Adverse Drug Reactions per Month",
    subtitle = expression(paste(lambda, " = 2.5 reactions/month")),
    x = "Number of Adverse Reactions",
    y = "Probability"
  ) +
  scale_x_continuous(breaks = k) +
  theme_minimal()
```

3.6.4. Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

lam = 2.5
```

3.7. The Central Limit Theorem

```
k = np.arange(0, 13)
probs = stats.poisson.pmf(k, lam)

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(k, probs, color="darkorange", alpha=0.8)
ax.axvline(lam, linestyle="--", color="firebrick")
ax.set_title(f"Poisson Distribution: Adverse Drug Reactions per Month (lambda = {lam})")
ax.set_xlabel("Number of Adverse Reactions")
ax.set_ylabel("Probability")
ax.set_xticks(k)
plt.tight_layout()
plt.show()
```

3.7. The Central Limit Theorem

The **Central Limit Theorem (CLT)** is one of the most remarkable results in all of statistics. It states:

If you take sufficiently large random samples from *any* population (regardless of the population's distribution), the distribution of the sample means will be approximately normal.

More precisely: if $X_1, X_2, \dots, X_n$ are independent and identically distributed random variables with mean μ and standard deviation σ , then the sample mean $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ has a distribution that approaches $N(\mu, \sigma^2/n)$ as n grows large.

3.7.1. Why Is This Important?

The CLT is the reason the normal distribution is so central to statistics. It means that:

3. Probability, Distributions, and the Language of Uncertainty

1. **We can use normal-based methods** (t-tests, confidence intervals, regression) even when the underlying data are not normally distributed, as long as our sample size is large enough.
2. **Sample means are more predictable than individual observations.** The standard deviation of the sample mean ($\sigma/\sqrt{n}$) shrinks as n grows, meaning larger samples give more precise estimates.

3.7.2. How Large Is “Large Enough”?

It depends on how non-normal the underlying distribution is:

- If the population is already roughly symmetric, $n = 15\text{--}30$ is often sufficient.
- If the population is heavily skewed (e.g., income, hospital length of stay), you may need $n = 50$ or more.
- For extremely skewed distributions, $n > 100$ may be needed.

3.7.3. Simulation Demonstration

Let us demonstrate the CLT visually. We will take samples from a **right-skewed** distribution (exponential, which looks nothing like a bell curve) and show that the distribution of sample means becomes increasingly normal as the sample size grows.

3.7.4. R

```
library(tidyverse)

set.seed(42)

# Parameters
```

3.7. The Central Limit Theorem

```
n_simulations <- 5000 # Number of samples to draw
sample_sizes <- c(1, 5, 30, 100)

# For each sample size, draw many samples and compute the mean of each
clt_data <- map_dfr(sample_sizes, function(n) {
  means <- replicate(n_simulations, mean(rexp(n, rate = 1)))
  tibble(
    sample_size = paste0("n = ", n),
    sample_mean = means
  )
})

# Convert to ordered factor so panels appear in the right order
clt_data$sample_size <- factor(clt_data$sample_size,
                              levels = paste0("n = ", sample_sizes))

ggplot(clt_data, aes(x = sample_mean)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 50, fill = "steelblue", alpha = 0.7) +
  geom_density(color = "firebrick", linewidth = 0.8) +
  facet_wrap(~sample_size, scales = "free") +
  labs(
    title = "Central Limit Theorem in Action",
    subtitle = "Sampling from an Exponential(1) distribution (mean = 1, right-skewed)",
    x = "Sample Mean",
    y = "Density"
  ) +
  theme_minimal()
```

3. Probability, Distributions, and the Language of Uncertainty

3.7.5. Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

n_simulations = 5000
sample_sizes = [1, 5, 30, 100]

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

for i, n in enumerate(sample_sizes):
    means = [np.mean(np.random.exponential(1, n)) for _ in range(n_simulations)]
    axes[i].hist(means, bins=50, density=True, color="steelblue", alpha=0.7)
    axes[i].set_title(f"n = {n}")
    axes[i].set_xlabel("Sample Mean")
    axes[i].set_ylabel("Density")

fig.suptitle(
    "Central Limit Theorem in Action\n"
    "Sampling from an Exponential(1) distribution (right-skewed)",
    fontsize=13
)
plt.tight_layout()
plt.show()
```

Notice how the distribution of sample means changes:

- When $n = 1$, the distribution of “means” is just the original exponential distribution — heavily right-skewed.
- At $n = 5$, the skewness is already reduced.

3.8. Bayes' Theorem: Updating Beliefs with Evidence

- At $n = 30$, the distribution looks approximately bell-shaped.
- At $n = 100$, the distribution is very close to a perfect normal curve, and it is much narrower (less variable).

This is the CLT in action. It does not matter that we started with a skewed distribution — the means converge to normality.

3.8. Bayes' Theorem: Updating Beliefs with Evidence

3.8.1. The Idea

Bayes' theorem provides a mathematically precise way to update your beliefs when you receive new evidence. In clinical medicine, one of the most important applications is interpreting diagnostic tests.

Suppose a patient tests positive for a disease. What is the probability they actually have the disease? Your intuition might say “very high,” but as we will see, the answer depends critically on how common the disease is in the population.

3.8.2. The Formula

$$P(A | B) = \frac{P(B | A) \times P(A)}{P(B)}$$

In the context of diagnostic testing:

- $P(\text{Disease} | \text{Positive test})$ = the probability the patient has the disease, given a positive test result. This is called the **positive predictive value (PPV)**.
- $P(\text{Positive test} | \text{Disease})$ = the probability of a positive test result if the patient truly has the disease. This is the test's **sensitivity** (true positive rate).

3. Probability, Distributions, and the Language of Uncertainty

- $P(\text{Disease})$ = the probability of having the disease before the test, based on the population prevalence or clinical judgment. This is the **prior probability** or **pre-test probability**.
- $P(\text{Positive test})$ = the overall probability of a positive test result, accounting for both true positives and false positives.

We can expand the denominator using the **law of total probability**:

$$P(\text{Positive test}) = P(\text{Positive} \mid \text{Disease}) \times P(\text{Disease}) + P(\text{Positive} \mid \text{No disease}) \times P(\text{No disease})$$

Note that $P(\text{Positive} \mid \text{No disease})$ is the **false positive rate**, which equals $1 - \text{specificity}$.

3.8.3. Key Diagnostic Test Terminology

Table 3.1.: Diagnostic test performance metrics

Term	Definition	Formula
Sensitivity	Probability of a positive test given disease is present	$P(+ \mid D)$
Specificity	Probability of a negative test given disease is absent	$P(- \mid \bar{D})$
Positive Predictive Value (PPV)	Probability of disease given a positive test	$P(D \mid +)$
Negative Predictive Value (NPV)	Probability of no disease given a negative test	$P(\bar{D} \mid -)$
Prevalence	Proportion of the population with the disease	$P(D)$

3.8.4. Clinical Example: HIV Screening

Let us work through a concrete example. Consider an HIV screening test with the following characteristics:

- **Sensitivity:** 99.7% (the test correctly identifies 99.7% of people who have HIV)
- **Specificity:** 99.5% (the test correctly identifies 99.5% of people who do not have HIV)
- **Prevalence:** 0.3% (the proportion of the general population with HIV)

These numbers look excellent. But what happens when a randomly selected person tests positive?

Using Bayes' theorem:

$$\begin{aligned} PPV = P(D \mid +) &= \frac{P(+ \mid D) \times P(D)}{P(+ \mid D) \times P(D) + P(+ \mid \bar{D}) \times P(\bar{D})} \\ &= \frac{0.997 \times 0.003}{0.997 \times 0.003 + 0.005 \times 0.997} \\ &= \frac{0.002991}{0.002991 + 0.004985} \\ &= \frac{0.002991}{0.007976} \approx 0.375 \end{aligned}$$

The positive predictive value is only about **37.5%**! This means that if a person from the general population tests positive, there is only about a 37.5% chance they actually have HIV. More than 6 out of 10 positive results are false positives.

3. Probability, Distributions, and the Language of Uncertainty

This counterintuitive result arises because the disease is rare. Even with a highly accurate test, the large number of healthy people (99.7% of the population) generates many false positives that overwhelm the true positives from the small number of infected people.

This is why HIV screening programs use **confirmatory testing**: a second, different test is performed on all positive results. It is also why screening is more effective in high-prevalence populations (e.g., among people with known risk factors).

3.8.5. The Role of Prevalence

The PPV depends heavily on prevalence. Let us compute the PPV across a range of prevalence values:

3.8.6. R

```
library(tidyverse)

sensitivity <- 0.997
specificity <- 0.995

prevalence <- seq(0.001, 0.30, by = 0.001)

ppv <- (sensitivity * prevalence) /
  (sensitivity * prevalence + (1 - specificity) * (1 - prevalence))

npv <- (specificity * (1 - prevalence)) /
  (specificity * (1 - prevalence) + (1 - sensitivity) * prevalence)

test_data <- tibble(
  prevalence = prevalence,
```

3.8. Bayes' Theorem: Updating Beliefs with Evidence

```
PPV = ppv,
NPV = npv
) |>
  pivot_longer(cols = c(PPV, NPV), names_to = "metric", values_to = "value")

ggplot(test_data, aes(x = prevalence, y = value, color = metric)) +
  geom_line(linewidth = 1.2) +
  scale_x_continuous(labels = scales::percent_format()) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title = "How Prevalence Affects PPV and NPV",
    subtitle = "Sensitivity = 99.7%, Specificity = 99.5%",
    x = "Disease Prevalence",
    y = "Predictive Value",
    color = "Metric"
  ) +
  theme_minimal() +
  scale_color_manual(values = c("PPV" = "steelblue", "NPV" = "firebrick"))
```

3.8.7. Python

```
import numpy as np
import matplotlib.pyplot as plt

sensitivity = 0.997
specificity = 0.995

prevalence = np.arange(0.001, 0.301, 0.001)

ppv = (sensitivity * prevalence) / (
    sensitivity * prevalence + (1 - specificity) * (1 - prevalence)
)
```

3. Probability, Distributions, and the Language of Uncertainty

```
npv = (specificity * (1 - prevalence)) / (
    specificity * (1 - prevalence) + (1 - sensitivity) * prevalence
)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(prevalence * 100, ppv * 100, color="steelblue", linewidth=1.5, label="PPV")
ax.plot(prevalence * 100, npv * 100, color="firebrick", linewidth=1.5, label="NPV")
ax.set_xlabel("Disease Prevalence (%)")
ax.set_ylabel("Predictive Value (%)")
ax.set_title("How Prevalence Affects PPV and NPV\nSensitivity = 99.7%, Specificity = 99.7%")
ax.legend()
ax.set_ylim(0, 105)
plt.tight_layout()
plt.show()
```

The plot shows a crucial pattern: when prevalence is very low, even an excellent test produces many false positives (low PPV). As prevalence increases, PPV rises. Conversely, NPV is very high when prevalence is low (a negative result is very reassuring) but decreases as disease becomes more common.

Clinical takeaway: Always consider the pre-test probability (prevalence in the relevant population) when interpreting a diagnostic test result. A positive mammogram in a 25-year-old woman with no risk factors means something very different from the same result in a 60-year-old woman with a family history of breast cancer.

3.9. Exercises

💡 Exercise 1.1: Z-Scores and the Normal Distribution

A laboratory reports that fasting blood glucose levels in healthy adults follow a normal distribution with a mean of 90 mg/dL and a standard deviation of 10 mg/dL.

1. What is the Z-score for a patient with a fasting glucose of 115 mg/dL?
2. What proportion of healthy adults have a fasting glucose above 115 mg/dL?
3. What fasting glucose value corresponds to the 95th percentile?
4. Plot the normal distribution and shade the area above 115 mg/dL.

3.10. R

3. Probability, Distributions, and the Language of Uncertainty

```
mu <- 90
sigma <- 10
patient_value <- 115

# 1. Calculate the Z-score
z_score <- # YOUR CODE HERE

# 2. Proportion above 115 (upper tail probability)
prop_above <- # YOUR CODE HERE (hint: use pnorm())

# 3. 95th percentile
percentile_95 <- # YOUR CODE HERE (hint: use qnorm())

# 4. Plot (modify the blood pressure example from the chapter)
# YOUR CODE HERE
```

3.11. Python

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

mu = 90
sigma = 10
patient_value = 115

# 1. Calculate the Z-score
z_score = # YOUR CODE HERE

# 2. Proportion above 115
prop_above = # YOUR CODE HERE (hint: use stats.norm.sf() or 1 - stats.norm.cdf())

# 3. 95th percentile
percentile_95 = # YOUR CODE HERE (hint: use stats.norm.ppf())

# 4. Plot
# YOUR CODE HERE
```

Solution

3.12. R

3. Probability, Distributions, and the Language of Uncertainty

```
library(tidyverse)

mu <- 90
sigma <- 10
patient_value <- 115

# 1. Z-score
z_score <- (patient_value - mu) / sigma
cat("Z-score:", z_score, "\n")
# Z-score: 2.5

# 2. Proportion above 115
prop_above <- 1 - pnorm(patient_value, mean = mu, sd = sigma)
# Equivalently: pnorm(patient_value, mean = mu, sd = sigma, lower.tail = FALSE)
cat("Proportion above 115:", round(prop_above, 4), "\n")
# About 0.0062 or 0.62%

# 3. 95th percentile
percentile_95 <- qnorm(0.95, mean = mu, sd = sigma)
cat("95th percentile:", round(percentile_95, 1), "mg/dL\n")
# About 106.4 mg/dL

# 4. Plot
x <- seq(mu - 4 * sigma, mu + 4 * sigma, length.out = 300)
y <- dnorm(x, mean = mu, sd = sigma)
plot_data <- tibble(x = x, y = y)

ggplot(plot_data, aes(x = x, y = y)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_area(data = plot_data |> filter(x >= patient_value),
            fill = "firebrick", alpha = 0.4) +
  geom_vline(xintercept = patient_value, linetype = "dashed", color = "firebrick") +
  annotate("text", x = 118, y = 0.02,
           label = paste0("Glucose = ", patient_value, " mg/dL\nZ = ",
                           (patient_value - mu) / sigma),
           color = "firebrick", hjust = 0) +
  labs(
    title = "Distribution of Fasting Blood Glucose",
    x = "Fasting Blood Glucose (mg/dL)", y = "Density"
  ) +
  theme_minimal()
```


3.13. Python

3. Probability, Distributions, and the Language of Uncertainty

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

mu = 90
sigma = 10
patient_value = 115

# 1. Z-score
z_score = (patient_value - mu) / sigma
print(f"Z-score: {z_score}")
# Z-score: 2.5

# 2. Proportion above 115
prop_above = 1 - stats.norm.cdf(patient_value, loc=mu, scale=sigma)
# Or equivalently: stats.norm.sf(patient_value, loc=mu, scale=sigma)
print(f"Proportion above 115: {prop_above:.4f}")
# About 0.0062 or 0.62%

# 3. 95th percentile
percentile_95 = stats.norm.ppf(0.95, loc=mu, scale=sigma)
print(f"95th percentile: {percentile_95:.1f} mg/dL")
# About 106.4 mg/dL

# 4. Plot
x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, 300)
y = stats.norm.pdf(x, loc=mu, scale=sigma)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(x, y, color="steelblue", linewidth=1.5)
x_fill = x[x >= patient_value]
y_fill = stats.norm.pdf(x_fill, loc=mu, scale=sigma)
ax.fill_between(x_fill, y_fill, color="firebrick", alpha=0.4)
ax.axvline(patient_value, linestyle="--", color="firebrick")
ax.text(117, 0.02, f"Glucose = {patient_value} mg/dL\nZ = {z_score}",
        color="firebrick")
ax.set_title("Distribution of Fasting Blood Glucose")
ax.set_xlabel("Fasting Blood Glucose (mg/dL)")
ax.set_ylabel("Density")
plt.tight_layout()
plt.show()
```

💡 Exercise 1.2: Binomial Distribution — Vaccine Efficacy

A COVID-19 vaccine has an efficacy of 90%, meaning that among people exposed to the virus, a vaccinated person has a 90% chance of NOT getting symptomatic disease (equivalently, a 10% chance of a breakthrough infection).

In a group of 25 vaccinated individuals who are all exposed to the virus:

1. What is the expected number of breakthrough infections?
2. What is the probability of zero breakthrough infections?
3. What is the probability of 5 or more breakthrough infections?
4. Plot the full probability distribution.

3.14. R

```
n <- 25
p <- 0.10 # probability of breakthrough infection

# 1. Expected number
expected <- # YOUR CODE HERE

# 2. P(X = 0)
prob_zero <- # YOUR CODE HERE (hint: dbinom())

# 3. P(X >= 5)
prob_five_or_more <- # YOUR CODE HERE (hint: use pbinom with lower.tail)

# 4. Plot the distribution
# YOUR CODE HERE
```

3.15. Python

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

n = 25
p = 0.10

# 1. Expected number
expected = # YOUR CODE HERE

# 2. P(X = 0)
prob_zero = # YOUR CODE HERE (hint: stats.binom.pmf())

# 3. P(X >= 5)
prob_five_or_more = # YOUR CODE HERE (hint: stats.binom.sf())

# 4. Plot
# YOUR CODE HERE
```

Solution

3.16. R

```

library(tidyverse)

n <- 25
p <- 0.10

# 1. Expected number
expected <- n * p
cat("Expected breakthrough infections:", expected, "\n")
# 2.5

# 2.  $P(X = 0)$ 
prob_zero <- dbinom(0, size = n, prob = p)
cat("P(X = 0):", round(prob_zero, 4), "\n")
# About 0.0718

# 3.  $P(X \geq 5)$ 
prob_five_or_more <- 1 - pbinom(4, size = n, prob = p)
# Equivalently: pbinom(4, size = n, prob = p, lower.tail = FALSE)
cat("P(X >= 5):", round(prob_five_or_more, 4), "\n")
# About 0.0980

# 4. Plot
outcomes <- tibble(
  k = 0:n,
  probability = dbinom(k, size = n, prob = p)
)

ggplot(outcomes, aes(x = k, y = probability)) +
  geom_col(aes(fill = k >= 5), alpha = 0.8) +
  scale_fill_manual(values = c("FALSE" = "steelblue", "TRUE" = "firebrick"),
    guide = "none") +
  geom_vline(xintercept = expected, linetype = "dashed") +
  labs(
    title = "Breakthrough Infections in 25 Vaccinated Individuals",
    subtitle = "Binomial(25, 0.10); red bars = 5 or more infections",
    x = "Number of Breakthrough Infections", y = "Probability"
  ) +
  scale_x_continuous(breaks = seq(0, 25, 2)) +
  theme_minimal()

```

3.17. Python

```

from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

n = 25
p = 0.10

# 1. Expected number
expected = n * p
print(f"Expected breakthrough infections: {expected}")
# 2.5

# 2.  $P(X = 0)$ 
prob_zero = stats.binom.pmf(0, n, p)
print(f"P(X = 0): {prob_zero:.4f}")
# About 0.0718

# 3.  $P(X \geq 5)$ 
prob_five_or_more = stats.binom.sf(4, n, p) # sf = survival function = 1 - cdf
print(f"P(X >= 5): {prob_five_or_more:.4f}")
# About 0.0980

# 4. Plot
k = np.arange(0, n + 1)
probs = stats.binom.pmf(k, n, p)
colors = ["firebrick" if ki >= 5 else "steelblue" for ki in k]

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(k, probs, color=colors, alpha=0.8)
ax.axvline(expected, linestyle="--", color="black")
ax.set_title("Breakthrough Infections in 25 Vaccinated Individuals\nBinomial(25, 0.10)")
ax.set_xlabel("Number of Breakthrough Infections")
ax.set_ylabel("Probability")
ax.set_xticks(range(0, 26, 2))
plt.tight_layout()
plt.show()

```

3. Probability, Distributions, and the Language of Uncertainty

💡 Exercise 1.3: Poisson Distribution — Emergency Department Visits

An emergency department receives an average of 8 trauma cases per day.

1. What is the probability of receiving exactly 8 trauma cases tomorrow?
2. What is the probability of receiving 12 or more?
3. On how many days per year (365 days) would you expect 0 trauma cases?
4. Create a bar plot of the $\text{Poisson}(8)$ distribution from 0 to 20.

3.18. R

```
lambda <- 8

# 1. P(X = 8)
prob_exactly_8 <- # YOUR CODE HERE (hint: dpois())

# 2. P(X >= 12)
prob_12_or_more <- # YOUR CODE HERE (hint: ppois with lower.tail)

# 3. Expected days with 0 cases in a year
prob_zero <- # YOUR CODE HERE
expected_zero_days <- # YOUR CODE HERE

# 4. Plot
# YOUR CODE HERE
```


3.19. Python

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

lam = 8

# 1. P(X = 8)
prob_exactly_8 = # YOUR CODE HERE (hint: stats.poisson.pmf())

# 2. P(X >= 12)
prob_12_or_more = # YOUR CODE HERE (hint: stats.poisson.sf())

# 3. Expected days with 0 cases
prob_zero = # YOUR CODE HERE
expected_zero_days = # YOUR CODE HERE

# 4. Plot
# YOUR CODE HERE
```

Solution

3.20. R

3. Probability, Distributions, and the Language of Uncertainty

```
library(tidyverse)

lambda <- 8

# 1. P(X = 8)
prob_exactly_8 <- dpois(8, lambda = lambda)
cat("P(X = 8):", round(prob_exactly_8, 4), "\n")
# About 0.1396

# 2. P(X >= 12)
prob_12_or_more <- ppois(11, lambda = lambda, lower.tail = FALSE)
cat("P(X >= 12):", round(prob_12_or_more, 4), "\n")
# About 0.1121

# 3. Expected days with 0 cases
prob_zero <- dpois(0, lambda = lambda)
expected_zero_days <- 365 * prob_zero
cat("P(X = 0):", format(prob_zero, scientific = TRUE), "\n")
cat("Expected zero-case days per year:", round(expected_zero_days, 3),
# P(0) is very small (~0.000335), so about 0.12 days/year

# 4. Plot
k <- 0:20
poisson_data <- tibble(
  k = k,
  probability = dpois(k, lambda = lambda)
)

ggplot(poisson_data, aes(x = k, y = probability)) +
  geom_col(fill = "darkorange", alpha = 0.8) +
  geom_vline(xintercept = lambda, linetype = "dashed", color = "firebrn
  labs(
    title = "Poisson Distribution: Trauma Cases per Day",
    subtitle = expression(paste(lambda, " = 8")),
    x = "Number of Trauma Cases", y = "Probability"
  ) +
  scale_x_continuous(breaks = k) +
  theme_minimal()
```

3.21. Python

3. Probability, Distributions, and the Language of Uncertainty

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

lam = 8

# 1. P(X = 8)
prob Exactly_8 = stats.poisson.pmf(8, lam)
print(f"P(X = 8): {prob Exactly_8:.4f}")
# About 0.1396

# 2. P(X >= 12)
prob 12_or_more = stats.poisson.sf(11, lam)
print(f"P(X >= 12): {prob 12_or_more:.4f}")
# About 0.1121

# 3. Expected days with 0 cases
prob_zero = stats.poisson.pmf(0, lam)
expected_zero_days = 365 * prob_zero
print(f"P(X = 0): {prob_zero:.6f}")
print(f"Expected zero-case days per year: {expected_zero_days:.3f}")
# Very small, about 0.12 days per year

# 4. Plot
k = np.arange(0, 21)
probs = stats.poisson.pmf(k, lam)

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(k, probs, color="darkorange", alpha=0.8)
ax.axvline(lam, linestyle="--", color="firebrick")
ax.set_title(f"Poisson Distribution: Trauma Cases per Day (lambda = {lam})")
ax.set_xlabel("Number of Trauma Cases")
ax.set_ylabel("Probability")
ax.set_xticks(k)
plt.tight_layout()
plt.show()
```

 Exercise 1.4: Bayes' Theorem — Interpreting a Cancer Screening Test

A breast cancer screening mammogram has the following characteristics:

- Sensitivity: 87% (detects 87% of actual cancers)
 - Specificity: 95% (correctly clears 95% of women without cancer)
 - Prevalence of breast cancer in women aged 50–59 undergoing screening: 2%
1. Calculate the positive predictive value (PPV): if a woman in this age group has a positive mammogram, what is the probability she actually has breast cancer?
 2. Calculate the negative predictive value (NPV).
 3. How does the PPV change if we consider a high-risk population where prevalence is 10%?
 4. Create a plot showing PPV as a function of prevalence (from 0.1% to 30%).

3.22. R

3. Probability, Distributions, and the Language of Uncertainty

```
sensitivity <- 0.87
specificity <- 0.95
prevalence <- 0.02

# 1. PPV
ppv <- # YOUR CODE HERE

# 2. NPV
npv <- # YOUR CODE HERE

# 3. PPV with prevalence = 10%
ppv_high_risk <- # YOUR CODE HERE

# 4. Plot PPV vs. prevalence
# YOUR CODE HERE
```

3.23. Python

```
import numpy as np
import matplotlib.pyplot as plt

sensitivity = 0.87
specificity = 0.95
prevalence = 0.02

# 1. PPV
ppv = # YOUR CODE HERE

# 2. NPV
npv = # YOUR CODE HERE

# 3. PPV with prevalence = 10%
ppv_high_risk = # YOUR CODE HERE

# 4. Plot PPV vs. prevalence
# YOUR CODE HERE
```

Solution

3.24. R

3. Probability, Distributions, and the Language of Uncertainty

```
library(tidyverse)

sensitivity <- 0.87
specificity <- 0.95
prevalence <- 0.02

# 1. PPV
ppv <- (sensitivity * prevalence) /
  (sensitivity * prevalence + (1 - specificity) * (1 - prevalence))
cat("PPV (prevalence = 2%):", round(ppv, 4), "\n")
# About 0.2623 or 26.2%

# 2. NPV
npv <- (specificity * (1 - prevalence)) /
  (specificity * (1 - prevalence) + (1 - sensitivity) * prevalence)
cat("NPV (prevalence = 2%):", round(npv, 4), "\n")
# About 0.9972 or 99.7%

# 3. PPV with high-risk prevalence
prev_high <- 0.10
ppv_high_risk <- (sensitivity * prev_high) /
  (sensitivity * prev_high + (1 - specificity) * (1 - prev_high))
cat("PPV (prevalence = 10%):", round(ppv_high_risk, 4), "\n")
# About 0.6588 or 65.9%

# 4. Plot
prev_range <- seq(0.001, 0.30, by = 0.001)
ppv_curve <- (sensitivity * prev_range) /
  (sensitivity * prev_range + (1 - specificity) * (1 - prev_range))

plot_data <- tibble(prevalence = prev_range, PPV = ppv_curve)

ggplot(plot_data, aes(x = prevalence, y = PPV)) +
  geom_line(color = "steelblue", linewidth = 1.2) +
  geom_point(data = tibble(prevalence = c(0.02, 0.10),
                           PPV = c(ppv, ppv_high_risk)),
             color = "firebrick", size = 3) +
  annotate("text", x = 0.04, y = ppv - 0.03,
           label = paste0("General pop (2%): PPV = ", round(ppv * 100,
                                                             1)),
           hjust = 0, color = "firebrick", size = 3.5) +
  annotate("text", x = 0.12, y = ppv_high_risk - 0.03,
           label = paste0("High risk (10%): PPV = ", round(ppv_high_risk * 100,
                                                             1)),
           hjust = 0, color = "firebrick", size = 3.5) +
  scale_x_continuous(labels = scales::percent_format()) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0,
                                                                    100)) +
  labs(
    title = "Mammography PPV Depends on Prevalence",
    subtitle = "Sensitivity = 87%, Specificity = 95%",
    x = "Prevalence",
```


3.25. Python

3. Probability, Distributions, and the Language of Uncertainty

```
import numpy as np
import matplotlib.pyplot as plt

sensitivity = 0.87
specificity = 0.95
prevalence = 0.02

# 1. PPV
ppv = (sensitivity * prevalence) / (
    sensitivity * prevalence + (1 - specificity) * (1 - prevalence)
)
print(f"PPV (prevalence = 2%): {ppv:.4f}")
# About 0.2623 or 26.2%

# 2. NPV
npv = (specificity * (1 - prevalence)) / (
    specificity * (1 - prevalence) + (1 - sensitivity) * prevalence
)
print(f"NPV (prevalence = 2%): {npv:.4f}")
# About 0.9972 or 99.7%

# 3. PPV with high-risk prevalence
prev_high = 0.10
ppv_high = (sensitivity * prev_high) / (
    sensitivity * prev_high + (1 - specificity) * (1 - prev_high)
)
print(f"PPV (prevalence = 10%): {ppv_high:.4f}")
# About 0.6588 or 65.9%

# 4. Plot
prev_range = np.arange(0.001, 0.301, 0.001)
ppv_curve = (sensitivity * prev_range) / (
    sensitivity * prev_range + (1 - specificity) * (1 - prev_range)
)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(prev_range * 100, ppv_curve * 100, color="steelblue", linewidth=2)
ax.plot([2, 10], [ppv * 100, ppv_high * 100], "o", color="firebrick",
        markersize=10)
ax.annotate(f"General pop (2%): PPV = {ppv*100:.1f}%",
            xy=(2, ppv*100), xytext=(5, ppv*100 - 5),
            color="firebrick", fontsize=9,
            arrowprops=dict(arrowstyle="->", color="firebrick"))
ax.annotate(f"High risk (10%): PPV = {ppv_high*100:.1f}%",
            xy=(10, ppv_high*100), xytext=(13, ppv_high*100 - 5),
            color="firebrick", fontsize=9,
            arrowprops=dict(arrowstyle="->", color="firebrick"))
ax.set_xlabel("Prevalence (%)")
ax.set_ylabel("Positive Predictive Value (%)")
ax.set_title("Mammography PPV Depends on Prevalence\nSensitivity = 87%
```

💡 Exercise 1.5: Central Limit Theorem — Hands-On Simulation

Demonstrate the Central Limit Theorem yourself by simulating sample means from a non-normal distribution.

1. Generate 10,000 individual observations from a **Poisson distribution** with $\lambda = 3$ (this distribution is right-skewed). Plot a histogram.
2. Now, for each of the following sample sizes ($n = 5, 15, 50, 200$), draw 5,000 random samples of that size, compute the mean of each sample, and plot the distribution of sample means.
3. Overlay a normal curve on the $n = 50$ histogram to show how well the normal approximation fits.

3.26. R

```
library(tidyverse)

set.seed(123)
lambda <- 3

# 1. Histogram of raw Poisson data
raw_data <- rpois(10000, lambda = lambda)
# YOUR CODE HERE: plot a histogram

# 2. CLT simulation for different sample sizes
sample_sizes <- c(5, 15, 50, 200)
# YOUR CODE HERE: for each n, draw 5000 samples, compute means, plot

# 3. Overlay normal curve on n = 50
# YOUR CODE HERE
```

3.27. Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(123)
lam = 3

# 1. Histogram of raw Poisson data
raw_data = np.random.poisson(lam, 10000)
# YOUR CODE HERE: plot a histogram

# 2. CLT simulation
sample_sizes = [5, 15, 50, 200]
# YOUR CODE HERE

# 3. Overlay normal curve
# YOUR CODE HERE
```

Solution

3.28. R

```

library(tidyverse)

set.seed(123)
lambda <- 3
n_sim <- 5000

# 1. Raw Poisson data
raw_data <- rpois(10000, lambda = lambda)
ggplot(tibble(x = raw_data), aes(x = x)) +
  geom_histogram(binwidth = 1, fill = "steelblue", color = "white", alpha = 0.8) +
  labs(title = "Raw Poisson(3) Data (right-skewed)",
       x = "Value", y = "Count") +
  theme_minimal()

# 2. CLT simulation
sample_sizes <- c(5, 15, 50, 200)

clt_data <- map_dfr(sample_sizes, function(n) {
  means <- replicate(n_sim, mean(rpois(n, lambda = lambda)))
  tibble(n = paste0("n = ", n), sample_mean = means)
})

clt_data$n <- factor(clt_data$n, levels = paste0("n = ", sample_sizes))

ggplot(clt_data, aes(x = sample_mean)) +
  geom_histogram(aes(y = after_stat(density)),
                bins = 40, fill = "steelblue", alpha = 0.7) +
  facet_wrap(~n, scales = "free_y") +
  labs(title = "CLT: Distribution of Sample Means from Poisson(3)",
       x = "Sample Mean", y = "Density") +
  theme_minimal()

# 3. Overlay normal curve on n = 50
n50_means <- replicate(n_sim, mean(rpois(50, lambda = lambda)))
theoretical_sd <- sqrt(lambda / 50)

ggplot(tibble(x = n50_means), aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)),
                bins = 40, fill = "steelblue", alpha = 0.7) +
  stat_function(fun = dnorm, args = list(mean = lambda, sd = theoretical_sd),
               color = "firebrick", linewidth = 1.2) +
  labs(title = "Sample Means (n=50) with Normal Approximation Overlay",
       subtitle = paste0("Theoretical: N(", lambda, ", ", round(theoretical_sd^2, 4), ", ",
       x = "Sample Mean", y = "Density") +
  theme_minimal()

```

3.29. Python

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(123)
lam = 3
n_sim = 5000

# 1. Raw Poisson data
raw_data = np.random.poisson(lam, 10000)
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(raw_data, bins=range(0, 15), color="steelblue", alpha=0.8, edgecolor="white")
ax.set_title("Raw Poisson(3) Data (right-skewed)")
ax.set_xlabel("Value")
ax.set_ylabel("Count")
plt.tight_layout()
plt.show()

# 2. CLT simulation
sample_sizes = [5, 15, 50, 200]
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

for i, n in enumerate(sample_sizes):
    means = [np.mean(np.random.poisson(lam, n)) for _ in range(n_sim)]
    axes[i].hist(means, bins=40, density=True, color="steelblue", alpha=0.7)
    axes[i].set_title(f"n = {n}")
    axes[i].set_xlabel("Sample Mean")
    axes[i].set_ylabel("Density")

fig.suptitle("CLT: Distribution of Sample Means from Poisson(3)", fontsize=13)
plt.tight_layout()
plt.show()

# 3. Overlay normal curve on n = 50
n50_means = [np.mean(np.random.poisson(lam, 50)) for _ in range(n_sim)]
theoretical_sd = np.sqrt(lam / 50)

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(n50_means, bins=40, density=True, color="steelblue", alpha=0.7, label="Simula
x_range = np.linspace(min(n50_means), max(n50_means), 200)
ax.plot(x_range, stats.norm.pdf(x_range, loc=lam, scale=theoretical_sd),
        color="firebrick", linewidth=1.5, label="Normal approx.")
ax.set_title("Sample Means (n=50) with Normal Approximation Overlay")
ax.set_xlabel("Sample Mean")
ax.set_ylabel("Density")
ax.legend()
plt.tight_layout()
plt.show()

```

3.30. Chapter Summary

In this chapter, we covered the foundational language of probability and statistics:

- **Probability** quantifies uncertainty. The frequentist interpretation focuses on long-run frequencies; the Bayesian interpretation treats probability as a degree of belief. Both are useful.
- **Random variables** can be discrete (countable values) or continuous (any value in a range).
- The **normal distribution** is the most important distribution in statistics, characterized by its mean and standard deviation. Z-scores standardize values relative to the distribution.
- The **binomial distribution** models the number of successes in a fixed number of independent trials — useful for modeling treatment response rates, disease prevalence, and more.
- The **Poisson distribution** models counts of rare events occurring at a constant rate — useful for adverse events, hospital admissions, and rare disease incidence.
- The **Central Limit Theorem** tells us that sample means are approximately normally distributed regardless of the underlying population distribution, as long as the sample size is large enough. This justifies the widespread use of normal-based statistical methods.
- **Bayes' theorem** provides a principled way to update probabilities in light of new evidence. In diagnostic testing, it reveals that the predictive value of a test depends critically on disease prevalence, not just sensitivity and specificity.

These concepts underpin everything that follows in this course. In the next chapter, we will build on this foundation to discuss statistical inference — how to draw conclusions about populations from samples.

3.31. References and Further Reading

- Wasserman, L. (2004). *All of Statistics: A Concise Course in Statistical Inference*. Springer. A rigorous but accessible introduction to probability and statistics. Chapters 1–3 cover the material in this chapter at a more mathematical level.
- Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian Data Analysis* (3rd ed.). CRC Press. The definitive textbook on Bayesian statistics. Chapter 1 provides an excellent introduction to Bayesian thinking.
- Harrell, F. E. (2015). *Regression Modeling Strategies: With Applications to Linear Models, Logistic and Ordinal Regression, and Survival Analysis* (2nd ed.). Springer. Chapter 2 discusses the role of probability and distributions in the context of regression modeling for health sciences.
- Altman, D. G., & Bland, J. M. (1994). “Diagnostic tests 1: Sensitivity and specificity.” *BMJ*, 308(6943), 1552. A classic short paper explaining sensitivity, specificity, and predictive values for a clinical audience.
- Altman, D. G., & Bland, J. M. (1994). “Diagnostic tests 2: Predictive values.” *BMJ*, 309(6947), 102. The companion paper on PPV and NPV, with a clear explanation of how prevalence affects predictive values.
- Gigerenzer, G., Gaissmaier, W., Kurz-Milcke, E., Schwartz, L. M., & Woloshin, S. (2007). “Helping doctors and patients make sense of health statistics.” *Psychological Science in the Public Interest*, 8(2), 53–96. An excellent discussion of how to communicate probabilistic information about diagnostic tests and treatments.
- Cetinkaya-Rundel, M., & Hardin, J. (2024). *Introduction to Modern Statistics* (2nd ed.). OpenIntro. Freely available at <https://openintro->

3. Probability, Distributions, and the Language of Uncertainty

ims2.netlify.app/. Chapters 3–4 provide an accessible introduction to probability and distributions with many worked examples.

- Rice, J. A. (2007). *Mathematical Statistics and Data Analysis* (3rd ed.). Thomson Brooks/Cole. A thorough treatment of probability distributions for students comfortable with calculus.
- Motulsky, H. (2018). *Intuitive Biostatistics: A Nonmathematical Guide to Statistical Thinking* (4th ed.). Oxford University Press. A superb introduction to biostatistics for medical professionals, with clear explanations of probability, distributions, and Bayesian reasoning.

4. Statistical Inference: What We Can and Cannot Conclude

4.1. Introduction

Imagine you run a clinical trial comparing a new antihypertensive drug to placebo. After 12 weeks, the treatment group's mean systolic blood pressure dropped by 8.3 mmHg more than the placebo group. But how confident should you be in that number? Would you see the same result if you repeated the trial? Could the difference be explained by chance alone? And even if the effect is “real,” is 8.3 mmHg enough to matter for patient health?

These are the questions that **statistical inference** answers. Inference is the bridge between the data you *observed* (your sample) and the truth you *want to know* (the population). This chapter will equip you to cross that bridge carefully, avoiding the many pitfalls that trip up even experienced researchers.

We will use a single running example throughout: a randomized controlled trial (RCT) of **Drug X** versus placebo for systolic blood pressure reduction. This trial enrolled 200 participants (100 per arm), measured blood pressure at baseline and 12 weeks, and found a mean difference of 8.3 mmHg (SD = 14.2 mmHg) favoring Drug X.

4. Statistical Inference: What We Can and Cannot Conclude

4.2. Point Estimates and Sampling Variability

4.2.1. What Is a Point Estimate?

A **point estimate** is a single number calculated from your sample that serves as your best guess for an unknown population parameter. Common examples:

What you want to know	Point estimate
Population mean blood pressure reduction	Sample mean ($\bar{x}$)
Population proportion who respond to treatment	Sample proportion ($\hat{p}$)
Population odds ratio for treatment effect	Sample odds ratio ($\widehat{OR}$)

In our trial, the point estimate for the mean difference in blood pressure reduction is **8.3 mmHg**. This is a single number – our best guess – but it comes with uncertainty.

4.2.2. Sampling Variability

If we ran the same trial again with 200 new participants from the same population, we would almost certainly get a *different* point estimate. Maybe 7.1 mmHg, or 9.8 mmHg, or 6.5 mmHg. This natural fluctuation from sample to sample is called **sampling variability**.

The key insight: **every point estimate is “wrong” in the sense that it almost never equals the exact population parameter.** The question is *how wrong* it might be.

4.2. Point Estimates and Sampling Variability

4.2.3. The Standard Error

The **standard error (SE)** quantifies how much a point estimate is expected to vary across repeated samples. For a sample mean:

$$SE(\bar{x}) = \frac{s}{\sqrt{n}}$$

where s is the sample standard deviation and n is the sample size.

For the difference between two independent means:

$$SE(\bar{x}_1 - \bar{x}_2) = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

In our trial, assuming equal SDs of 14.2 mmHg in each group:

$$SE = \sqrt{\frac{14.2^2}{100} + \frac{14.2^2}{100}} = \sqrt{2.016 + 2.016} = \sqrt{4.033} \approx 2.01 \text{ mmHg}$$

This tells us: if we repeated this trial many times, the sample mean difference would typically vary by about 2 mmHg from trial to trial.

! Important

Standard deviation vs. standard error: The standard deviation (SD = 14.2) describes variability among *individual patients*. The standard error (SE = 2.01) describes variability among *sample means*. They answer different questions. The SD is always larger than the SE because means are less variable than individual observations.

4. Statistical Inference: What We Can and Cannot Conclude

4.3. Confidence Intervals: What They Actually Mean

4.3.1. Constructing a 95% Confidence Interval

A **95% confidence interval (CI)** for our mean difference is:

$$\bar{x} \pm t^* \times SE = 8.3 \pm 1.97 \times 2.01 = (4.34, 12.26) \text{ mmHg}$$

where t^* is the critical value from the t -distribution (approximately 1.97 for large samples).

4.3.2. The Correct Interpretation

Here is one of the most commonly misunderstood concepts in statistics.

Common Misconception

Wrong: “There is a 95% probability that the true mean difference lies between 4.34 and 12.26 mmHg.”

Right: “If we repeated this trial many times and computed a 95% CI each time, approximately 95% of those intervals would contain the true mean difference.”

Why does this distinction matter? The true population parameter is a fixed (unknown) number – it does not have a probability of being “in” or “out” of an interval. The *interval* is the random quantity, not the parameter. Each time you run a new study, you get a new interval. Some of those intervals capture the truth; some do not. A 95% CI means that the *procedure* works 95% of the time.

In practice, a useful way to think about it: the confidence interval gives you a range of plausible values for the population parameter, given the

4.4. P-values: What They Are, What They Are Not

data you observed. Values inside the interval are reasonably compatible with your data; values outside are not.

4.3.3. What the Width Tells You

The width of a confidence interval reflects **precision**. A narrow CI means you have a precise estimate; a wide CI means substantial uncertainty. Width depends on:

1. **Sample size** – larger n gives narrower CIs
2. **Variability** – less variable data gives narrower CIs
3. **Confidence level** – 99% CIs are wider than 95% CIs (more confidence requires more hedging)

In our trial, the CI of (4.34, 12.26) is moderately wide – the true effect could be as small as about 4 mmHg or as large as 12 mmHg. This matters clinically: a 4 mmHg reduction and a 12 mmHg reduction may lead to very different treatment decisions.

4.4. P-values: What They Are, What They Are Not

4.4.1. Definition

A **p-value** is the probability of observing a test statistic as extreme as (or more extreme than) the one you calculated, *assuming the null hypothesis is true*.

For our trial, the null hypothesis is $H_0 : \mu_{\text{Drug X}} - \mu_{\text{Placebo}} = 0$ (no difference). The test statistic is:

$$t = \frac{8.3 - 0}{2.01} = 4.13$$

4. Statistical Inference: What We Can and Cannot Conclude

With 198 degrees of freedom, $p < 0.001$.

4.4.2. What a P-value Is

- The p-value measures the **compatibility between your data and the null hypothesis**
- A small p-value means your data would be unlikely if the null hypothesis were true
- It quantifies evidence against H_0 , not evidence *for* any particular alternative

4.4.3. What a P-value Is NOT

The American Statistical Association published a landmark statement in 2016 (Wasserstein & Lazar, 2016) because p-values are so widely misinterpreted. Here are the key points:

P-value Misconceptions – From the ASA Statement

1. **A p-value is NOT the probability that the null hypothesis is true.** A p-value of 0.03 does *not* mean there is a 3% chance the drug doesn't work.
2. **A p-value is NOT the probability that the result was produced by chance.** It is calculated *assuming* chance is the only explanation.
3. **Statistical significance does NOT mean clinical or practical importance.** A tiny, meaningless effect can have $p < 0.001$ with a large enough sample.
4. **A large p-value does NOT mean the null hypothesis is true.** It means you failed to find strong evidence against it – absence of evidence is not evidence of absence.
5. **The p-value does NOT measure the size of an effect.** It

4.4. *P*-values: What They Are, What They Are Not

mixes effect size with sample size. Always report the effect size and confidence interval alongside the *p*-value.

4.4.4. The “Statistical Significance” Problem in Medicine

The threshold of $p < 0.05$ is deeply entrenched in medical research. But there is nothing magical about 0.05. It is an arbitrary convention, originally proposed as a convenient rule of thumb by Ronald Fisher, not as a rigid decision boundary.

Problems with dichotomizing at $p = 0.05$:

- A result with $p = 0.049$ is treated as fundamentally different from $p = 0.051$, even though they are essentially identical in evidential strength.
- Journals preferentially publish “significant” results, creating **publication bias**.
- Researchers may unconsciously (or consciously) engage in **p-hacking** – trying multiple analyses until one crosses the 0.05 threshold.
- In large clinical databases, nearly everything is “statistically significant” because of enormous sample sizes, regardless of clinical relevance.

A better approach: Report the point estimate, confidence interval, and *p*-value. Interpret them together. Judge results by their clinical importance, not just their *p*-value. As the ASA statement concludes: “No single index should substitute for scientific reasoning.”

4. Statistical Inference: What We Can and Cannot Conclude

4.5. Effect Sizes That Matter Clinically

4.5.1. Why Effect Sizes Matter

Statistical significance tells you whether an effect is likely to be non-zero. It says nothing about whether the effect is *large enough to matter*. In medicine, this distinction is critical: a drug that lowers blood pressure by 0.5 mmHg might achieve $p < 0.001$ in a trial of 50,000 people, but no clinician would prescribe it based on that effect.

4.5.2. Cohen's d (Standardized Mean Difference)

Cohen's d expresses the difference between two means in standard deviation units:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}}$$

For our trial: $d = 8.3/14.2 = 0.58$, which is conventionally considered a “medium” effect.

Cohen's d	Interpretation
0.2	Small
0.5	Medium
0.8	Large

i Note

These benchmarks (from Cohen, 1988) are rough guidelines, not rigid rules. A “small” effect by Cohen's standards might be highly clinically meaningful (e.g., a small reduction in mortality), while a “large” effect

might be trivial in another context. Always interpret effect sizes in the clinical context.

4.5.3. Odds Ratios and Risk Ratios

For binary outcomes (e.g., death, hospitalization, disease incidence), effect sizes are typically reported as:

Risk Ratio (Relative Risk):

$$RR = \frac{P(\text{event} \mid \text{treatment})}{P(\text{event} \mid \text{control})}$$

An RR of 0.75 means the treatment group had 75% the risk of the control group, i.e., a 25% relative risk reduction.

Odds Ratio:

$$OR = \frac{\text{odds of event in treatment}}{\text{odds of event in control}} = \frac{p_1/(1-p_1)}{p_2/(1-p_2)}$$

Odds ratios are the natural output of logistic regression. When the outcome is rare (< 10%), the OR approximates the RR. When the outcome is common, the OR exaggerates the effect compared to the RR.

Risk Difference (Absolute Risk Reduction):

$$RD = P(\text{event} \mid \text{control}) - P(\text{event} \mid \text{treatment})$$

If 15% of control patients are hospitalized vs. 10% of treatment patients, the RD = 0.05 (or 5 percentage points).

4. Statistical Inference: What We Can and Cannot Conclude

4.5.4. Number Needed to Treat (NNT)

The NNT is perhaps the most clinician-friendly effect size:

$$NNT = \frac{1}{RD} = \frac{1}{|p_1 - p_2|}$$

Using our example: $NNT = 1/0.05 = 20$. This means you need to treat 20 patients to prevent one additional hospitalization. A low NNT is desirable. For reference:

- Statins for secondary prevention of MI: NNT around 30-80
- Aspirin for acute MI: NNT around 40
- Antibiotics for strep pharyngitis to prevent rheumatic fever: NNT around 4,000

4.5.5. Statistically Significant but Clinically Meaningless

Consider a large pragmatic trial ($n = 20,000$) of a new diabetes drug that lowers HbA1c by 0.1% compared to the current standard of care, with $p = 0.002$. Is this clinically meaningful?

Almost certainly not. A 0.1% reduction in HbA1c is below the threshold most endocrinologists consider clinically relevant (typically 0.3-0.5%). The p-value is tiny because the sample size is enormous – even a trivial true effect becomes “significant” with enough data.

This is why you should **always** report and interpret effect sizes (with confidence intervals), not just p-values.

4.6. Type I and Type II Errors

4.6.1. The Two Kinds of Mistakes

When making a decision based on a hypothesis test, there are two ways to be wrong:

	H_0 is true (no effect)	H_0 is false (real effect)
Reject H_0	Type I error (α)	Correct (Power)
Fail to reject H_0	Correct	Type II error (β)

- **Type I error (false positive):** You conclude the drug works, but it actually doesn't. Probability = α (typically set at 0.05).
- **Type II error (false negative):** You conclude the drug doesn't work, but it actually does. Probability = β .

4.6.2. Statistical Power

Power = $1 - \beta$ = the probability of correctly detecting a real effect.

Power depends on:

1. **Effect size** – larger effects are easier to detect
2. **Sample size** – more data provides more power
3. **Significance level (α)** – higher α gives more power (but more false positives)
4. **Variability** – less noise makes signals easier to detect

In clinical trials, we typically aim for **80% power** (i.e., $\beta = 0.20$). This means we accept a 20% chance of missing a true effect – a remarkably high false-negative rate that is often underappreciated.

4. Statistical Inference: What We Can and Cannot Conclude

Clinical Implication

Many clinical trials are **underpowered** – they have too few participants to detect realistic effect sizes. A “negative” trial (one that fails to reach $p < 0.05$) does not necessarily mean the treatment is ineffective. It may simply mean the trial was too small. Always check the confidence interval: if it includes both clinically meaningful and null effects, the trial was inconclusive, not negative.

4.6.3. A Power Calculation Example

Suppose you are planning a trial of Drug X and expect a 5 mmHg difference (SD = 15 mmHg). How many participants per group do you need for 80% power at $\alpha = 0.05$?

$$n = \frac{2(z_{\alpha/2} + z_{\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1.96 + 0.84)^2 \times 15^2}{5^2} = \frac{2 \times 7.84 \times 225}{25} \approx 141 \text{ per group}$$

You need approximately 141 participants per group, or 282 total.

4.7. Multiple Testing

4.7.1. The Problem

When you perform multiple statistical tests, the probability of at least one false positive increases rapidly. If you test 20 independent hypotheses at $\alpha = 0.05$, the probability of at least one Type I error is:

$$1 - (1 - 0.05)^{20} = 1 - 0.95^{20} = 1 - 0.358 = 0.642$$

4.7. Multiple Testing

That is a **64.2% chance** of at least one false positive. This is a massive problem in:

- **Genomics/GWAS:** Testing millions of genetic variants simultaneously
- **Clinical trials with multiple endpoints:** Primary, secondary, and exploratory outcomes
- **Subgroup analyses:** Testing treatment effects in men vs. women, old vs. young, etc.

4.7.2. Bonferroni Correction

The simplest fix: divide α by the number of tests.

$$\alpha_{\text{adjusted}} = \frac{\alpha}{m}$$

where m is the number of tests. If you run 20 tests, you require $p < 0.05/20 = 0.0025$ for each test.

Pros: Simple, controls the **family-wise error rate** (FWER) – the probability of *any* false positives.

Cons: Very conservative. As m grows, it becomes extremely difficult to detect real effects. With 1 million SNPs in a GWAS, you need $p < 5 \times 10^{-8}$.

4.7.3. False Discovery Rate (FDR) Control

The **Benjamini-Hochberg procedure** (1995) controls the **false discovery rate** – the expected proportion of rejected hypotheses that are false positives. This is less stringent than Bonferroni and more appropriate when you expect many true positives among your tests.

Steps:

4. Statistical Inference: What We Can and Cannot Conclude

1. Rank all m p-values from smallest to largest: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$
2. Find the largest k such that $p_{(k)} \leq \frac{k}{m} \times q$, where q is the desired FDR (e.g., 0.05)
3. Reject all hypotheses with $p \leq p_{(k)}$

In genomics, FDR control at $q = 0.05$ means: among all the genes we declare “significant,” we expect no more than 5% to be false positives. This is often a more useful guarantee than Bonferroni’s promise of *no* false positives.

4.7.4. When Does Multiple Testing Matter?

Scenario	Correction needed?
Pre-specified primary endpoint in a clinical trial	No (single test)
One primary + several secondary endpoints	Yes
Subgroup analyses	Yes (or treat as exploratory)
Genome-wide association study	Yes (millions of tests)
Exploratory data analysis	No formal correction, but be transparent

4.8. Exercises



Exercise 2.1: Confidence Interval Simulation

Simulate 100 trials, each with $n=50$ per group, true mean difference = 5, $SD = 10$. Compute a 95% CI for each trial. What proportion of CIs contain the true value of 5?

4.9. R

4. Statistical Inference: What We Can and Cannot Conclude

```
set.seed(42)
n_sims <- 100
n_per_group <- 50
true_diff <- 5
sd_val <- 10

contains_true <- numeric(n_sims)

for (i in 1:n_sims) {
  group1 <- rnorm(n_per_group, mean = 0, sd = sd_val)
  group2 <- rnorm(n_per_group, mean = true_diff, sd = sd_val)

  test_result <- t.test(group2, group1)
  ci <- test_result$conf.int

  contains_true[i] <- (ci[1] <= true_diff & ci[2] >= true_diff)
}

cat("Proportion of CIs containing true value:", mean(contains_true), "\n")

# Bonus: visualize the CIs
library(ggplot2)
ci_data <- data.frame(
  sim = 1:n_sims,
  lower = numeric(n_sims),
  upper = numeric(n_sims),
  contains = logical(n_sims)
)

set.seed(42)
for (i in 1:n_sims) {
  group1 <- rnorm(n_per_group, mean = 0, sd = sd_val)
  group2 <- rnorm(n_per_group, mean = true_diff, sd = sd_val)
  test_result <- t.test(group2, group1)
  ci_data$lower[i] <- test_result$conf.int[1]
  ci_data$upper[i] <- test_result$conf.int[2]
106 ci_data$contains[i] <- (ci_data$lower[i] <= true_diff & ci_data$upper[i]
  }

ggplot(ci_data, aes(x = sim, y = (lower + upper) / 2, color = contains)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.3) +
  geom_hline(yintercept = true_diff, linetype = "dashed", color = "blue") +
  scale_color_manual(values = c("red", "darkgreen")) +
  labs(x = "Simulation", y = "Mean Difference",
       title = "95% Confidence Intervals from 100 Simulated Trials") +
  theme_minimal()
```

4.10. Python

4. Statistical Inference: What We Can and Cannot Conclude

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

np.random.seed(42)
n_sims = 100
n_per_group = 50
true_diff = 5
sd_val = 10

contains_true = []
lower_bounds = []
upper_bounds = []

for i in range(n_sims):
    group1 = np.random.normal(0, sd_val, n_per_group)
    group2 = np.random.normal(true_diff, sd_val, n_per_group)

    diff = np.mean(group2) - np.mean(group1)
    se = np.sqrt(np.var(group1, ddof=1)/n_per_group + np.var(group2, ddof=1)/n_per_group)
    t_crit = stats.t.ppf(0.975, df=n_per_group*2 - 2)

    lower = diff - t_crit * se
    upper = diff + t_crit * se

    lower_bounds.append(lower)
    upper_bounds.append(upper)
    contains_true.append(lower <= true_diff <= upper)

print(f"Proportion of CIs containing true value: {np.mean(contains_true):.2f}")

# Visualize
fig, ax = plt.subplots(figsize=(10, 6))
for i in range(n_sims):
    color = "green" if contains_true[i] else "red"
    ax.plot([i, i], [lower_bounds[i], upper_bounds[i]], color=color, linewidth=2)

ax.axhline(y=true_diff, color="blue", linestyle="--", label="True difference")
ax.set_xlabel("Simulation")
ax.set_ylabel("Mean Difference")
ax.set_title("95% Confidence Intervals from 100 Simulated Trials")
ax.legend()
plt.tight_layout()
plt.show()
```

Solution

You should find that approximately 95 out of 100 CIs contain the true value of 5. The exact number will vary due to randomness, but it should be close to 95. The red intervals in the plot are the ones that “missed” – they do not contain the true value. This is the correct interpretation of a 95% CI: the *procedure* captures the truth 95% of the time.

 Exercise 2.2: P-value Interpretation

A clinical trial reports: “The new drug reduced 30-day mortality from 12% to 9% ($p = 0.04$).” For each statement below, indicate whether it is TRUE or FALSE and explain why.

1. There is a 96% probability that the drug truly reduces mortality.
2. There is a 4% probability the result is due to chance.
3. If the drug had no effect, we would see a difference this large or larger about 4% of the time.
4. The drug reduces mortality by 3 percentage points.

4.11. R

4. Statistical Inference: What We Can and Cannot Conclude

```
# This is a conceptual exercise, but let's verify the numbers:
p_control <- 0.12
p_treatment <- 0.09

risk_difference <- p_control - p_treatment
risk_ratio <- p_treatment / p_control
nnt <- 1 / risk_difference

cat("Risk Difference:", risk_difference, "\n")
cat("Risk Ratio:", round(risk_ratio, 3), "\n")
cat("NNT:", round(nnt, 1), "\n")
```

4.12. Python

```
# This is a conceptual exercise, but let's verify the numbers:
p_control = 0.12
p_treatment = 0.09

risk_difference = p_control - p_treatment
risk_ratio = p_treatment / p_control
nnt = 1 / risk_difference

print(f"Risk Difference: {risk_difference}")
print(f"Risk Ratio: {risk_ratio:.3f}")
print(f"NNT: {nnt:.1f}")
```

Solution

1. **FALSE.** The p-value does not give the probability that the drug works. It does not tell you the probability of any hypothesis being true or false.

2. **FALSE.** The p-value is not the probability that the result is “due to chance.” It is calculated *assuming* chance is the only explanation.
3. **TRUE.** This is the correct definition of a p-value: if the null hypothesis (no difference) were true, we would observe a result this extreme or more extreme approximately 4% of the time.
4. **TRUE (as a point estimate).** The absolute risk reduction is $12\% - 9\% = 3$ percentage points. The $\text{NNT} = 1/0.03 = 33.3$, meaning you need to treat about 34 patients to prevent one death. Whether this is clinically meaningful depends on the drug’s cost, side effects, and the patient population.

Exercise 2.3: Multiple Testing Correction

You are analyzing gene expression data and have tested 1,000 genes for differential expression between a treatment and control group. You find 60 genes with $p < 0.05$. Apply both Bonferroni correction and BH-FDR correction. How many genes remain significant under each approach?

4.13. R

4. Statistical Inference: What We Can and Cannot Conclude

```
# Simulate 1000 p-values: 950 null (uniform), 50 truly differentially expressed
set.seed(123)
n_genes <- 1000
n_true <- 50

# Null p-values are uniformly distributed
p_null <- runif(n_genes - n_true, 0, 1)

# True effects: p-values will tend to be small
p_true <- rbeta(n_true, 1, 20) # skewed toward 0

p_values <- c(p_null, p_true)

# How many nominally significant?
cat("Nominally significant (p < 0.05):", sum(p_values < 0.05), "\n")

# Bonferroni correction
p_bonferroni <- p.adjust(p_values, method = "bonferroni")
cat("Significant after Bonferroni:", sum(p_bonferroni < 0.05), "\n")

# BH-FDR correction
p_fdr <- p.adjust(p_values, method = "BH")
cat("Significant after BH-FDR:", sum(p_fdr < 0.05), "\n")

# Compare
cat("\nBonferroni is much more conservative.\n")
cat("FDR retains more discoveries while controlling the false discovery rate\n")
```


4.14. Python

```
import numpy as np
from statsmodels.stats.multitest import multipletests

np.random.seed(123)
n_genes = 1000
n_true = 50

# Null p-values (uniform) and true effect p-values (small)
p_null = np.random.uniform(0, 1, n_genes - n_true)
p_true = np.random.beta(1, 20, n_true)

p_values = np.concatenate([p_null, p_true])

print(f"Nominally significant (p < 0.05): {np.sum(p_values < 0.05)}")

# Bonferroni correction
reject_bonf, pvals_bonf, _, _ = multipletests(p_values, alpha=0.05, method='bonferroni')
print(f"Significant after Bonferroni: {np.sum(reject_bonf)}")

# BH-FDR correction
reject_fdr, pvals_fdr, _, _ = multipletests(p_values, alpha=0.05, method='fdr_bh')
print(f"Significant after BH-FDR: {np.sum(reject_fdr)}")

print("\nBonferroni is much more conservative.")
print("FDR retains more discoveries while controlling the false discovery rate.")
```

Solution

With the simulated data (seed = 123), you should see approximately:

4. Statistical Inference: What We Can and Cannot Conclude

- **Nominally significant:** ~60-110 genes (includes both true and false positives)
- **Bonferroni:** ~15-30 genes (very conservative, few false positives but many missed true effects)
- **BH-FDR:** ~30-50 genes (moderate, good balance of sensitivity and false positive control)

The Bonferroni correction controls the family-wise error rate (probability of *any* false positives), while BH-FDR controls the expected proportion of false positives among rejections. In genomics, where you expect many true positives, FDR is usually more appropriate because Bonferroni's stringency causes you to miss too many real discoveries.

Exercise 2.4: Power Analysis

You are planning a trial to test whether a lifestyle intervention reduces HbA1c in type 2 diabetes patients. You expect a 0.4% reduction (SD = 1.2%). Compute the required sample size per group for 80% power at $\alpha = 0.05$. Then recompute for 90% power. How does the sample size change?

4.15. R

```

# Using the power.t.test function
# 80% power
result_80 <- power.t.test(
  delta = 0.4,      # expected difference
  sd = 1.2,         # standard deviation
  sig.level = 0.05,
  power = 0.80,
  type = "two.sample",
  alternative = "two.sided"
)
cat("Sample size per group (80% power):", ceiling(result_80$n), "\n")

# 90% power
result_90 <- power.t.test(
  delta = 0.4,
  sd = 1.2,
  sig.level = 0.05,
  power = 0.90,
  type = "two.sample",
  alternative = "two.sided"
)
cat("Sample size per group (90% power):", ceiling(result_90$n), "\n")

cat("\nIncreasing power from 80% to 90% requires about",
    round((ceiling(result_90$n) / ceiling(result_80$n) - 1) * 100),
    "% more participants per group.\n")

```

4.16. Python

4. Statistical Inference: What We Can and Cannot Conclude

```
from statsmodels.stats.power import TTestIndPower

analysis = TTestIndPower()

# Cohen's d = delta / sd = 0.4 / 1.2
effect_size = 0.4 / 1.2

# 80% power
n_80 = analysis.solve_power(effect_size=effect_size, alpha=0.05, power=0.80,
                           alternative='two-sided')
print(f"Sample size per group (80% power): {int(np.ceil(n_80))}")

# 90% power
n_90 = analysis.solve_power(effect_size=effect_size, alpha=0.05, power=0.90,
                           alternative='two-sided')
print(f"Sample size per group (90% power): {int(np.ceil(n_90))}")

print(f"\nIncreasing from 80% to 90% power requires about "
      f"{int(round((n_90/n_80 - 1) * 100))}% more participants per group.")
```

Solution

With an effect size of 0.4% and SD of 1.2% (Cohen's $d = 0.33$):

- **80% power:** approximately 143 per group (286 total)
- **90% power:** approximately 191 per group (382 total)

Going from 80% to 90% power requires roughly 33% more participants. This illustrates the diminishing returns of increasing power – each additional percentage point of

power costs more participants. Most trials settle for 80% power as a reasonable compromise.

4.17. Key Takeaways

1. **Point estimates are always uncertain.** Report confidence intervals, not just point estimates.
2. **Confidence intervals describe the precision of your estimate.** They tell you the range of plausible values for the population parameter.
3. **P-values measure compatibility with the null hypothesis.** They are not the probability that the null is true, nor the probability of a chance finding.
4. **Statistical significance is not clinical significance.** Always report and interpret effect sizes.
5. **NNT is the most clinician-friendly effect size** for binary outcomes.
6. **Underpowered studies produce inconclusive results,** not evidence that a treatment is ineffective.
7. **Multiple testing inflates false positives.** Use Bonferroni (conservative) or FDR (less conservative) corrections as appropriate.

4.18. References and Further Reading

- Wasserstein, R. L., & Lazar, N. A. (2016). The ASA statement on p-values: Context, process, and purpose. *The American Statistician*, 70(2), 129-133.
- Wasserstein, R. L., Schirm, A. L., & Lazar, N. A. (2019). Moving to a world beyond “ $p < 0.05$.” *The American Statistician*, 73(sup1), 1-19.

4. Statistical Inference: What We Can and Cannot Conclude

- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.
- Vittinghoff, E., Glidden, D. V., Shiboski, S. C., & McCulloch, C. E. (2012). *Regression Methods in Biostatistics: Linear, Logistic, Survival, and Repeated Measures Models* (2nd ed.). Springer.
- Steyerberg, E. W. (2019). *Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating* (2nd ed.). Springer.
- Smits, L. J. M., et al. (2026). Recommendations for clinical prediction model development, validation, and updating. *BMJ*.
- Greenland, S., Senn, S. J., Rothman, K. J., et al. (2016). Statistical tests, P values, confidence intervals, and power: A guide to misinterpretations. *European Journal of Epidemiology*, 31(4), 337-350.
- Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289-300.
- Altman, D. G., & Bland, J. M. (1995). Absence of evidence is not evidence of absence. *BMJ*, 311(7003), 485.

5. Regression: The Workhorse of Clinical Research

5.1. Introduction

If you read clinical research papers, you will encounter regression in nearly every one. It is by far the most commonly used statistical method in medicine. Whether researchers are adjusting for confounders in an observational study, building a risk prediction model, or estimating the effect of a treatment, regression is the tool they reach for.

This chapter covers the foundations: linear regression for continuous outcomes and logistic regression for binary outcomes. We will work through clinical examples in detail, discuss assumptions and diagnostics, and build your intuition for interpreting regression output – a skill you will use throughout your career.

5.2. Why Regression?

Regression serves two distinct (but related) purposes in clinical research:

1. **Estimating adjusted effects.** In an observational study comparing smokers to non-smokers, smokers are also older, more likely to be male, and more likely to drink alcohol. Simple comparisons confound the smoking effect with these other variables. Regression lets you

5. Regression: The Workhorse of Clinical Research

estimate the effect of smoking *while holding age, sex, and alcohol constant* – what we call “adjusting for confounders.”

2. **Predicting outcomes.** A cardiologist wants to estimate a patient’s 10-year risk of coronary heart disease (CHD) based on their age, sex, smoking status, blood pressure, and cholesterol. Regression can combine these predictors into a single risk score. The Framingham Risk Score, one of the most widely used tools in medicine, is essentially a logistic regression model.

These two goals require different approaches to model building and evaluation, as we will see in later chapters. For now, we focus on the mechanics.

5.3. Simple Linear Regression

5.3.1. The Setup

Suppose we have data from 150 adults and want to understand the relationship between **age** (in years) and **systolic blood pressure** (SBP, in mmHg). We suspect that blood pressure increases with age.

Simple linear regression fits a straight line through the data:

$$\text{SBP}_i = \beta_0 + \beta_1 \times \text{Age}_i + \epsilon_i$$

where:

- β_0 is the **intercept** – the predicted SBP when Age = 0 (often not directly meaningful)
- β_1 is the **slope** – the predicted change in SBP for each one-year increase in age
- ϵ_i is the **residual** – the difference between the observed and predicted SBP for person i

5.3. Simple Linear Regression

We assume: $\epsilon_i \sim N(0, \sigma^2)$ – the residuals are normally distributed with mean zero and constant variance.

5.3.2. Fitting the Line

The method of **ordinary least squares (OLS)** finds the values of β_0 and β_1 that minimize the sum of squared residuals:

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

This has a closed-form solution:

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

5.3.3. Interpreting the Output

Suppose our regression yields:

$$\widehat{\text{SBP}} = 90.5 + 0.65 \times \text{Age}$$

Intercept (90.5 mmHg): The predicted SBP when Age = 0. This is a mathematical necessity, not a clinical statement – no one in our study is zero years old. The intercept anchors the line but is rarely interpreted on its own.

Slope (0.65 mmHg/year): For each additional year of age, systolic blood pressure is predicted to increase by 0.65 mmHg, on average. This is the key quantity of interest. A 10-year age difference corresponds to a predicted 6.5 mmHg difference in SBP.

5. Regression: The Workhorse of Clinical Research

! Important

Correlation is not causation. The slope tells you about the *association* between age and blood pressure. It does not mean that aging *causes* blood pressure to rise by exactly 0.65 mmHg per year. There may be confounders (e.g., weight gain, decreased physical activity, dietary changes) that both increase with age and raise blood pressure. To estimate causal effects, you need additional assumptions or study designs (e.g., randomized trials, instrumental variables).

5.3.4. Hypothesis Testing for the Slope

The standard test asks: is β_1 significantly different from zero?

$$H_0 : \beta_1 = 0 \quad (\text{no linear relationship})$$

$$H_A : \beta_1 \neq 0$$

The test statistic is: $t = \hat{\beta}_1 / SE(\hat{\beta}_1)$

If the output gives $\hat{\beta}_1 = 0.65$, $SE = 0.08$, then $t = 0.65/0.08 = 8.13$, which is highly significant ($p < 0.001$). But remember from Chapter 2: report the confidence interval for β_1 as well. Here: $0.65 \pm 1.96 \times 0.08 = (0.49, 0.81)$. We are quite confident that each year of age is associated with a 0.49 to 0.81 mmHg increase in SBP.

5.4. Multiple Linear Regression

5.4.1. Adding Predictors

In practice, we rarely care about just one predictor. Let us add **sex** (0 = female, 1 = male) and **BMI** (kg/m²) to our model:

5.4. Multiple Linear Regression

$$\text{SBP}_i = \beta_0 + \beta_1 \times \text{Age}_i + \beta_2 \times \text{Male}_i + \beta_3 \times \text{BMI}_i + \epsilon_i$$

5.4.2. Interpreting “Adjusted” Coefficients

Suppose the output is:

Variable	Coefficient	SE	p-value
Intercept	72.3	6.1	< 0.001
Age	0.58	0.07	< 0.001
Male	4.2	1.8	0.021
BMI	0.92	0.21	< 0.001

Each coefficient is interpreted as the effect of that variable **holding all other variables constant** (also called “adjusting for” or “controlling for” the other variables):

- **Age (0.58):** Each additional year of age is associated with a 0.58 mmHg higher SBP, *after adjusting for sex and BMI*. Notice this is slightly lower than the 0.65 from simple regression – some of the apparent age effect was actually due to BMI increasing with age.
- **Male (4.2):** Males have, on average, 4.2 mmHg higher SBP than females, *after adjusting for age and BMI*.
- **BMI (0.92):** Each 1 kg/m² increase in BMI is associated with a 0.92 mmHg higher SBP, *after adjusting for age and sex*.

i Note

The phrase “after adjusting for” is critical. It means: if we compare two people who are the same age and sex, but differ by 1 unit of BMI, we predict that the person with the higher BMI has 0.92 mmHg higher

5. Regression: The Workhorse of Clinical Research

SBP. This is the power of multiple regression – it lets you isolate the association with one variable while holding others constant.

5.5. Assumptions of Linear Regression

Linear regression relies on four key assumptions, sometimes remembered by the acronym **LINE**:

5.5.1. 1. Linearity

The relationship between each predictor and the outcome must be approximately linear (after accounting for other predictors). A curved relationship will produce biased estimates and poor predictions.

How to check: Plot residuals vs. each predictor, or residuals vs. fitted values. Look for systematic patterns (curves, U-shapes). If you see curvature, consider adding polynomial terms (e.g., Age²), using splines, or transforming the variable.

5.5.2. 2. Independence

Observations must be independent of each other. This is violated when you have:

- Repeated measures on the same patient
- Patients clustered within hospitals
- Time-series data

Violations do not bias coefficients but severely affect standard errors, p-values, and confidence intervals. If you have non-independent data, you need methods like mixed models or GEE (covered in later chapters).

5.5.3. 3. Normality of Residuals

The residuals (ϵ_i) should be approximately normally distributed. This matters most for small samples. With large samples ($n > 30$ -50 per group), the Central Limit Theorem makes inference robust to non-normality.

How to check: Q-Q plot (residuals plotted against theoretical normal quantiles). Points should fall approximately on a straight line.

5.5.4. 4. Equal Variance (Homoscedasticity)

The spread of residuals should be roughly constant across all levels of the predicted values. If residuals “fan out” as predicted values increase (heteroscedasticity), standard errors are biased.

How to check: Plot residuals vs. fitted values. The spread should be roughly uniform. If it fans out, consider transforming the outcome (e.g., log transformation) or using robust standard errors.

5.5.5. Diagnostic Plots

The standard set of diagnostic plots includes:

1. **Residuals vs. Fitted** – checks linearity and homoscedasticity
2. **Q-Q Plot** – checks normality of residuals
3. **Scale-Location** – another check for homoscedasticity
4. **Residuals vs. Leverage** – identifies influential observations (Cook’s distance)

Practical Advice

No real dataset perfectly satisfies all assumptions. The question is whether violations are severe enough to invalidate your conclusions.

5. Regression: The Workhorse of Clinical Research

Linear regression is reasonably robust to moderate violations, especially with large samples. However, extreme non-linearity or strong heteroscedasticity can seriously distort your results.

5.6. Logistic Regression for Binary Outcomes

5.6.1. Why Not Linear Regression for Binary Outcomes?

Many outcomes in medicine are binary: alive/dead, disease/no disease, hospitalized/not. You might think you could code the outcome as 0/1 and use linear regression. But this causes problems:

- Predicted values can fall below 0 or above 1, which are impossible for probabilities
- The residuals are not normally distributed
- The variance is not constant (it depends on the predicted probability)

Logistic regression solves these problems by modeling the **log-odds** (logit) of the outcome:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

where $p = P(Y = 1)$ is the probability of the event.

5.6.2. The Logit Link

The **logit** function transforms a probability (bounded between 0 and 1) into a value that can range from $-\infty$ to $+\infty$:

5.6. Logistic Regression for Binary Outcomes

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

The inverse (the **logistic function**) converts back:

$$p = \frac{e^{\beta_0 + \beta_1 x_1 + \dots}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots}} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots)}}$$

This ensures predicted probabilities always fall between 0 and 1.

5.6.3. Interpreting Coefficients as Odds Ratios

In logistic regression, the coefficients are on the log-odds scale, which is not intuitive. We exponentiate them to get **odds ratios (OR)**:

$$OR = e^{\beta}$$

5.6.4. Clinical Example: Predicting 10-year CHD Risk

Let us build a model predicting 10-year coronary heart disease (CHD) risk from age, sex, smoking status, total cholesterol, and systolic blood pressure. This is similar to the Framingham Risk Score.

Suppose our logistic regression output is:

Variable	Coefficient (β)	SE	OR (e^{β})	95% CI for OR	p-value
Intercept	-8.45	0.92	—	—	< 0.001
Age (per year)	0.065	0.012	1.067	(1.042, 1.093)	< 0.001

5. Regression: The Workhorse of Clinical Research

Variable	Coefficient (β)	SE	OR (e^β)	95% CI for OR	p-value
Male	0.72	0.19	2.054	(1.414, 2.985)	< 0.001
Current smoker	0.58	0.21	1.786	(1.182, 2.698)	0.006
Total cholesterol (per 10 mg/dL)	0.11	0.04	1.116	(1.032, 1.207)	0.006
SBP (per 10 mmHg)	0.18	0.05	1.197	(1.085, 1.321)	< 0.001

Interpretation:

- **Age (OR = 1.067):** Each additional year of age is associated with 6.7% higher odds of CHD, adjusted for sex, smoking, cholesterol, and SBP.
- **Male (OR = 2.054):** Males have approximately twice the odds of developing CHD compared to females, after adjustment.
- **Current smoker (OR = 1.786):** Current smokers have about 79% higher odds of CHD compared to non-smokers, after adjustment.
- **Total cholesterol (OR = 1.116 per 10 mg/dL):** Each 10 mg/dL increase in total cholesterol is associated with 11.6% higher odds of CHD.
- **SBP (OR = 1.197 per 10 mmHg):** Each 10 mmHg increase in SBP is associated with about 20% higher odds of CHD.

5.6. Logistic Regression for Binary Outcomes

5.6.5. Predicted Probabilities

You can use the model to compute individual risk. For a 55-year-old male smoker with cholesterol of 240 mg/dL and SBP of 150 mmHg:

$$\begin{aligned}\text{logit}(p) &= -8.45 + 0.065(55) + 0.72(1) + 0.58(1) + 0.11(24) + 0.18(15) \\ &= -8.45 + 3.575 + 0.72 + 0.58 + 2.64 + 2.70 = 1.765\end{aligned}$$

$$p = \frac{e^{1.765}}{1 + e^{1.765}} = \frac{5.84}{6.84} = 0.854$$

Wait – that is an 85.4% predicted probability of CHD, which seems too high. This is because our coefficients are illustrative; real Framingham coefficients would produce more calibrated predictions. The important point is the *method*: plug in patient values, compute the linear predictor, and convert to a probability using the logistic function.

i Note

Odds ratios vs. risk ratios: Remember from Chapter 2 that odds ratios exaggerate the effect when the outcome is common. If the baseline risk is 5%, an OR of 2.0 corresponds to a risk ratio of approximately 1.9. But if the baseline risk is 40%, an OR of 2.0 corresponds to a risk ratio of only about 1.4. In logistic regression, you always get ORs; converting to risk ratios requires additional steps. For common outcomes, consider using log-binomial regression or Poisson regression with robust standard errors to obtain risk ratios directly.

5.7. Model Fit

5.7.1. R-squared (Linear Regression)

R^2 is the proportion of variance in the outcome explained by the predictors:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

- $R^2 = 0$: the model explains none of the variance (no better than the mean)
- $R^2 = 1$: the model explains all the variance (perfect prediction)

In clinical research, R^2 values are often modest (0.10 - 0.40) because human biology is variable. An R^2 of 0.25 means your model explains 25% of the variance in blood pressure – the remaining 75% is due to factors not in your model, measurement error, and inherent biological variability.

Adjusted R^2 penalizes for the number of predictors, preventing the illusion of improvement from adding noise variables:

$$R^2_{\text{adj}} = 1 - \frac{(1 - R^2)(n - 1)}{n - k - 1}$$

where k is the number of predictors. Always report adjusted R^2 when comparing models with different numbers of predictors.

5.7.2. AIC (Akaike Information Criterion)

AIC balances model fit against complexity:

$$AIC = -2\log(L) + 2k$$

5.8. Categorical Predictors and Dummy Variables

where L is the likelihood and k is the number of parameters. Lower AIC is better. AIC is useful for comparing non-nested models (models that are not subsets of each other). Unlike R^2 , AIC has no absolute interpretation – it is only meaningful relative to other models fit to the same data.

5.7.3. Deviance (Logistic Regression)

In logistic regression, we do not have a traditional R^2 . Instead, we use **deviance**, which plays a similar role to the sum of squares in linear regression:

$$\text{Deviance} = -2 \log(L)$$

A **likelihood ratio test** compares the deviance of your model to a null model (intercept only). Pseudo- R^2 measures (e.g., Nagelkerke, McFadden) attempt to provide an R^2 -like summary, but they do not have the same clean interpretation as linear R^2 .

5.8. Categorical Predictors and Dummy Variables

5.8.1. The Problem

Regression requires numeric predictors. How do you include a variable like **race** (White, Black, Hispanic, Asian, Other)?

5.8.2. Dummy (Indicator) Variables

Create $k - 1$ binary variables for a categorical predictor with k levels. One level is chosen as the **reference category**:

5. Regression: The Workhorse of Clinical Research

Patient	White	Black	Hispanic	Asian
White patient	1	0	0	0
Black patient	0	1	0	0
Hispanic patient	0	0	1	0
Asian patient	0	0	0	1
Other patient	0	0	0	0

Here, “Other” is the reference category (all dummies = 0). Each coefficient represents the difference between that group and the reference group.



Warning

The choice of reference category changes the coefficients but NOT the model’s predictions or overall fit. It only changes which comparisons are directly reported. Choose a reference category that makes clinical sense – often the most common group or the control/standard-of-care group.

5.8.3. Interpreting Dummy Variable Coefficients

If the coefficient for Black (vs. Other) is 3.8, it means: “On average, Black patients have 3.8 mmHg higher SBP than Other patients, holding all other variables constant.”

Most statistical software handles this automatically. In R, factors are automatically converted to dummy variables. In Python (statsmodels or scikit-learn), you may need to create them explicitly or use formula notation.

5.9. Interaction Terms

5.9.1. When Effects Depend on Each Other

An **interaction** occurs when the effect of one predictor depends on the level of another predictor. For example:

- The effect of a drug might differ between men and women
- The effect of age on blood pressure might be steeper for smokers than non-smokers
- The effect of exercise on weight loss might depend on baseline BMI

5.9.2. Specifying an Interaction

To test whether the age-SBP relationship differs by sex:

$$\text{SBP} = \beta_0 + \beta_1 \text{Age} + \beta_2 \text{Male} + \beta_3 (\text{Age} \times \text{Male}) + \epsilon$$

The interaction term β_3 captures *how much the slope of Age differs for males vs. females*.

5.9.3. Interpreting Interaction Terms

Suppose we get:

Variable	Coefficient
Intercept	85.0
Age	0.50
Male	-5.0
Age x Male	0.20

5. Regression: The Workhorse of Clinical Research

For **females** (Male = 0):

$$\text{SBP} = 85.0 + 0.50 \times \text{Age}$$

For **males** (Male = 1):

$$\begin{aligned}\text{SBP} &= 85.0 + 0.50 \times \text{Age} + (-5.0)(1) + 0.20 \times \text{Age}(1) \\ &= 80.0 + 0.70 \times \text{Age}\end{aligned}$$

So the age slope is 0.50 mmHg/year for females but 0.70 mmHg/year for males. The interaction coefficient (0.20) is the *difference in slopes*. Males' blood pressure increases more steeply with age than females'.

! Important

Main effects rule: When you include an interaction term, you must also include both “main effects” (the individual terms). Never include Age $\times$ Male without also including Age and Male separately. Dropping main effects changes the interpretation of the interaction term in misleading ways.

💡 When to Test for Interactions

Do not test every possible interaction – this leads to multiple testing problems and overfitting. Only test interactions that are:

1. **Pre-specified** based on clinical reasoning (e.g., “We hypothesize the drug works differently in men vs. women”)
2. **Scientifically plausible** (there is a biological mechanism for effect modification)
3. **Adequately powered** (interaction tests require much larger samples than main effect tests – roughly 4x the sample size)

5.10. Exercises

Exercise 3.1: Simple Linear Regression

Using simulated clinical data, fit a simple linear regression of systolic blood pressure on age. Interpret the slope and intercept. Create a scatterplot with the regression line.

5.11. R

5. Regression: The Workhorse of Clinical Research

```
# Simulate clinical data
set.seed(42)
n <- 150
age <- round(runif(n, 30, 75))
sbp <- 85 + 0.6 * age + rnorm(n, 0, 12)

clinical_data <- data.frame(age = age, sbp = sbp)

# Fit the model
model <- lm(sbp ~ age, data = clinical_data)
summary(model)

# Interpret
cat("\nIntercept:", coef(model)[1], "mmHg\n")
cat("Slope:", coef(model)[2], "mmHg per year of age\n")
cat("R-squared:", summary(model)$r.squared, "\n")

# Confidence interval for the slope
confint(model, "age", level = 0.95)

# Scatterplot with regression line
library(ggplot2)
ggplot(clinical_data, aes(x = age, y = sbp)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = TRUE, color = "blue") +
  labs(x = "Age (years)", y = "Systolic Blood Pressure (mmHg)",
       title = "Age vs. Systolic Blood Pressure") +
  theme_minimal()
```


5.12. Python

5. Regression: The Workhorse of Clinical Research

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

np.random.seed(42)
n = 150
age = np.random.uniform(30, 75, n).round()
sbp = 85 + 0.6 * age + np.random.normal(0, 12, n)

clinical_data = pd.DataFrame({'age': age, 'sbp': sbp})

# Fit the model
X = sm.add_constant(clinical_data['age'])
model = sm.OLS(clinical_data['sbp'], X).fit()
print(model.summary())

print(f"\nIntercept: {model.params['const']:.2f} mmHg")
print(f"Slope: {model.params['age']:.3f} mmHg per year of age")
print(f"R-squared: {model.rsquared:.3f}")
print(f"95% CI for slope: ({model.conf_int().loc['age', 0]:.3f}, "
      f"{model.conf_int().loc['age', 1]:.3f})")

# Scatterplot with regression line
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(age, sbp, alpha=0.5)
age_range = np.linspace(30, 75, 100)
predicted = model.params['const'] + model.params['age'] * age_range
ax.plot(age_range, predicted, 'b-', linewidth=2, label='Regression line')
ax.set_xlabel('Age (years)')
ax.set_ylabel('Systolic Blood Pressure (mmHg)')
ax.set_title('Age vs. Systolic Blood Pressure')
ax.legend()
plt.tight_layout()
plt.show()
```

Solution

You should find a slope of approximately 0.6 mmHg/year (close to the true value used in simulation), an intercept around 85, and R^2 around 0.40-0.50. The interpretation: each additional year of age is associated with about 0.6 mmHg higher systolic blood pressure. The intercept (approximately 85) represents the predicted SBP at age 0, which is an extrapolation and not clinically meaningful. The 95% CI for the slope should be entirely positive, confirming the association.

 Exercise 3.2: Multiple Regression and Confounding

Add BMI and sex to the model from Exercise 3.1. Compare the age coefficient before and after adjustment. Does it change? What does this tell you about confounding?

5.13. R

5. Regression: The Workhorse of Clinical Research

```
set.seed(42)
n <- 150
age <- round(runif(n, 30, 75))
sex <- rbinom(n, 1, 0.5) # 1 = male
# BMI increases slightly with age (confounding)
bmi <- 22 + 0.08 * age + 2 * sex + rnorm(n, 0, 3)
# SBP depends on age, sex, AND BMI
sbp <- 70 + 0.45 * age + 4 * sex + 1.0 * bmi + rnorm(n, 0, 10)

df <- data.frame(age = age, sex = factor(sex, labels = c("Female", "Male")),
                 bmi = bmi, sbp = sbp)

# Unadjusted model (age only)
model_unadj <- lm(sbp ~ age, data = df)
cat("=== Unadjusted Model ===\n")
cat("Age coefficient:", coef(model_unadj)["age"], "\n")
cat("R-squared:", summary(model_unadj)$r.squared, "\n\n")

# Adjusted model (age + sex + BMI)
model_adj <- lm(sbp ~ age + sex + bmi, data = df)
cat("=== Adjusted Model ===\n")
summary(model_adj)
cat("\nAge coefficient (unadjusted):", round(coef(model_unadj)["age"], 3),
cat("Age coefficient (adjusted):", round(coef(model_adj)["age"], 3), "\n")
cat("Change:", round(coef(model_unadj)["age"] - coef(model_adj)["age"], 3),
```

5.14. Python

```

import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

np.random.seed(42)
n = 150
age = np.random.uniform(30, 75, n).round()
sex = np.random.binomial(1, 0.5, n)
# BMI increases slightly with age (confounding)
bmi = 22 + 0.08 * age + 2 * sex + np.random.normal(0, 3, n)
# SBP depends on age, sex, AND BMI
sbp = 70 + 0.45 * age + 4 * sex + 1.0 * bmi + np.random.normal(0, 10, n)

df = pd.DataFrame({'age': age, 'sex': sex, 'bmi': bmi, 'sbp': sbp})

# Unadjusted model
model_unadj = smf.ols('sbp ~ age', data=df).fit()
print("=== Unadjusted Model ===")
print(f"Age coefficient: {model_unadj.params['age']:.3f}")
print(f"R-squared: {model_unadj.rsquared:.3f}\n")

# Adjusted model
model_adj = smf.ols('sbp ~ age + C(sex) + bmi', data=df).fit()
print("=== Adjusted Model ===")
print(model_adj.summary())
print(f"\nAge coefficient (unadjusted): {model_unadj.params['age']:.3f}")
print(f"Age coefficient (adjusted): {model_adj.params['age']:.3f}")
print(f"Change: {model_unadj.params['age'] - model_adj.params['age']:.3f}")

```

Solution

The unadjusted age coefficient should be notably larger

5. Regression: The Workhorse of Clinical Research

than the adjusted coefficient. In the simulation, the true age effect is 0.45, but the unadjusted estimate is inflated because BMI increases with age (confounding path: Age $\rightarrow$ BMI $\rightarrow$ SBP). After adjusting for BMI and sex, the age coefficient moves closer to the true value of 0.45. This demonstrates **confounding**: the unadjusted age coefficient captured not only the direct effect of age but also the indirect effect through BMI. Adjusted R^2 should also be substantially higher in the multiple regression model.

Exercise 3.3: Logistic Regression

Using simulated data, fit a logistic regression predicting 10-year CHD risk from age, sex, smoking, and cholesterol. Compute and interpret the odds ratios.

5.15. R

5.10. Exercises

```
set.seed(42)
n <- 500

age <- round(runif(n, 35, 75))
male <- rbinom(n, 1, 0.5)
smoker <- rbinom(n, 1, 0.25)
cholesterol <- round(rnorm(n, 220, 40))

# Generate CHD outcome (binary) based on logistic model
log_odds <- -7 + 0.06 * age + 0.5 * male + 0.4 * smoker + 0.008 * cholesterol
prob_chd <- plogis(log_odds) # inverse logit
chd <- rbinom(n, 1, prob_chd)

df <- data.frame(age, male = factor(male), smoker = factor(smoker),
                 cholesterol, chd)

cat("CHD prevalence:", mean(chd), "\n\n")

# Fit logistic regression
model <- glm(chd ~ age + male + smoker + cholesterol,
             data = df, family = binomial)
summary(model)

# Odds ratios with 95% CI
or_table <- data.frame(
  OR = exp(coef(model)),
  Lower = exp(confint(model))[, 1],
  Upper = exp(confint(model))[, 2]
)
cat("\nOdds Ratios:\n")
print(round(or_table[-1, ], 3)) # Exclude intercept

# Predicted probability for a specific patient
new_patient <- data.frame(age = 60, male = factor(1), smoker = factor(1),
                          cholesterol = 260)
pred_prob <- predict(model, newdata = new_patient, type = "response")
cat("\nPredicted CHD probability for 60-year-old male smoker with",
    "cholesterol 260:", round(pred_prob, 3), "\n")
```

5. *Regression: The Workhorse of Clinical Research*

5.16. Python


```

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from scipy.special import expit

np.random.seed(42)
n = 500

age = np.random.uniform(35, 75, n).round()
male = np.random.binomial(1, 0.5, n)
smoker = np.random.binomial(1, 0.25, n)
cholesterol = np.random.normal(220, 40, n).round()

# Generate CHD outcome
log_odds = -7 + 0.06 * age + 0.5 * male + 0.4 * smoker + 0.008 * cholesterol
prob_chd = expit(log_odds)
chd = np.random.binomial(1, prob_chd)

df = pd.DataFrame({
    'age': age, 'male': male, 'smoker': smoker,
    'cholesterol': cholesterol, 'chd': chd
})

print(f"CHD prevalence: {chd.mean():.3f}\n")

# Fit logistic regression
model = smf.logit('chd ~ age + male + smoker + cholesterol', data=df).fit()
print(model.summary())

# Odds ratios with 95% CI
params = model.params[1:] # exclude intercept
conf = model.conf_int().iloc[1:]
or_table = pd.DataFrame({
    'OR': np.exp(params),
    'Lower 95% CI': np.exp(conf[0]),
    'Upper 95% CI': np.exp(conf[1])
})

print("\nOdds Ratios:")
print(or_table.round(3))

# Predicted probability for a specific patient
new_patient = pd.DataFrame({
    'age': [60], 'male': [1], 'smoker': [1], 'cholesterol': [260]
})

pred_prob = model.predict(new_patient)
print(f"\nPredicted CHD probability for 60yo male smoker, chol=260: {pred_prob.values[0]:.3f}")

```

5. Regression: The Workhorse of Clinical Research

Solution

The logistic regression should recover coefficients close to the true values used in simulation (age: 0.06, male: 0.5, smoker: 0.4, cholesterol: 0.008). The odds ratios should be approximately:

- Age (per year): OR around 1.06 – each year of age increases CHD odds by about 6%
- Male: OR around 1.65 – males have about 65% higher odds of CHD
- Smoker: OR around 1.50 – smokers have about 50% higher odds of CHD
- Cholesterol (per mg/dL): OR around 1.008 – each 1 mg/dL increase in cholesterol increases CHD odds by 0.8% (or about 8% per 10 mg/dL)

The predicted probability for a high-risk patient (60-year-old male smoker, cholesterol 260) should be noticeably higher than the population average.

Exercise 3.4: Regression Diagnostics

Using the linear regression model from Exercise 3.1, create diagnostic plots to assess model assumptions. Identify any potential violations.

5.17. R

```

set.seed(42)
n <- 150
age <- round(runif(n, 30, 75))
sbp <- 85 + 0.6 * age + rnorm(n, 0, 12)
df <- data.frame(age = age, sbp = sbp)

model <- lm(sbp ~ age, data = df)

# Standard diagnostic plots (2x2)
par(mfrow = c(2, 2))
plot(model)
par(mfrow = c(1, 1))

# Or using ggplot2 for prettier diagnostics
library(ggplot2)
library(patchwork)

diag_data <- data.frame(
  fitted = fitted(model),
  residuals = resid(model),
  std_residuals = rstandard(model)
)

p1 <- ggplot(diag_data, aes(x = fitted, y = residuals)) +
  geom_point(alpha = 0.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(se = FALSE, color = "blue") +
  labs(title = "Residuals vs Fitted", x = "Fitted Values", y = "Residuals") +
  theme_minimal()

p2 <- ggplot(diag_data, aes(sample = std_residuals)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  labs(title = "Normal Q-Q Plot", x = "Theoretical Quantiles",
    y = "Standardized Residuals") +
  theme_minimal()

p1 + p2

```

5. *Regression: The Workhorse of Clinical Research*

5.18. Python

```

import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)
n = 150
age = np.random.uniform(30, 75, n).round()
sbp = 85 + 0.6 * age + np.random.normal(0, 12, n)
df = pd.DataFrame({'age': age, 'sbp': sbp})

model = smf.ols('sbp ~ age', data=df).fit()

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1. Residuals vs Fitted
axes[0, 0].scatter(model.fittedvalues, model.resid, alpha=0.5)
axes[0, 0].axhline(y=0, color='red', linestyle='--')
axes[0, 0].set_xlabel('Fitted Values')
axes[0, 0].set_ylabel('Residuals')
axes[0, 0].set_title('Residuals vs Fitted')

# 2. Q-Q Plot
stats.probplot(model.resid, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('Normal Q-Q Plot')

# 3. Scale-Location
std_resid = np.sqrt(np.abs(model.get_influence().resid_studentized_internal))
axes[1, 0].scatter(model.fittedvalues, std_resid, alpha=0.5)
axes[1, 0].set_xlabel('Fitted Values')
axes[1, 0].set_ylabel('sqrt(|Standardized Residuals|)')
axes[1, 0].set_title('Scale-Location')

# 4. Residuals vs Leverage
sm.graphics.influence_plot(model, ax=axes[1, 1], criterion="cooks")
axes[1, 1].set_title('Residuals vs Leverage')

plt.tight_layout()
plt.show()

# Formal test for normality
shapiro_stat, shapiro_p = stats.shapiro(model.resid)
print(f"Shapiro-Wilk test for normality: W={shapiro_stat:.4f}, p={shapiro_p:.4f}")

```

5. Regression: The Workhorse of Clinical Research

Solution

Since the data were simulated from a correctly specified linear model with normal errors, the diagnostic plots should look well-behaved:

1. **Residuals vs. Fitted:** Points should be randomly scattered around zero with no systematic pattern. No evidence of non-linearity or heteroscedasticity.
2. **Q-Q Plot:** Points should fall approximately along the diagonal line, confirming normality.
3. **Scale-Location:** The spread of residuals should be roughly constant across fitted values.
4. **Residuals vs. Leverage:** No points should have both high leverage and large residuals (high Cook's distance).

In real clinical data, you will rarely see such clean diagnostics. Common violations include: non-linearity (curved pattern in residuals vs. fitted), heteroscedasticity (fanning out of residuals), and heavy tails (deviations at the ends of the Q-Q plot).

💡 Exercise 3.5: Interaction Terms

Test whether the effect of age on SBP differs between males and females by adding an interaction term. Interpret the interaction coefficient and create a visualization.

5.19. R

```

set.seed(42)
n <- 300
age <- round(runif(n, 30, 75))
sex <- factor(rbinom(n, 1, 0.5), labels = c("Female", "Male"))

# True interaction: steeper age slope for males
sbp <- 80 + 0.45 * age +
  ifelse(sex == "Male", -8 + 0.25 * age, 0) +
  rnorm(n, 0, 10)

df <- data.frame(age = age, sex = sex, sbp = sbp)

# Model WITHOUT interaction
model_no_int <- lm(sbp ~ age + sex, data = df)
cat("=== Model without interaction ===\n")
summary(model_no_int)

# Model WITH interaction
model_int <- lm(sbp ~ age * sex, data = df)
cat("\n=== Model with interaction ===\n")
summary(model_int)

# Is the interaction significant?
anova(model_no_int, model_int)

# Visualization
library(ggplot2)
ggplot(df, aes(x = age, y = sbp, color = sex)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(x = "Age (years)", y = "Systolic Blood Pressure (mmHg)",
       title = "Age-SBP Relationship by Sex",
       subtitle = "Steeper slope for males indicates an interaction") +
  theme_minimal() +
  scale_color_manual(values = c("Female" = "coral", "Male" = "steelblue"))

```

5. *Regression: The Workhorse of Clinical Research*

5.20. Python


```

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt

np.random.seed(42)
n = 300
age = np.random.uniform(30, 75, n).round()
sex = np.random.binomial(1, 0.5, n) # 1 = male

# True interaction: steeper age slope for males
sbp = 80 + 0.45 * age + (-8 + 0.25 * age) * sex + np.random.normal(0, 10, n)

df = pd.DataFrame({'age': age, 'sex': sex, 'sbp': sbp})

# Model WITHOUT interaction
model_no_int = smf.ols('sbp ~ age + C(sex)', data=df).fit()
print("=== Model without interaction ===")
print(model_no_int.summary())

# Model WITH interaction
model_int = smf.ols('sbp ~ age * C(sex)', data=df).fit()
print("\n=== Model with interaction ===")
print(model_int.summary())

# Compare models (likelihood ratio test)
from scipy.stats import chi2
lr_stat = -2 * (model_no_int.llf - model_int.llf)
lr_p = chi2.sf(lr_stat, df=1)
print(f"\nLikelihood ratio test: chi2={lr_stat:.2f}, p={lr_p:.4f}")

# Visualization
fig, ax = plt.subplots(figsize=(8, 6))
for s, label, color in [(0, 'Female', 'coral'), (1, 'Male', 'steelblue')]:
    mask = df['sex'] == s
    ax.scatter(df.loc[mask, 'age'], df.loc[mask, 'sbp'],
               alpha=0.3, color=color, label=label)
    age_range = np.linspace(30, 75, 100)
    pred = model_int.params['Intercept'] + model_int.params['age'] * age_range
    if s == 1:
        pred += model_int.params['C(sex)[T.1]'] + model_int.params['age:C(sex)[T.1]'] * age_range
    ax.plot(age_range, pred, color=color, linewidth=2)

ax.set_xlabel('Age (years)')
ax.set_ylabel('Systolic Blood Pressure (mmHg)')
ax.set_title('Age-SBP Relationship by Sex')
ax.legend()
plt.tight_layout()
plt.show()

```

5. Regression: The Workhorse of Clinical Research

Solution

The interaction term should be statistically significant and close to the true value of 0.25. Interpretation:

- **Female age slope:** approximately 0.45 mmHg/year (the main effect of age)
- **Male age slope:** approximately $0.45 + 0.25 = 0.70$ mmHg/year (main effect + interaction)
- **Interaction coefficient (~0.25):** Males' blood pressure increases 0.25 mmHg/year *faster* than females' blood pressure

The visualization should show two regression lines with noticeably different slopes – the male line is steeper. The lines may cross at some point, which reflects the negative main effect of male sex combined with the positive interaction.

This has clinical implications: if the age-SBP relationship truly differs by sex, then risk prediction models should account for this interaction rather than assuming a uniform age effect.

5.21. Key Takeaways

1. **Simple linear regression** fits a line; the slope is the predicted change in the outcome per unit increase in the predictor.
2. **Multiple regression coefficients** are “adjusted” – they represent the effect of one variable holding others constant.
3. **Always check assumptions** with diagnostic plots. Violations can invalidate your inferences.
4. **Logistic regression** is for binary outcomes. Exponentiate coefficients to get odds ratios.

5.22. References and Further Reading

5. **Categorical predictors** require dummy variables. The coefficients represent differences from the reference category.
6. **Interaction terms** allow effects to differ across subgroups. Only test interactions that are pre-specified and scientifically plausible.
7. **Model fit statistics** (R^2 , AIC, deviance) help you evaluate and compare models, but no single metric tells the whole story.

5.22. References and Further Reading

- Vittinghoff, E., Glidden, D. V., Shiboski, S. C., & McCulloch, C. E. (2012). *Regression Methods in Biostatistics: Linear, Logistic, Survival, and Repeated Measures Models* (2nd ed.). Springer.
- Steyerberg, E. W. (2019). *Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating* (2nd ed.). Springer.
- Smits, L. J. M., et al. (2026). Recommendations for clinical prediction model development, validation, and updating. *BMJ*.
- Wasserstein, R. L., & Lazar, N. A. (2016). The ASA statement on p-values: Context, process, and purpose. *The American Statistician*, 70(2), 129-133.
- Harrell, F. E. (2015). *Regression Modeling Strategies* (2nd ed.). Springer.
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression* (3rd ed.). Wiley.
- Gelman, A., & Hill, J. (2007). *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press.
- D'Agostino, R. B., Vasan, R. S., Pencina, M. J., et al. (2008). General cardiovascular risk profile for use in primary care: The Framingham Heart Study. *Circulation*, 117(6), 743-753.

Part II.

Advanced Statistical Methods

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation Fails

i Learning Objectives

By the end of this chapter, you will be able to:

1. Explain why categorising continuous variables leads to information loss and biased estimates
2. Recognise when a linear term is insufficient for modelling a continuous predictor
3. Describe fractional polynomials and their role in flexible modelling
4. Understand how splines work, including the concepts of knots, cubic splines, and restricted cubic splines
5. Implement and interpret restricted cubic spline models in both R and Python
6. Compare linear, categorical, and spline models using appropriate fit statistics
7. Apply the practical recommendations from Lopez-Ayala et al. (2025) in your own analyses

i What is different about this chapter

This chapter introduces tools you probably haven't seen before. The pace is faster than the previous chapters, and the concepts (splines, knot placement) may feel unfamiliar. Budget extra time here.

Why it matters for your research: If you build a prediction model or run a regression without properly handling continuous variables, your results will be less accurate — or wrong. Systematic reviews show that most published clinical prediction models categorise continuous variables inappropriately. This chapter teaches you the right way.

6.1. Introduction

Imagine you are studying the relationship between body mass index (BMI) and mortality. You have data on thousands of patients, and you need to build a regression model. What do you do with BMI?

If you are like most researchers in clinical medicine, you probably split BMI into categories: underweight (<18.5), normal ($18.5\text{--}24.9$), overweight ($25\text{--}29.9$), and obese ($30+$). You then include these categories as dummy variables in your regression model. This approach feels natural — it mirrors how we think clinically, and the results are easy to present in a table.

This is almost always the wrong approach.

Categorising continuous variables is one of the most common and most damaging analytic practices in clinical research. It throws away information, introduces bias, reduces statistical power, and can lead to completely wrong conclusions about the shape of relationships between predictors and outcomes.

This chapter draws heavily on a landmark tutorial by Lopez-Ayala et al. ([lopezayala2025?](#)), published in the *BMJ* in 2025, which provides a comprehensive guide to modelling continuous variables correctly. We will

6.2. *The Problem with Categorising Continuous Variables*

follow their framework closely, supplementing it with additional clinical examples and implementation guidance.

6.2. The Problem with Categorising Continuous Variables

6.2.1. What Happens When You Categorise

When you convert a continuous variable into categories, you make three implicit assumptions, all of which are almost certainly wrong:

1. **Within each category, the relationship is flat.** A patient with a BMI of 18.6 is treated identically to a patient with a BMI of 24.8 — despite a difference of over 6 kg/m².
2. **At the category boundary, there is a sudden jump.** A patient with a BMI of 24.9 is treated as fundamentally different from a patient with a BMI of 25.0 — despite a difference of only 0.1 kg/m².
3. **The cut-points are meaningful.** The chosen boundaries (often based on clinical convention or percentiles) capture the true biology of the relationship.

None of these assumptions hold for most biological relationships. Biology is smooth, not stepped.

6.2.2. The Consequences Are Serious

Lopez-Ayala et al. (lopezayala2025?) enumerate several consequences of categorisation:

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

- **Loss of statistical power.** Categorising a continuous variable into two groups is equivalent to discarding roughly one-third of your data (royston2006?). Even with more categories, substantial information is lost.
- **Biased effect estimates.** Because the relationship within each category is forced to be flat, the estimated effects are systematically wrong. The direction and magnitude of the bias depend on where you place the cut-points.
- **Residual confounding.** When a categorised variable is used as a confounder in an adjusted model, the adjustment is incomplete. The true confounding effect varies smoothly, but your model only adjusts in crude steps. This is sometimes called *residual confounding due to categorisation*.
- **Arbitrary cut-points change conclusions.** Different choices of cut-points can lead to different — even contradictory — conclusions from the same data. This is a form of researcher degrees of freedom that inflates false positive rates.
- **Loss of predictive accuracy.** Prediction models that categorise continuous predictors perform worse than those that model them appropriately.



A Common But Flawed Justification

Researchers often justify categorisation by saying “it’s easier to interpret” or “clinicians think in categories.” While clinical decision-making does involve thresholds (e.g., treat if blood pressure > 140 mmHg), the *modelling* of relationships should reflect reality, not convenience. You can always convert a continuous model’s output into clinically actionable categories *after* modelling — but you cannot recover the information lost by categorising *before* modelling.

6.2.3. A Visual Demonstration

Consider the true relationship between a biomarker (say, C-reactive protein) and the log-odds of a diagnosis. Suppose the true relationship is U-shaped: both very low and very high values are associated with increased risk.

If you categorise CRP into tertiles (low, medium, high), you will estimate a *monotonically increasing* relationship — completely missing the U-shape. The low-risk group is contaminated by some high-risk individuals at the low end, and the comparison between groups obscures the smooth curve.

```
library(ggplot2)

set.seed(42)
x <- seq(0, 10, length.out = 300)
# True U-shaped relationship
y_true <- 0.3 * (x - 5)^2 - 2

df <- data.frame(x = x, y_true = y_true)

# Categorise into tertiles
df$category <- cut(df$x, breaks = quantile(df$x, probs = c(0, 1/3, 2/3, 1)),
                  include.lowest = TRUE, labels = c("Low", "Medium", "High"))
cat_means <- aggregate(y_true ~ category, data = df, FUN = mean)

df$y_cat <- cat_means$y_true[match(df$category, cat_means$category)]

ggplot(df, aes(x = x)) +
  geom_line(aes(y = y_true), linewidth = 1.2, colour = "#2c3e50") +
  geom_step(aes(y = y_cat), linewidth = 1, colour = "#e74c3c", linetype = "dashed") +
  labs(x = "Biomarker Level", y = "Log-Odds of Outcome",
       subtitle = "Black curve = true relationship; Red steps = categorised estimate") +
  theme_minimal(base_size = 14) +
  theme(plot.subtitle = element_text(size = 11))
```

6.3. When Linear Terms Are Not Enough

6.3.1. The Linearity Assumption

In standard logistic or linear regression, including a continuous predictor x as a simple linear term assumes:

$$\text{logit}(p) = \beta_0 + \beta_1 x$$

This means that each one-unit increase in x is associated with the same change in the log-odds, regardless of where on the x scale you start. For BMI, this would mean that going from 20 to 21 has the same effect as going from 35 to 36.

For many clinical variables, this assumption is **wrong**:

- **BMI and mortality**: U-shaped or J-shaped. Both very low and very high BMI are associated with increased mortality.
- **Alcohol consumption and cardiovascular risk**: J-shaped. Moderate consumption may be associated with lower risk than either abstinence or heavy drinking (though this remains debated).
- **Age and many outcomes**: Often non-linear, with steeper increases at older ages.
- **Blood pressure and stroke risk**: The relationship steepens at higher pressures.
- **CSF glucose and bacterial meningitis**: A threshold effect where low values are strongly predictive but the relationship flattens at higher values (we will explore this in detail below).

6.3.2. Detecting Non-Linearity

Before choosing a modelling strategy, it helps to assess whether non-linearity is present:

6.4. Fractional Polynomials

1. **Visual inspection.** Plot the predictor against the outcome using a loess (locally estimated scatterplot smoothing) curve or a generalised additive model (GAM) smooth. If the relationship is not a straight line, you need a non-linear term.
2. **Likelihood ratio test.** Fit a model with a linear term and a model with a non-linear term (e.g., a spline). Compare them using a likelihood ratio test. A significant result indicates that the non-linear model fits substantially better.
3. **AIC/BIC comparison.** Compare the Akaike Information Criterion (AIC) or Bayesian Information Criterion (BIC) of the linear and non-linear models. Lower values indicate better fit (penalised for complexity).
4. **Residual plots.** Plot residuals against the predictor. A systematic pattern (e.g., a curve) suggests non-linearity.

Practical Rule

As Lopez-Ayala et al. ([lopezayala2025?](#)) recommend: when in doubt, use a non-linear term. The cost of modelling a truly linear relationship with a spline is minimal (you lose very little power), but the cost of forcing a non-linear relationship into a linear term can be severe (biased estimates, wrong conclusions).

6.4. Fractional Polynomials

6.4.1. The Idea

Fractional polynomials (FPs), developed by Royston and Altman ([royston1994?](#)), extend conventional polynomials by allowing *fractional*

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

and *negative* powers. Instead of being restricted to $x, x^2, x^3, \dots$, fractional polynomials choose from the set of powers:

$$\{-2, -1, -0.5, 0, 0.5, 1, 2, 3\}$$

where x^0 is defined as $\log(x)$.

A **first-degree fractional polynomial** (FP1) has the form:

$$\beta_0 + \beta_1 x^{p_1}$$

A **second-degree fractional polynomial** (FP2) has the form:

$$\beta_0 + \beta_1 x^{p_1} + \beta_2 x^{p_2}$$

where p_1 and p_2 are selected from the allowed power set. If $p_1 = p_2$, the second term becomes $x^{p_1} \log(x)$ (the so-called “repeated powers” convention).

6.4.2. How FPs Work

The algorithm searches through all combinations of powers from the allowed set and selects the combination that best fits the data (by maximum likelihood or AIC). For an FP2, there are 36 possible combinations of powers, making the search computationally straightforward.

The key advantage of fractional polynomials is that they can capture a wide range of curve shapes — monotone increasing, monotone decreasing, U-shaped, J-shaped, S-shaped — with only one or two parameters beyond the linear term.

6.4.3. Strengths and Limitations

Strengths:

- Parsimonious: only 1–2 additional parameters
- Can capture many common curve shapes
- Well-established methodology with formal model selection procedures (the FP selection algorithm, also called MFP for multivariable fractional polynomials)
- Particularly useful when sample size is modest

Limitations:

- Behaviour at the extremes of the predictor range can be erratic (the curves are governed by global functions)
- Cannot capture complex local features (e.g., a bump in the middle of the range)
- The set of allowed shapes, while broad, is still constrained

Lopez-Ayala et al. ([lopezayala2025?](#)) note that fractional polynomials and splines are *complementary* approaches, and in many cases they give similar results. However, for most applications in clinical research, **restricted cubic splines have become the recommended default** due to their greater flexibility and better behaviour at the boundaries.

6.5. Splines: The Recommended Approach

6.5.1. What is a spline, in plain language?

Think of a spline as a flexible ruler. A straight line forces the same slope everywhere. A spline lets the curve bend at specific points (called **knots**), producing a smooth curve that follows the data's actual shape. The result is a single smooth function — not separate lines stitched together.

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

You have already seen the problem a spline solves: in Chapter 5, we used a straight line to model the relationship between age and blood pressure. That works if the relationship is genuinely linear. But many clinical relationships are not — BMI and mortality is U-shaped, age and surgical risk accelerates in the elderly. A spline captures these patterns.

6.5.2. What Is a Spline?

A spline is a piecewise polynomial function — a curve made up of polynomial segments joined smoothly at specific points called **knots**. The idea is simple: rather than fitting one global function to the entire range of a predictor, you fit separate polynomial segments in different intervals and then connect them smoothly.

Think of it like bending a flexible ruler: the ruler is smooth everywhere, but you can create complex curves by bending it at specific points.

6.5.3. Knots

Knots are the points along the predictor's range where the polynomial segments connect. The number and placement of knots control the flexibility of the spline:

- **More knots** = more flexible curve (can capture more complex shapes)
- **Fewer knots** = smoother, more constrained curve

How to choose knot locations:

The standard recommendation, following Harrell (**harrell2015?**), is to place knots at **fixed percentiles** of the predictor's distribution:

6.5. Splines: The Recommended Approach

Number of Knots	Percentile Locations
3	10th, 50th, 90th
4	5th, 35th, 65th, 95th
5	5th, 27.5th, 50th, 72.5th, 95th

This data-driven placement ensures that knots are spread across the range of observed data, with more knots where there are more observations.

6.5.4. Cubic Splines

A **cubic spline** uses third-degree (cubic) polynomial segments between knots. The segments are constrained to have continuous first and second derivatives at each knot, ensuring the resulting curve is smooth.

A cubic spline with k knots uses $k + 3$ parameters (degrees of freedom) — the cubic polynomial in the first segment, plus one additional parameter for each knot beyond the first. This can become expensive in terms of degrees of freedom when many knots are used.

6.5.5. Restricted Cubic Splines (RCS)

A **restricted cubic spline** (also called a **natural cubic spline**) adds one more constraint: the function is forced to be **linear** beyond the outermost knots (i.e., in the tails of the distribution).

This constraint is motivated by the fact that there are typically few observations in the tails, so we have little information to estimate complex curves there. Forcing linearity in the tails:

- Prevents wild oscillations at the extremes
- Reduces the number of parameters (an RCS with k knots uses $k - 1$ parameters)

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

- Makes extrapolation more sensible (linear extrapolation is more defensible than cubic extrapolation)

! The Recommended Default

Lopez-Ayala et al. ([lopezayala2025?](#)) and Harrell ([harrell2015?](#)) recommend **restricted cubic splines with 3 to 5 knots** as the default approach for modelling continuous predictors. With 4 knots (3 degrees of freedom), you can capture the vast majority of biologically plausible curve shapes. Use 5 knots (4 df) if you suspect a particularly complex relationship and have adequate sample size.

An RCS with 3 knots (2 df) can capture monotone and simple U/J-shaped curves. This is a good starting point when sample size is limited or when you do not expect a complex relationship.

6.5.6. Why Restricted Cubic Splines?

To summarise the case for RCS:

1. **Flexibility.** They can capture a wide variety of non-linear shapes without requiring you to specify the shape in advance.
2. **Parsimony.** With $k - 1$ parameters for k knots, they are relatively economical. A 4-knot RCS adds only 2 parameters beyond a linear term.
3. **Good boundary behaviour.** The linearity constraint in the tails prevents overfitting where data are sparse.
4. **Interpretability through plots.** While the individual coefficients of a spline are not directly interpretable, the resulting curve is highly interpretable when plotted.
5. **Well supported by software.** The `rms` package in R and various Python libraries make implementation straightforward.

6.5.7. Interpreting Spline Models

This is a critical point that many researchers initially find confusing:

You cannot interpret the individual regression coefficients of a spline model. An RCS with 4 knots produces 3 coefficients (e.g., `rcs(bmi, 4)`, `rcs(bmi, 4)'`, `rcs(bmi, 4)''`), but these individual numbers are meaningless in isolation. They define the *shape* of the curve jointly.

Instead, you interpret spline models through:

1. **Plots.** Plot the predicted outcome (or log-odds, or hazard ratio) as a function of the predictor. This shows you the entire shape of the relationship.
2. **Specific contrasts.** You can calculate the predicted difference in outcome between any two values of the predictor (e.g., the odds ratio for BMI = 35 vs BMI = 25).
3. **Overall tests.** Use a chunk test (Wald test or likelihood ratio test) with $k - 1$ degrees of freedom to test whether the predictor has *any* association with the outcome. Use a test with $k - 2$ degrees of freedom to test specifically for *non-linearity* (i.e., whether the relationship departs from a straight line).

```
library(rms)
library(ggplot2)

set.seed(123)
n <- 500
x <- runif(n, 20, 45)
# True non-linear relationship (U-shaped in BMI)
logit_p <- -8 + 0.05 * (x - 30)^2
p <- plogis(logit_p)
y <- rbinom(n, 1, p)
```

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

```
dd <- datadist(x)
options(datadist = "dd")

fit <- lrm(y ~ rcs(x, 4))
knot_locs <- attr(rcs(x, 4), "parms")

# Prediction
x_pred <- seq(20, 45, length.out = 200)
pred <- Predict(fit, x = x_pred)

ggplot(as.data.frame(pred), aes(x = x, y = yhat)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2, fill = "#3498db") +
  geom_line(linewidth = 1.2, colour = "#2c3e50") +
  geom_point(data = data.frame(x = knot_locs, y = rep(min(pred$yhat), length.out = 4)),
    aes(x = x, y = y), shape = 17, size = 3, colour = "#e74c3c") +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
  labs(x = "BMI (kg/m\u00b2)", y = "Log-Odds of Outcome",
    subtitle = "RCS with 4 knots; triangles mark knot positions") +
  theme_minimal(base_size = 14)
```

6.6. Clinical Example: CSF Glucose and Bacterial Meningitis

This example is adapted from Lopez-Ayala et al. ([lopezayala2025?](#)), who use it to illustrate the full workflow for modelling a continuous predictor.

6.6.1. The Clinical Question

In patients presenting with suspected meningitis, cerebrospinal fluid (CSF) glucose is one of several laboratory values used to distinguish bacterial from viral meningitis. Low CSF glucose is a hallmark of bacterial infection, as

6.6. Clinical Example: CSF Glucose and Bacterial Meningitis

bacteria consume glucose. But what is the precise shape of this relationship? And is it appropriate to use a simple cut-point (e.g., CSF glucose < 2.2 mmol/L)?

6.6.2. Step-by-Step Analysis

Step 1: Visualise the raw relationship.

Before fitting any model, plot the outcome against the predictor using a smoothing function (loess or GAM). This gives you a preliminary sense of the shape.

```
library(ggplot2)
library(rms)

set.seed(2025)
n <- 400
csf_glucose <- rlnorm(n, meanlog = log(3), sdlog = 0.6)
csf_glucose <- pmin(csf_glucose, 10)

# True relationship: steep drop in probability below ~2, then flat
logit_p <- 3 - 4 * pmax(0, 2.5 - csf_glucose) - 0.3 * csf_glucose
p_bact <- plogis(logit_p)
bacterial <- rbinom(n, 1, p_bact)

df_csf <- data.frame(csf_glucose = csf_glucose, bacterial = bacterial)

ggplot(df_csf, aes(x = csf_glucose, y = bacterial)) +
  geom_jitter(height = 0.05, alpha = 0.3, width = 0.05) +
  geom_smooth(method = "loess", se = TRUE, colour = "#2c3e50") +
  labs(x = "CSF Glucose (mmol/L)", y = "Probability of Bacterial Meningitis",
       subtitle = "Loess smooth suggests a non-linear (threshold-like) relationship") +
  theme_minimal(base_size = 14)
```

Step 2: Fit competing models.

We fit three models and compare them:

- **Model 1 (Linear):** `bacterial ~ csf_glucose`
- **Model 2 (Categorised):** `bacterial ~ I(csf_glucose < 2.2)`
(a common clinical cut-point)
- **Model 3 (RCS):** `bacterial ~ rcs(csf_glucose, 4)`

```
library(rms)

dd_csf <- datadist(df_csf)
options(datadist = "dd_csf")

# Model 1: Linear
fit_linear <- lrm(bacterial ~ csf_glucose, data = df_csf)

# Model 2: Categorised
df_csf$glucose_low <- as.numeric(df_csf$csf_glucose < 2.2)
fit_cat <- lrm(bacterial ~ glucose_low, data = df_csf)

# Model 3: RCS with 4 knots
fit_rcs <- lrm(bacterial ~ rcs(csf_glucose, 4), data = df_csf)

# Compare AIC
aic_vals <- c(
  Linear = AIC(fit_linear),
  Categorised = AIC(fit_cat),
  RCS_4knots = AIC(fit_rcs)
)

cat("AIC Comparison:\n")
print(round(aic_vals, 1))
cat("\nLower AIC = better fit (penalised for complexity)\n")
```

6.6. Clinical Example: CSF Glucose and Bacterial Meningitis

Step 3: Test for non-linearity.

```
# The anova() function in rms tests the overall effect and non-linearity separately
cat("ANOVA for RCS model:\n")
print(anova(fit_rcs))
```

The ANOVA output from `rms` provides two key tests:

- **Total effect** (all spline terms combined): tests whether CSF glucose has *any* association with the outcome
- **Nonlinear component** (spline terms beyond the linear): tests whether the relationship departs from linearity

If the non-linear component is significant, this confirms that a linear term is insufficient.

Step 4: Plot the fitted curves from all three models.

```
x_seq <- seq(min(df_csf$csf_glucose), max(df_csf$csf_glucose), length.out = 200)

# Predictions from each model
pred_linear <- predict(fit_linear, newdata = data.frame(csf_glucose = x_seq),
                      type = "fitted")
pred_cat <- predict(fit_cat,
                   newdata = data.frame(glucose_low = as.numeric(x_seq < 2.2)),
                   type = "fitted")

pred_rcs_obj <- Predict(fit_rcs, csf_glucose = x_seq)

plot_df <- data.frame(
  csf_glucose = rep(x_seq, 3),
  probability = c(pred_linear, pred_cat, as.data.frame(pred_rcs_obj)$yhat),
  Model = rep(c("Linear", "Categorised (<2.2)", "RCS (4 knots)"), each = length(x_seq))
)
```

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

```
# Convert RCS from log-odds to probability for fair comparison
plot_df$probability[plot_df$Model == "RCS (4 knots)"] <-
  plogis(plot_df$probability[plot_df$Model == "RCS (4 knots)"])

ggplot(plot_df, aes(x = csf_glucose, y = probability, colour = Model)) +
  geom_line(linewidth = 1.1) +
  scale_colour_manual(values = c("Linear" = "#3498db",
                                "Categorised (<2.2)" = "#e74c3c",
                                "RCS (4 knots)" = "#2c3e50")) +
  labs(x = "CSF Glucose (mmol/L)", y = "Predicted Probability of Bacterial M")
  theme_minimal(base_size = 14) +
  theme(legend.position = "bottom")
```

6.6.3. What This Example Teaches Us

1. The **linear model** misses the threshold-like nature of the relationship. It estimates a constant decrease in probability per unit increase in glucose, which underestimates the risk at very low glucose levels and overestimates it at intermediate levels.
2. The **categorised model** captures the broad direction (low glucose = higher risk) but loses all nuance. It cannot tell you that a glucose of 0.5 mmol/L carries a very different risk than a glucose of 2.0 mmol/L — both are simply “low.”
3. The **RCS model** captures the steep decline in risk at low glucose levels and the flattening at higher levels, providing the most accurate and informative picture.

6.7. Practical Recommendations

Lopez-Ayala et al. (lopezayala2025?) provide a practical framework (their Box 2) that we adapt here:

Workflow for Modelling Continuous Predictors

1. **Keep continuous variables continuous.** Do not categorise unless there is a strong *a priori* biological or clinical reason (and even then, think twice).
2. **Start by visualising** the predictor–outcome relationship using a smoother (loess, GAM, or a spline with many knots).
3. **Fit a restricted cubic spline** with 3–5 knots as the default modelling strategy. Use 3 knots when sample size is small or the relationship is expected to be simple; 4–5 knots when sample size is adequate and the relationship may be complex.
4. **Test for non-linearity** using a likelihood ratio test or the Wald test from `anova(fit)` in `rms`. If non-linearity is not significant, you *may* simplify to a linear term — but the spline model will give very similar results anyway, so simplification is optional.
5. **Present results as plots** showing the predicted outcome across the range of the predictor, with confidence intervals. Supplement with specific contrasts (e.g., odds ratio for value A vs value B) as needed.
6. **Do not report individual spline coefficients** in tables. They are not interpretable. Instead, report the overall test for the predictor and any non-linearity test, along with a figure.
7. **Use AIC or likelihood ratio tests** to compare models (linear vs spline, different numbers of knots).

6.8. Implementation

6.8.1. R: Using the rms Package

The `rms` package by Frank Harrell is the gold standard for modelling continuous variables with restricted cubic splines.

```
# Install and load required packages
# install.packages(c("rms", "ggplot2"))
library(rms)
library(ggplot2)

# Step 1: Prepare data and set datadist (required by rms)
dd <- datadist(your_data)
options(datadist = "dd")

# Step 2: Fit a logistic regression with RCS
# rcs(variable, number_of_knots)
fit <- lrm(outcome ~ rcs(predictor, 4) + confounder1 + confounder2,
           data = your_data)

# Step 3: Examine the model
print(fit)           # Overall model summary
anova(fit)           # Tests for each predictor (total + non-linearity)

# Step 4: Plot the relationship
p <- Predict(fit, predictor) # Predictions across range of predictor
plot(p)               # rms built-in plotting

# Or use ggplot for more control
pred_df <- as.data.frame(p)
ggplot(pred_df, aes(x = predictor, y = yhat)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
```

6.8. Implementation

```
geom_line(linewidth = 1) +  
labs(x = "Predictor", y = "Log-Odds") +  
theme_minimal()  
  
# Step 5: Specific contrasts  
# Odds ratio for predictor = 30 vs predictor = 25 (reference)  
summary(fit, predictor = c(25, 30))
```

6.8.2. Python: Using patsy and statsmodels

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import statsmodels.api as sm  
from patsy import dmatrix, bs, cr  
  
# --- Method 1: Using patsy's cr() for natural (restricted) cubic splines ---  
  
# Create the spline basis using patsy formula  
# cr() creates a natural cubic spline (restricted cubic spline)  
# df = number of knots - 1 (so df=3 means 4 knots)  
formula = "cr(predictor, df=3)"  
spline_basis = dmatrix(formula, data=your_data, return_type='dataframe')  
  
# Fit logistic regression  
X = pd.concat([spline_basis, your_data[['confounder1', 'confounder2']]], axis=1)  
model = sm.GLM(your_data['outcome'], X, family=sm.families.Binomial())  
result = model.fit()  
print(result.summary())  
  
# --- Method 2: Using scikit-learn's SplineTransformer ---
```

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

```
from sklearn.preprocessing import SplineTransformer

# Create spline features
spline_transformer = SplineTransformer(
    n_knots=4,
    degree=3,
    extrapolation='linear' # This mimics the RCS constraint
)
X_spline = spline_transformer.fit_transform(
    your_data[['predictor']].values
)

# Fit logistic regression
model = sm.GLM(your_data['outcome'],
               sm.add_constant(X_spline),
               family=sm.families.Binomial())
result = model.fit()

# --- Plotting the fitted relationship ---

x_range = np.linspace(your_data['predictor'].min(),
                      your_data['predictor'].max(), 200)

# Generate predictions
X_pred = dmatrix(formula, data=pd.DataFrame({'predictor': x_range}),
                  return_type='dataframe')
pred_logodds = result.predict(X_pred)

plt.figure(figsize=(8, 5))
plt.plot(x_range, pred_logodds, 'k-', linewidth=2)
plt.xlabel('Predictor')
plt.ylabel('Predicted Probability')
plt.title('Non-linear relationship modelled with RCS')
```

```
plt.tight_layout()
plt.show()
```

6.9. Comparison with Other Approaches

Table 6.1.: Comparison of approaches for modelling continuous variables

Approach	Pros	Cons
Linear term	Simple, 1 df	Assumes linearity — often wrong
Categorisation	“Easy” to present	Information loss, bias, arbitrary cut-points
Quadratic polynomial	Can capture U-shapes, 2 df	Symmetric, poor boundary behaviour
Fractional polynomials	Flexible, parsimonious	Global functions; limited local flexibility
Restricted cubic splines	Flexible, good boundary behaviour, well-supported	Coefficients not directly interpretable
GAM (smoothing splines)	Very flexible, automatic smoothing	Can overfit; penalty selection needed

For most clinical research, **restricted cubic splines with 3–5 knots** represent the best balance of flexibility, parsimony, and practical usability.

6.10. Exercises

6.10.1. Exercise 1: Recognising the Problem

Consider a study of the association between systolic blood pressure (SBP) and stroke risk. The researchers categorise SBP into: normal (<120), elevated ($120\text{--}129$), Stage 1 hypertension ($130\text{--}139$), and Stage 2 hypertension (≥ 140).

Questions:

- What assumptions does this categorisation impose on the SBP–stroke relationship?
- A patient with SBP of 131 mmHg is classified identically to a patient with SBP of 139 mmHg. Why is this problematic?
- Suggest a better modelling approach and explain how you would implement it.

6.10.2. Exercise 2: Fitting and Interpreting Spline Models

Using the built-in PBC dataset (primary biliary cholangitis) from the `survival` package in R, model the relationship between serum bilirubin and transplant status (a binary outcome: transplanted or not).

6.10.2.1. R

```
library(rms)
library(ggplot2)

# Load the PBC data
data(pbc, package = "survival")
pbc <- pbc[!is.na(pbc$trt), ] # Remove incomplete cases
```

6.10. Exercises

```
# Create a binary outcome: transplanted (status == 1) vs not
pbc$transplant <- as.numeric(pbc$status == 1)

# Set up datadist
dd <- datadist(pbc)
options(datadist = "dd")

# Task 1: Fit a logistic regression with bilirubin as a linear term
fit_linear <- lrm(transplant ~ bili, data = pbc)

# Task 2: Fit a logistic regression with bilirubin modelled using RCS (4 knots)
fit_rcs <- lrm(transplant ~ rcs(bili, 4), data = pbc)

# Task 3: Test for non-linearity
anova(fit_rcs)

# Task 4: Compare AIC
AIC(fit_linear)
AIC(fit_rcs)

# Task 5: Plot the relationship
p <- Predict(fit_rcs, bili)
plot(p, xlab = "Serum Bilirubin (mg/dL)",
      ylab = "Log-Odds of Transplant")

# Task 6: What does the plot tell you about the bilirubin-transplant
#          relationship? Is it linear?
```

6.10.2.2. Python

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from patsy import dmatrix

# Simulate data similar to PBC bilirubin-transplant relationship
np.random.seed(42)
n = 300
bili = np.random.exponential(scale=2, size=n)
bili = np.clip(bili, 0.3, 25)
logit_p = -3 + 0.1 * bili + 0.005 * bili**2 # Non-linear true relationship
prob = 1 / (1 + np.exp(-logit_p))
transplant = np.random.binomial(1, prob)

df = pd.DataFrame({'bili': bili, 'transplant': transplant})

# Task 1: Fit logistic regression with linear bilirubin
X_linear = sm.add_constant(df[['bili']])
fit_linear = sm.GLM(df['transplant'], X_linear,
                    family=sm.families.Binomial()).fit()
print("Linear model:")
print(fit_linear.summary())

# Task 2: Create spline basis and fit logistic regression with splines
spline_basis = dmatrix("cr(bili, df=3)", data=df, return_type='dataframe')
fit_spline = sm.GLM(df['transplant'], spline_basis,
                    family=sm.families.Binomial()).fit()
print("\nSpline model:")
print(fit_spline.summary())

# Task 3: Compare AIC
print(f"\nAIC Linear: {fit_linear.aic:.1f}")
```



```

print(f"AIC Spline: {fit_spline.aic:.1f}")

# Task 4: Plot the predicted log-odds across bilirubin range
bili_range = np.linspace(df['bili'].min(), df['bili'].max(), 200)
spline_pred = dmatrix("cr(bili_range, df=3)",
                      {"bili_range": bili_range},
                      return_type='dataframe')
pred_prob = fit_spline.predict(spline_pred)

plt.figure(figsize=(8, 5))
plt.plot(bili_range, pred_prob, 'k-', linewidth=2)
plt.xlabel('Serum Bilirubin (mg/dL)')
plt.ylabel('Predicted Probability of Transplant')
plt.title('Non-linear effect of bilirubin on transplant (RCS)')
plt.tight_layout()
plt.show()

```

6.10.3. Exercise 3: Categorisation vs Splines Head-to-Head

Simulate data with a known non-linear (U-shaped) relationship and compare the performance of categorised and spline models.

6.10.3.1. R

```

library(rms)
library(ggplot2)

set.seed(2025)
n <- 1000
x <- rnorm(n, mean = 50, sd = 10)

```

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorise

```
# True U-shaped relationship
logit_p <- -5 + 0.004 * (x - 50)^2
y <- rbinom(n, 1, plogis(logit_p))

sim_data <- data.frame(x = x, y = y)
dd <- datadist(sim_data)
options(datadist = "dd")

# Model 1: Categorise into quartiles
sim_data$x_cat <- cut(sim_data$x,
                      breaks = quantile(sim_data$x, c(0, 0.25, 0.5, 0.75, 1)),
                      include.lowest = TRUE)
fit_cat <- lrm(y ~ x_cat, data = sim_data)

# Model 2: Linear term
fit_lin <- lrm(y ~ x, data = sim_data)

# Model 3: RCS with 5 knots
fit_rcs <- lrm(y ~ rcs(x, 5), data = sim_data)

# Questions:
# a) Compare AIC of the three models. Which fits best?
# b) Plot all three fitted curves on the same graph along with the
#    true logit function. Which model best recovers the truth?
# c) What happens if you change the quartile cut-points to tertiles?
#    Does it affect the categorised model's conclusions?

cat("AIC values:\n")
cat("Categorised:", AIC(fit_cat), "\n")
cat("Linear:      ", AIC(fit_lin), "\n")
cat("RCS(5):      ", AIC(fit_rcs), "\n")
```

6.10.3.2. Python

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from patsy import dmatrix
from scipy.special import expit

np.random.seed(2025)
n = 1000
x = np.random.normal(50, 10, size=n)

# True U-shaped relationship
logit_p = -5 + 0.004 * (x - 50)**2
y = np.random.binomial(1, expit(logit_p))

df = pd.DataFrame({'x': x, 'y': y})

# Model 1: Categorise into quartiles
df['x_cat'] = pd.qcut(df['x'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
X_cat = pd.get_dummies(df['x_cat'], drop_first=True).astype(float)
X_cat = sm.add_constant(X_cat)
fit_cat = sm.GLM(df['y'], X_cat, family=sm.families.Binomial()).fit()

# Model 2: Linear
X_lin = sm.add_constant(df[['x']])
fit_lin = sm.GLM(df['y'], X_lin, family=sm.families.Binomial()).fit()

# Model 3: RCS
X_rcs = dmatrix("cr(x, df=4)", data=df, return_type='dataframe')
fit_rcs = sm.GLM(df['y'], X_rcs, family=sm.families.Binomial()).fit()

```

6. Modelling Non-Linear Relationships: Splines, Fractional Polynomials, and Why Categorisation

```
print(f"AIC Categorised: {fit_cat.aic:.1f}")
print(f"AIC Linear:      {fit_lin.aic:.1f}")
print(f"AIC RCS:         {fit_rcs.aic:.1f}")

# Plot all three fitted curves
x_range = np.linspace(df['x'].min(), df['x'].max(), 200)

# True curve
true_logit = -5 + 0.004 * (x_range - 50)**2
true_prob = expit(true_logit)

# RCS predictions
X_rcs_pred = dmatrix("cr(x_range, df=4)", {"x_range": x_range},
                    return_type='dataframe')
rcs_prob = fit_rcs.predict(X_rcs_pred)

# Linear predictions
X_lin_pred = sm.add_constant(pd.DataFrame({'x': x_range}))
lin_prob = fit_lin.predict(X_lin_pred)

plt.figure(figsize=(10, 6))
plt.plot(x_range, true_prob, 'k--', linewidth=2, label='True relationship')
plt.plot(x_range, rcs_prob, 'b-', linewidth=2, label='RCS')
plt.plot(x_range, lin_prob, 'r-', linewidth=1.5, label='Linear')
plt.xlabel('Predictor (x)')
plt.ylabel('Predicted Probability')
plt.legend()
plt.title('Comparing modelling strategies for a U-shaped relationship')
plt.tight_layout()
plt.show()
```

6.11. Summary

Key Concept	Details
Do not categorise	Categorising continuous variables loses information, introduces bias, and reduces power
Linear terms	Assume a constant effect per unit change; often wrong for biological variables
Fractional polynomials	Use non-integer powers for flexible global curves; parsimonious but limited locally
Splines	Piecewise polynomials joined at knots; flexible and locally adaptive
Restricted cubic splines	Splines forced to be linear in the tails; recommended default with 3–5 knots
Interpretation	Use plots and contrasts, not individual coefficients
Testing	Overall association test ($k - 1$ df); non-linearity test ($k - 2$ df)

References

7. Penalised Regression: Ridge, LASSO, and Elastic Net

i Learning Objectives

By the end of this chapter, you will be able to:

1. Explain why standard regression fails when the number of predictors is large relative to sample size
2. Describe the bias-variance tradeoff and how penalised regression exploits it
3. Distinguish between Ridge (L2), LASSO (L1), and Elastic Net penalties
4. Explain what the tuning parameters λ and α control
5. Use cross-validation to select optimal penalty parameters
6. Interpret regularisation path plots (coefficient traces)
7. Implement penalised regression in both R (`glmnet`) and Python (`sklearn`)
8. Choose the appropriate method for a given clinical research problem

7.1. Introduction: Why Standard Regression Is Not Enough

Throughout the earlier chapters, we have used standard regression — ordinary least squares (OLS) for continuous outcomes, logistic regression for binary outcomes, Cox regression for time-to-event data. These methods work well when the number of predictors is small relative to the sample size and the predictors are not highly correlated.

But modern clinical research increasingly involves situations where these assumptions break down:

- **Genomics and proteomics.** A study might measure expression levels of 20,000 genes in 200 patients. There are 100 times more predictors than observations.
- **Radiomic features.** A CT scan can be summarised by hundreds of texture and shape features. A typical study might have 300 features and 150 patients.
- **Electronic health records.** A prediction model might draw on hundreds of laboratory values, medications, diagnoses, and demographic variables.
- **Multi-centre clinical prediction.** Even with a modest number of clinical predictors, small samples in some centres can lead to unstable estimates.

In all these situations, standard regression either **fails entirely** (when $p > n$) or produces **unreliable, overfitted** models (when p is close to n).

Penalised regression — also known as regularised regression or shrinkage methods — solves this problem by adding a penalty to the regression coefficients, pulling them toward zero. This trades a small amount of bias for a large reduction in variance, typically producing models that predict much better on new data.

7.2. The Problem: Overfitting and Instability

7.2.1. What Happens with Too Many Predictors

Consider a logistic regression model for predicting 30-day mortality after cardiac surgery. You have 200 patients (40 deaths) and want to use 50 candidate predictors (age, comorbidities, lab values, surgical variables, etc.).

With standard logistic regression, this model is in serious trouble. A common rule of thumb (events per variable, or EPV) suggests you need at least 10–20 events per predictor (**steyerberg2019?**). With 40 events and 50 predictors, your EPV is 0.8 — far below the minimum.

What goes wrong?

1. **Overfitting.** The model fits the training data very well — perhaps perfectly — but performs terribly on new patients. It has memorised the noise in the training data rather than learning the true signal.
2. **Unstable coefficients.** Small changes in the data (adding or removing a few patients) cause large changes in the estimated coefficients. Some coefficients may be absurdly large.
3. **Separation.** In logistic regression, if a predictor perfectly separates the outcomes (e.g., all patients with a certain lab value died), the maximum likelihood estimate for that coefficient is infinite.
4. **Non-convergence.** When $p > n$, the standard regression algorithm simply does not converge — there is no unique solution.

7.2.2. A Demonstration

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
library(ggplot2)

set.seed(42)
n <- 100
p <- 40

# True model: only 5 predictors matter
X <- matrix(rnorm(n * p), n, p)
colnames(X) <- paste0("X", 1:p)
true_beta <- c(rep(1, 5), rep(0, p - 5))
logit_p <- X %*% true_beta
y <- rbinom(n, 1, plogis(logit_p))

# Fit standard logistic regression (suppress warnings about non-convergence)
fit_standard <- suppressWarnings(
  glm(y ~ X, family = binomial())
)

coef_df <- data.frame(
  predictor = paste0("X", 1:p),
  estimated = coef(fit_standard)[-1], # Remove intercept
  true = true_beta
)

ggplot(coef_df, aes(x = reorder(predictor, -abs(estimated)))) +
  geom_col(aes(y = estimated), fill = "#3498db", alpha = 0.7) +
  geom_point(aes(y = true), colour = "#e74c3c", size = 3) +
  coord_flip() +
  labs(x = "Predictor", y = "Coefficient Value",
       subtitle = "Blue bars = estimated; Red dots = true values. Many coeff.",
       theme_minimal(base_size = 12)
```

7.3. The Bias-Variance Tradeoff

The fundamental insight behind penalised regression is the **bias-variance tradeoff**.

7.3.1. Prediction Error Decomposition

The expected prediction error of a model can be decomposed into three components:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- **Bias** is the systematic error introduced by simplifying assumptions. A model that is too simple (e.g., a linear model for a non-linear relationship) has high bias.
- **Variance** is the sensitivity of the model to the specific training data. A model that is too complex (e.g., 50 predictors with 40 events) has high variance.
- **Irreducible noise** is the inherent randomness in the outcome that no model can capture.

7.3.2. The Tradeoff

Standard regression (OLS, maximum likelihood) is **unbiased** — it gives the best estimates *on average*. But in high-dimensional or small-sample settings, this unbiasedness comes at the cost of enormous variance. The estimated coefficients are unbiased but wildly imprecise.

Penalised regression deliberately introduces a small amount of bias — shrinking coefficients toward zero — in exchange for a dramatic reduction in variance. The net effect is a model with **lower total prediction error** despite being biased.

7. Penalised Regression: Ridge, LASSO, and Elastic Net

Think of it this way: if the true coefficient is 0.5, an unbiased estimator might give you estimates of -2.3, 3.1, 0.8, -1.5, etc. across different samples (unbiased on average, but highly variable). A biased estimator that consistently gives you 0.3 is much closer to the truth in any given sample, even though it is systematically too low.

```
library(ggplot2)

lambda <- seq(0.01, 5, length.out = 200)
bias2 <- 2 * (1 - exp(-0.5 * lambda))
variance <- 3 * exp(-lambda)
total <- bias2 + variance + 0.5

df_bv <- data.frame(
  lambda = rep(lambda, 3),
  error = c(bias2, variance, total),
  component = rep(c("Bias2", "Variance", "Total Error"), each = length(lambda))
)

ggplot(df_bv, aes(x = lambda, y = error, colour = component)) +
  geom_line(linewidth = 1.2) +
  scale_colour_manual(values = c("Bias2" = "#e74c3c", "Variance" = "#3498db",
                                "Total Error" = "#2c3e50")) +
  geom_vline(xintercept = lambda[which.min(total)], linetype = "dashed", colour = "red") +
  annotate("text", x = lambda[which.min(total)] + 0.3, y = max(total) * 0.9,
         label = "Optimal\npenalty", size = 3.5) +
  labs(x = "Penalty Strength (lambda)", y = "Error",
       colour = "Component") +
  scale_x_reverse() +
  theme_minimal(base_size = 14) +
  theme(legend.position = "bottom")
```

7.4. The Penalised Regression Framework

7.4.1. The General Idea

In standard regression, we estimate coefficients by minimising a loss function (e.g., residual sum of squares for linear regression, negative log-likelihood for logistic regression):

$$\hat{\beta} = \arg \min_{\beta} \{\text{Loss}(\beta)\}$$

In penalised regression, we add a **penalty term** that grows with the size of the coefficients:

$$\hat{\beta} = \arg \min_{\beta} \{\text{Loss}(\beta) + \lambda \cdot \text{Penalty}(\beta)\}$$

The penalty term discourages large coefficients. The tuning parameter $\lambda \geq 0$ controls the strength of the penalty:

- $\lambda = 0$: no penalty, equivalent to standard regression
- $\lambda \rightarrow \infty$: all coefficients shrunk to zero (null model)
- Intermediate λ : the sweet spot, chosen by cross-validation

The three main methods differ only in the form of the penalty.

! Standardisation Is Essential

Before applying penalised regression, all predictors must be **standardised** (centred to mean zero, scaled to unit variance). This is because the penalty treats all coefficients equally — if one predictor is measured in milligrams and another in kilograms, the unstandardised coefficients will be on very different scales, and the penalty will disproportionately shrink the one with the smaller scale.

7. Penalised Regression: Ridge, LASSO, and Elastic Net

Both `glmnet` (R) and `sklearn` (Python) standardise internally by default, but you should be aware of this, especially when interpreting the output.

7.5. Ridge Regression (L2 Penalty)

7.5.1. Definition

Ridge regression adds the **squared sum of coefficients** (L2 norm) as the penalty:

$$\hat{\beta}_{\text{Ridge}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

Note that the **intercept** β_0 is typically **not penalised** — only the predictor coefficients $\beta_1, \dots, \beta_p$ are shrunk.

7.5.2. Key Properties

1. **Shrinks coefficients toward zero, but never exactly to zero.** No matter how large λ is, the Ridge coefficients remain non-zero (though they can be extremely small). This means Ridge regression does *not* perform variable selection — all predictors remain in the model.
2. **Closed-form solution.** The Ridge estimate has an elegant closed-form:

$$\hat{\beta}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

The $\lambda \mathbf{I}$ term added to $\mathbf{X}^T \mathbf{X}$ makes the matrix invertible even when $p > n$ or when predictors are highly correlated. This is why Ridge

7.6. LASSO Regression (L1 Penalty)

regression is sometimes described as “regularising” an ill-conditioned problem.

3. **Handles multicollinearity gracefully.** When predictors are highly correlated (e.g., systolic and diastolic blood pressure, or multiple related inflammatory markers), standard regression produces wildly unstable coefficients that can have the wrong sign. Ridge regression stabilises these estimates by shrinking correlated predictors toward each other.

7.5.3. When to Use Ridge

- You believe **many predictors** contribute to the outcome (none is truly zero)
- Predictors are **highly correlated** (multicollinearity)
- You care about **prediction accuracy** more than variable selection
- Example: predicting ICU mortality using dozens of physiological measurements, most of which carry some information

7.6. LASSO Regression (L1 Penalty)

7.6.1. Definition

The LASSO (Least Absolute Shrinkage and Selection Operator) uses the **sum of absolute values** (L1 norm) as the penalty:

$$\hat{\beta}_{\text{LASSO}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

7. Penalised Regression: Ridge, LASSO, and Elastic Net

7.6.2. Key Properties

1. **Shrinks some coefficients to exactly zero.** This is the defining feature of the LASSO. When λ is large enough, some coefficients are forced to zero, effectively removing those predictors from the model. The LASSO thus performs **automatic variable selection**.
2. **No closed-form solution.** Unlike Ridge, the LASSO requires iterative optimisation algorithms (coordinate descent is the standard approach in `glmnet`).
3. **Selects at most n predictors** (when $p > n$). The LASSO can select at most $\min(n, p)$ predictors. When you have far more predictors than observations, this is a useful property.
4. **Struggles with correlated predictors.** When two predictors are highly correlated, the LASSO tends to pick one and drop the other arbitrarily. This can lead to **unstable variable selection** — which predictor is kept can change across different samples.

7.6.3. Why Sparsity Matters Clinically

In clinical research, we often want to identify the **most important predictors** from a larger set of candidates. This is common in:

- **Clinical prediction model development.** A parsimonious model with 5–10 predictors is more practical than one with 50.
- **Biomarker discovery.** From a panel of 100 candidate biomarkers, which ones are truly associated with the outcome?
- **Risk factor identification.** Among many potential lifestyle, genetic, and environmental factors, which are the strongest?

The LASSO's ability to set coefficients exactly to zero makes it a natural tool for these tasks.

⚠ LASSO Is Not a Substitute for Domain Knowledge

While the LASSO performs automatic variable selection, the results should always be interpreted in light of clinical knowledge. A variable dropped by the LASSO is not necessarily unimportant — it may be correlated with another retained variable, or the sample may be too small to detect its effect. Similarly, a variable retained by the LASSO is not necessarily causal.

Use the LASSO as a tool to *complement*, not replace, clinical reasoning about which variables matter.

7.6.4. The Geometry of L1 vs L2

The difference between Ridge and LASSO can be understood geometrically. The L2 penalty constrains coefficients to lie within a sphere (in two dimensions, a circle). The L1 penalty constrains coefficients to lie within a diamond (in two dimensions, a square rotated 45 degrees).

The key insight is that the diamond has **corners** on the axes. When the contours of the loss function touch the constraint region, they are much more likely to hit a corner of the diamond (where one coefficient is exactly zero) than a point on the smooth surface of the sphere. This is why the LASSO produces exact zeros and Ridge does not.

```
library(ggplot2)

theta <- seq(0, 2 * pi, length.out = 200)

# L2 constraint (circle)
l2_x <- cos(theta)
l2_y <- sin(theta)

# L1 constraint (diamond)
```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
l1_x <- c(1, 0, -1, 0, 1)
l1_y <- c(0, 1, 0, -1, 0)

# Loss function contours (ellipse centered at (1.5, 1))
ellipse_x <- function(a, cx = 1.5, cy = 1) cx + a * 1.2 * cos(theta)
ellipse_y <- function(a, cx = 1.5, cy = 1) cy + a * 0.8 * sin(theta)

plot_df <- rbind(
  data.frame(x = l2_x, y = l2_y, group = "L2 (Ridge)"),
  data.frame(x = l1_x, y = l1_y, group = "L1 (LASSO)")
)

ggplot() +
  # Constraint regions
  geom_polygon(data = data.frame(x = l2_x, y = l2_y),
    aes(x, y), fill = "#3498db", alpha = 0.15) +
  geom_path(data = data.frame(x = l2_x, y = l2_y),
    aes(x, y), colour = "#3498db", linewidth = 1.2) +
  geom_polygon(data = data.frame(x = l1_x, y = l1_y),
    aes(x, y), fill = "#e74c3c", alpha = 0.15) +
  geom_path(data = data.frame(x = l1_x, y = l1_y),
    aes(x, y), colour = "#e74c3c", linewidth = 1.2) +
  # Loss contours
  geom_path(data = data.frame(x = ellipse_x(0.8), y = ellipse_y(0.8)),
    aes(x, y), colour = "grey40", linetype = "dashed") +
  geom_path(data = data.frame(x = ellipse_x(1.15), y = ellipse_y(1.15)),
    aes(x, y), colour = "grey40", linetype = "dashed") +
  geom_path(data = data.frame(x = ellipse_x(1.5), y = ellipse_y(1.5)),
    aes(x, y), colour = "grey40", linetype = "dashed") +
  # OLS solution
  geom_point(aes(x = 1.5, y = 1), size = 3, colour = "black") +
  annotate("text", x = 1.7, y = 1.15, label = "OLS\nsolution", size = 3) +
  # LASSO solution (hits corner)
```

7.7. Elastic Net: The Best of Both Worlds

```
geom_point(aes(x = 0, y = 1), size = 3, colour = "#e74c3c") +  
annotate("text", x = -0.35, y = 1.1, label = "LASSO", size = 3, colour = "#e74c3c") +  
# Ridge solution  
geom_point(aes(x = 0.65, y = 0.75), size = 3, colour = "#3498db") +  
annotate("text", x = 0.65, y = 0.55, label = "Ridge", size = 3, colour = "#3498db") +  
coord_fixed() +  
labs(x = expression(beta[1]), y = expression(beta[2])) +  
theme_minimal(base_size = 14) +  
xlim(-1.8, 3) + ylim(-1.5, 2.5)
```

7.7. Elastic Net: The Best of Both Worlds

7.7.1. Definition

The Elastic Net combines both L1 and L2 penalties:

$$\hat{\beta}_{\text{EN}} = \arg \min_{\beta} \left\{ \text{Loss}(\beta) + \lambda \left[\alpha \sum_{j=1}^p |\beta_j| + \frac{(1-\alpha)}{2} \sum_{j=1}^p \beta_j^2 \right] \right\}$$

The parameter $\alpha \in [0, 1]$ controls the **mix** of L1 and L2:

- $\alpha = 1$: pure LASSO
- $\alpha = 0$: pure Ridge
- $0 < \alpha < 1$: Elastic Net (a blend)

7.7.2. Why Combine?

The Elastic Net addresses the LASSO's limitations while retaining its strengths:

7. Penalised Regression: Ridge, LASSO, and Elastic Net

1. **Correlated predictors.** When predictors are correlated, the LASSO picks one and drops the rest. The Elastic Net tends to **keep correlated predictors together** (the “grouping effect”), either including all of them or excluding all of them. This is often more scientifically reasonable.
2. **More than n predictors.** The LASSO selects at most n predictors. The Elastic Net can select more, which is useful in genomics where thousands of genes may be relevant.
3. **Stability.** The Elastic Net generally produces more stable results than the LASSO across different samples.

7.7.3. Choosing Alpha

In practice, α is often chosen by **cross-validation** alongside λ . A common approach:

- Try $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9, 1.0\}$
- For each α , use cross-validation to find the optimal λ
- Select the (α, λ) combination with the best cross-validated performance

Alternatively, $\alpha = 0.5$ is a reasonable default that balances L1 and L2 equally.

7.8. Choosing Lambda: Cross-Validation

7.8.1. The Cross-Validation Procedure

The tuning parameter λ controls the penalty strength. We cannot estimate it from the data using the same approach as the regression coefficients —

7.8. Choosing Lambda: Cross-Validation

it must be chosen externally. **K-fold cross-validation** is the standard approach:

1. Divide the data into K folds (typically $K = 10$).
2. For each value of λ on a grid:
 - a. For each fold, fit the model on the other $K - 1$ folds and predict the held-out fold.
 - b. Calculate the prediction error (e.g., mean squared error, deviance, or misclassification rate).
3. Average the prediction error across folds for each λ .
4. Select the λ that minimises the average prediction error.

7.8.2. Lambda.min vs Lambda.1se

Cross-validation typically identifies two candidate values of λ :

- **lambda.min**: the value of λ that minimises the cross-validated error. This gives the model with the best predictive performance.
- **lambda.1se**: the largest λ whose cross-validated error is within one standard error of the minimum. This gives a **simpler, more parsimonious model** that is nearly as good as the best.

Which Lambda to Use?

In clinical prediction modelling, **lambda.min** is often preferred when predictive accuracy is the primary goal. In biomarker discovery or variable selection, **lambda.1se** is often preferred because it gives a more parsimonious model with fewer selected variables.

Steyerberg (**steyerberg2019?**) recommends reporting results for both and discussing the trade-off between parsimony and performance.

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
library(glmnet)

set.seed(42)
n <- 200
p <- 30
X <- matrix(rnorm(n * p), n, p)
true_beta <- c(rep(1.5, 5), rep(0, 25))
y <- rbinom(n, 1, plogis(X %*% true_beta))

cv_fit <- cv.glmnet(X, y, family = "binomial", alpha = 1, nfolds = 10)
plot(cv_fit)
```

7.9. Regularisation Paths

A **regularisation path** (also called a coefficient trace or coefficient path) shows how each coefficient changes as λ varies from large (strong penalty) to small (weak penalty).

```
library(glmnet)

fit_lasso <- glmnet(X, y, family = "binomial", alpha = 1)
plot(fit_lasso, xvar = "lambda", label = TRUE)
abline(v = log(cv_fit$lambda.min), lty = 2, col = "blue")
abline(v = log(cv_fit$lambda.1se), lty = 2, col = "red")
legend("topright", legend = c("lambda.min", "lambda.1se"),
      col = c("blue", "red"), lty = 2, cex = 0.8)
```

7.9.1. How to Read a Regularisation Path

- **Right side** (large λ): strong penalty, all coefficients near zero.

7.10. Clinical Example: Breast Cancer Diagnosis from Image Features

- **Left side** (small λ): weak penalty, coefficients approach their OLS values.
- **Order of entry**: predictors that become non-zero first (at the largest λ) are the most important — they require the least “loosening” of the penalty to enter the model.
- **Sign and magnitude**: the sign and eventual magnitude reveal the direction and strength of each predictor’s association.

This plot is extremely useful for understanding the relative importance of predictors and the stability of the model across different penalty strengths.

7.10. Clinical Example: Breast Cancer Diagnosis from Image Features

7.10.1. The Dataset

The Wisconsin Breast Cancer Diagnostic dataset contains 569 biopsy samples with 30 image-derived features (mean, standard error, and worst values for 10 characteristics: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension). The outcome is the diagnosis: malignant (M) or benign (B).

With 30 predictors and 569 observations, this is a setting where penalised regression may improve upon standard logistic regression, particularly because many of the features are highly correlated (e.g., radius, perimeter, and area are nearly collinear).

7.10.2. Analysis in R

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
library(glmnet)
library(ggplot2)

# Load the dataset (available in mlbench or we can use the built-in version)
# install.packages("mlbench")
data(BreastCancer, package = "mlbench")

# Prepare the data
bc <- BreastCancer[complete.cases(BreastCancer), ]
bc$diagnosis <- ifelse(bc$Class == "malignant", 1, 0)

# Extract numeric predictors (columns 2-10 in BreastCancer)
X <- as.matrix(apply(bc[, 2:10], 2, as.numeric))
y <- bc$diagnosis

# Fit LASSO with cross-validation
set.seed(42)
cv_lasso <- cv.glmnet(X, y, family = "binomial", alpha = 1, nfolds = 10)

cat("Lambda.min:", round(cv_lasso$lambda.min, 4), "\n")
cat("Lambda.1se:", round(cv_lasso$lambda.1se, 4), "\n\n")

# Coefficients at lambda.1se (sparser model)
coef_1se <- coef(cv_lasso, s = "lambda.1se")
cat("Coefficients at lambda.1se:\n")
print(round(coef_1se[coef_1se[, 1] != 0, , drop = FALSE], 4))

# Coefficients at lambda.min (best predictive model)
coef_min <- coef(cv_lasso, s = "lambda.min")
cat("\nCoefficients at lambda.min:\n")
print(round(coef_min[coef_min[, 1] != 0, , drop = FALSE], 4))
```


7.10. Clinical Example: Breast Cancer Diagnosis from Image Features

```
# Compare LASSO, Ridge, and Elastic Net
set.seed(42)
cv_ridge <- cv.glmnet(X, y, family = "binomial", alpha = 0, nfolds = 10)
cv_enet <- cv.glmnet(X, y, family = "binomial", alpha = 0.5, nfolds = 10)

cat("\n--- Cross-Validated Deviance (at lambda.min) ---\n")
cat("Ridge:      ", round(min(cv_ridge$cvm), 4), "\n")
cat("Elastic Net:", round(min(cv_enet$cvm), 4), "\n")
cat("LASSO:      ", round(min(cv_lasso$cvm), 4), "\n")
```

7.10.3. Comparing the Three Methods

```
library(ggplot2)

# Extract coefficients at lambda.min for each method
coef_ridge <- as.vector(coef(cv_ridge, s = "lambda.min"))[-1]
coef_enet <- as.vector(coef(cv_enet, s = "lambda.min"))[-1]
coef_lasso <- as.vector(coef(cv_lasso, s = "lambda.min"))[-1]

pred_names <- colnames(X)

comp_df <- data.frame(
  predictor = rep(pred_names, 3),
  coefficient = c(coef_ridge, coef_enet, coef_lasso),
  method = rep(c("Ridge", "Elastic Net", "LASSO"), each = length(pred_names))
)

ggplot(comp_df, aes(x = predictor, y = coefficient, fill = method)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_fill_manual(values = c("Ridge" = "#3498db", "Elastic Net" = "#2ecc71",
    "LASSO" = "#e74c3c")) +
```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
coord_flip() +  
labs(x = "Predictor", y = "Coefficient (standardised)",  
     fill = "Method") +  
theme_minimal(base_size = 13) +  
theme(legend.position = "bottom")
```

7.10.4. Interpretation

Several patterns emerge from this analysis:

1. **Ridge** retains all 9 predictors, distributing the coefficients across correlated features.
2. **LASSO** selects a subset — typically those features that carry the most unique information.
3. **Elastic Net** behaves intermediately, selecting somewhat more predictors than LASSO while still shrinking some to zero.
4. The cross-validated deviance is similar across all three methods, confirming that penalised regression is robust to the choice of penalty type in this dataset.

7.11. When to Use Which Method

The following decision framework can guide your choice:

Figure 7.1.: Decision flowchart for choosing a penalised regression method.

7.11.1. Summary Table

Table 7.1.: When to use Ridge, LASSO, or Elastic Net

Scenario	Recommended Method	Rationale
Few predictors, large sample	Standard regression (or Ridge for mild shrinkage)	Penalisation not needed, but mild shrinkage can still improve predictions
Many predictors, want variable selection	LASSO	Sets unimportant predictors to exactly zero
Many correlated predictors, want variable selection	Elastic Net	Groups correlated predictors together
Many correlated predictors, want best prediction	Ridge	Keeps all predictors, handles collinearity well
Genomics / high-dimensional ($p \gg n$)	Elastic Net or LASSO	Need sparsity; Elastic Net more stable
Clinical prediction model	Ridge or Elastic Net	Stability and prediction are key

7.12. Implementation

7.12.1. R: Using `glmnet`

The `glmnet` package is the standard implementation for penalised regression in R. It is fast (uses coordinate descent), handles logistic and Cox regression as well as linear, and has excellent cross-validation support.

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
library(glmnet)

# --- Step 1: Prepare data ---
# X must be a matrix (not a data frame)
# y must be a vector (numeric for linear, 0/1 for logistic)
X <- as.matrix(your_data[, predictor_columns])
y <- your_data$outcome

# --- Step 2: Fit with cross-validation ---
# alpha = 1 for LASSO, 0 for Ridge, 0-1 for Elastic Net
# family = "gaussian" for linear, "binomial" for logistic, "cox" for survival
set.seed(42) # For reproducibility of CV folds
cv_fit <- cv.glmnet(X, y, family = "binomial", alpha = 1, nfolds = 10)

# --- Step 3: Examine the CV curve ---
plot(cv_fit) # Shows deviance vs log(lambda)

# --- Step 4: Extract coefficients ---
# At lambda.min (best prediction)
coef(cv_fit, s = "lambda.min")

# At lambda.1se (most parsimonious)
coef(cv_fit, s = "lambda.1se")

# --- Step 5: Make predictions ---
predictions <- predict(cv_fit, newx = X_test, s = "lambda.min",
                       type = "response") # Gives probabilities

# --- Step 6: Regularisation path ---
fit <- glmnet(X, y, family = "binomial", alpha = 1)
plot(fit, xvar = "lambda", label = TRUE)

# --- Step 7: Compare alpha values for Elastic Net ---
```

7.12. Implementation

```
alphas <- c(0, 0.25, 0.5, 0.75, 1)
cv_results <- lapply(alphas, function(a) {
  cv.glmnet(X, y, family = "binomial", alpha = a, nfolds = 10)
})
# Find the alpha with lowest minimum CV error
min_errors <- sapply(cv_results, function(fit) min(fit$cvm))
best_alpha <- alphas[which.min(min_errors)]
cat("Best alpha:", best_alpha, "\n")
```

7.12.2. Python: Using scikit-learn

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

# --- Step 1: Prepare data ---
X = your_data[predictor_columns].values
y = your_data['outcome'].values

# StandardScaler (sklearn's LogisticRegression does not auto-scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# --- Step 2: LASSO (L1) with cross-validation ---
# In sklearn, the parameter C = 1/lambda (inverse regularisation strength)
# penalty='l1' for LASSO, 'l2' for Ridge, 'elasticnet' for Elastic Net

lasso_cv = LogisticRegressionCV(
```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
penalty='l1',
solver='saga',          # Required for L1 penalty
Cs=50,                  # Number of C values to try (more = finer grid)
cv=10,                  # 10-fold CV
scoring='neg_log_loss',
max_iter=10000,
random_state=42
)
lasso_cv.fit(X_scaled, y)

print(f"Best C (= 1/lambda): {lasso_cv.C_[0]:.4f}")
print(f"Best lambda: {1/lasso_cv.C_[0]:.4f}")
print(f"\nNon-zero coefficients:")
for name, coef in zip(predictor_columns, lasso_cv.coef_[0]):
    if abs(coef) > 1e-6:
        print(f" {name}: {coef:.4f}")

# --- Step 3: Ridge (L2) with cross-validation ---
ridge_cv = LogisticRegressionCV(
    penalty='l2',
    solver='lbfgs',
    Cs=50,
    cv=10,
    scoring='neg_log_loss',
    max_iter=10000,
    random_state=42
)
ridge_cv.fit(X_scaled, y)

# --- Step 4: Elastic Net with cross-validation ---
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import GridSearchCV
```

7.12. Implementation

```
# sklearn's Elastic Net for classification uses SGDClassifier
# or LogisticRegression with penalty='elasticnet'
enet_cv = LogisticRegressionCV(
    penalty='elasticnet',
    solver='saga',
    Cs=50,
    cv=10,
    l1_ratios=[0.1, 0.25, 0.5, 0.75, 0.9], # alpha values to try
    scoring='neg_log_loss',
    max_iter=10000,
    random_state=42
)
enet_cv.fit(X_scaled, y)

print(f"Best l1_ratio (alpha): {enet_cv.l1_ratio_[0]:.2f}")
print(f"Best C (1/lambda): {enet_cv.C_[0]:.4f}")

# --- Step 5: Regularisation path plot ---
from sklearn.linear_model import lasso_path

# For linear regression (use LogisticRegression loop for classification)
Cs = np.logspace(-3, 2, 100)
coefs = []
for C in Cs:
    model = LogisticRegression(penalty='l1', C=C, solver='saga',
                              max_iter=10000)
    model.fit(X_scaled, y)
    coefs.append(model.coef_[0])

coefs = np.array(coefs)
lambdas = 1.0 / Cs

plt.figure(figsize=(10, 6))
```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
for j in range(X.shape[1]):
    plt.plot(np.log(lambdas), coefs[:, j],
             label=predictor_columns[j] if X.shape[1] <= 15 else None)
plt.xlabel('log(lambda)')
plt.ylabel('Coefficient')
plt.title('LASSO Regularisation Path')
plt.axvline(x=np.log(1/lasso_cv.C_[0]), color='k', linestyle='--',
            label='CV optimal lambda')
if X.shape[1] <= 15:
    plt.legend(loc='best', fontsize=8)
plt.tight_layout()
plt.show()
```

7.13. Common Pitfalls



Pitfalls to Avoid

1. **Forgetting to standardise.** If you do not standardise predictors, the penalty will disproportionately affect predictors with larger scales. `glmnet` and `sklearn` handle this internally, but be aware when interpreting coefficients — they are on the standardised scale unless you back-transform.
2. **Interpreting LASSO-selected variables as “important.”** A variable dropped by the LASSO may be correlated with a retained variable. It is not necessarily unimportant — it is just redundant *given the other predictors in the model*.
3. **Reporting p-values for LASSO-selected variables.** After variable selection by LASSO, the standard errors and p-values from refitting an unpenalised model on the selected variables

7.14. Penalised Regression in Prediction Modelling

are **invalid**. This is because the selection step has already used the data, inflating the apparent significance. This is sometimes called “post-selection inference” and is an active area of research.

4. **Using the same data for tuning and evaluation.** The cross-validation used to select λ should be considered part of model building. To honestly assess model performance, you need a **separate** test set or nested cross-validation.
5. **Ignoring the 1se rule.** Researchers often default to `lambda.min`, but `lambda.1se` frequently gives a much simpler model with negligibly worse performance. Always consider both.
6. **Applying penalised regression when standard regression suffices.** If you have 10 predictors and 1000 observations, standard regression is fine. Penalisation will not hurt (much), but it adds complexity without clear benefit unless you are specifically developing a prediction model.

7.14. Penalised Regression in Prediction Modelling

In the context of clinical prediction models — which we discuss in later chapters — penalised regression plays a specific role:

1. **Shrinkage corrects for optimism.** Prediction models developed on finite samples are always overfitted to some degree. The apparent performance (e.g., C-statistic, calibration) is too optimistic. Penalisation provides a principled way to shrink coefficients, correcting for this optimism *during* model development rather than *after*.
2. **Alternative to backward selection.** Traditional backward step-wise selection is known to inflate Type I error rates, produce unstable

7. Penalised Regression: Ridge, LASSO, and Elastic Net

models, and overestimate effect sizes. LASSO and Elastic Net provide a more principled approach to variable selection with better statistical properties.

3. **Uniform shrinkage vs differential shrinkage.** A simple post-hoc shrinkage factor (as recommended by Steyerberg (steyerberg2019?)) applies the same shrinkage to all coefficients. Penalised regression allows *differential* shrinkage — strong effects are shrunk less than weak effects — which is more efficient.

7.15. Exercises

7.15.1. Exercise 1: Conceptual Questions

- a) A colleague tells you: “I used LASSO and it removed age from my model predicting mortality. This means age is not a risk factor for death.” Explain why this interpretation is incorrect.
- b) You fit a Ridge regression model to predict length of hospital stay using 20 clinical variables. The cross-validated RMSE is 2.1 days. You then fit a standard linear regression on the same data and get $\text{RMSE} = 1.8$ days. Your colleague says the standard model is better because it has lower RMSE. What is wrong with this comparison?
- c) Explain in one sentence why the LASSO produces exact zeros but Ridge does not.

7.15.2. Exercise 2: Predicting Diabetes Onset

Using the Pima Indians Diabetes dataset (available in R and Python), build penalised regression models to predict diabetes onset.

7.15.2.1. R

```

library(glmnet)
library(mlbench)

# Load data
data(PimaIndiansDiabetes2, package = "mlbench")
pima <- na.omit(PimaIndiansDiabetes2)

# Prepare data
X <- as.matrix(pima[, 1:8])
y <- ifelse(pima$diabetes == "pos", 1, 0)

# Task 1: Fit a LASSO model with 10-fold cross-validation
set.seed(123)
cv_lasso <- cv.glmnet(X, y, family = "binomial", alpha = 1, nfolds = 10)

# Task 2: Plot the cross-validation curve
plot(cv_lasso)

# Task 3: Which variables are selected at lambda.1se?
coef(cv_lasso, s = "lambda.1se")

# Task 4: Fit Ridge and Elastic Net (alpha = 0.5) models
cv_ridge <- cv.glmnet(X, y, family = "binomial", alpha = 0, nfolds = 10)
cv_enet <- cv.glmnet(X, y, family = "binomial", alpha = 0.5, nfolds = 10)

# Task 5: Compare the three methods:
#   - How many variables does each select (at lambda.1se)?
#   - What is the cross-validated deviance for each?
#   - Plot the regularisation path for the LASSO model

# Task 6: Which method would you recommend for this problem, and why?

```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

7.15.2.2. Python

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_diabetes # Note: this is regression
import matplotlib.pyplot as plt

# Load the Pima Indians Diabetes dataset
# Available via: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes
# Or use the following URL:
url = ("https://raw.githubusercontent.com/jbrownlee/Datasets/master/"
      "pima-indians-diabetes.data.csv")
columns = ['pregnancies', 'glucose', 'blood_pressure', 'skin_thickness',
          'insulin', 'bmi', 'diabetes_pedigree', 'age', 'outcome']
pima = pd.read_csv(url, names=columns)

# Remove rows with zero values in columns where zero is not meaningful
cols_no_zero = ['glucose', 'blood_pressure', 'skin_thickness',
                'insulin', 'bmi']
pima[cols_no_zero] = pima[cols_no_zero].replace(0, np.nan)
pima = pima.dropna()

X = pima[columns[:-1]].values
y = pima['outcome'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Task 1: Fit LASSO with cross-validation
lasso_cv = LogisticRegressionCV(
```

```

    penalty='l1', solver='saga', Cs=50, cv=10,
    scoring='neg_log_loss', max_iter=10000, random_state=42
)
lasso_cv.fit(X_scaled, y)

print("LASSO selected features:")
for name, coef in zip(columns[:-1], lasso_cv.coef_[0]):
    if abs(coef) > 1e-6:
        print(f" {name}: {coef:.4f}")

# Task 2: Fit Ridge with cross-validation
ridge_cv = LogisticRegressionCV(
    penalty='l2', solver='lbfgs', Cs=50, cv=10,
    scoring='neg_log_loss', max_iter=10000, random_state=42
)
ridge_cv.fit(X_scaled, y)

# Task 3: Fit Elastic Net with cross-validation
enet_cv = LogisticRegressionCV(
    penalty='elasticnet', solver='saga', Cs=50, cv=10,
    l1_ratios=[0.25, 0.5, 0.75], scoring='neg_log_loss',
    max_iter=10000, random_state=42
)
enet_cv.fit(X_scaled, y)

# Task 4: Compare cross-validated scores
for name, model in [('LASSO', lasso_cv), ('Ridge', ridge_cv),
                    ('Elastic Net', enet_cv)]:
    scores = cross_val_score(model, X_scaled, y, cv=10,
                              scoring='neg_log_loss')
    print(f"{name}: Mean CV log-loss = {-scores.mean():.4f} "
          f" (+/- {scores.std():.4f})")

```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
# Task 5: Plot regularisation path for LASSO
Cs = np.logspace(-2, 2, 80)
coefs = []
for C in Cs:
    model = LogisticRegressionCV(penalty='l1', solver='saga', Cs=[C],
                                cv=5, max_iter=10000, random_state=42)
    model.fit(X_scaled, y)
    coefs.append(model.coef_[0])

coefs = np.array(coefs)
plt.figure(figsize=(10, 6))
for j, name in enumerate(columns[:-1]):
    plt.plot(np.log10(1/Cs), coefs[:, j], label=name)
plt.xlabel('log10(lambda)')
plt.ylabel('Coefficient')
plt.title('LASSO Regularisation Path - Pima Diabetes')
plt.legend(loc='best')
plt.tight_layout()
plt.show()
```

7.15.3. Exercise 3: The Effect of Correlation

This exercise demonstrates how LASSO and Ridge handle correlated predictors differently.

7.15.3.1. R

```
library(glmnet)
library(MASS)

set.seed(2025)
```

```

n <- 200

# Create 4 correlated predictors (correlation ~ 0.9)
Sigma <- matrix(0.9, 4, 4)
diag(Sigma) <- 1
X_corr <- mvrnorm(n, mu = rep(0, 4), Sigma = Sigma)

# Add 6 independent predictors (noise)
X_indep <- matrix(rnorm(n * 6), n, 6)
X <- cbind(X_corr, X_indep)
colnames(X) <- c(paste0("Corr_", 1:4), paste0("Noise_", 1:6))

# True model: all 4 correlated predictors have effect = 0.5
true_beta <- c(rep(0.5, 4), rep(0, 6))
y <- rbinom(n, 1, plogis(X %*% true_beta))

# Task 1: Fit LASSO and examine which variables are selected
cv_lasso <- cv.glmnet(X, y, family = "binomial", alpha = 1)
cat("LASSO coefficients (lambda.1se):\n")
print(round(as.matrix(coef(cv_lasso, s = "lambda.1se")), 3))

# Task 2: Fit Ridge and examine coefficients
cv_ridge <- cv.glmnet(X, y, family = "binomial", alpha = 0)
cat("\nRidge coefficients (lambda.1se):\n")
print(round(as.matrix(coef(cv_ridge, s = "lambda.1se")), 3))

# Task 3: Fit Elastic Net (alpha = 0.5)
cv_enet <- cv.glmnet(X, y, family = "binomial", alpha = 0.5)
cat("\nElastic Net coefficients (lambda.1se):\n")
print(round(as.matrix(coef(cv_enet, s = "lambda.1se")), 3))

# Questions:
# a) Does LASSO select all 4 correlated predictors, or does it pick

```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
# just one or two? Why?
# b) How does Ridge handle the correlated predictors differently?
# c) Does Elastic Net improve on the LASSO in terms of selecting the
# correlated predictors?
# d) Run the analysis 10 times with different seeds. How stable is the
# LASSO variable selection compared to Ridge and Elastic Net?
```

7.15.3.2. Python

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from scipy.special import expit

np.random.seed(2025)
n = 200

# Create 4 correlated predictors (correlation ~ 0.9)
mean = np.zeros(4)
cov = np.full((4, 4), 0.9)
np.fill_diagonal(cov, 1.0)
X_corr = np.random.multivariate_normal(mean, cov, size=n)

# Add 6 independent noise predictors
X_indep = np.random.randn(n, 6)
X = np.hstack([X_corr, X_indep])

feature_names = [f"Corr_{i+1}" for i in range(4)] + \
                 [f"Noise_{i+1}" for i in range(6)]

# True model: all 4 correlated predictors contribute
```



```

true_beta = np.array([0.5]*4 + [0.0]*6)
y = np.random.binomial(1, expit(X @ true_beta))

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Task 1: LASSO
lasso = LogisticRegressionCV(penalty='l1', solver='saga', Cs=50,
                             cv=10, max_iter=10000, random_state=42)
lasso.fit(X_scaled, y)
print("LASSO coefficients:")
for name, coef in zip(feature_names, lasso.coef_[0]):
    print(f" {name}: {coef:.4f}")

# Task 2: Ridge
ridge = LogisticRegressionCV(penalty='l2', solver='lbfgs', Cs=50,
                             cv=10, max_iter=10000, random_state=42)
ridge.fit(X_scaled, y)
print("\nRidge coefficients:")
for name, coef in zip(feature_names, ridge.coef_[0]):
    print(f" {name}: {coef:.4f}")

# Task 3: Elastic Net
enet = LogisticRegressionCV(penalty='elasticnet', solver='saga', Cs=50,
                             cv=10, l1_ratios=[0.5], max_iter=10000,
                             random_state=42)
enet.fit(X_scaled, y)
print("\nElastic Net coefficients:")
for name, coef in zip(feature_names, enet.coef_[0]):
    print(f" {name}: {coef:.4f}")

# Task 4: Stability analysis - run 10 times with different seeds
print("\n--- Stability Analysis (LASSO) ---")

```

7. Penalised Regression: Ridge, LASSO, and Elastic Net

```
for seed in range(10):
    np.random.seed(seed)
    idx = np.random.choice(n, n, replace=True) # Bootstrap sample
    lasso_boot = LogisticRegressionCV(penalty='l1', solver='saga',
                                     Cs=20, cv=5, max_iter=10000,
                                     random_state=42)
    lasso_boot.fit(X_scaled[idx], y[idx])
    selected = [feature_names[j] for j in range(10)
               if abs(lasso_boot.coef_[0][j]) > 1e-6]
    print(f" Seed {seed}: {selected}")
```

7.16. Summary

Concept	Details
The problem	Standard regression fails or overfits when p is large relative to n
Bias-variance tradeoff	Adding bias (shrinkage) reduces variance, lowering total prediction error
Ridge (L2)	Shrinks all coefficients toward zero; never exactly zero; handles collinearity
LASSO (L1)	Shrinks some coefficients to exactly zero; automatic variable selection
Elastic Net	Combines L1 and L2; groups correlated predictors; alpha controls the mix
Lambda	Penalty strength; chosen by cross-validation
lambda.min	Best predictive performance

Concept	Details
lambda.1se	Most parsimonious model within 1 SE of the best
Standardise first	Essential so the penalty treats all predictors equally
Post-selection inference	P-values after LASSO selection are invalid without special methods

References

8. Survival Analysis: Time-to-Event Data in Medicine

8.1. Why Ordinary Regression Fails for Time-to-Event Data

In medicine, we frequently ask questions like: *How long until a patient relapses after chemotherapy?* *How long does a hip replacement last before revision surgery is needed?* *What is the median survival time after a Stage IV lung cancer diagnosis?*

These are **time-to-event** questions. The outcome is not simply “did the event happen?” (which would be logistic regression) nor is it a continuous measurement at a fixed time point (which would be linear regression). The outcome is *how long* until something happens.

You might think: “Why not just use linear regression with time as the outcome?” There is a fundamental problem — **censoring**.

8.1.1. The Censoring Problem

Imagine a clinical trial that follows 100 patients for 5 years after a cancer diagnosis. At the end of the study:

- 40 patients have died (we know their exact survival time).
- 30 patients are still alive at year 5 (we know they survived *at least* 5 years, but we do not know when they will die).

8. Survival Analysis: Time-to-Event Data in Medicine

- 20 patients moved away or were lost to follow-up at various time points.
- 10 patients withdrew from the study.

For the last 60 patients, we have **incomplete information**. We know they survived *at least* until the last time we observed them, but not how much longer. This is **right censoring** — the event of interest (death) may occur to the right of (after) our last observation.

If we simply excluded these 60 patients, we would introduce severe bias — we would systematically underestimate survival times because the long-survivors are disproportionately censored. If we treated their last-observed time as their event time, we would also underestimate survival. Survival analysis methods were developed precisely to handle this problem.

8.1.2. Types of Censoring

- **Right censoring** (most common): The event has not yet occurred when observation ends. The true event time is to the right of the censoring time.
- **Left censoring**: The event occurred *before* observation began (e.g., a patient already had the disease at enrollment, but we do not know when it started).
- **Interval censoring**: The event is known to have occurred within an interval but the exact time is unknown (e.g., a tumor detected at a scheduled 6-month scan — it appeared sometime between the last scan and this one).

In this chapter, we focus primarily on right censoring, which dominates clinical research.

8.2. The Kaplan-Meier Estimator

The **Kaplan-Meier (KM) estimator** is the workhorse of survival analysis. It provides a non-parametric estimate of the survival function $S(t) = P(T > t)$, the probability of surviving beyond time t .

8.2.1. How It Works

The KM estimator calculates survival probability at each time point where an event occurs:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right)$$

where:

- t_i is a time at which at least one event occurred,
- d_i is the number of events at time t_i ,
- n_i is the number of individuals at risk just before time t_i (alive and uncensored).

The key insight is that censored individuals contribute to the **risk set** (n_i) up until the moment they are censored, and then they drop out. They are not discarded — they contribute all the information they can.

8.2.2. Clinical Example: Primary Biliary Cholangitis (PBC)

We will use the classic PBC dataset from the Mayo Clinic, which is included in R's **survival** package. This dataset contains 418 patients with primary biliary cholangitis (PBC), a chronic liver disease. Patients were followed from enrollment until death, transplant, or censoring.

8. Survival Analysis: Time-to-Event Data in Medicine

8.2.2.1. R

```
# Load the PBC dataset
data(pbc, package = "survival")

# Prepare data - status: 0 = censored, 1 = transplant, 2 = dead
# For now, treat death as the event and censor everything else
pbc_clean <- pbc %>%
  filter(!is.na(trt)) %>%
  mutate(
    status_binary = ifelse(status == 2, 1, 0),
    time_years = time / 365.25,
    trt_label = factor(trt, labels = c("D-penicillamine", "Placebo"))
  )

cat("N =", nrow(pbc_clean), "\n")
cat("Events (deaths):", sum(pbc_clean$status_binary), "\n")
cat("Censored:", sum(pbc_clean$status_binary == 0), "\n")

# Fit overall KM curve
km_overall <- survfit(Surv(time_years, status_binary) ~ 1, data = pbc_clean)
print(km_overall)

# Plot KM curve with confidence intervals
ggsurvplot(
  km_overall,
  data = pbc_clean,
  conf.int = TRUE,
  risk.table = TRUE,
  xlab = "Time (years)",
  ylab = "Survival probability",
  title = "Overall Survival in PBC Patients",
  ggtheme = theme_minimal()
```



```
)
```

8.2.2.2. Python

```
import pandas as pd
import numpy as np
from lifelines import KaplanMeierFitter
from lifelines.statistics import logrank_test
import matplotlib.pyplot as plt

# Load PBC data (from R via CSV or use lifelines datasets)
# We'll replicate the data preparation
from lifelines.datasets import load_rossi # placeholder - in practice, load PBC
# For demonstration, we use the PBC data exported from R or a similar dataset
# Here we simulate the structure:
import subprocess

# In practice, you would load the PBC data. We use lifelines' built-in loader:
try:
    pbc = pd.read_csv("data/pbc.csv")
except FileNotFoundError:
    # Create from R's dataset structure for demonstration
    from lifelines.datasets import load_regression_dataset
    print("Note: Using lifelines example data for demonstration.")
    print("In practice, load the PBC dataset from the survival R package.")

# Kaplan-Meier estimation
kmf = KaplanMeierFitter()

# Example with generic time-to-event data
np.random.seed(42)
n = 312
```

8. Survival Analysis: Time-to-Event Data in Medicine

```
time_years = np.random.exponential(scale=8, size=n)
event_observed = np.random.binomial(1, 0.45, size=n)

kmf.fit(time_years, event_observed=event_observed, label="PBC Patients")

print(kmf.summary.head(10))
print(f"\nMedian survival time: {kmf.median_survival_time_: .2f} years")

fig, ax = plt.subplots(figsize=(8, 5))
kmf.plot_survival_function(ax=ax, ci_show=True)
ax.set_xlabel("Time (years)")
ax.set_ylabel("Survival probability")
ax.set_title("Kaplan-Meier Survival Estimate")
plt.tight_layout()
plt.show()
```

8.2.3. Interpreting the KM Curve

Key features to examine:

1. **Median survival time:** the time at which $\hat{S}(t) = 0.5$ (where the curve crosses the 50% line). If the curve never drops to 50%, the median is undefined.
2. **Survival at specific time points:** e.g., “5-year survival is 65%.”
3. **Shape of the curve:** A steep early drop suggests high early mortality. A long flat tail suggests most survivors do well long-term.
4. **Confidence bands:** Wider bands mean more uncertainty, often because fewer patients remain at risk.
5. **Censoring marks** (tick marks on the curve): Heavy censoring near the tail means the late estimates are unreliable.

8.2.4. Confidence Intervals for the KM Estimator

The pointwise confidence interval for $\hat{S}(t)$ is typically computed via **Greenwood's formula**, which estimates the variance of $\hat{S}(t)$:

$$\widehat{\text{Var}}[\hat{S}(t)] = \hat{S}(t)^2 \sum_{t_i \leq t} \frac{d_i}{n_i(n_i - d_i)}$$

A 95% confidence interval is then $\hat{S}(t) \pm 1.96\sqrt{\widehat{\text{Var}}[\hat{S}(t)]}$. In practice, log-log transformations are preferred because they guarantee the interval stays within $[0, 1]$.

8.3. The Log-Rank Test

The **log-rank test** compares survival curves between two or more groups. It is the most widely used test for this purpose.

The null hypothesis is that survival is identical across groups: $H_0 : S_1(t) = S_2(t)$ for all t .

The test statistic compares the observed number of events in each group to the number expected under the null hypothesis, summed over all event times. It is most powerful when the **proportional hazards assumption** holds — that is, when one group has a consistently higher (or lower) hazard than the other across time.

8.3.0.1. R

8. Survival Analysis: Time-to-Event Data in Medicine

```
# Compare survival between treatment groups
km_by_trt <- survfit(Surv(time_years, status_binary) ~ trt_label, data = pbc_

# Log-rank test
logrank <- survdiff(Surv(time_years, status_binary) ~ trt_label, data = pbc_
print(logrank)
```

```
ggsurvplot(
  km_by_trt,
  data = pbc_clean,
  pval = TRUE,
  conf.int = TRUE,
  risk.table = TRUE,
  xlab = "Time (years)",
  ylab = "Survival probability",
  legend.title = "Treatment",
  palette = c("#E7B800", "#2E9FDF"),
  ggtheme = theme_minimal()
)
```

8.3.0.2. Python

```
from lifelines import KaplanMeierFitter
from lifelines.statistics import logrank_test
import numpy as np

np.random.seed(42)

# Simulate two treatment groups
n_per_group = 156
time_trt = np.random.exponential(scale=8.5, size=n_per_group)
event_trt = np.random.binomial(1, 0.42, size=n_per_group)
```

8.3. The Log-Rank Test

```
time_placebo = np.random.exponential(scale=7.5, size=n_per_group)
event_placebo = np.random.binomial(1, 0.48, size=n_per_group)

# Log-rank test
results = logrank_test(time_trt, time_placebo, event_trt, event_placebo)
print(f"Log-rank test statistic: {results.test_statistic:.3f}")
print(f"p-value: {results.p_value:.4f}")

# Plot both curves
fig, ax = plt.subplots(figsize=(8, 5))
kmf_trt = KaplanMeierFitter()
kmf_trt.fit(time_trt, event_trt, label="D-penicillamine")
kmf_trt.plot_survival_function(ax=ax)

kmf_placebo = KaplanMeierFitter()
kmf_placebo.fit(time_placebo, event_placebo, label="Placebo")
kmf_placebo.plot_survival_function(ax=ax)

ax.set_xlabel("Time (years)")
ax.set_ylabel("Survival probability")
ax.set_title("Survival by Treatment Group")
plt.tight_layout()
plt.show()
```

i When the Log-Rank Test is Weak

The log-rank test has low power when survival curves cross (e.g., one treatment is better early but worse later). In such cases, consider the **weighted log-rank test** (e.g., Fleming-Harrington) or the **restricted mean survival time (RMST)** approach.

8.4. Cox Proportional Hazards Model

The **Cox proportional hazards (PH) model** is the most widely used regression model for survival data. Introduced by Sir David Cox in 1972, it models the **hazard function** — the instantaneous rate of the event occurring at time t , given survival to that point:

$$h(t|X) = h_0(t) \exp(\beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p)$$

where:

- $h(t|X)$ is the hazard at time t for an individual with covariates X ,
- $h_0(t)$ is the **baseline hazard** (an unspecified function of time),
- $\beta_1, \dots, \beta_p$ are regression coefficients,
- $\exp(\beta_j)$ is the **hazard ratio** for a one-unit increase in X_j .

8.4.1. Key Properties

1. **Semi-parametric:** The baseline hazard $h_0(t)$ is left unspecified — the model makes no assumption about the shape of the hazard over time. Only the relative effect of covariates is estimated.
2. **Proportional hazards assumption:** The ratio of hazards for any two individuals is constant over time. If patient A has twice the hazard of patient B at time 1, they also have twice the hazard at time 5.
3. **Hazard ratios:** $HR = \exp(\beta)$. An $HR > 1$ means increased hazard (worse prognosis); $HR < 1$ means decreased hazard (better prognosis); $HR = 1$ means no effect.

8.4.2. Fitting the Cox Model

8.4.2.1. R

```
# Fit Cox model with multiple predictors
cox_model <- coxph(
  Surv(time_years, status_binary) ~ age + sex + albumin + bili + edema,
  data = pbc_clean
)

summary(cox_model)
```

```
# Forest plot of hazard ratios
ggforest(cox_model, data = pbc_clean)
```

8.4.2.2. Python

```
from lifelines import CoxPHFitter
import pandas as pd
import numpy as np

# Create a simulated clinical dataset
np.random.seed(42)
n = 312
df = pd.DataFrame({
    'time_years': np.random.exponential(scale=8, size=n),
    'event': np.random.binomial(1, 0.45, size=n),
    'age': np.random.normal(50, 10, size=n),
    'sex': np.random.binomial(1, 0.5, size=n),
    'albumin': np.random.normal(3.5, 0.5, size=n),
    'bili': np.random.exponential(scale=3, size=n),
```

8. Survival Analysis: Time-to-Event Data in Medicine

```
'edema': np.random.choice([0, 0.5, 1], size=n, p=[0.75, 0.15, 0.10])
})

cph = CoxPHFitter()
cph.fit(df, duration_col='time_years', event_col='event',
        formula='age + sex + albumin + bili + edema')

cph.print_summary()

fig, ax = plt.subplots(figsize=(8, 5))
cph.plot(ax=ax)
ax.set_title("Cox PH Model - Hazard Ratios")
plt.tight_layout()
plt.show()
```

8.4.3. Interpreting Hazard Ratios

Consider a Cox model result where bilirubin has $\hat{\beta} = 0.15$ (HR = 1.16, 95% CI: 1.10–1.23, $p < 0.001$).

Interpretation: “For each 1 mg/dL increase in serum bilirubin, the hazard of death increases by 16% (HR = 1.16, 95% CI: 1.10–1.23), adjusting for age, sex, albumin, and edema.”



Hazard Ratios Are Not Risk Ratios

A hazard ratio of 2.0 does **not** mean “twice the risk of death.” It means the instantaneous rate of death is twice as high at any given moment. Over long follow-up, the cumulative risk difference depends on the baseline hazard. Be precise in your language when reporting results.

8.5. Checking the Proportional Hazards Assumption

The proportional hazards assumption is critical. If it is violated, the hazard ratios from the Cox model do not have a stable interpretation — the “effect” of a covariate depends on when you look.

8.5.1. Schoenfeld Residuals

The most common diagnostic tool is the **Schoenfeld residual test**. Under proportional hazards, the scaled Schoenfeld residuals for a covariate should show no trend over time. A significant trend indicates violation.

8.5.1.1. R

```
# Test proportional hazards assumption
ph_test <- cox.zph(cox_model)
print(ph_test)
```

```
# Plot Schoenfeld residuals
par(mfrow = c(2, 3))
plot(ph_test)
```

8.5.1.2. Python

```
# Check proportional hazards assumption in lifelines
cph.check_assumptions(df, p_value_threshold=0.05, show_plots=True)
```

8.5.2. What To Do When PH Is Violated

If the PH assumption is violated for a covariate:

1. **Stratify** on the offending variable: `coxph(Surv(time, event) ~ x1 + strata(x2), data = df)`. This allows the baseline hazard to differ by strata of `x2` without estimating an HR for it.
2. **Include a time interaction**: Allow the effect to change over time.
3. **Use time-varying covariates** (see below).
4. **Use an alternative model**: Accelerated failure time (AFT) models or parametric models (Weibull, log-normal).

8.6. Time-Varying Covariates

In many clinical settings, covariates change over the course of follow-up. A patient's lab values, treatment status, or disease stage may evolve. The standard Cox model assumes covariates are fixed at baseline, but the model can be extended to handle **time-varying covariates**.

The idea is to split each patient's follow-up into intervals. Within each interval, the covariate values are fixed, but they can change between intervals. In R, this is accomplished using the `tmerge()` function from the `survival` package or by manually creating a "counting process" data format with start and stop times:

```
# Conceptual example - counting process format
# PatientID  start  stop  event  lab_value
#      1      0     3     0     2.1
#      1      3     7     0     3.4
#      1      7    10     1     5.8    <- event at time 10
```

This is an advanced topic. The key takeaway is that the Cox model is flexible enough to accommodate covariates that change over time, but the

data preparation is more complex and requires careful thought about when measurements were taken.

8.7. Competing Risks

Standard survival analysis assumes that the event of interest is the only possible event, or that censoring is **non-informative** (i.e., censored patients have the same future risk as those who remain under observation).

Competing risks arise when another event prevents the event of interest from being observed. Classic examples:

- Studying time to heart transplant rejection, but the patient dies of infection first.
- Studying time to cancer recurrence, but the patient dies of cardiovascular disease.
- Studying time to hospital readmission, but the patient dies before readmission.

In these situations, treating the competing event as simple censoring is **wrong** — it inflates the estimated probability of the event of interest (because death is not “non-informative” censoring; dead patients cannot experience the event).

8.7.1. Two Approaches

1. **Cause-specific hazard models:** Fit separate Cox models for each event type, censoring the competing events. This estimates the rate of each event among those still at risk.
2. **Fine-Gray subdistribution hazard model:** Directly models the cumulative incidence function, keeping patients who experienced the competing event in the risk set. This is often more appropriate for clinical prediction.

8. Survival Analysis: Time-to-Event Data in Medicine

Rule of Thumb for Competing Risks

If you are interested in **etiology** (what causes the event?), use cause-specific hazards. If you are interested in **prediction** (what is the probability of the event by time t ?), use the Fine-Gray model and report cumulative incidence functions rather than 1 minus KM.

8.8. Full Clinical Example: PBC Prognostic Model

Let us build a complete survival model for the PBC dataset, following the steps a clinical researcher would take.

8.8.0.1. R

```
# Step 1: Explore the data
pbc_model <- pbc_clean %>%
  select(time_years, status_binary, age, sex, albumin, bili,
         protime, stage, edema, trt_label) %>%
  drop_na()

cat("Complete cases:", nrow(pbc_model), "\n")
cat("Events:", sum(pbc_model$status_binary), "\n")
cat("Median follow-up:", median(pbc_model$time_years), "years\n")

# Step 2: Fit multivariable Cox model
cox_full <- coxph(
  Surv(time_years, status_binary) ~ age + sex + albumin + log(bili) +
    protime + stage + edema + trt_label,
  data = pbc_model
)
summary(cox_full)
```

8.8. Full Clinical Example: PBC Prognostic Model

```
# Step 3: Assess model discrimination
cat("Concordance (C-index):", summary(cox_full)$concordance[1], "\n")
cat("This means the model correctly ranks survival times",
    round(summary(cox_full)$concordance[1] * 100, 1),
    "% of the time.\n")
```

```
# Step 4: Predicted survival for new patients
new_patients <- data.frame(
  age = c(40, 65),
  sex = c("f", "f"),
  albumin = c(3.8, 2.5),
  bili = c(1.0, 10.0),
  protime = c(10.5, 13.0),
  stage = c(2, 4),
  edema = c(0, 1),
  trt_label = c("Placebo", "Placebo")
)

pred_surv <- survfit(cox_full, newdata = new_patients)

plot(pred_surv, col = c("blue", "red"), lwd = 2,
     xlab = "Time (years)", ylab = "Survival probability",
     main = "Predicted Survival: Low-risk vs High-risk Patient")
legend("topright", c("Low risk", "High risk"),
     col = c("blue", "red"), lwd = 2)
```

8.8.0.2. Python

```
from lifelines import CoxPHFitter, KaplanMeierFitter
import pandas as pd
```

8. Survival Analysis: Time-to-Event Data in Medicine

```
import numpy as np

np.random.seed(42)
n = 276

# Simulate a PBC-like dataset
pbc_sim = pd.DataFrame({
    'time_years': np.abs(np.random.exponential(7, n) +
                        np.random.normal(0, 1, n)),
    'event': np.random.binomial(1, 0.45, n),
    'age': np.random.normal(50, 10, n),
    'sex': np.random.binomial(1, 0.12, n), # mostly female in PBC
    'albumin': np.random.normal(3.5, 0.4, n),
    'log_bili': np.random.normal(0.5, 1.0, n),
    'protime': np.random.normal(11, 1, n),
    'stage': np.random.choice([1, 2, 3, 4], n, p=[0.05, 0.25, 0.40, 0.30]),
    'edema': np.random.choice([0, 0.5, 1], n, p=[0.75, 0.15, 0.10]),
    'treatment': np.random.binomial(1, 0.5, n)
})

# Fit Cox model
cph = CoxPHFitter()
cph.fit(pbc_sim, duration_col='time_years', event_col='event',
        formula='age + sex + albumin + log_bili + protime + stage + edema + treatment')

cph.print_summary()
print(f"\nConcordance index: {cph.concordance_index_:.3f}")

# Predict survival for specific patient profiles
low_risk = pd.DataFrame({
    'age': [40], 'sex': [0], 'albumin': [3.8], 'log_bili': [0.0],
    'protime': [10.5], 'stage': [2], 'edema': [0], 'treatment': [0]
})
```

```

high_risk = pd.DataFrame({
    'age': [65], 'sex': [0], 'albumin': [2.5], 'log_bili': [2.3],
    'protime': [13.0], 'stage': [4], 'edema': [1], 'treatment': [0]
})

fig, ax = plt.subplots(figsize=(8, 5))
cph.predict_survival_function(low_risk).plot(ax=ax, label='Low risk', color='blue')
cph.predict_survival_function(high_risk).plot(ax=ax, label='High risk', color='red')
ax.set_xlabel("Time (years)")
ax.set_ylabel("Survival probability")
ax.set_title("Predicted Survival: Low-risk vs High-risk Patient")
ax.legend()
plt.tight_layout()
plt.show()

```

8.9. Exercises

8.9.1. Exercise 1: Kaplan-Meier Curves by Disease Stage

Using the PBC dataset, create Kaplan-Meier curves stratified by disease stage (1–4). Perform a log-rank test to compare survival across stages. Report the median survival time for each stage.

8.9.1.1. R Starter Code

```

# Fit KM by stage
km_stage <- survfit(Surv(time_years, status_binary) ~ stage, data = pbc_clean)

# Your code: plot the curves and perform the log-rank test

```

8. Survival Analysis: Time-to-Event Data in Medicine

```
# Hint: use ggsurvplot() with pval = TRUE
# Hint: use survdiff() for the formal test
```

8.9.1.2. Python Starter Code

```
# Fit KM for each stage and plot
# Your code here
# Hint: loop over stages, fit KaplanMeierFitter for each
# Use logrank_test() for pairwise comparisons
```

8.9.2. Exercise 2: Build and Evaluate a Cox Model

Build a Cox proportional hazards model for PBC survival using age, bilirubin (log-transformed), albumin, prothrombin time, and edema as predictors.

1. Report the hazard ratios and 95% confidence intervals.
2. Check the proportional hazards assumption. Which covariates, if any, violate it?
3. Calculate the concordance index. How well does the model discriminate?
4. Predict the 5-year survival probability for a 55-year-old woman with bilirubin = 3 mg/dL, albumin = 3.2 g/dL, prothrombin time = 11 seconds, and no edema.

8.9.2.1. R Starter Code

```
# Your code here
# cox_model <- coxph(Surv(time_years, status_binary) ~ ..., data = pbc_clean)
```



```
# summary(cox_model)
# cox.zph(cox_model)
```

8.9.2.2. Python Starter Code

```
# Your code here
# cph = CoxPHFitter()
# cph.fit(...)
# cph.check_assumptions(...)
```

8.9.3. Exercise 3: Competing Risks Thinking

Consider a study of time to kidney transplant rejection in 500 recipients. During follow-up, some patients die before experiencing rejection (a competing risk).

1. Explain why treating death as censoring would bias the results. What direction would the bias go?
2. Which approach would you use if your goal were to estimate the probability of rejection within 2 years — cause-specific hazards or Fine-Gray? Why?
3. (Bonus) In R, use the `cmprsk` or `tidycmprsk` package to fit a Fine-Gray model. In Python, explore `lifelines` or `scikit-survival` for competing risk methods.

8.10. Summary

8. Survival Analysis: Time-to-Event Data in Medicine

Concept	What It Does	Key Assumption
Kaplan-Meier	Non-parametric survival estimate	Non-informative censoring
Log-rank test	Compares survival curves	Proportional hazards (most powerful)
Cox PH model	Regression for survival data	Proportional hazards, non-informative censoring
Fine-Gray model	Competing risks regression	Models subdistribution hazard

Survival analysis is fundamental to clinical research. The methods in this chapter — Kaplan-Meier estimation, log-rank testing, and Cox regression — form the backbone of nearly every clinical trial and prognostic study. The proportional hazards assumption must always be checked, and competing risks should be considered whenever another event can prevent observation of the event of interest.

8.11. References and Further Reading

The foundational paper on proportional hazards regression is Cox (1972), “Regression Models and Life-Tables,” published in the *Journal of the Royal Statistical Society, Series B*, which introduced the semi-parametric model that now bears his name. This paper is one of the most cited in all of statistics.

For a comprehensive and accessible introduction to survival analysis in the context of this course, see Smits et al. (2026), which covers Kaplan-Meier estimation, the Cox model, and competing risks with modern clinical examples. Their treatment of model diagnostics and the proportional hazards assumption is particularly practical.

8.11. References and Further Reading

Bradburn et al. provide an excellent tutorial series on survival analysis for medical researchers, published in the *British Journal of Cancer*. Their series covers Kaplan-Meier methods, the Cox model, and advanced topics in a format accessible to clinicians without extensive statistical training.

For hands-on implementation, the `survival` and `survminer` packages in R are the standard tools. Therneau and Grambsch's *Modeling Survival Data: Extending the Cox Model* (Springer, 2000) is the definitive reference for the R `survival` package. In Python, the `lifelines` library by Cameron Davidson-Pilon provides an excellent API for survival analysis, and its online documentation includes worked clinical examples.

For competing risks specifically, Fine and Gray (1999), "A Proportional Hazards Model for the Subdistribution of a Competing Risk," in the *Journal of the American Statistical Association*, is the key methodological reference. Austin and Fine (2017) provide practical guidance on when and how to use competing risk methods in clinical research.

Part III.

Bayesian Methods

9. Bayesian Inference: A Different Way to Think About Evidence

9.1. Introduction

You have now completed the core statistical methods that form the backbone of most clinical research: regression, splines, penalised regression, and survival analysis. All of these methods share a common philosophical framework — they are **frequentist**. In this part of the course, we step back to explore a fundamentally different way of thinking about evidence.

Every statistical method you have learned so far in this course has been **frequentist**. In frequentist statistics, probability refers to long-run frequencies: “if we repeated this experiment infinitely many times, the parameter would fall in our 95% confidence interval 95% of the time.” That is a mathematically precise statement, but it is not the question most clinicians actually want answered. What clinicians want to know is: *given the data I have in front of me, what is the probability that this treatment works?*

Bayesian inference answers that question directly. It treats probability as a measure of **uncertainty** — a degree of belief — rather than a long-run frequency. This chapter introduces the Bayesian framework from first principles, shows you how it works with a medical example you already understand (diagnostic testing), and then builds toward the computational machinery (MCMC) that makes modern Bayesian analysis possible.

9. Bayesian Inference: A Different Way to Think About Evidence

i Why now?

Bayesian methods are increasingly required in clinical research. The US FDA released updated guidance in 2026 on the use of Bayesian statistics in medical device trials, and Bayesian adaptive trial designs are now standard in oncology. Understanding this framework is no longer optional for quantitative researchers in medicine and public health.

i Notation used in this chapter

- $P(A | B)$ means “the probability of A, given that B is true”
- θ (theta) is the parameter we want to estimate (e.g., a treatment effect)
- D stands for “data” — the observations we have collected
- $\propto$ means “is proportional to”

9.2. Two Philosophies of Probability

9.2.1. The Frequentist View

In the frequentist framework:

- **Parameters are fixed** but unknown constants.
- **Data are random** — they vary from sample to sample.
- **Probability** describes the long-run frequency of events across hypothetical replications.
- A 95% confidence interval means: if we repeated this study infinitely, 95% of the resulting intervals would contain the true parameter.

The frequentist approach does **not** assign a probability to the parameter itself. You cannot say “there is a 95% probability that the true effect lies in

this interval” — even though nearly everyone, including many researchers, interprets confidence intervals that way.

9.2.2. The Bayesian View

In the Bayesian framework:

- **Parameters are uncertain** and described by probability distributions.
- **Data are fixed** — once observed, they do not change.
- **Probability** quantifies our degree of belief or uncertainty about unknown quantities.
- A 95% credible interval means exactly what it sounds like: there is a 95% probability that the parameter lies in this interval, given the data and the model.

Table 9.1.: Comparison of frequentist and Bayesian frameworks

Feature	Frequentist	Bayesian
What is probability?	Long-run frequency	Degree of belief
Parameters	Fixed, unknown	Random variables with distributions
Data	Random (varies across replications)	Fixed (observed once)
Inference	P-values, confidence intervals	Posterior distributions, credible intervals
Prior information	Not formally incorporated	Formally incorporated via priors

9.3. Bayes' Theorem: You Already Know This

Bayes' theorem is the mathematical engine of Bayesian inference. You have seen it before in epidemiology, even if it was not called by that name.

$$P(\theta | D) = \frac{P(D | \theta) P(\theta)}{P(D)}$$

Where:

- $P(\theta | D)$ is the **posterior** — our updated belief about the parameter θ after seeing data D .
- $P(D | \theta)$ is the **likelihood** — how probable the observed data are under a particular value of θ .
- $P(\theta)$ is the **prior** — our belief about θ before seeing the data.
- $P(D)$ is the **marginal likelihood** (or evidence) — a normalising constant that ensures the posterior integrates to 1.

In words: **Posterior** $\propto$ **Likelihood** $\times$ **Prior**.

9.3.1. A Medical Example: COVID-19 Rapid Testing

Suppose a new rapid antigen test for COVID-19 has:

- **Sensitivity** = 85% (probability of a positive test given truly infected)
- **Specificity** = 98% (probability of a negative test given truly uninfected)

A patient in your emergency department tests positive. What is the probability they actually have COVID-19?

This depends on the **prevalence** (the prior probability of disease). Let us work through two scenarios.

Scenario 1: High prevalence (20% — during a winter surge)

9.4. Prior Distributions

$$\begin{aligned} P(\text{COVID} \mid +) &= \frac{P(+ \mid \text{COVID}) \times P(\text{COVID})}{P(+)} \\ &= \frac{0.85 \times 0.20}{(0.85 \times 0.20) + (0.02 \times 0.80)} = \frac{0.17}{0.17 + 0.016} = \frac{0.17}{0.186} \approx 0.914 \end{aligned}$$

A positive test in a high-prevalence setting gives a 91.4% posterior probability of disease.

Scenario 2: Low prevalence (1% — summer, low community transmission)

$$P(\text{COVID} \mid +) = \frac{0.85 \times 0.01}{(0.85 \times 0.01) + (0.02 \times 0.99)} = \frac{0.0085}{0.0085 + 0.0198} = \frac{0.0085}{0.0283} \approx 0.300$$

The same positive test now gives only a 30% posterior probability. The **prior matters**.

This is Bayesian reasoning in its simplest form: your conclusion depends on what you knew before you saw the evidence. Every Bayesian analysis extends this logic to continuous parameters and complex models.

9.4. Prior Distributions

The prior distribution encodes what we know (or do not know) about a parameter before collecting data. Choosing an appropriate prior is one of the most important — and most debated — steps in Bayesian analysis.

9.4.1. Types of Priors

Uninformative (flat) priors assign equal probability to all values. For example, a uniform distribution over $(-\infty, +\infty)$ for a regression coefficient. These are sometimes called “objective” priors because they appear to let the data speak for themselves.

- In practice, truly flat priors are improper (they do not integrate to 1) and can cause computational problems.
- They are rarely truly uninformative — a flat prior on a log-odds scale is not flat on the probability scale.

Weakly informative priors gently constrain parameters to reasonable ranges without being strongly opinionated. For example, a $\text{Normal}(0, 10)$ prior on a log-odds regression coefficient says: “I think the coefficient is probably somewhere between -20 and $+20$ on the log-odds scale, but I am not very sure.”

- These are the **recommended default** for most applied work.
- They prevent the model from exploring absurd parameter values while remaining flexible.
- Andrew Gelman’s recommendation: use priors that rule out implausible values but remain vague within the plausible range.

Informative priors incorporate genuine prior knowledge. For example, if previous meta-analyses show that a drug reduces mortality by 10–30%, you might use a prior centred on that range.

- These are especially valuable in small-sample settings (paediatric trials, rare diseases).
- They must be justified and subjected to sensitivity analysis.

9.4.2. The Beta Distribution as a Conjugate Prior

When estimating a proportion (e.g., a treatment response rate), the Beta distribution is a natural prior because it is **conjugate** to the binomial likelihood. This means the posterior is also a Beta distribution, which makes calculations simple.

The Beta distribution is parameterised by two shape parameters, α and β :

- Beta(1, 1) — uniform on $[0, 1]$, an uninformative prior.
- Beta(2, 2) — a gentle hump centred at 0.5, weakly informative.
- Beta(10, 30) — centred at 0.25, moderately informative (e.g., prior belief that the response rate is around 25%).

If the prior is Beta(α_0, β_0) and we observe y successes in n trials, the posterior is:

$$\theta \mid y \sim \text{Beta}(\alpha_0 + y, \beta_0 + n - y)$$

9.4.3. R

```
library(tidyverse)

# Observed data: 8 successes out of 20 trials
y <- 8; n <- 20

# Define three priors
priors <- tibble(
  prior_name = c("Uninformative: Beta(1,1)",
                 "Weakly informative: Beta(2,2)",
                 "Informative: Beta(10,30)"),
  alpha0 = c(1, 2, 10),
```

9. Bayesian Inference: A Different Way to Think About Evidence

```
beta0 = c(1, 2, 30)
)

theta <- seq(0, 1, length.out = 500)

# Build plotting data
plot_data <- priors %>%
  rowwise() %>%
  reframe(
    prior_name = prior_name,
    theta = theta,
    Prior = dbeta(theta, alpha0, beta0),
    Posterior = dbeta(theta, alpha0 + y, beta0 + n - y)
  ) %>%
  pivot_longer(cols = c(Prior, Posterior),
    names_to = "Distribution", values_to = "Density")

ggplot(plot_data, aes(x = theta, y = Density, colour = Distribution,
  linetype = Distribution)) +
  geom_line(linewidth = 1) +
  facet_wrap(~ prior_name, scales = "free_y") +
  scale_colour_manual(values = c("Prior" = "steelblue", "Posterior" = "firebr
  labs(x = expression(theta ~ "(response rate)"),
    y = "Density",
    title = "Effect of Prior Choice on the Posterior",
    subtitle = "Data: 8/20 patients responded") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
```

9.4.4. Python

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

y, n = 8, 20
theta = np.linspace(0, 1, 500)

priors = [
    ("Uninformative: Beta(1,1)", 1, 1),
    ("Weakly informative: Beta(2,2)", 2, 2),
    ("Informative: Beta(10,30)", 10, 30),
]

fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=False)

for ax, (name, a0, b0) in zip(axes, priors):
    prior_density = beta.pdf(theta, a0, b0)
    post_density = beta.pdf(theta, a0 + y, b0 + n - y)
    ax.plot(theta, prior_density, label="Prior", color="steelblue", linewidth=1.5)
    ax.plot(theta, post_density, label="Posterior", color="firebrick", linewidth=1.5)
    ax.set_title(name, fontsize=10)
    ax.set_xlabel(r"$\theta$ (response rate)")
    ax.set_ylabel("Density")
    ax.legend(fontsize=9)

fig.suptitle("Effect of Prior Choice on the Posterior\nData: 8/20 patients responded",
             fontsize=12, y=1.04)
plt.tight_layout()
plt.show()

```

Notice how with enough data, the posteriors converge regardless of the

9. Bayesian Inference: A Different Way to Think About Evidence

prior. This is a key property: **the data overwhelm the prior** as the sample size grows. With small samples, however, the prior has real influence — which is precisely why choosing it carefully matters.

9.5. The Likelihood

The likelihood function $P(D | \theta)$ describes how probable the observed data are for each possible value of the parameter. It is the same quantity that appears in maximum likelihood estimation (MLE), but in the Bayesian framework it is multiplied by the prior rather than maximised alone.

For our binomial example (8 responses in 20 patients):

$$L(\theta) = \binom{20}{8} \theta^8 (1 - \theta)^{12}$$

The likelihood peaks at $\hat{\theta}_{\text{MLE}} = 8/20 = 0.40$. The Bayesian posterior will be centred near this value but pulled slightly toward the prior — a phenomenon called **shrinkage**.

9.6. The Posterior Distribution

The posterior is the complete answer to a Bayesian analysis. It is not a single number; it is a full probability distribution over the parameter. From the posterior you can extract:

- **Point estimates:** the posterior mean, median, or mode.
- **Credible intervals:** ranges containing the parameter with a specified probability.
- **Probabilities of hypotheses:** e.g., $P(\theta > 0.3 | \text{data})$.

9.6.1. Credible Intervals vs Confidence Intervals

This distinction is critical and frequently confused.

A 95% frequentist confidence interval means: if the experiment were repeated many times, 95% of the computed intervals would contain the true parameter. It does **not** mean there is a 95% probability the true parameter is in *this* interval.

A 95% Bayesian credible interval means: given the data and the model, there is a 95% probability that the parameter lies in this interval.

The Bayesian credible interval says what most people *think* a confidence interval says. This interpretability is one of the major practical advantages of Bayesian inference.

9.6.2. R

```
# Posterior from weakly informative prior Beta(2,2) + data (8/20)
alpha_post <- 2 + 8
beta_post  <- 2 + 12

# 95% credible interval (equal-tailed)
ci_95 <- qbeta(c(0.025, 0.975), alpha_post, beta_post)
cat("Posterior mean:", alpha_post / (alpha_post + beta_post), "\n")
cat("95% credible interval:", round(ci_95, 3), "\n")

# Probability that response rate exceeds 30%
prob_gt_30 <- 1 - pbeta(0.30, alpha_post, beta_post)
cat("P(theta > 0.30 | data):", round(prob_gt_30, 3), "\n")
```

9. Bayesian Inference: A Different Way to Think About Evidence

9.6.3. Python

```
from scipy.stats import beta

alpha_post = 2 + 8
beta_post = 2 + 12

ci_95 = beta.ppf([0.025, 0.975], alpha_post, beta_post)
print(f"Posterior mean: {alpha_post / (alpha_post + beta_post):.3f}")
print(f"95% credible interval: [{ci_95[0]:.3f}, {ci_95[1]:.3f}]")

prob_gt_30 = 1 - beta.cdf(0.30, alpha_post, beta_post)
print(f"P(theta > 0.30 | data): {prob_gt_30:.3f}")
```

9.7. Markov Chain Monte Carlo (MCMC)

9.7.1. Why We Need It

The Beta-Binomial example above was analytically tractable because the Beta prior is conjugate to the binomial likelihood. In real research, models have many parameters and non-conjugate priors. The posterior cannot be written in closed form.

MCMC is a family of algorithms that **draw samples from the posterior distribution** even when we cannot compute it analytically. The key idea: rather than calculating $P(\theta \mid D)$ everywhere, we construct a random walk through parameter space that, over many steps, visits regions in proportion to their posterior probability.

The most widely used MCMC algorithm today is the **No-U-Turn Sampler (NUTS)**, a variant of Hamiltonian Monte Carlo (HMC). It is the default in Stan, brms, PyMC, and most modern Bayesian software.

9.7.2. How It Works Conceptually

1. Start at some initial parameter value θ_0 .
2. Propose a new value θ^* based on the current position.
3. Evaluate the posterior density at θ^* relative to θ_0 .
4. Accept or reject the proposal based on a probability rule.
5. Repeat thousands of times.

The resulting **chain** of θ values is a sample from the posterior distribution. We typically run multiple chains (e.g., 4) and discard the initial “warm-up” period.

9.7.3. Diagnosing Convergence

MCMC is only useful if the chains have converged to the target posterior. Three key diagnostics:

Trace plots show the sampled values over iterations. A well-behaved trace plot looks like a “hairy caterpillar” — the chain rapidly explores the parameter space with no trends or drifts. If you see long excursions, stuck chains, or clear trends, the chain has not converged.

$\hat{R}$ (**R-hat**) compares the variance within each chain to the variance between chains. If all chains are sampling from the same distribution, $\hat{R} \approx 1.0$. The standard threshold is $\hat{R} < 1.01$. Values above 1.05 indicate serious convergence problems.

Effective sample size (ESS) estimates how many independent samples the chains are equivalent to. Autocorrelation in MCMC means that consecutive draws are correlated, so 4,000 MCMC draws might contain only the equivalent of 1,000 independent draws. For reliable posterior summaries, aim for $\text{ESS} > 400$ per chain for bulk statistics and $\text{ESS} > 400$ for tail quantiles.

9.7.4. R

```
set.seed(42)

# Simulate a well-behaved chain (approximately normal draws)
good_chain <- cumsum(rnorm(2000, 0, 0.1)) * 0
good_chain[1] <- rnorm(1, 5, 1)
for (i in 2:2000) {
  good_chain[i] <- good_chain[i-1] + rnorm(1, 0, 0.3)
  # Metropolis-like: pull toward mean of 5
  good_chain[i] <- good_chain[i] + 0.1 * (5 - good_chain[i])
}

# Simulate a poorly-mixing chain (high autocorrelation)
bad_chain <- numeric(2000)
bad_chain[1] <- 2
for (i in 2:2000) {
  bad_chain[i] <- bad_chain[i-1] + rnorm(1, 0, 0.02)
}

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(good_chain, type = "l", col = "steelblue",
     main = "Good mixing", xlab = "Iteration", ylab = expression(theta),
     cex.main = 1.1)
plot(bad_chain, type = "l", col = "firebrick",
     main = "Poor mixing", xlab = "Iteration", ylab = expression(theta),
     cex.main = 1.1)
```

9.7.5. Python

```
import numpy as np
import matplotlib.pyplot as plt
```

9.7. Markov Chain Monte Carlo (MCMC)

```
np.random.seed(42)

# Good chain: rapidly mixing
good_chain = np.zeros(2000)
good_chain[0] = np.random.normal(5, 1)
for i in range(1, 2000):
    good_chain[i] = good_chain[i-1] + np.random.normal(0, 0.3)
    good_chain[i] += 0.1 * (5 - good_chain[i])

# Bad chain: high autocorrelation, slow drift
bad_chain = np.zeros(2000)
bad_chain[0] = 2.0
for i in range(1, 2000):
    bad_chain[i] = bad_chain[i-1] + np.random.normal(0, 0.02)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(good_chain, color="steelblue", linewidth=0.5)
axes[0].set_title("Good mixing")
axes[0].set_xlabel("Iteration")
axes[0].set_ylabel(r"$\theta$")

axes[1].plot(bad_chain, color="firebrick", linewidth=0.5)
axes[1].set_title("Poor mixing")
axes[1].set_xlabel("Iteration")
axes[1].set_ylabel(r"$\theta$")

plt.tight_layout()
plt.show()
```

9. Bayesian Inference: A Different Way to Think About Evidence

9.7.6. A first taste of Bayesian software

You do not need to write MCMC samplers yourself. Modern software handles this entirely. Here is what fitting a Bayesian logistic regression looks like in practice:

9.8. R

```
library(rstanarm)
fit <- stan_glm(chd_10yr ~ age + sex + smoking,
               data = framingham,
               family = binomial(),
               prior = normal(0, 2.5), # weakly informative
               chains = 4, iter = 2000, seed = 42)
summary(fit)
```

9.9. Python

```
import bambi as bmb
model = bmb.Model("chd_10yr ~ age + sex + smoking",
                  data=framingham, family="bernoulli")
results = model.fit(draws=2000, chains=4, random_seed=42)
results.summary()
```

The next chapter covers this in full detail, including how to check your model and interpret the output.

9.10. When Bayesian Methods Add Value

Bayesian inference is not always necessary, but there are situations where it provides genuine advantages over frequentist methods:

Small samples. When data are scarce (rare diseases, paediatric populations, early-phase trials), informative priors allow you to borrow strength from previous studies. A frequentist analysis of 12 patients yields wide confidence intervals and little power; a Bayesian analysis that incorporates prior evidence can produce more precise and clinically useful estimates.

Prior evidence is available. If there are five previous randomised trials of a drug, it would be wasteful to ignore that evidence when analysing a sixth. Bayesian methods provide a principled framework for incorporating it.

Rare events. When estimating the probability of a rare adverse event (e.g., 0 events in 200 patients), the frequentist MLE is zero — clearly an underestimate. A Bayesian analysis with an appropriate prior yields a plausible non-zero estimate.

Sequential updating. In adaptive clinical trials, data are analysed repeatedly as they accumulate. Bayesian methods handle this naturally: today’s posterior becomes tomorrow’s prior. Frequentist methods require complex corrections for multiple looks at the data.

Direct probability statements. Clinicians want to know “what is the probability that this treatment is effective?” Bayesian methods answer that question directly: $P(\text{treatment effect} > 0 \mid \text{data})$.

! Bayesian methods are not a magic fix

Bayesian inference does not rescue a poorly designed study, does not eliminate bias, and does not make small samples equivalent to large ones. The prior helps, but it also introduces subjectivity.

9. Bayesian Inference: A Different Way to Think About Evidence

Sensitivity analyses showing how results change under different priors are essential.

9.11. Practical Example: Estimating a Surgical Complication Rate

A hospital performs a new minimally invasive cardiac procedure. In the first 15 cases, 2 patients experience a major complication. The national average for the traditional open procedure is approximately 12%. What can we say about this hospital's complication rate?

9.11.1. R

```
# Data
y <- 2 # complications
n <- 15 # procedures

# Prior: weakly informative, centred near the national average of 12%
# Beta(3, 22) has mean 3/25 = 0.12 and is moderately concentrated
alpha_prior <- 3
beta_prior <- 22

# Posterior
alpha_post <- alpha_prior + y
beta_post <- beta_prior + n - y

theta <- seq(0, 0.5, length.out = 500)

plot_df <- tibble(
  theta = theta,
```


9.11. Practical Example: Estimating a Surgical Complication Rate

```
Prior      = dbeta(theta, alpha_prior, beta_prior),
Likelihood = dbeta(theta, y + 1, n - y + 1), # normalised for visualisation
Posterior  = dbeta(theta, alpha_post, beta_post)
) %>%
  pivot_longer(-theta, names_to = "Component", values_to = "Density")

ggplot(plot_df, aes(x = theta, y = Density, colour = Component)) +
  geom_line(linewidth = 1.2) +
  scale_colour_manual(values = c("Prior" = "steelblue",
                                "Likelihood" = "grey50",
                                "Posterior" = "firebrick")) +
  labs(x = "Complication rate",
       y = "Density",
       title = "Bayesian Estimation of Surgical Complication Rate",
       subtitle = "2 complications in 15 procedures; Prior: Beta(3, 22)") +
  geom_vline(xintercept = 0.12, linetype = "dashed", colour = "grey40") +
  annotate("text", x = 0.135, y = max(dbeta(theta, alpha_post, beta_post)) * 0.9,
         label = "National avg = 12%", size = 3.5, hjust = 0) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")

# Posterior summaries
cat("Posterior mean:", round(alpha_post / (alpha_post + beta_post), 3), "\n")
cat("95% credible interval:",
    round(qbeta(c(0.025, 0.975), alpha_post, beta_post), 3), "\n")
cat("P(rate > 0.20 | data):",
    round(1 - pbeta(0.20, alpha_post, beta_post), 3), "\n")
```

9.11.2. Python

```
import numpy as np
import matplotlib.pyplot as plt
```

9. Bayesian Inference: A Different Way to Think About Evidence

```
from scipy.stats import beta

y, n = 2, 15
alpha_prior, beta_prior = 3, 22
alpha_post = alpha_prior + y
beta_post_ = beta_prior + n - y

theta = np.linspace(0, 0.5, 500)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(theta, beta.pdf(theta, alpha_prior, beta_prior),
        label="Prior", color="steelblue", linewidth=1.5)
ax.plot(theta, beta.pdf(theta, y + 1, n - y + 1),
        label="Likelihood (normalised)", color="grey", linewidth=1.5)
ax.plot(theta, beta.pdf(theta, alpha_post, beta_post_),
        label="Posterior", color="firebrick", linewidth=1.5)
ax.axvline(0.12, linestyle="--", color="grey", linewidth=1)
ax.text(0.135, ax.get_ylim()[1] * 0.5, "National avg = 12%", fontsize=9)
ax.set_xlabel("Complication rate")
ax.set_ylabel("Density")
ax.set_title("Bayesian Estimation of Surgical Complication Rate\n"
            "2 complications in 15 procedures; Prior: Beta(3, 22)")
ax.legend()
plt.tight_layout()
plt.show()

ci = beta.ppf([0.025, 0.975], alpha_post, beta_post_)
print(f"Posterior mean: {alpha_post / (alpha_post + beta_post_) : .3f}")
print(f"95% credible interval: [{ci[0] : .3f}, {ci[1] : .3f}]")
print(f"P(rate > 0.20 | data): {1 - beta.cdf(0.20, alpha_post, beta_post_) : .3f}")
```

The posterior mean is pulled between the MLE ($2/15 = 0.133$) and the prior mean (0.12), settling near 0.125. With only 15 observations, the prior

meaningfully influences the result — and that is appropriate, because we have genuine prior information about surgical complication rates.

9.12. Exercises

9.12.1. Exercise 1: Diagnostic Test Updating

A screening mammogram has a sensitivity of 90% and a specificity of 92%. Among women aged 50–59, the prevalence of breast cancer is approximately 2%.

- Calculate the posterior probability of breast cancer given a positive mammogram using Bayes' theorem.
- If the patient then receives a confirmatory ultrasound (sensitivity 95%, specificity 85%), use the posterior from (a) as the new prior and calculate the updated posterior probability after a positive ultrasound.
- What does this sequential updating illustrate about the Bayesian framework?

9.12.2. R

```
# Part (a): Mammogram
sens_mammo <- 0.90
spec_mammo <- 0.92
prevalence <- 0.02

post_mammo <- (sens_mammo * prevalence) /
  (sens_mammo * prevalence + (1 - spec_mammo) * (1 - prevalence))
cat("P(cancer | positive mammogram):", round(post_mammo, 4), "\n")

# Part (b): Ultrasound, using posterior from (a) as new prior
```

9. Bayesian Inference: A Different Way to Think About Evidence

```
sens_us <- 0.95
spec_us <- 0.85
prior_us <- post_mammo

post_us <- (sens_us * prior_us) /
  (sens_us * prior_us + (1 - spec_us) * (1 - prior_us))
cat("P(cancer | positive mammogram AND positive ultrasound):",
    round(post_us, 4), "\n")
```

9.12.3. Python

```
# Part (a)
sens_mammo, spec_mammo, prevalence = 0.90, 0.92, 0.02
post_mammo = (sens_mammo * prevalence) / \
  (sens_mammo * prevalence + (1 - spec_mammo) * (1 - prevalence))
print(f"P(cancer | positive mammogram): {post_mammo:.4f}")

# Part (b)
sens_us, spec_us = 0.95, 0.85
post_us = (sens_us * post_mammo) / \
  (sens_us * post_mammo + (1 - spec_us) * (1 - post_mammo))
print(f"P(cancer | pos mammo AND pos ultrasound): {post_us:.4f}")
```

9.12.4. Exercise 2: Prior Sensitivity Analysis

A Phase II trial of a new antibiotic enrolls 30 patients with resistant urinary tract infections. Twenty-one patients achieve clinical cure.

- Using three different priors — $\text{Beta}(1,1)$, $\text{Beta}(5,5)$, and $\text{Beta}(20,10)$ — compute the posterior mean and 95% credible interval for the cure rate.

- b. Plot all three posteriors on the same graph.
- c. Which prior has the most influence on the posterior? Why?

9.12.5. R

```

y <- 21; n <- 30
priors <- list(
  list(name = "Beta(1,1)", a = 1, b = 1),
  list(name = "Beta(5,5)", a = 5, b = 5),
  list(name = "Beta(20,10)", a = 20, b = 10)
)

theta <- seq(0, 1, length.out = 500)
plot_df <- tibble()

for (p in priors) {
  a_post <- p$a + y
  b_post <- p$b + n - y
  ci <- qbeta(c(0.025, 0.975), a_post, b_post)
  cat(sprintf("%s -> Posterior mean: %.3f, 95%% CI: [%.3f, %.3f]\n",
    p$name, a_post / (a_post + b_post), ci[1], ci[2]))
  plot_df <- bind_rows(plot_df,
    tibble(theta = theta, Density = dbeta(theta, a_post, b_post),
      Prior = p$name))
}

ggplot(plot_df, aes(x = theta, y = Density, colour = Prior)) +
  geom_line(linewidth = 1.2) +
  labs(x = "Cure rate", y = "Density",
    title = "Posterior Distributions Under Different Priors",
    subtitle = "21/30 patients cured") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")

```

9.12.6. Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta

y, n = 21, 30
priors = [("Beta(1,1)", 1, 1), ("Beta(5,5)", 5, 5), ("Beta(20,10)", 20, 10)]
theta = np.linspace(0, 1, 500)

fig, ax = plt.subplots(figsize=(8, 5))
for name, a0, b0 in priors:
    a_post, b_post = a0 + y, b0 + n - y
    ci = beta.ppf([0.025, 0.975], a_post, b_post)
    mean_post = a_post / (a_post + b_post)
    print(f"{name} -> Mean: {mean_post:.3f}, 95% CI: [{ci[0]:.3f}, {ci[1]:.3f}]")
    ax.plot(theta, beta.pdf(theta, a_post, b_post), label=name, linewidth=1.5)

ax.set_xlabel("Cure rate")
ax.set_ylabel("Density")
ax.set_title("Posterior Distributions Under Different Priors\n21/30 patients")
ax.legend()
plt.tight_layout()
plt.show()
```

9.12.7. Exercise 3: Interpreting MCMC Diagnostics

You fit a Bayesian logistic regression model and obtain the following diagnostics for the treatment effect parameter:

9.13. Summary

- Chain 1 R-hat: 1.08
 - Chain 2 R-hat: 1.08
 - Bulk ESS: 150
 - Tail ESS: 85
 - The trace plot shows two chains that appear to be exploring different regions of the parameter space.
- a. Is there evidence of convergence problems? Explain which diagnostics are concerning and why.
 - b. What steps would you take to fix these problems?
 - c. Would you trust the posterior summaries from this model? Why or why not?

9.13. Summary

Bayesian inference provides a coherent framework for updating beliefs in light of evidence. Its core components — the prior, the likelihood, and the posterior — are connected by Bayes' theorem. While simple conjugate models (like Beta-Binomial) can be solved analytically, most real-world models require MCMC sampling. The key practical advantages of Bayesian methods include direct probability statements, principled incorporation of prior knowledge, and natural handling of sequential data — all of which are particularly valuable in clinical research.

In the next chapter, we will move from theory to practice: fitting Bayesian regression and hierarchical models using modern software.



Key Takeaways

1. Bayesian inference treats parameters as uncertain and data as fixed — the reverse of frequentist thinking.
2. The posterior is proportional to the likelihood times the prior.
3. Credible intervals have the intuitive interpretation people mis-

9. Bayesian Inference: A Different Way to Think About Evidence

takenly attribute to confidence intervals.

4. MCMC allows us to sample from complex posteriors; always check convergence with trace plots, R-hat, and ESS.
5. Bayesian methods shine when samples are small, prior evidence exists, or direct probability statements are needed.

9.14. References and Further Reading

The foundational textbook for applied Bayesian analysis is McElreath's *Statistical Rethinking* (2nd edition, 2020), which teaches Bayesian inference through simulation and coding rather than mathematical derivation. It is highly recommended for students coming from a non-statistical background. For a more comprehensive and mathematically rigorous treatment, Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin's *Bayesian Data Analysis* (3rd edition, 2013), commonly known as BDA3, remains the standard reference; a free PDF is available from the authors' website. Kruschke's *Doing Bayesian Data Analysis* (2nd edition, 2015) offers a gentle introduction with extensive worked examples in R.

For the clinical and regulatory context, the FDA's 2010 guidance document *Guidance for the Use of Bayesian Statistics in Medical Device Clinical Trials* provides a framework for incorporating Bayesian methods in regulatory submissions, and updated guidance released in 2026 reflects the growing acceptance of Bayesian adaptive designs. Spiegelhalter, Abrams, and Myles' *Bayesian Approaches to Clinical Trials and Health-Care Evaluation* (2004) is particularly relevant for public health students, covering prior elicitation, adaptive designs, and cost-effectiveness analysis. The Stan Development Team's documentation (mc-stan.org) includes excellent case studies and practical tutorials that complement any of the above textbooks.

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications

10.1. Introduction

The previous chapter introduced the conceptual foundations of Bayesian inference: priors, likelihoods, posteriors, and MCMC. This chapter puts those ideas to work. We will fit Bayesian regression models, learn the critical model-checking steps that distinguish careful Bayesian analysis from careless button-pressing, and then tackle the most powerful application of the Bayesian framework in clinical research: **hierarchical (multilevel) models**.

Hierarchical models deserve special attention because they solve a problem that arises constantly in medicine. Clinical data are almost always grouped: patients within hospitals, measurements within patients, sites within multi-centre trials. Frequentist mixed-effects models handle this, but the Bayesian hierarchical framework does it more flexibly and with clearer interpretation. By the end of this chapter, you will understand partial pooling, shrinkage, and why these ideas matter for clinical decision-making.

10.2. Bayesian Linear Regression

10.2.1. How It Differs from Frequentist Regression

In frequentist ordinary least squares (OLS), we obtain point estimates of regression coefficients and standard errors. A 95% confidence interval is a statement about long-run coverage.

In Bayesian linear regression, each coefficient has a **full posterior distribution**. We obtain not just an estimate and an interval, but the entire shape of our uncertainty. This is the same linear model:

$$y_i = \beta_0 + \beta_1 x_{1i} + \dots + \beta_p x_{pi} + \epsilon_i, \quad \epsilon_i \sim \text{Normal}(0, \sigma^2)$$

But now we place priors on all parameters: $\beta_0, \beta_1, \dots, \beta_p, \sigma$. The MCMC sampler then generates draws from the joint posterior $P(\beta_0, \beta_1, \dots, \beta_p, \sigma \mid \mathbf{y})$.

10.2.2. Clinical Example: Predicting Systolic Blood Pressure

We will model systolic blood pressure (SBP) as a function of age, BMI, and whether the patient is on antihypertensive medication.

10.2.3. R

```
library(brms)

# Simulate clinical data
set.seed(123)
n <- 200
bp_data <- tibble(
```

10.2. Bayesian Linear Regression

```
age = round(rnorm(n, 55, 12)),
bmi = round(rnorm(n, 28, 5), 1),
on_meds = rbinom(n, 1, 0.4),
sbp = round(90 + 0.5 * age + 0.8 * bmi - 8 * on_meds + rnorm(n, 0, 12))
)

# Fit Bayesian linear regression with weakly informative priors
fit_bp <- brm(
  sbp ~ age + bmi + on_meds,
  data = bp_data,
  family = gaussian(),
  prior = c(
    prior(normal(0, 20), class = "b"),      # weakly informative for slopes
    prior(normal(120, 30), class = "Intercept"), # centred on typical SBP
    prior(exponential(0.1), class = "sigma")  # positive scale parameter
  ),
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = 42,
  silent = 2,
  refresh = 0
)

summary(fit_bp)
```

```
library(bayesplot)
mcmc_areas(fit_bp, pars = c("b_age", "b_bmi", "b_on_meds"),
  prob = 0.95, prob_outer = 0.99) +
  labs(title = "Posterior Distributions of Regression Coefficients",
    subtitle = "Shaded region = 95% credible interval") +
  theme_minimal(base_size = 13)
```

10.2.4. Python

```

import numpy as np
import pandas as pd
import pymc as pm
import arviz as az

np.random.seed(123)
n = 200
bp_data = pd.DataFrame({
    'age': np.round(np.random.normal(55, 12, n)),
    'bmi': np.round(np.random.normal(28, 5, n), 1),
    'on_meds': np.random.binomial(1, 0.4, n)
})
bp_data['sbp'] = np.round(
    90 + 0.5 * bp_data['age'] + 0.8 * bp_data['bmi']
    - 8 * bp_data['on_meds'] + np.random.normal(0, 12, n)
)

with pm.Model() as bp_model:
    # Priors
    intercept = pm.Normal("Intercept", mu=120, sigma=30)
    b_age = pm.Normal("b_age", mu=0, sigma=20)
    b_bmi = pm.Normal("b_bmi", mu=0, sigma=20)
    b_meds = pm.Normal("b_on_meds", mu=0, sigma=20)
    sigma = pm.Exponential("sigma", lam=0.1)

    # Linear model
    mu = intercept + b_age * bp_data['age'].values + \
        b_bmi * bp_data['bmi'].values + \
        b_meds * bp_data['on_meds'].values

    # Likelihood

```

10.3. Bayesian Logistic Regression

```
y_obs = pm.Normal("sbp", mu=mu, sigma=sigma, observed=bp_data['sbp'].values)

# Sample
trace_bp = pm.sample(1000, tune=1000, chains=4, random_seed=42,
                     progressbar=False)

print(az.summary(trace_bp, var_names=["Intercept", "b_age", "b_bmi",
                                     "b_on_meds", "sigma"]))

az.plot_forest(trace_bp, var_names=["b_age", "b_bmi", "b_on_meds"],
               combined=True, hdi_prob=0.95, figsize=(8, 4))
import matplotlib.pyplot as plt
plt.title("Posterior Distributions of Regression Coefficients\n95% HDI")
plt.tight_layout()
plt.show()
```

The output shows the **posterior mean**, **standard deviation** (the Bayesian analogue of the standard error), and the **95% credible interval** for each coefficient. Notice that the on-medication effect has a credible interval that is entirely below zero, providing direct evidence that medication lowers SBP.

10.3. Bayesian Logistic Regression

For binary outcomes (e.g., 30-day readmission, treatment response), we use Bayesian logistic regression. The model is identical to the frequentist version except that all parameters receive priors:

$$\text{logit}(P(y_i = 1)) = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}$$

10.3.1. R

```

# Simulate readmission data
set.seed(456)
n <- 300
readmit_data <- tibble(
  age = round(rnorm(n, 65, 10)),
  charlson = rpois(n, 2),
  los = rpois(n, 5) + 1, # length of stay in days
  readmit_30 = rbinom(n, 1,
    plogis(-3 + 0.02 * round(rnorm(n, 65, 10)) +
      0.3 * rpois(n, 2) + 0.1 * (rpois(n, 5) + 1)))
)

fit_readmit <- brm(
  readmit_30 ~ age + charlson + los,
  data = readmit_data,
  family = bernoulli(link = "logit"),
  prior = c(
    prior(normal(0, 2.5), class = "b"), # weakly informative on log-odds
    prior(normal(0, 5), class = "Intercept")
  ),
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = 42,
  silent = 2,
  refresh = 0
)

# Posterior summary on the odds ratio scale
posterior_samples <- as_draws_df(fit_readmit)
or_summary <- posterior_samples %>%

```

10.3. Bayesian Logistic Regression

```
transmute(
  OR_age = exp(b_age),
  OR_charlson = exp(b_charlson),
  OR_los = exp(b_los)
) %>%
summarise(across(everything(),
  list(mean = mean,
        q2.5 = ~quantile(., 0.025),
        q97.5 = ~quantile(., 0.975))))

cat("Posterior odds ratios (mean [95% CrI]):\n")
cat(sprintf(" Age:      %.3f [%.3f, %.3f]\n",
  or_summary$OR_age_mean, or_summary$OR_age_q2.5, or_summary$OR_age_q97.5))
cat(sprintf(" Charlson: %.3f [%.3f, %.3f]\n",
  or_summary$OR_charlson_mean, or_summary$OR_charlson_q2.5,
  or_summary$OR_charlson_q97.5))
cat(sprintf(" LOS:      %.3f [%.3f, %.3f]\n",
  or_summary$OR_los_mean, or_summary$OR_los_q2.5, or_summary$OR_los_q97.5))
```

10.3.2. Python

```
import numpy as np
import pandas as pd
import pymc as pm
import arviz as az

np.random.seed(456)
n = 300
readmit_data = pd.DataFrame({
  'age': np.round(np.random.normal(65, 10, n)),
  'charlson': np.random.poisson(2, n),
  'los': np.random.poisson(5, n) + 1
```

```

}))

from scipy.special import expit
lp = -3 + 0.02 * readmit_data['age'] + 0.3 * readmit_data['charlson'] + \
      0.1 * readmit_data['los']
readmit_data['readmit_30'] = np.random.binomial(1, expit(lp))

with pm.Model() as readmit_model:
    intercept = pm.Normal("Intercept", mu=0, sigma=5)
    b_age      = pm.Normal("b_age", mu=0, sigma=2.5)
    b_charl    = pm.Normal("b_charlson", mu=0, sigma=2.5)
    b_los      = pm.Normal("b_los", mu=0, sigma=2.5)

    logit_p = intercept + b_age * readmit_data['age'].values + \
                  b_charl * readmit_data['charlson'].values + \
                  b_los * readmit_data['los'].values

    y_obs = pm.Bernoulli("readmit", logit_p=logit_p,
                        observed=readmit_data['readmit_30'].values)
    trace_readmit = pm.sample(1000, tune=1000, chains=4, random_seed=42,
                             progressbar=False)

# Compute odds ratios from posterior
posterior = az.extract(trace_readmit)
for var in ["b_age", "b_charlson", "b_los"]:
    or_vals = np.exp(posterior[var].values)
    print(f"OR {var}: {or_vals.mean():.3f} "
          f"[{np.percentile(or_vals, 2.5):.3f}, "
          f"{np.percentile(or_vals, 97.5):.3f}]")

```


10.4. Prior Predictive Checks

A prior predictive check asks: **before seeing any data, does the model generate plausible outcomes?** This is a critical step that is often skipped. If your priors allow the model to predict systolic blood pressures of 500 mmHg or negative values, your priors are too vague.

The procedure:

1. Sample parameter values from the priors (not the posterior).
2. Simulate data from the model using those parameter values.
3. Plot the simulated data and check whether it covers a plausible range.

10.4.1. R

```
# Generate prior predictive samples
pp_check_prior <- brm(
  sbp ~ age + bmi + on_meds,
  data = bp_data,
  family = gaussian(),
  prior = c(
    prior(normal(0, 20), class = "b"),
    prior(normal(120, 30), class = "Intercept"),
    prior(exponential(0.1), class = "sigma")
  ),
  sample_prior = "only",
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = 42,
  silent = 2,
  refresh = 0
)
```

```
pp_samples <- posterior_predict(pp_check_prior)

# Plot density of prior predictive samples vs observed data
tibble(
  simulated = as.vector(pp_samples[sample(nrow(pp_samples), 50), ]),
  type = "Prior predictive"
) %>%
  ggplot(aes(x = simulated)) +
  geom_density(fill = "steelblue", alpha = 0.3) +
  geom_density(data = bp_data, aes(x = sbp), fill = "firebrick", alpha = 0.3) +
  labs(x = "Systolic Blood Pressure (mmHg)",
       y = "Density",
       title = "Prior Predictive Check",
       subtitle = "Blue = simulated from priors; Red = observed data") +
  xlim(-100, 350) +
  theme_minimal(base_size = 13)
```

10.4.2. Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
n_sim = 2000

# Simulate from priors
intercepts = np.random.normal(120, 30, n_sim)
b_ages     = np.random.normal(0, 20, n_sim)
b_bmis     = np.random.normal(0, 20, n_sim)
b_meds_arr = np.random.normal(0, 20, n_sim)
sigmas     = np.random.exponential(10, n_sim)
```

10.5. Posterior Predictive Checks

```
# Pick a "typical" patient: age=55, bmi=28, on_meds=0
sim_sbp = intercepts + b_ages * 55 + b_bmis * 28 + b_meds_arr * 0
sim_sbp += np.random.normal(0, sigmas)

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(sim_sbp, bins=80, density=True, alpha=0.4, color="steelblue",
        label="Prior predictive", range=(-500, 800))
ax.hist(bp_data['sbp'].values, bins=30, density=True, alpha=0.4,
        color="firebrick", label="Observed data")
ax.set_xlabel("Systolic Blood Pressure (mmHg)")
ax.set_ylabel("Density")
ax.set_title("Prior Predictive Check\nBlue = simulated from priors; Red = observed")
ax.set_xlim(-200, 400)
ax.legend()
plt.tight_layout()
plt.show()
```

If the prior predictive distribution is wildly broader than any plausible clinical range, consider tightening the priors. If it is too narrow and excludes plausible values, widen them. This iterative process happens **before** fitting the model to data.

10.5. Posterior Predictive Checks

After fitting, we ask: **does the fitted model generate data that look like the observed data?** This is the Bayesian equivalent of residual diagnostics.

The procedure:

1. Draw parameter values from the posterior.
2. Simulate new data from the model using those posterior draws.

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications

3. Compare the distribution of simulated data to the observed data.

If the model is well specified, the simulated data should be nearly indistinguishable from the real data in terms of summary statistics (mean, variance, range, shape).

10.5.1. R

```
pp_check(fit_bp, type = "dens_overlay", ndraws = 100) +  
  labs(title = "Posterior Predictive Check",  
        subtitle = "Light blue = simulated from posterior; Dark = observed") +  
  theme_minimal(base_size = 13)
```

10.5.2. Python

```
import pymc as pm  
import arviz as az  
import matplotlib.pyplot as plt  
  
# Using the previously fitted model  
with bp_model:  
    ppc = pm.sample_posterior_predictive(trace_bp, random_seed=42,  
                                       progressbar=False)  
  
az.plot_ppc(az.from_pymc3(posterior_predictive=ppc,  
                        observed_data={"sbp": bp_data['sbp'].values}),  
            num_pp_samples=100, figsize=(8, 5))  
plt.title("Posterior Predictive Check")  
plt.tight_layout()  
plt.show()
```

10.6. Bayesian Hierarchical (Multilevel) Models

10.6.1. Why They Matter

Consider a multi-site clinical trial testing a new antihypertensive drug across 12 hospitals. The treatment effect might vary from site to site due to differences in patient populations, clinical protocols, or local practice patterns. How should we estimate the treatment effect at each site?

There are three naive approaches:

1. **Complete pooling**: ignore the site structure and estimate a single treatment effect. This assumes all sites are identical — clearly wrong.
2. **No pooling**: estimate a separate treatment effect at each site. This ignores that the sites are studying the same drug. Small sites will have wildly imprecise estimates.
3. **Partial pooling (hierarchical model)**: the Bayesian approach. Each site has its own treatment effect, but those effects are drawn from a shared distribution. Sites with small samples are **pulled toward the overall mean**, while sites with large samples retain their individual estimates.

This pulling toward the grand mean is called **shrinkage**, and it is one of the most powerful ideas in applied statistics.

10.6.2. The Model Structure

A Bayesian hierarchical model for treatment effects across J hospitals:

$$y_{ij} = \beta_0 + \beta_1 \text{treatment}_{ij} + u_j + \epsilon_{ij}$$

where:

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications

- $u_j \sim \text{Normal}(0, \tau^2)$ is the random effect for hospital j
- τ is the between-hospital standard deviation (estimated from the data)
- $\epsilon_{ij} \sim \text{Normal}(0, \sigma^2)$ is the residual error

In the Bayesian framework, τ also gets a prior. A common choice is a half-Cauchy or half-normal distribution, which allows for substantial between-group variation while gently regularising toward zero.

10.6.3. Clinical Example: Treatment Effects Across Hospitals

10.6.4. R

```
library(brms)

# Simulate multi-site trial data
set.seed(789)
n_hospitals <- 12
patients_per_hospital <- c(15, 20, 25, 30, 35, 40, 50, 60, 80, 100, 120, 150)
true_grand_effect <- -8 # mmHg reduction in SBP
tau_true <- 3           # between-hospital SD

hospital_effects <- rnorm(n_hospitals, 0, tau_true)

trial_data <- map2_dfr(1:n_hospitals, patients_per_hospital, function(j, nj)
  tibble(
    hospital = factor(j),
    treatment = rep(0:1, length.out = nj),
    sbp = 140 + (true_grand_effect + hospital_effects[j]) * treatment +
      rnorm(nj, 0, 15)
  )
})
```

10.6. Bayesian Hierarchical (Multilevel) Models

```
# Fit hierarchical model
fit_hier <- brm(
  sbp ~ treatment + (treatment | hospital),
  data = trial_data,
  prior = c(
    prior(normal(0, 20), class = "b"),
    prior(normal(140, 30), class = "Intercept"),
    prior(cauchy(0, 5), class = "sd"),
    prior(exponential(0.1), class = "sigma")
  ),
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = 42,
  silent = 2,
  refresh = 0,
  control = list(adapt_delta = 0.95)
)

# Extract hospital-specific treatment effects (partial pooling)
ranefs <- ranef(fit_hier)$hospital[, , "treatment"]
grand_mean <- fixef(fit_hier)["treatment", "Estimate"]

# No-pooling estimates (separate OLS per hospital)
no_pool <- trial_data %>%
  group_by(hospital) %>%
  summarise(no_pool_est = coef(lm(sbp ~ treatment))[2],
            n = n(), .groups = "drop") %>%
  mutate(
    hospital_num = as.numeric(hospital),
    partial_pool_est = grand_mean + ranefs[, "Estimate"]
  )
```

```
# Plot shrinkage
ggplot(no_pool, aes(y = reorder(hospital, n))) +
  geom_point(aes(x = no_pool_est, colour = "No pooling"), size = 3) +
  geom_point(aes(x = partial_pool_est, colour = "Partial pooling"), size = 3) +
  geom_vline(xintercept = grand_mean, linetype = "dashed", colour = "grey40") +
  annotate("text", x = grand_mean + 0.5, y = 12.5,
    label = paste0("Grand mean = ", round(grand_mean, 1)),
    hjust = 0, size = 3.5) +
  geom_segment(aes(x = no_pool_est, xend = partial_pool_est,
    yend = reorder(hospital, n)),
    arrow = arrow(length = unit(0.15, "cm")),
    colour = "grey60") +
  scale_colour_manual(values = c("No pooling" = "steelblue",
    "Partial pooling" = "firebrick")) +
  labs(x = "Treatment Effect (mmHg change in SBP)",
    y = "Hospital (ordered by sample size)",
    colour = "Estimate Type",
    title = "Shrinkage in a Hierarchical Model",
    subtitle = "Arrows show how partial pooling pulls estimates toward the grand mean") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
```

10.6.5. Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(789)
n_hospitals = 12
patients = [15, 20, 25, 30, 35, 40, 50, 60, 80, 100, 120, 150]
grand_effect = -8
```


10.6. Bayesian Hierarchical (Multilevel) Models

```
tau = 3

hospital_effects = np.random.normal(0, tau, n_hospitals)

rows = []
for j in range(n_hospitals):
    nj = patients[j]
    trt = np.tile([0, 1], nj // 2 + 1)[:nj]
    sbp = 140 + (grand_effect + hospital_effects[j]) * trt + \
        np.random.normal(0, 15, nj)
    for i in range(nj):
        rows.append({'hospital': j, 'treatment': trt[i], 'sbp': sbp[i]})

trial_data = pd.DataFrame(rows)

# No-pooling estimates
no_pool = []
for j in range(n_hospitals):
    df_j = trial_data[trial_data['hospital'] == j]
    trt_mean = df_j[df_j['treatment'] == 1]['sbp'].mean()
    ctl_mean = df_j[df_j['treatment'] == 0]['sbp'].mean()
    no_pool.append(trt_mean - ctl_mean)

no_pool = np.array(no_pool)
grand_mean_est = np.mean(no_pool)

# Simulate partial pooling (shrinkage toward grand mean)
# Shrinkage factor depends on sample size: larger sites shrink less
shrinkage_factor = np.array([15 / (15 + nj) for nj in patients])
partial_pool = grand_mean_est + (1 - shrinkage_factor) * (no_pool - grand_mean_est)

# Plot
fig, ax = plt.subplots(figsize=(10, 6))
```

```

order = np.argsort(patients)
y_pos = np.arange(n_hospitals)

for i, idx in enumerate(order):
    ax.plot([no_pool[idx], partial_pool[idx]], [i, i],
            color="grey", linewidth=1, zorder=1)
    ax.annotate("", xy=(partial_pool[idx], i), xytext=(no_pool[idx], i),
                arrowprops=dict(arrowstyle="->", color="grey"))

ax.scatter(no_pool[order], y_pos, color="steelblue", s=60, zorder=2,
           label="No pooling")
ax.scatter(partial_pool[order], y_pos, color="firebrick", s=60, zorder=2,
           label="Partial pooling")
ax.axvline(grand_mean_est, linestyle="--", color="grey", linewidth=1)
ax.set_yticks(y_pos)
ax.set_yticklabels([f"Hospital {order[i]+1}\n(n={patients[order[i]]})"
                    for i in range(n_hospitals)], fontsize=9)
ax.set_xlabel("Treatment Effect (mmHg)")
ax.set_title("Shrinkage in a Hierarchical Model\n"
             "Arrows show estimates pulled toward the grand mean")
ax.legend()
plt.tight_layout()
plt.show()

```

10.6.6. Understanding Shrinkage

The figure above illustrates the key behaviour of hierarchical models:

- **Small hospitals** (few patients) are pulled strongly toward the grand mean. Their individual data are noisy, so the model relies more on the shared information from other sites.

10.7. The Practical Bayesian Workflow

- **Large hospitals** (many patients) retain estimates close to their no-pooling values. Their data are informative enough to override the group-level information.
- **The grand mean** itself is estimated from all sites, so every site contributes to it.

This is **partial pooling** — a principled compromise between treating all sites as identical (complete pooling) and treating them as completely independent (no pooling). It is particularly valuable when some sites have very few patients, where no-pooling estimates would be unreliable.

i Shrinkage is not bias

Shrinkage toward the grand mean might seem like it introduces bias. In a narrow sense it does: individual site estimates are biased toward the mean. But this bias is traded for reduced variance, and the overall mean squared error is lower. This is the bias-variance trade-off that we discussed in earlier chapters, now appearing naturally in the Bayesian framework.

10.6.7. Varying Slopes

The model above includes a varying (random) slope for treatment by hospital. This means we are estimating not just “does the treatment effect vary across hospitals?” but also “by how much?” The between-hospital standard deviation τ directly quantifies this heterogeneity. If the 95% credible interval for τ excludes zero, there is evidence of meaningful variation across sites.

10.7. The Practical Bayesian Workflow

A complete Bayesian analysis follows a structured workflow:

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications

10.7.1. Step 1: Specify the Model

- Write down the likelihood (data-generating process).
- Choose priors for all parameters.
- Justify your choices: are the priors weakly informative? Are they based on prior studies?

10.7.2. Step 2: Prior Predictive Check

- Simulate data from the priors alone.
- Verify that the implied range of outcomes is clinically plausible.
- Iterate if necessary: tighten or widen priors.

10.7.3. Step 3: Fit the Model

- Run MCMC (typically 4 chains, 1000–2000 post-warmup iterations each).
- Check for warnings (divergent transitions, max treedepth warnings).

10.7.4. Step 4: Diagnose Convergence

- Inspect trace plots: should look like “hairy caterpillars”.
- Check $\hat{R} < 1.01$ for all parameters.
- Check ESS > 400 for both bulk and tail.
- If any diagnostic fails, do **not** interpret the results. Fix the model first.

10.7.5. Step 5: Posterior Predictive Check

- Simulate data from the fitted posterior.
- Compare to observed data: distributions, summary statistics, patterns.
- If the model is badly misspecified, revise the likelihood or priors.

10.7.6. Step 6: Summarise and Report

- Report posterior means (or medians), credible intervals, and relevant posterior probabilities.
- Include sensitivity analyses: how do results change under alternative priors?
- Report MCMC diagnostics alongside the results.

Reporting Template

“We estimated the treatment effect using a Bayesian [model type] with [prior specification]. The posterior mean treatment effect was X (95% credible interval: [L, U]). The posterior probability that the treatment effect exceeds the clinically meaningful threshold of Y was Z%. All chains converged ($\hat{R} < 1.01$, bulk ESS > [value], tail ESS > [value]).”

10.8. Implementation: Software Choices

10.8.1. R Ecosystem

brms (Bayesian Regression Models using Stan) is the recommended package for applied Bayesian analysis in R. It uses the familiar R formula syntax and compiles models in Stan behind the scenes. Essentially, anything you

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications

can fit with `lme4` can be fit with `brms`, but with priors and full posterior inference.

`rstanarm` provides pre-compiled Stan models for common regression types. It is faster to start (no compilation time) but less flexible than `brms`.

Both produce Stan code that you can inspect, which is excellent for learning.

10.8.2. Python Ecosystem

`PyMC` (version 5+) is the most mature probabilistic programming library in Python. It uses the JAX or NumPy backends and provides NUTS sampling out of the box.

`Bambi` (BAyesian Model-Building Interface) is the Python analogue of `brms`: it provides a formula-based interface on top of `PyMC`, making it easy to specify complex models without writing raw `PyMC` code.

10.8.3. R (`brms`)

```
library(brms)

# Specify and fit
fit <- brm(
  outcome ~ predictor1 + predictor2 + (1 | group),
  data = my_data,
  family = gaussian(),
  prior = c(
    prior(normal(0, 10), class = "b"),
    prior(normal(0, 20), class = "Intercept"),
    prior(cauchy(0, 5), class = "sd"),
    prior(exponential(0.1), class = "sigma")
  )
)
```

10.8. Implementation: Software Choices

```
),
  chains = 4, iter = 2000, warmup = 1000, seed = 42
)

# Diagnose
plot(fit)           # trace plots
summary(fit)        # R-hat, ESS
pp_check(fit)       # posterior predictive check
conditional_effects(fit) # visualise effects

# Posterior probability of meaningful effect
hypothesis(fit, "predictor1 > 0.5")
```

10.8.4. Python (bambi)

```
import bambi as bmb
import arviz as az

# Specify and fit
model = bmb.Model("outcome ~ predictor1 + predictor2 + (1 | group)",
                  data=my_data, family="gaussian")
results = model.fit(draws=1000, tune=1000, chains=4, random_seed=42)

# Diagnose
az.plot_trace(results)  # trace plots
print(az.summary(results)) # R-hat, ESS
az.plot_ppc(results)    # posterior predictive check

# Posterior probability
posterior = az.extract(results)
prob = (posterior['predictor1'] > 0.5).mean()
print(f"P(predictor1 > 0.5 | data) = {prob:.3f}")
```

10.9. Exercises

10.9.1. Exercise 1: Bayesian Logistic Regression for ICU Mortality

A dataset contains 500 ICU admissions with the following variables: age, APACHE II score, mechanical ventilation status (yes/no), and 28-day mortality (outcome).

- Simulate a dataset with plausible parameter values.
- Fit a Bayesian logistic regression model using weakly informative priors.
- Perform a prior predictive check: do the priors imply plausible mortality rates?
- Report posterior odds ratios with 95% credible intervals.
- Calculate $P(\text{OR}_{\text{APACHE}} > 1.10 \mid \text{data})$ — the probability that each unit increase in APACHE score increases the odds of death by more than 10%.

10.9.2. R

```
set.seed(101)
n <- 500
icu_data <- tibble(
  age = round(rnorm(n, 62, 15)),
  apache = round(rnorm(n, 18, 7)),
  ventilated = rbinom(n, 1, 0.35),
  mortality = rbinom(n, 1,
    plogis(-4.5 + 0.02 * round(rnorm(n, 62, 15)) +
      0.12 * round(rnorm(n, 18, 7)) + 0.8 * rbinom(n, 1, 0.35)))
)
```



```

fit_icu <- brm(
  mortality ~ age + apache + ventilated,
  data = icu_data,
  family = bernoulli(),
  prior = c(
    prior(normal(0, 2.5), class = "b"),
    prior(normal(0, 5), class = "Intercept")
  ),
  chains = 4, iter = 2000, warmup = 1000, seed = 42,
  silent = 2, refresh = 0
)

# Posterior OR for APACHE
post <- as_draws_df(fit_icu)
or_apache <- exp(post$b_apache)
cat(sprintf("OR APACHE: %.3f [%.3f, %.3f]\n",
  mean(or_apache), quantile(or_apache, 0.025), quantile(or_apache, 0.975)))
cat(sprintf("P(OR > 1.10 | data): %.3f\n", mean(or_apache > 1.10)))

```

10.9.3. Python

```

import numpy as np
import pandas as pd
import pymc as pm
import arviz as az
from scipy.special import expit

np.random.seed(101)
n = 500
icu_data = pd.DataFrame({
  'age': np.round(np.random.normal(62, 15, n)),
  'apache': np.round(np.random.normal(18, 7, n)),

```

```

        'ventilated': np.random.binomial(1, 0.35, n)
    })
    lp = -4.5 + 0.02 * icu_data['age'] + 0.12 * icu_data['apache'] + \
        0.8 * icu_data['ventilated']
    icu_data['mortality'] = np.random.binomial(1, expit(lp))

    with pm.Model() as icu_model:
        intercept = pm.Normal("Intercept", 0, 5)
        b_age = pm.Normal("b_age", 0, 2.5)
        b_apache = pm.Normal("b_apache", 0, 2.5)
        b_vent = pm.Normal("b_ventilated", 0, 2.5)

        logit_p = intercept + b_age * icu_data['age'].values + \
            b_apache * icu_data['apache'].values + \
            b_vent * icu_data['ventilated'].values

        y = pm.Bernoulli("mort", logit_p=logit_p,
                        observed=icu_data['mortality'].values)
        trace = pm.sample(1000, tune=1000, chains=4, random_seed=42,
                        progressbar=False)

    post = az.extract(trace)
    or_apache = np.exp(post['b_apache'].values)
    print(f"OR APACHE: {or_apache.mean():.3f} "
          f"[{np.percentile(or_apache, 2.5):.3f}, "
          f"{np.percentile(or_apache, 97.5):.3f}]")
    print(f"P(OR > 1.10 | data): {(or_apache > 1.10).mean():.3f}")

```

10.9.4. Exercise 2: Hierarchical Model for Multi-Site Drug Trial

Twelve hospitals participate in a trial comparing a new statin to standard care. The primary outcome is change in LDL cholesterol (mg/dL) at 12

weeks. Hospitals vary in patient volume from 20 to 200 patients.

- a. Simulate data where the true average treatment effect is -25 mg/dL with between-hospital SD of 5 mg/dL.
- b. Fit a Bayesian hierarchical model with random intercepts and random slopes for treatment by hospital.
- c. Create a shrinkage plot showing how hospital-specific estimates are pulled toward the grand mean.
- d. Compare the hierarchical model estimates to separate hospital-by-hospital OLS regressions. Which approach is more appropriate and why?

10.9.5. Exercise 3: Prior Sensitivity for Rare Events

A new surgical technique is tested in 40 patients. Zero patients experience a major adverse event.

- a. Compute the posterior distribution for the adverse event rate using three priors: $\text{Beta}(1,1)$, $\text{Beta}(0.5, 0.5)$ (Jeffreys prior), and $\text{Beta}(1, 9)$ (informative: prior belief $\sim 10\%$ rate).
- b. Report the posterior mean and 95% upper credible bound for each prior.
- c. A frequentist would report the point estimate as $0/40 = 0$. Explain why the Bayesian estimates are more useful for regulatory decision-making.

10.10. Summary

Bayesian regression models extend familiar linear and logistic regression by placing priors on parameters and yielding full posterior distributions rather than point estimates. The practical workflow — specify, check priors, fit, diagnose, check posteriors, report — ensures rigorous and transparent

10. Applied Bayesian Modelling: Regression, Hierarchical Models, and Clinical Applications

analysis. Hierarchical models, with their partial pooling and shrinkage properties, are especially powerful for multi-site clinical studies where borrowing information across groups improves estimation. Modern software (brms in R, PyMC/bambi in Python) makes these methods accessible to applied researchers.

Key Takeaways

1. Bayesian regression yields full posterior distributions for all coefficients, not just point estimates.
2. Prior and posterior predictive checks are essential quality-control steps.
3. Hierarchical models perform partial pooling: small groups shrink toward the grand mean, large groups retain their individual estimates.
4. Shrinkage reduces mean squared error by trading a small amount of bias for a large reduction in variance.
5. Always report MCMC diagnostics ($\hat{R}$, ESS, trace plots) alongside posterior summaries.

10.11. References and Further Reading

The primary reference for the brms package is Buerkner’s 2017 paper “brms: An R Package for Bayesian Multilevel Models Using Stan” published in the *Journal of Statistical Software*, which provides a thorough overview of the formula syntax, prior specification, and model families available. For the hierarchical modelling concepts presented in this chapter, Gelman and Hill’s *Data Analysis Using Regression and Multilevel/Hierarchical Models* (2006) remains the classic applied reference, and Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin’s *Bayesian Data Analysis* (3rd edition, 2013) provides the theoretical foundations.

10.11. References and Further Reading

McElreath's *Statistical Rethinking* (2nd edition, 2020) devotes several excellent chapters to multilevel models with a focus on intuition and simulation. For the clinical applications of hierarchical Bayesian models, the FDA's 2010 guidance on Bayesian statistics in medical device trials and its 2026 update provide the regulatory perspective. Berry, Carlin, Lee, and Muller's *Bayesian Adaptive Methods for Clinical Trials* (2010) covers the design and analysis of adaptive trials using Bayesian methods, including hierarchical borrowing across subgroups and historical controls.

On the Python side, the PyMC documentation (pymc.io) includes a growing library of case studies, and Osvaldo Martin's *Bayesian Analysis with Python* (3rd edition, 2024) provides a practical guide to PyMC for applied researchers. The ArviZ documentation (arviz-devs.github.io/arviz) covers posterior diagnostics and visualisation in detail. For the concept of shrinkage and partial pooling, Efron and Morris's classic 1977 paper "Stein's Paradox in Statistics" in *Scientific American* remains a wonderfully accessible introduction, showing that shrinkage estimators outperform individual estimates even in simple settings.

Part IV.

Dimensionality Reduction and Unsupervised Learning

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

11.1. Introduction

A single patient encounter can generate hundreds of variables: vital signs, lab results, imaging features, medication lists, genomic markers, clinical notes. When the number of features (p) is large relative to the number of observations (n), standard statistical methods break down. Regression models become unstable, distances between data points become uninformative, and visualisation is impossible.

Dimensionality reduction addresses this by projecting high-dimensional data into a lower-dimensional space while preserving as much meaningful structure as possible. This chapter covers three complementary methods: **PCA** (principal component analysis), **t-SNE** (t-distributed stochastic neighbour embedding), and **UMAP** (uniform manifold approximation and projection). Each serves a different purpose, and understanding when to use which is a core skill in modern data analysis.

11.2. The Curse of Dimensionality

The “curse of dimensionality” refers to a collection of phenomena that arise when data live in high-dimensional spaces. The most important for clinical

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

data analysis:

Distances become meaningless. In high dimensions, the distance between the nearest and farthest data points converges. If all points are approximately equidistant, distance-based methods (k-nearest neighbours, clustering) lose their ability to discriminate.

Sparsity. The volume of a high-dimensional space grows exponentially with the number of dimensions. Data points become increasingly sparse — you need exponentially more observations to maintain the same data density. A study with 200 patients and 500 gene expression features has data that are extraordinarily sparse.

Overfitting. With many features, models can find spurious patterns in the noise. A logistic regression with 200 predictors and 100 observations is almost guaranteed to overfit, even with regularisation.

Visualisation is impossible. Humans perceive at most three spatial dimensions. To explore the structure of a 50-dimensional dataset, we must reduce it to 2 or 3 dimensions.

11.3. PCA: Principal Component Analysis

PCA is the oldest and most widely used dimensionality reduction technique. It finds the directions (linear combinations of the original variables) along which the data vary the most.

11.3.1. The Core Idea

Given a data matrix $\mathbf{X}$ with n observations and p variables:

1. **Centre** the data: subtract the mean of each variable.
2. **Compute the covariance matrix** $\mathbf{C} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$.

11.3. PCA: Principal Component Analysis

3. **Find the eigenvectors** of **C**. These are the **principal components** — the directions of maximum variance.
4. **The eigenvalues** tell you how much variance each component captures.

The first principal component (PC1) captures the most variance, PC2 captures the second most (orthogonal to PC1), and so on. By keeping only the first k components, you reduce the data from p dimensions to k dimensions.

11.3.2. Key Properties

- PCA is a **linear** method: each component is a weighted sum of the original variables.
- It is **deterministic**: running PCA twice on the same data gives the same result (up to sign flips).
- It preserves **global structure** (distances and variance) well, but may miss non-linear patterns.

11.3.3. Choosing the Number of Components

Scree plot. Plot the eigenvalues (or proportion of variance explained) against the component number. Look for an “elbow” — a point where the curve flattens, suggesting that additional components contribute little.

Cumulative variance threshold. Keep enough components to explain a target percentage of total variance (commonly 80–90%).

Kaiser criterion. Keep components with eigenvalues > 1 (only meaningful when data are standardised).

11.3.4. Clinical Example: Metabolic Panel PCA

A common application: reducing a panel of correlated lab values to a few interpretable dimensions. Consider a dataset with glucose, HbA1c, triglycerides, LDL, HDL, creatinine, ALT, and albumin.

11.3.5. R

```
set.seed(42)
n <- 300

# Simulate correlated metabolic data
# Create correlation structure: glucose/HbA1c/triglycerides cluster;
# creatinine/albumin cluster
library(MASS)
Sigma <- matrix(c(
  1.0, 0.7, 0.5, 0.3, -0.2, 0.1, 0.2, -0.1,
  0.7, 1.0, 0.4, 0.2, -0.2, 0.1, 0.1, -0.1,
  0.5, 0.4, 1.0, 0.4, -0.3, 0.1, 0.3, -0.1,
  0.3, 0.2, 0.4, 1.0, -0.5, 0.1, 0.2, 0.0,
  -0.2, -0.2, -0.3, -0.5, 1.0, -0.1, -0.1, 0.2,
  0.1, 0.1, 0.1, 0.1, -0.1, 1.0, 0.1, -0.3,
  0.2, 0.1, 0.3, 0.2, -0.1, 0.1, 1.0, -0.2,
  -0.1, -0.1, -0.1, 0.0, 0.2, -0.3, -0.2, 1.0
), nrow = 8)

z <- mvrnorm(n, mu = rep(0, 8), Sigma = Sigma)
metabolic <- tibble(
  glucose      = round(z[,1] * 30 + 100),
  hba1c        = round(z[,2] * 1.0 + 5.8, 1),
  triglycerides = round(z[,3] * 50 + 150),
  ldl          = round(z[,4] * 30 + 120),
```

11.3. PCA: Principal Component Analysis

```
hdl      = round(z[,5] * 12 + 55),
creatinine = round(z[,6] * 0.3 + 1.0, 2),
alt      = round(z[,7] * 15 + 30),
albumin  = round(z[,8] * 0.4 + 4.0, 1)
)

# PCA on scaled data
pca_result <- prcomp(metabolic, scale. = TRUE)

# Scree plot and biplot side by side
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Scree plot
var_explained <- pca_result$sdev^2 / sum(pca_result$sdev^2)
barplot(var_explained, names.arg = paste0("PC", 1:8),
        col = "steelblue", main = "Scree Plot",
        ylab = "Proportion of Variance", xlab = "Component",
        ylim = c(0, 0.4))
abline(h = 1/8, lty = 2, col = "firebrick") # Kaiser criterion line
text(9, 1/8 + 0.015, "Kaiser threshold", col = "firebrick", cex = 0.8)

# Biplot
biplot(pca_result, scale = 0, cex = 0.6,
       col = c("grey70", "firebrick"),
       main = "PCA Biplot: Metabolic Panel")
```

11.3.6. Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

```
from sklearn.decomposition import PCA

np.random.seed(42)
n = 300

Sigma = np.array([
    [1.0, 0.7, 0.5, 0.3, -0.2, 0.1, 0.2, -0.1],
    [0.7, 1.0, 0.4, 0.2, -0.2, 0.1, 0.1, -0.1],
    [0.5, 0.4, 1.0, 0.4, -0.3, 0.1, 0.3, -0.1],
    [0.3, 0.2, 0.4, 1.0, -0.5, 0.1, 0.2, 0.0],
    [-0.2, -0.2, -0.3, -0.5, 1.0, -0.1, -0.1, 0.2],
    [0.1, 0.1, 0.1, 0.1, -0.1, 1.0, 0.1, -0.3],
    [0.2, 0.1, 0.3, 0.2, -0.1, 0.1, 1.0, -0.2],
    [-0.1, -0.1, -0.1, 0.0, 0.2, -0.3, -0.2, 1.0]
])

z = np.random.multivariate_normal(np.zeros(8), Sigma, n)
labels = ["glucose", "hba1c", "triglycerides", "ldl",
          "hdl", "creatinine", "alt", "albumin"]
metabolic = pd.DataFrame(z, columns=labels)

# Scale and fit PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(metabolic)
pca = PCA()
scores = pca.fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scree plot
axes[0].bar(range(1, 9), pca.explained_variance_ratio_,
            color="steelblue", edgecolor="white")
axes[0].axhline(y=1/8, color="firebrick", linestyle="--", label="Kaiser three
```

11.3. PCA: Principal Component Analysis

```
axes[0].set_xlabel("Component")
axes[0].set_ylabel("Proportion of Variance")
axes[0].set_title("Scree Plot")
axes[0].set_xticks(range(1, 9))
axes[0].legend()

# Biplot
loadings = pca.components_[1:2].T # first 2 PCs
scale_factor = 3
axes[1].scatter(scores[:, 0], scores[:, 1], alpha=0.3, s=10, color="grey")
for i, lab in enumerate(labels):
    axes[1].annotate("", xy=(loadings[i, 0]*scale_factor,
                             loadings[i, 1]*scale_factor),
                    xytext=(0, 0),
                    arrowprops=dict(arrowstyle="->", color="firebrick", lw=1.5))
    axes[1].text(loadings[i, 0]*scale_factor*1.15,
                 loadings[i, 1]*scale_factor*1.15,
                 lab, fontsize=9, color="firebrick", ha="center")
axes[1].set_xlabel(f"PC1 ({{pca.explained_variance_ratio_[0]:.1%}} var)")
axes[1].set_ylabel(f"PC2 ({{pca.explained_variance_ratio_[1]:.1%}} var)")
axes[1].set_title("PCA Biplot: Metabolic Panel")
axes[1].axhline(0, color="grey", linewidth=0.5)
axes[1].axvline(0, color="grey", linewidth=0.5)

plt.tight_layout()
plt.show()
```

11.3.7. Interpreting the Biplot

The biplot overlays two things:

- **Points** (grey): individual patients projected onto the first two principal components.

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

- **Arrows** (red): the original variables. The direction and length of each arrow show how strongly and in which direction that variable contributes to each PC.

In our metabolic panel:

- **PC1** is dominated by glucose, HbA1c, and triglycerides pointing in one direction, with HDL pointing in the opposite direction. This is a “metabolic syndrome” axis: patients to the right tend to have higher glucose and triglycerides and lower HDL.
- **PC2** separates creatinine and albumin, capturing a “renal/hepatic” dimension.

This kind of interpretation is exactly what makes PCA useful in clinical research: it reveals which variables “travel together” and suggests underlying biological dimensions.

11.4. t-SNE: Non-Linear Dimensionality Reduction

11.4.1. The Core Idea

PCA preserves global, linear structure. But many datasets contain non-linear patterns — clusters, manifolds, or curves — that PCA cannot capture well. **t-SNE** (t-distributed stochastic neighbour embedding) was designed specifically for **visualisation** of high-dimensional data in 2 or 3 dimensions.

t-SNE works in two steps:

1. In the high-dimensional space, compute the probability that any two points are “neighbours” (using a Gaussian kernel). Nearby points have high probability; distant points have low probability.

11.4. *t*-SNE: Non-Linear Dimensionality Reduction

2. In the low-dimensional (2D) embedding, try to arrange points so that the same neighbourhood probabilities are preserved, using a **t-distribution** kernel (heavier tails than a Gaussian, which helps prevent crowding).

The algorithm minimises the KL divergence between the high-dimensional and low-dimensional probability distributions using gradient descent.

11.4.2. The Perplexity Parameter

Perplexity (typically 5–50) controls how many neighbours each point considers. It roughly corresponds to the effective number of local neighbours.

- **Low perplexity** (5–10): focuses on very local structure. May produce many small, tight clusters.
- **High perplexity** (30–50): considers broader neighbourhoods. Produces smoother, more global layouts.

There is no single correct value; try several and see how the structure changes.

11.4.3. Critical Pitfalls and Misinterpretations

 **t-SNE is for exploration, not inference**

t-SNE is a visualisation tool. It is not a statistical test. Do not draw conclusions about statistical significance from a t-SNE plot.

Distances between clusters are meaningless. Two clusters that appear far apart may not actually be more different than two clusters that appear close. t-SNE distorts global distances to preserve local neighbourhoods.

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

Cluster sizes are meaningless. A tight cluster in t-SNE space does not mean those points are more similar than a spread-out cluster.

Different runs give different layouts. t-SNE uses stochastic optimisation. Running it twice with different seeds gives different-looking plots (though the same clusters should emerge).

It does not scale well. Standard t-SNE is $O(n^2)$, making it slow for datasets with more than ~10,000 observations. Approximations (Barnes-Hut, FIt-SNE) help.

11.4.4. Clinical Example: Visualising Patient Phenotypes

11.4.5. R

```
library(Rtsne)

set.seed(42)

# Simulate 3 patient subtypes with 20 features
n_per_group <- 100
p <- 20

group1 <- matrix(rnorm(n_per_group * p, mean = 0, sd = 1), ncol = p)
group2 <- matrix(rnorm(n_per_group * p, mean = 2, sd = 1.2), ncol = p)
group3 <- matrix(rnorm(n_per_group * p, mean = -1, sd = 0.8), ncol = p)
# Make groups differ in specific feature subsets
group2[, 1:5] <- group2[, 1:5] + 3
group3[, 10:15] <- group3[, 10:15] - 4

X <- rbind(group1, group2, group3)
labels <- factor(rep(c("Type A", "Type B", "Type C"), each = n_per_group))
```

11.4. *t*-SNE: Non-Linear Dimensionality Reduction

```
# Run t-SNE with two perplexity values
tsne_low <- Rtsne(X, perplexity = 10, dims = 2, verbose = FALSE, max_iter = 1000)
tsne_high <- Rtsne(X, perplexity = 40, dims = 2, verbose = FALSE, max_iter = 1000)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
cols <- c("steelblue", "firebrick", "forestgreen")

plot(tsne_low$Y, col = cols[labels], pch = 16, cex = 0.8,
     main = "t-SNE (perplexity = 10)", xlab = "t-SNE 1", ylab = "t-SNE 2")
legend("topright", levels(labels), col = cols, pch = 16, cex = 0.8)

plot(tsne_high$Y, col = cols[labels], pch = 16, cex = 0.8,
     main = "t-SNE (perplexity = 40)", xlab = "t-SNE 1", ylab = "t-SNE 2")
legend("topright", levels(labels), col = cols, pch = 16, cex = 0.8)
```

11.4.6. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

np.random.seed(42)
n_per = 100
p = 20

g1 = np.random.normal(0, 1, (n_per, p))
g2 = np.random.normal(2, 1.2, (n_per, p))
g3 = np.random.normal(-1, 0.8, (n_per, p))
g2[:, :5] += 3
g3[:, 10:15] -= 4

X = np.vstack([g1, g2, g3])
```

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

```
labels = np.repeat(["Type A", "Type B", "Type C"], n_per)
colors = {"Type A": "steelblue", "Type B": "firebrick", "Type C": "forestgreen"}

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, perp in zip(axes, [10, 40]):
    tsne = TSNE(n_components=2, perplexity=perp, random_state=42, max_iter=1000)
    emb = tsne.fit_transform(X)
    for lab in ["Type A", "Type B", "Type C"]:
        mask = labels == lab
        ax.scatter(emb[mask, 0], emb[mask, 1], c=colors[lab],
                  s=15, alpha=0.7, label=lab)
    ax.set_title(f"t-SNE (perplexity = {perp})")
    ax.set_xlabel("t-SNE 1")
    ax.set_ylabel("t-SNE 2")
    ax.legend(fontsize=9)

plt.tight_layout()
plt.show()
```

Notice how perplexity changes the visual appearance but the three groups are consistently separated. This is reassuring — the cluster structure is real, not an artefact of a particular perplexity setting.

11.5. UMAP: Uniform Manifold Approximation and Projection

11.5.1. How UMAP Differs from t-SNE

UMAP (McInnes, Healy, and Melville, 2018) is a newer method that has largely replaced t-SNE in many applications. Key advantages:

11.5. UMAP: Uniform Manifold Approximation and Projection

Table 11.1.: Comparison of t-SNE and UMAP

Feature	t-SNE	UMAP
Speed	Slow ($O(n^2)$ or $O(n \log n)$ approximate)	Fast (typically 3–10x faster)
Global structure	Poorly preserved	Better preserved
Reproducibility	Stochastic, varies across runs	More stable (with fixed seed)
Scalability	Struggles above ~50k points	Handles millions of points
Embedding dimensions	Typically 2–3	2 to arbitrary d
Use beyond visualisation	Rarely	Commonly used as a preprocessing step

UMAP works by constructing a topological representation (a weighted graph) of the data in high dimensions, then optimising a low-dimensional layout that preserves the graph structure.

11.5.2. Key Parameters

n_neighbors (analogous to perplexity in t-SNE): controls the balance between local and global structure. Small values (5–15) emphasise local clusters; large values (50–200) emphasise global connectivity.

min_dist: controls how tightly points are packed in the embedding. Small values (0.0–0.1) produce tight, well-separated clusters; large values (0.5–1.0) produce a more uniform spread.

11.5.3. When to Use Which Method

- **PCA**: use for initial exploration, as a preprocessing step (reduce to 20–50 PCs before t-SNE/UMAP), or when you need interpretable components. Use when linear relationships are sufficient.
- **t-SNE**: use for detailed visualisation of moderate-sized datasets (< 10,000 points) when you want to emphasise local cluster structure. Avoid for quantitative downstream analysis.
- **UMAP**: use as your default non-linear method. Faster, better global structure, and can serve as a preprocessing step for clustering.

11.5.4. R

```
library(Rtsne)
library(uwot)

set.seed(42)

pca_scores <- prcomp(X, scale. = TRUE)$x[, 1:2]
tsne_emb   <- Rtsne(X, perplexity = 30, dims = 2, verbose = FALSE)$Y
umap_emb   <- umap(X, n_neighbors = 15, min_dist = 0.1, verbose = FALSE)

par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
cols <- c("steelblue", "firebrick", "forestgreen")[as.numeric(labels)]

plot(pca_scores, col = cols, pch = 16, cex = 0.8,
     main = "PCA", xlab = "PC1", ylab = "PC2")

plot(tsne_emb, col = cols, pch = 16, cex = 0.8,
     main = "t-SNE (perplexity = 30)", xlab = "t-SNE 1", ylab = "t-SNE 2")
```

11.5. UMAP: Uniform Manifold Approximation and Projection

```
plot(umap_emb, col = cols, pch = 16, cex = 0.8,  
     main = "UMAP (n_neighbors = 15)", xlab = "UMAP 1", ylab = "UMAP 2")
```

11.5.5. Python

```
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.decomposition import PCA  
from sklearn.manifold import TSNE  
from umap import UMAP  
  
pca_2d = PCA(n_components=2).fit_transform(X)  
tsne_2d = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X)  
umap_2d = UMAP(n_components=2, n_neighbors=15, min_dist=0.1,  
               random_state=42).fit_transform(X)  
  
fig, axes = plt.subplots(1, 3, figsize=(16, 5))  
titles = ["PCA", "t-SNE (perplexity=30)", "UMAP (n_neighbors=15)"]  
embeddings = [pca_2d, tsne_2d, umap_2d]  
  
for ax, emb, title in zip(axes, embeddings, titles):  
    for lab, col in colors.items():  
        mask = labels == lab  
        ax.scatter(emb[mask, 0], emb[mask, 1], c=col, s=15, alpha=0.7, label=lab)  
    ax.set_title(title)  
    ax.set_xlabel("Dim 1")  
    ax.set_ylabel("Dim 2")  
    ax.legend(fontsize=8)  
  
plt.tight_layout()  
plt.show()
```

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

All three methods recover the three groups, but with different visual characteristics. PCA shows the groups with some overlap (because the separation is partly non-linear). t-SNE and UMAP separate them more clearly, with UMAP preserving more of the relative distances between clusters.

11.6. Practical Workflow: Scale, Reduce, Visualise, Interpret

A standard workflow for high-dimensional clinical data:

1. **Scale** the data. PCA, t-SNE, and UMAP are all sensitive to the scale of variables. Always standardise (mean = 0, SD = 1) before applying any dimensionality reduction.
2. **Reduce with PCA first.** If the original data have hundreds or thousands of features, first reduce to 20–50 principal components. This removes noise and speeds up t-SNE/UMAP dramatically.
3. **Visualise with UMAP (or t-SNE).** Apply the non-linear method to the PCA-reduced data. Colour points by known labels (diagnosis, treatment group) or by continuous variables (lab values, scores).
4. **Interpret cautiously.** Look for clusters, gradients, or outliers. Generate hypotheses — but remember that these are visualisations, not tests. Validate any apparent structure with formal statistical methods.

11.6.1. R

11.6. Practical Workflow: Scale, Reduce, Visualise, Interpret

```
library(uwot)

set.seed(42)

# Simulate high-dimensional data: 500 patients, 100 features
n <- 500; p <- 100
group <- sample(c("Healthy", "Mild", "Severe"), n, replace = TRUE,
               prob = c(0.4, 0.35, 0.25))

X_hd <- matrix(rnorm(n * p), ncol = p)
# Add signal in first few features
X_hd[group == "Mild", 1:10] <- X_hd[group == "Mild", 1:10] + 1.5
X_hd[group == "Severe", 1:10] <- X_hd[group == "Severe", 1:10] + 3
X_hd[group == "Severe", 11:20] <- X_hd[group == "Severe", 11:20] - 2

# Step 1: Scale
X_scaled <- scale(X_hd)

# Step 2: PCA to 20 components
pca_20 <- prcomp(X_scaled)$x[, 1:20]

# Step 3: UMAP on PCA-reduced data
umap_result <- umap(pca_20, n_neighbors = 15, min_dist = 0.1, verbose = FALSE)

# Step 4: Visualise
plot_df <- tibble(
  UMAP1 = umap_result[, 1],
  UMAP2 = umap_result[, 2],
  Group = factor(group, levels = c("Healthy", "Mild", "Severe"))
)

ggplot(plot_df, aes(x = UMAP1, y = UMAP2, colour = Group)) +
  geom_point(alpha = 0.6, size = 1.5) +
```

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

```
scale_colour_manual(values = c("Healthy" = "steelblue",
                               "Mild" = "goldenrod",
                               "Severe" = "firebrick")) +
labs(title = "UMAP After PCA Reduction",
     subtitle = "500 patients, 100 features → 20 PCs → 2D UMAP") +
theme_minimal(base_size = 13) +
theme(legend.position = "bottom")
```

11.6.2. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from umap import UMAP

np.random.seed(42)
n, p = 500, 100
group = np.random.choice(["Healthy", "Mild", "Severe"], n, p=[0.4, 0.35, 0.25])

X_hd = np.random.normal(0, 1, (n, p))
X_hd[group == "Mild", :10] += 1.5
X_hd[group == "Severe", :10] += 3
X_hd[group == "Severe", 10:20] -= 2

# Workflow
X_scaled = StandardScaler().fit_transform(X_hd)
pca_20 = PCA(n_components=20).fit_transform(X_scaled)
umap_2d = UMAP(n_neighbors=15, min_dist=0.1, random_state=42).fit_transform(pca_20)

fig, ax = plt.subplots(figsize=(8, 6))
color_map = {"Healthy": "steelblue", "Mild": "goldenrod", "Severe": "firebrick"}
```

```

for g in ["Healthy", "Mild", "Severe"]:
    mask = group == g
    ax.scatter(umap_2d[mask, 0], umap_2d[mask, 1],
               c=color_map[g], s=12, alpha=0.6, label=g)
ax.set_xlabel("UMAP 1")
ax.set_ylabel("UMAP 2")
ax.set_title("UMAP After PCA Reduction\n500 patients, 100 features → 20 PCs → 2D")
ax.legend()
plt.tight_layout()
plt.show()

```

11.7. Exercises

11.7.1. Exercise 1: PCA on Clinical Lab Data

Using the simulated metabolic panel data from this chapter (or a real dataset if available):

- Perform PCA on the standardised data.
- Create a scree plot and determine how many components to retain using the cumulative variance threshold (80%) and the Kaiser criterion.
- Examine the loadings of the first two PCs. Which variables load most strongly on each? Propose a clinical interpretation.
- Create a biplot coloured by a simulated diabetes status variable. Do diabetic patients separate along PC1?

11.7.2. R

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

```
# Add diabetes status (correlated with glucose/HbA1c)
set.seed(99)
metabolic$diabetes <- factor(ifelse(metabolic$glucose > 110 & metabolic$hba1c > 10,
                                   "Diabetic", "Non-diabetic"))

pca_fit <- prcomp(metabolic %>% select(-diabetes), scale. = TRUE)

# (a) & (b) Scree plot with cumulative variance
var_prop <- pca_fit$sdev^2 / sum(pca_fit$sdev^2)
cum_var <- cumsum(var_prop)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
barplot(var_prop, names.arg = paste0("PC", 1:8), col = "steelblue",
        main = "Variance Explained", ylab = "Proportion")
abline(h = 1/8, col = "firebrick", lty = 2)

plot(1:8, cum_var, type = "b", pch = 16, col = "steelblue",
     main = "Cumulative Variance", xlab = "Components",
     ylab = "Cumulative proportion", ylim = c(0, 1))
abline(h = 0.80, col = "firebrick", lty = 2)
text(6, 0.82, "80% threshold", col = "firebrick", cex = 0.8)

cat("Components for 80% variance:", which(cum_var >= 0.80)[1], "\n")

# (c) Loadings
cat("\nPC1 loadings:\n")
print(round(sort(pca_fit$rotation[, 1], decreasing = TRUE), 3))
cat("\nPC2 loadings:\n")
print(round(sort(pca_fit$rotation[, 2], decreasing = TRUE), 3))
```

11.7.3. Python

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Recreate metabolic data (using earlier arrays)
diabetes = ["Diabetic" if metabolic.iloc[i]['glucose'] > 110 and
            metabolic.iloc[i]['hba1c'] > 6.2 else "Non-diabetic"
            for i in range(len(metabolic))]

X = metabolic[['glucose', 'hba1c', 'triglycerides', 'ldl', 'hdl',
               'creatinine', 'alt', 'albumin']].values
X_s = StandardScaler().fit_transform(X)
pca = PCA().fit(X_s)

cum_var = np.cumsum(pca.explained_variance_ratio_)
n_80 = np.argmax(cum_var >= 0.80) + 1
print(f"Components for 80% variance: {n_80}")

# Loadings
feat_names = ['glucose', 'hba1c', 'triglycerides', 'ldl', 'hdl',
              'creatinine', 'alt', 'albumin']
for pc in [0, 1]:
    loadings = pca.components_[pc]
    order = np.argsort(np.abs(loadings))[:, :-1]
    print(f"\nPC{pc+1} loadings:")
    for idx in order:
        print(f"  {feat_names[idx]:>15s}: {loadings[idx]:+.3f}")

```

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

11.7.4. Exercise 2: t-SNE Sensitivity to Perplexity

Using the three-group simulated dataset:

- Run t-SNE with perplexity values of 5, 15, 30, and 50.
- Create a 2x2 panel of the resulting embeddings.
- At which perplexity value do the three groups first become clearly separated?
- Are the distances between clusters consistent across perplexity values? What does this tell you about interpreting t-SNE?

11.7.5. Exercise 3: UMAP Parameter Exploration

Using the same dataset:

- Run UMAP with `n_neighbors` in {5, 15, 50, 100} while holding `min_dist` = 0.1.
- Run UMAP with `min_dist` in {0.0, 0.1, 0.5, 1.0} while holding `n_neighbors` = 15.
- Create a panel of plots for each parameter sweep.
- Describe how each parameter affects the visual appearance. Which combination would you recommend for this dataset?

11.7.6. Exercise 4: Full Workflow on Simulated Genomic Data

Simulate a dataset of 1,000 patients with 500 gene expression features and 4 cancer subtypes.

- Scale the data and perform PCA. How many PCs are needed for 80% variance?
- Apply UMAP to the first 30 PCs. Colour by cancer subtype. Are the subtypes visually separable?
- Repeat with t-SNE. Compare the two visualisations.

- d. One of the four subtypes is rare (5% of patients). Can you still identify it in the UMAP plot?

11.8. Summary

Dimensionality reduction transforms high-dimensional clinical data into interpretable, lower-dimensional representations. PCA provides linear, interpretable components that are ideal for initial exploration and pre-processing. t-SNE and UMAP offer non-linear embeddings that reveal complex structure invisible to PCA, though their outputs must be interpreted with care. The standard workflow — scale, reduce with PCA, embed with UMAP, interpret cautiously — is a reliable starting point for any high-dimensional dataset in clinical research.

Key Takeaways

1. Always scale your data before dimensionality reduction.
2. PCA is linear, deterministic, and interpretable. Use it first.
3. t-SNE is for visualisation only. Distances between clusters are meaningless; cluster sizes are meaningless.
4. UMAP is faster, preserves more global structure, and can serve as a preprocessing step for downstream analysis.
5. Non-linear methods are exploratory. Validate any apparent structure with formal statistical methods or domain expertise.

11.9. References and Further Reading

The foundational reference for UMAP is McInnes, Healy, and Melville’s 2018 paper “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” available on arXiv (1802.03426). For t-SNE, van der Maaten and Hinton’s 2008 paper in the *Journal of Machine*

11. PCA, t-SNE, and UMAP: Making Sense of High-Dimensional Data

Learning Research introduced the method, and Wattenberg, Viegas, and Johnson’s 2016 interactive article “How to Use t-SNE Effectively” on Distill.pub remains the best practical guide to understanding its behaviour and pitfalls.

For PCA in a clinical context, Jolliffe and Cadima’s 2016 tutorial “Principal Component Analysis: A Review and Recent Developments” in the *Philosophical Transactions of the Royal Society A* provides a modern overview. Becht et al.’s 2019 paper “Dimensionality Reduction for Visualizing Single-Cell Data Using UMAP” in *Nature Biotechnology* demonstrates the advantages of UMAP over t-SNE in a biological context that generalises well to other high-dimensional clinical data.

A comprehensive 2024 review by Xiang et al. in *BMC Bioinformatics* compares PCA, t-SNE, UMAP, and newer methods (PHATE, TriMap) across multiple benchmarks, providing guidance on method selection for different data types. For the mathematical foundations, Murphy’s *Probabilistic Machine Learning: An Introduction* (2022) covers PCA in depth, while the UMAP documentation (umap-learn.readthedocs.io) includes excellent tutorials with worked examples. James, Witten, Hastie, and Tibshirani’s *An Introduction to Statistical Learning* (2nd edition, 2021) provides an accessible introduction to PCA that is well suited for students with a biostatistics background.

12. Clustering: Discovering Patient Subgroups

12.1. Introduction

Every classification method we have studied so far is **supervised**: you train a model on labelled data and use it to predict labels for new observations. Clustering is fundamentally different. It is **unsupervised** — there are no labels. The goal is to discover natural groupings in the data that you did not know existed.

In clinical research, clustering is used to identify **patient phenotypes** — subgroups of patients who share similar clinical profiles but may not correspond to any established diagnostic category. These phenotypes can reveal heterogeneity within a disease, suggest new treatment targets, or identify patients who respond differently to therapy. Sepsis, ARDS, heart failure, and asthma have all been re-examined through clustering analyses that revealed clinically meaningful subtypes hidden within traditional diagnostic labels.

This chapter covers three major clustering algorithms (K-means, hierarchical clustering, and DBSCAN), methods for choosing the number of clusters and validating results, and the integration of clustering with the dimensionality reduction methods from the previous chapter.

12. Clustering: Discovering Patient Subgroups

! Clusters can be artefacts

Clustering algorithms will always produce clusters, even in completely random data. The existence of clusters in your output does not mean they are real. Validation — both statistical and clinical — is essential.

12.2. K-Means Clustering

12.2.1. The Algorithm

K-means is the simplest and most widely used clustering algorithm. Given k (the number of clusters), it works as follows:

1. **Initialise:** randomly place k cluster centroids in the feature space.
2. **Assign:** assign each data point to the nearest centroid (using Euclidean distance).
3. **Update:** recompute each centroid as the mean of all points assigned to it.
4. **Repeat** steps 2–3 until assignments stop changing (convergence).

K-means minimises the **within-cluster sum of squares (WCSS)**: the total squared distance from each point to its assigned centroid.

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

12.2.2. Choosing K: Elbow Method, Silhouette Scores, Gap Statistic

The most important decision in K-means is the number of clusters k . There is no single correct answer, but several heuristics help.

12.2. K-Means Clustering

Elbow method. Plot WCSS against k . As k increases, WCSS always decreases (more clusters = points closer to centroids). Look for an “elbow” where the rate of decrease sharply changes. The elbow suggests that adding more clusters yields diminishing returns.

Silhouette scores. For each point, the silhouette score measures how much closer it is to its own cluster than to the nearest other cluster. Values range from -1 (misclassified) to $+1$ (well clustered). The average silhouette score across all points summarises clustering quality. Higher is better; choose k that maximises the average silhouette score.

Gap statistic. Compares the WCSS of your data to the WCSS of data drawn from a reference null distribution (uniform). The gap is largest at the k where your data’s structure most exceeds what would be expected by chance.

12.2.3. Limitations of K-Means

- **Assumes spherical, equally sized clusters.** K-means uses Euclidean distance, so it finds globular clusters. Elongated, ring-shaped, or irregularly shaped clusters will be poorly recovered.
- **Sensitive to initialisation.** Different random starting points can give different results. Modern implementations (K-means++) use smart initialisation to mitigate this, and running the algorithm multiple times with different seeds is standard practice.
- **Sensitive to outliers.** A single extreme point can pull a centroid substantially.
- **Requires scaled data.** Features on different scales will dominate the distance calculations. Always standardise before running K-means.

12. Clustering: Discovering Patient Subgroups

12.2.4. Clinical Example: Patient Phenotyping from Lab Data

12.2.5. R

```
library(cluster)

set.seed(42)
n <- 400

# Simulate 3 patient phenotypes with distinct lab profiles
pheno1 <- tibble( # Metabolic syndrome phenotype
  glucose = rnorm(n * 0.35, 140, 20),
  crp      = rnorm(n * 0.35, 5, 2),
  albumin  = rnorm(n * 0.35, 3.5, 0.3),
  wbc      = rnorm(n * 0.35, 8, 2)
)
pheno2 <- tibble( # Inflammatory phenotype
  glucose = rnorm(n * 0.35, 100, 15),
  crp      = rnorm(n * 0.35, 15, 4),
  albumin  = rnorm(n * 0.35, 3.0, 0.4),
  wbc      = rnorm(n * 0.35, 14, 3)
)
pheno3 <- tibble( # Healthy-ish phenotype
  glucose = rnorm(n * 0.30, 90, 10),
  crp      = rnorm(n * 0.30, 2, 1),
  albumin  = rnorm(n * 0.30, 4.2, 0.2),
  wbc      = rnorm(n * 0.30, 7, 1.5)
)

lab_data <- bind_rows(pheno1, pheno2, pheno3)
lab_scaled <- scale(lab_data)

# Elbow and silhouette plots
```

12.2. K-Means Clustering

```
wcss <- numeric(10)
sil_avg <- numeric(10)
for (k in 2:10) {
  km <- kmeans(lab_scaled, centers = k, nstart = 25)
  wcss[k] <- km$tot.withinss
  sil_avg[k] <- mean(silhouette(km$cluster, dist(lab_scaled))[, 3])
}

par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))

# Elbow plot
plot(2:10, wcss[2:10], type = "b", pch = 16, col = "steelblue",
     xlab = "Number of clusters (k)", ylab = "Within-cluster SS",
     main = "Elbow Method")

# Silhouette plot
plot(2:10, sil_avg[2:10], type = "b", pch = 16, col = "firebrick",
     xlab = "Number of clusters (k)", ylab = "Average silhouette",
     main = "Silhouette Scores")

# K-means with k=3, visualise on first 2 PCs
km3 <- kmeans(lab_scaled, centers = 3, nstart = 25)
pca2 <- prcomp(lab_scaled)$x[, 1:2]
plot(pca2, col = c("steelblue", "firebrick", "forestgreen")[km3$cluster],
     pch = 16, cex = 0.7,
     main = "K-means (k=3) on PCA", xlab = "PC1", ylab = "PC2")
```

12.2.6. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

12. Clustering: Discovering Patient Subgroups

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

np.random.seed(42)

# Simulate phenotypes
g1 = np.column_stack([np.random.normal(140, 20, 140),
                      np.random.normal(5, 2, 140),
                      np.random.normal(3.5, 0.3, 140),
                      np.random.normal(8, 2, 140)])
g2 = np.column_stack([np.random.normal(100, 15, 140),
                      np.random.normal(15, 4, 140),
                      np.random.normal(3.0, 0.4, 140),
                      np.random.normal(14, 3, 140)])
g3 = np.column_stack([np.random.normal(90, 10, 120),
                      np.random.normal(2, 1, 120),
                      np.random.normal(4.2, 0.2, 120),
                      np.random.normal(7, 1.5, 120)])

X = np.vstack([g1, g2, g3])
X_scaled = StandardScaler().fit_transform(X)

ks = range(2, 11)
wcss = []
sils = []
for k in ks:
    km = KMeans(n_clusters=k, n_init=25, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
    sils.append(silhouette_score(X_scaled, km.labels_))

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
```

12.2. K-Means Clustering

```
axes[0].plot(list(ks), wcss, "o-", color="steelblue")
axes[0].set_xlabel("Number of clusters (k)")
axes[0].set_ylabel("Within-cluster SS")
axes[0].set_title("Elbow Method")

axes[1].plot(list(ks), sils, "o-", color="firebrick")
axes[1].set_xlabel("Number of clusters (k)")
axes[1].set_ylabel("Average silhouette")
axes[1].set_title("Silhouette Scores")

km3 = KMeans(n_clusters=3, n_init=25, random_state=42).fit(X_scaled)
pca2 = PCA(n_components=2).fit_transform(X_scaled)
colors = ["steelblue", "firebrick", "forestgreen"]
for c in range(3):
    mask = km3.labels_ == c
    axes[2].scatter(pca2[mask, 0], pca2[mask, 1], c=colors[c],
                    s=12, alpha=0.6, label=f"Cluster {c+1}")
axes[2].set_xlabel("PC1")
axes[2].set_ylabel("PC2")
axes[2].set_title("K-means (k=3) on PCA")
axes[2].legend()

plt.tight_layout()
plt.show()
```

Both the elbow and silhouette plots suggest $k = 3$ — consistent with our simulation of three phenotypes. In practice, the signal is rarely this clean, and multiple values of k may appear reasonable. Domain knowledge should guide the final choice.

12.3. Hierarchical Clustering

12.3.1. Agglomerative (Bottom-Up) Clustering

Hierarchical clustering does not require specifying k in advance. The **agglomerative** approach starts with each observation as its own cluster and iteratively merges the closest pair of clusters until all observations belong to a single cluster.

The result is a **dendrogram** — a tree-like diagram that shows the merging history. You choose the number of clusters by “cutting” the dendrogram at a height that produces a meaningful number of groups.

12.3.2. Linkage Methods

The key decision is how to define the distance between two clusters (each containing multiple points):

Single linkage. Distance = minimum distance between any two points in the two clusters. Tends to produce long, chain-like clusters. Sensitive to outliers and noise.

Complete linkage. Distance = maximum distance between any two points. Produces compact, roughly equal-sized clusters but can be distorted by outliers.

Average linkage. Distance = average of all pairwise distances. A compromise between single and complete.

Ward’s method. Merges the pair of clusters that causes the smallest increase in total WCSS. Tends to produce compact, spherical clusters of similar size. **This is usually the best default choice.**

12.3.3. R

```
# Use a smaller subset for readable dendrogram
set.seed(42)
idx <- sample(nrow(lab_scaled), 80)
hc <- hclust(dist(lab_scaled[idx, ]), method = "ward.D2")

# Plot dendrogram
plot(hc, labels = FALSE, main = "Hierarchical Clustering (Ward's Method)",
      xlab = "", sub = "", hang = -1, cex = 0.6)
rect.hclust(hc, k = 3, border = c("steelblue", "firebrick", "forestgreen"))

par(mfrow = c(1, 4), mar = c(2, 2, 3, 1))
methods <- c("single", "complete", "average", "ward.D2")
titles <- c("Single", "Complete", "Average", "Ward's")

for (i in seq_along(methods)) {
  hc_i <- hclust(dist(lab_scaled[idx, ]), method = methods[i])
  plot(hc_i, labels = FALSE, main = titles[i], xlab = "", sub = "",
        hang = -1, cex = 0.5)
  rect.hclust(hc_i, k = 3, border = "firebrick")
}
```

12.3.4. Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

np.random.seed(42)
idx = np.random.choice(len(X_scaled), 80, replace=False)
```

12. Clustering: Discovering Patient Subgroups

```
X_sub = X_scaled[idx]

Z = linkage(X_sub, method='ward')

fig, ax = plt.subplots(figsize=(12, 5))
dendrogram(Z, ax=ax, no_labels=True, color_threshold=Z[-3, 2])
ax.set_title("Hierarchical Clustering (Ward's Method)")
ax.set_xlabel("Patients")
ax.set_ylabel("Distance")
ax.axhline(y=Z[-3, 2], color='firebrick', linestyle='--', linewidth=1)
ax.text(5, Z[-3, 2] + 0.5, "Cut for k=3", color="firebrick", fontsize=10)
plt.tight_layout()
plt.show()
```

```
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
methods = ['single', 'complete', 'average', 'ward']
titles = ['Single', 'Complete', 'Average', "Ward's"]

for ax, method, title in zip(axes, methods, titles):
    Z_i = linkage(X_sub, method=method)
    dendrogram(Z_i, ax=ax, no_labels=True, color_threshold=0)
    ax.set_title(title)
    ax.set_xlabel("")

plt.tight_layout()
plt.show()
```

12.3.5. Reading a Dendrogram

The dendrogram encodes the full merging history:

12.4. DBSCAN: Density-Based Clustering

- The **height** of each horizontal line represents the distance at which two clusters were merged.
- **Long vertical lines** before a merge indicate well-separated clusters.
- **Short vertical lines** indicate merges of similar clusters — less clear separation.

Cutting the dendrogram at different heights gives different numbers of clusters. A large “gap” between merging heights (a long vertical line) suggests a natural number of clusters.

12.4. DBSCAN: Density-Based Clustering

12.4.1. When K-Means Fails

K-means assumes clusters are spherical and roughly equal in size. In clinical data, clusters may be irregularly shaped, overlap, or contain noise (outlier patients who do not belong to any group). **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) addresses these limitations.

12.4.2. How It Works

DBSCAN defines clusters as dense regions separated by sparse regions. Two parameters control its behaviour:

- **eps** (ε): the radius of the neighbourhood around each point.
- **min_samples**: the minimum number of points required within radius ε to form a dense region.

The algorithm classifies each point as:

1. **Core point**: has at least **min_samples** neighbours within radius ε .
2. **Border point**: within ε of a core point but does not have enough neighbours to be core itself.

12. Clustering: Discovering Patient Subgroups

3. **Noise point:** neither core nor border. These are **outliers** — DBSCAN explicitly identifies them.

Clusters are formed by connecting core points that are within ε of each other.

12.4.3. Advantages and Limitations

Advantages:

- Does not require specifying k in advance.
- Can find arbitrarily shaped clusters.
- Identifies outliers explicitly.

Limitations:

- Sensitive to the choice of ε and `min_samples`.
- Struggles when clusters have very different densities.
- Not ideal for very high-dimensional data (use after dimensionality reduction).

12.4.4. R

```
library(dbscan)

set.seed(42)

# Create data with two crescent-shaped clusters + noise
n_each <- 150
theta1 <- seq(0, pi, length.out = n_each)
theta2 <- seq(0, pi, length.out = n_each)

x1 <- cos(theta1) + rnorm(n_each, 0, 0.08)
```

12.4. DBSCAN: Density-Based Clustering

```
y1 <- sin(theta1) + rnorm(n_each, 0, 0.08)
x2 <- 1 - cos(theta2) + rnorm(n_each, 0, 0.08)
y2 <- 1 - sin(theta2) - 0.5 + rnorm(n_each, 0, 0.08)

# Add noise
x_noise <- runif(30, -0.5, 2)
y_noise <- runif(30, -1, 1.5)

crescent_data <- rbind(
  cbind(x1, y1),
  cbind(x2, y2),
  cbind(x_noise, y_noise)
)

# Compare K-means and DBSCAN
km_crescents <- kmeans(crescent_data, centers = 2, nstart = 25)
db_crescents <- dbscan(crescent_data, eps = 0.15, minPts = 5)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
cols_km <- c("steelblue", "firebrick")[km_crescents$cluster]
plot(crescent_data, col = cols_km, pch = 16, cex = 0.7,
     main = "K-means (k=2)", xlab = "x", ylab = "y")

# DBSCAN: cluster 0 = noise (grey), others = colours
db_cols <- c("grey50", "steelblue", "firebrick", "forestgreen")
plot(crescent_data,
     col = db_cols[db_crescents$cluster + 1],
     pch = ifelse(db_crescents$cluster == 0, 4, 16),
     cex = 0.7,
     main = "DBSCAN (eps=0.15)", xlab = "x", ylab = "y")
legend("topright", c("Noise", "Cluster 1", "Cluster 2"),
     col = db_cols[1:3], pch = c(4, 16, 16), cex = 0.8)
```

12.4.5. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN

np.random.seed(42)
n_each = 150

theta1 = np.linspace(0, np.pi, n_each)
theta2 = np.linspace(0, np.pi, n_each)

x1 = np.cos(theta1) + np.random.normal(0, 0.08, n_each)
y1 = np.sin(theta1) + np.random.normal(0, 0.08, n_each)
x2 = 1 - np.cos(theta2) + np.random.normal(0, 0.08, n_each)
y2 = 1 - np.sin(theta2) - 0.5 + np.random.normal(0, 0.08, n_each)

x_noise = np.random.uniform(-0.5, 2, 30)
y_noise = np.random.uniform(-1, 1.5, 30)

X_crescent = np.vstack([
    np.column_stack([x1, y1]),
    np.column_stack([x2, y2]),
    np.column_stack([x_noise, y_noise])
])

km = KMeans(n_clusters=2, n_init=25, random_state=42).fit(X_crescent)
db = DBSCAN(eps=0.15, min_samples=5).fit(X_crescent)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

colors_km = ["steelblue" if c == 0 else "firebrick" for c in km.labels_]
axes[0].scatter(X_crescent[:, 0], X_crescent[:, 1], c=colors_km, s=10)
```

```

axes[0].set_title("K-means (k=2)")

color_map = {-1: "grey", 0: "steelblue", 1: "firebrick"}
colors_db = [color_map.get(c, "forestgreen") for c in db.labels_]
markers = ["x" if c == -1 else "o" for c in db.labels_]
for c_val in set(db.labels_):
    mask = db.labels_ == c_val
    m = "x" if c_val == -1 else "o"
    label = "Noise" if c_val == -1 else f"Cluster {c_val+1}"
    axes[1].scatter(X_crescent[mask, 0], X_crescent[mask, 1],
                    c=color_map.get(c_val, "forestgreen"),
                    marker=m, s=15, label=label)
axes[1].set_title("DBSCAN (eps=0.15)")
axes[1].legend(fontsize=9)

plt.tight_layout()
plt.show()

```

The figure shows a classic example: K-means splits the crescent-shaped clusters incorrectly, while DBSCAN identifies them perfectly and correctly labels the noise points.

12.5. Cluster Validation

12.5.1. Internal Metrics

Internal metrics evaluate clustering quality using only the data itself (no external labels).

Silhouette score. For each point i , define $a(i)$ as the mean distance to other points in the same cluster and $b(i)$ as the mean distance to points in the nearest other cluster. The silhouette score is:

12. Clustering: Discovering Patient Subgroups

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Values range from -1 to $+1$. A high average silhouette score indicates well-separated, compact clusters.

Calinski-Harabasz index. The ratio of between-cluster variance to within-cluster variance (higher is better). It favours compact, well-separated clusters.

12.5.2. Stability-Based Validation

Internal metrics can be misleading — they can be high even for meaningless clusters. **Bootstrap stability** provides a stronger test:

1. Resample the data with replacement many times.
2. Re-run the clustering on each bootstrap sample.
3. Measure how often the same points end up in the same cluster.

If clusters are stable across resamples, they likely reflect real structure. If membership changes dramatically, the clusters may be artefacts.

12.5.3. R

```
library(cluster)

km3 <- kmeans(lab_scaled, centers = 3, nstart = 25)
sil <- silhouette(km3$cluster, dist(lab_scaled))

plot(sil, col = c("steelblue", "firebrick", "forestgreen"),
     main = "Silhouette Plot (k = 3)",
```



```
border = NA)
cat("Average silhouette width:", round(mean(sil[, 3]), 3), "\n")
```

12.5.4. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score, silhouette_samples

km3 = KMeans(n_clusters=3, n_init=25, random_state=42).fit(X_scaled)
sil_vals = silhouette_samples(X_scaled, km3.labels_)
avg_sil = silhouette_score(X_scaled, km3.labels_)

fig, ax = plt.subplots(figsize=(8, 5))
y_lower = 0
colors = ["steelblue", "firebrick", "forestgreen"]
for c in range(3):
    c_sil = np.sort(sil_vals[km3.labels_ == c])
    y_upper = y_lower + len(c_sil)
    ax.barh(range(y_lower, y_upper), c_sil, height=1.0,
            color=colors[c], edgecolor="none")
    y_lower = y_upper + 5

ax.axvline(avg_sil, color="black", linestyle="--",
          label=f"Average: {avg_sil:.3f}")
ax.set_xlabel("Silhouette Coefficient")
ax.set_ylabel("Patients (sorted within cluster)")
ax.set_title("Silhouette Plot (k=3)")
ax.legend()
plt.tight_layout()
plt.show()
```

12.6. Clinical Applications

12.6.1. Patient Phenotyping

The most impactful use of clustering in clinical research is identifying disease subtypes that were previously unrecognised. Notable examples:

- **ARDS phenotypes.** Calfee et al. (2014) applied latent class analysis to two ARDS clinical trial datasets and identified two phenotypes — “hyperinflammatory” and “hypoinflammatory” — with different mortality rates and different responses to treatment. The hyperinflammatory phenotype had higher IL-6, higher vasopressor use, and worse outcomes.
- **Sepsis endotypes.** Seymour et al. (2019) used K-means clustering on EHR data from over 20,000 sepsis patients and identified four phenotypes (alpha, beta, gamma, delta) with distinct organ dysfunction patterns and 28-day mortality rates ranging from 5% to 40%.
- **Heart failure subtypes.** Shah et al. (2015) applied hierarchical clustering to echocardiographic and clinical data from HFpEF patients and identified three phenotypes with different pathophysiology and prognosis.

12.6.2. Disease Subtyping from Lab Panels

A practical workflow for phenotyping:

1. Select clinically relevant variables (labs, vitals, demographics).
2. Handle missing data (imputation or exclusion).
3. Standardise all variables.
4. Reduce dimensionality (PCA to 10–20 components if many variables).
5. Apply multiple clustering algorithms (K-means, hierarchical) and compare.

12.7. Integrating Clustering with PCA/UMAP

6. Validate: silhouette scores, stability analysis, and — most importantly — clinical interpretation.
7. Compare clusters on outcomes (mortality, length of stay, treatment response).

12.7. Integrating Clustering with PCA/UMAP

The standard workflow for high-dimensional clinical data combines dimensionality reduction and clustering:

12.7.1. R

```
library(uwot)
library(cluster)

set.seed(42)

# High-dimensional data: 500 patients, 50 features, 4 subtypes
n <- 500; p <- 50
true_subtype <- sample(1:4, n, replace = TRUE, prob = c(0.3, 0.3, 0.25, 0.15))
X_hd <- matrix(rnorm(n * p), ncol = p)

# Add subtype-specific signals
for (s in 1:4) {
  feature_start <- (s - 1) * 8 + 1
  feature_end <- min(s * 8, p)
  X_hd[true_subtype == s, feature_start:feature_end] <-
    X_hd[true_subtype == s, feature_start:feature_end] + 2.5
}

# Step 1: Scale
```

12. Clustering: Discovering Patient Subgroups

```
X_s <- scale(X_hd)

# Step 2: PCA to 15 components
pca_15 <- prcomp(X_s)$x[, 1:15]

# Step 3: K-means on PCA scores
km4 <- kmeans(pca_15, centers = 4, nstart = 50)

# Step 4: UMAP for visualisation
umap_2d <- umap(pca_15, n_neighbors = 15, min_dist = 0.1, verbose = FALSE)

plot_df <- tibble(
  UMAP1 = umap_2d[, 1],
  UMAP2 = umap_2d[, 2],
  Cluster = factor(km4$cluster),
  True_subtype = factor(true_subtype)
)

p1 <- ggplot(plot_df, aes(x = UMAP1, y = UMAP2, colour = Cluster)) +
  geom_point(alpha = 0.6, size = 1.2) +
  scale_colour_manual(values = c("steelblue", "firebrick",
                                "forestgreen", "goldenrod")) +
  labs(title = "K-means Clusters on UMAP") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

p2 <- ggplot(plot_df, aes(x = UMAP1, y = UMAP2, colour = True_subtype)) +
  geom_point(alpha = 0.6, size = 1.2) +
  scale_colour_manual(values = c("steelblue", "firebrick",
                                "forestgreen", "goldenrod")) +
  labs(title = "True Subtypes on UMAP", colour = "Subtype") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```

```
library(patchwork)
p1 + p2
```

12.7.2. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from umap import UMAP

np.random.seed(42)
n, p = 500, 50
true_sub = np.random.choice(4, n, p=[0.3, 0.3, 0.25, 0.15])
X_hd = np.random.normal(0, 1, (n, p))

for s in range(4):
    start = s * 8
    end = min((s + 1) * 8, p)
    X_hd[true_sub == s, start:end] += 2.5

X_s = StandardScaler().fit_transform(X_hd)
pca_15 = PCA(n_components=15).fit_transform(X_s)
km4 = KMeans(n_clusters=4, n_init=50, random_state=42).fit(pca_15)
umap_2d = UMAP(n_neighbors=15, min_dist=0.1, random_state=42).fit_transform(pca_15)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors_4 = ["steelblue", "firebrick", "forestgreen", "goldenrod"]

for c in range(4):
```

12. Clustering: Discovering Patient Subgroups

```
mask = km4.labels_ == c
axes[0].scatter(umap_2d[mask, 0], umap_2d[mask, 1],
                c=colors_4[c], s=10, alpha=0.6, label=f"Cluster {c+1}")
axes[0].set_title("K-means Clusters on UMAP")
axes[0].legend(fontsize=8)

for s in range(4):
    mask = true_sub == s
    axes[1].scatter(umap_2d[mask, 0], umap_2d[mask, 1],
                   c=colors_4[s], s=10, alpha=0.6, label=f"Subtype {s+1}")
axes[1].set_title("True Subtypes on UMAP")
axes[1].legend(fontsize=8)

plt.tight_layout()
plt.show()
```

Comparing the two panels lets you assess how well the clustering algorithm recovered the true subtypes. In practice you do not know the true subtypes — the comparison would instead be against clinical outcomes, treatment response, or other external validation.

12.8. Cautionary Notes

 Clusters are hypotheses, not facts

Finding three clusters in your ICU dataset does not mean there are three types of ICU patients. It means that your data, with your chosen features, using your chosen algorithm, with your chosen parameters, can be partitioned into three groups. Whether those groups are biologically meaningful requires external validation.

Always validate clinically. Clusters should differ on outcomes, biomarkers, or treatment response — not just on the features used to create them. Clustering on lab values and then showing that the clusters differ on lab values is circular reasoning.

Try multiple algorithms. If K-means, hierarchical clustering, and DBSCAN all find the same groups, you have more confidence that the structure is real. If they disagree, the structure may be fragile.

Report negative results. Many clustering analyses find no meaningful structure. This is a legitimate finding — it means the disease population is more homogeneous than expected.

Beware of overfitting to noise. With enough features, clustering will always find patterns. Use stability analysis and, if possible, validate clusters in an independent dataset.

12.9. Exercises

12.9.1. Exercise 1: K-Means on Simulated Patient Data

Simulate a dataset of 600 patients with 6 clinical variables (heart rate, respiratory rate, temperature, systolic BP, creatinine, lactate) drawn from 3 underlying phenotypes.

- a. Scale the data and apply K-means for $k = 2, 3, 4, 5, 6$.
- b. Create elbow and silhouette plots. Which k appears optimal?
- c. Visualise the $k = 3$ solution using PCA.
- d. Profile the clusters: compute the mean of each variable within each cluster. Do the profiles make clinical sense?

12. Clustering: Discovering Patient Subgroups

12.9.2. R

```
set.seed(42)
n <- 600

# Phenotype 1: Septic shock (high HR, RR, lactate, low SBP)
p1 <- tibble(hr = rnorm(200, 115, 12), rr = rnorm(200, 28, 5),
             temp = rnorm(200, 38.5, 0.8), sbp = rnorm(200, 80, 12),
             creat = rnorm(200, 2.5, 0.8), lactate = rnorm(200, 5, 2))
# Phenotype 2: Stable critical (moderate vitals)
p2 <- tibble(hr = rnorm(200, 90, 10), rr = rnorm(200, 20, 3),
             temp = rnorm(200, 37.2, 0.5), sbp = rnorm(200, 110, 15),
             creat = rnorm(200, 1.2, 0.3), lactate = rnorm(200, 1.5, 0.5))
# Phenotype 3: Febrile, preserved hemodynamics
p3 <- tibble(hr = rnorm(200, 100, 10), rr = rnorm(200, 22, 4),
             temp = rnorm(200, 39.2, 0.7), sbp = rnorm(200, 120, 10),
             creat = rnorm(200, 1.0, 0.2), lactate = rnorm(200, 2.0, 0.8))

dat <- bind_rows(p1, p2, p3)
dat_scaled <- scale(dat)

# Elbow + silhouette
library(cluster)
wcss <- sil <- numeric(6)
for (k in 2:6) {
  km <- kmeans(dat_scaled, k, nstart = 25)
  wcss[k] <- km$tot.withinss
  sil[k] <- mean(silhouette(km$cluster, dist(dat_scaled))[, 3])
}

par(mfrow = c(1, 2))
plot(2:6, wcss[2:6], type = "b", pch = 16, col = "steelblue",
     xlab = "k", ylab = "WCSS", main = "Elbow Method")
```



```

plot(2:6, sil[2:6], type = "b", pch = 16, col = "firebrick",
     xlab = "k", ylab = "Avg Silhouette", main = "Silhouette")

# Profile clusters
km3 <- kmeans(dat_scaled, 3, nstart = 25)
dat$cluster <- km3$cluster
cat("\nCluster profiles (original scale):\n")
dat %>%
  group_by(cluster) %>%
  summarise(across(hr:lactate, mean), .groups = "drop") %>%
  print()

```

12.9.3. Python

```

import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

np.random.seed(42)

p1 = np.column_stack([np.random.normal(115,12,200), np.random.normal(28,5,200),
                      np.random.normal(38.5,0.8,200), np.random.normal(80,12,200),
                      np.random.normal(2.5,0.8,200), np.random.normal(5,2,200)])
p2 = np.column_stack([np.random.normal(90,10,200), np.random.normal(20,3,200),
                      np.random.normal(37.2,0.5,200), np.random.normal(110,15,200),
                      np.random.normal(1.2,0.3,200), np.random.normal(1.5,0.5,200)])
p3 = np.column_stack([np.random.normal(100,10,200), np.random.normal(22,4,200),
                      np.random.normal(39.2,0.7,200), np.random.normal(120,10,200),
                      np.random.normal(1.0,0.2,200), np.random.normal(2.0,0.8,200)])

```

12. Clustering: Discovering Patient Subgroups

```
X = np.vstack([p1, p2, p3])
cols = ['hr', 'rr', 'temp', 'sbp', 'creat', 'lactate']
df = pd.DataFrame(X, columns=cols)
X_s = StandardScaler().fit_transform(X)

for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=25, random_state=42).fit(X_s)
    sil = silhouette_score(X_s, km.labels_)
    print(f"k={k}: WCSS={km.inertia_:.0f}, Silhouette={sil:.3f}")

km3 = KMeans(n_clusters=3, n_init=25, random_state=42).fit(X_s)
df['cluster'] = km3.labels_
print("\nCluster profiles:")
print(df.groupby('cluster')[cols].mean().round(2))
```

12.9.4. Exercise 2: Hierarchical Clustering with Different Linkage Methods

Using the same dataset from Exercise 1:

- Apply hierarchical clustering with single, complete, average, and Ward's linkage.
- Cut each dendrogram at $k = 3$ and compare the resulting cluster assignments.
- Which linkage method produces clusters most similar to the K-means solution? Use the adjusted Rand index (ARI) to quantify agreement.
- Which linkage method would you recommend for clinical data and why?

12.9.5. Exercise 3: DBSCAN for Outlier Detection

Add 30 “outlier” patients to the Exercise 1 dataset: patients with extreme values across multiple variables (e.g., HR = 180, lactate = 15, creatinine = 8).

- a. Run K-means with $k = 3$. Where do the outliers end up?
- b. Run DBSCAN with appropriate `eps` and `min_samples`. How many outliers does it identify?
- c. Compare the cluster assignments for non-outlier patients between K-means and DBSCAN.
- d. In a clinical phenotyping study, which approach would you prefer for handling outliers?

12.9.6. Exercise 4: Full Pipeline — PCA + Clustering + UMAP Visualisation

Simulate a dataset of 800 patients with 30 clinical variables and 4 underlying subtypes.

- a. Standardise the data and apply PCA. Use a scree plot to choose the number of components.
- b. Run K-means ($k = 2, 3, 4, 5$) on the PCA scores. Use silhouette scores to choose k .
- c. Visualise the chosen clustering on a UMAP embedding.
- d. Profile the clusters: compute mean values for each variable and present in a table.
- e. Discuss: how would you validate these clusters in a real clinical study?

12.10. Summary

Clustering is a powerful tool for discovering patient subgroups, but it requires careful application and sceptical interpretation. K-means is simple and effective for spherical clusters but requires specifying k and is sensitive to outliers. Hierarchical clustering provides a dendrogram that reveals the full structure of merging and does not require pre-specifying k . DBSCAN handles irregular cluster shapes and identifies outliers. No single method is best for all situations; using multiple methods and checking for consistency is good practice. Above all, clusters are statistical constructs — they must be validated against clinical outcomes and replicated in independent datasets before they can inform patient care.

Key Takeaways

1. Always scale your data before clustering.
2. K-means is fast and effective but assumes spherical clusters and requires choosing k .
3. Hierarchical clustering (Ward's method) is a solid default that does not require pre-specifying k .
4. DBSCAN excels when clusters are irregularly shaped or when outlier detection matters.
5. Validate clusters with internal metrics (silhouette), stability analysis, and — most importantly — clinical outcomes.
6. Clustering will always find groups, even in random data. The burden of proof is on you to show the groups are meaningful.

12.11. References and Further Reading

The most clinically impactful clustering study in recent critical care literature is Seymour et al.'s 2019 paper “Derivation, Validation, and Potential Treatment Implications of Novel Clinical Phenotypes for Sepsis” in *JAMA*,

12.11. References and Further Reading

which identified four sepsis phenotypes using K-means clustering on electronic health record data. Calfee et al.'s 2014 paper "Subphenotypes in Acute Respiratory Distress Syndrome: Latent Class Analysis of Data From Two Randomised Controlled Trials" in *The Lancet Respiratory Medicine* applied a related method (latent class analysis) to identify ARDS subtypes with differential treatment response. Shah et al.'s 2015 paper in *Circulation* demonstrated the use of hierarchical clustering for heart failure with preserved ejection fraction phenotyping.

For the statistical foundations, Hastie, Tibshirani, and Friedman's *The Elements of Statistical Learning* (2nd edition, 2009) provides comprehensive coverage of K-means, hierarchical clustering, and spectral methods. James, Witten, Hastie, and Tibshirani's *An Introduction to Statistical Learning* (2nd edition, 2021) offers a more accessible treatment suitable for students. The original DBSCAN paper by Ester et al. (1996) is surprisingly readable and remains the standard reference for the algorithm.

For practical guidance, a 2024 review by Rodriguez et al. in *Critical Care Medicine* synthesises the rapidly growing literature on clustering-based phenotyping in critical illness, covering methodological best practices and the gap between statistical clusters and actionable clinical subtypes. Hennig et al.'s *Handbook of Cluster Analysis* (2016) is the most comprehensive reference, covering both methodology and applications. For validation, the *clValid* R package and its accompanying paper by Brock et al. (2008) in the *Journal of Statistical Software* provide tools for comparing clustering algorithms using internal, stability-based, and biological metrics.

Part V.

Introduction to Machine Learning

13. What Is Machine Learning? Core Concepts for Clinical Researchers

13.1. What Machine Learning Is (and Is Not)

You have already done machine learning — you just did not call it that. The dimensionality reduction (PCA, t-SNE, UMAP) and clustering (K-means, hierarchical, DBSCAN) methods from the previous chapters are forms of **unsupervised** machine learning. This part of the course focuses on **supervised** machine learning: algorithms that learn to predict an outcome from labelled examples. We also introduce the vocabulary, workflow, and evaluation framework that the ML community uses, which differs in emphasis from the statistical tradition you have been working in so far.

Machine learning (ML) is a collection of algorithms that learn patterns from data to make predictions or discover structure — without being explicitly programmed with rules. That is the textbook definition. Here is a more practical one for clinical researchers:

Machine learning is pattern recognition at scale. Given enough examples of inputs and outputs, an ML algorithm finds the mapping between them. A logistic regression that predicts 30-day readmission from 5 clinical variables is, technically, machine learning. An algorithm that reads chest X-rays and flags pneumonia is also machine learning. The difference is one of complexity and scale, not of kind.

13.1.1. Common Misconceptions

Let us address several misconceptions head-on:

1. **“ML is always better than regression.”** False. For many clinical prediction tasks with modest sample sizes and well-understood relationships, logistic or Cox regression performs just as well as or better than complex ML models — and is far more interpretable. A 2019 systematic review by Christodoulou et al. in the *Journal of Clinical Epidemiology* found that ML models did not consistently outperform logistic regression for clinical prediction.
2. **“ML finds truth in data automatically.”** False. ML finds patterns — including spurious ones. Without careful validation, feature engineering, and domain knowledge, ML models can learn noise, artifacts, and biases in the data.
3. **“ML doesn’t need assumptions.”** Misleading. ML models may not require linearity or normality assumptions, but they make other assumptions: that the training data is representative of future data, that the features capture the relevant information, that the data is not systematically biased.
4. **“More data always helps.”** Mostly true, but not always. More biased data produces a more confidently wrong model. Data quality matters at least as much as data quantity.
5. **“Deep learning is always the best approach.”** False. For tabular clinical data (the kind in most EHR studies), gradient-boosted trees often outperform deep learning. Deep learning excels with images, text, and time series.

13.1.2. How is ML different from what I already know?

If you have worked through the regression chapters, you already know machine learning — you just did not call it that. Logistic regression is a machine learning algorithm. The difference is mainly one of emphasis:

- **Traditional statistics** focuses on understanding relationships (what is the effect of smoking on lung cancer risk, adjusted for age?).
- **Machine learning** focuses on prediction accuracy (given these 50 variables, what is the probability this patient will be readmitted within 30 days?).

The tools overlap substantially. The new concepts in this chapter — cross-validation, regularisation, ensemble methods — are techniques for improving prediction performance when you have many variables and complex relationships.

13.2. Supervised vs Unsupervised vs Reinforcement Learning

13.2.1. Supervised Learning

In **supervised learning**, the algorithm learns from labeled examples: input-output pairs where the “right answer” is known.

- **Classification:** The output is a category. *Is this skin lesion malignant or benign? Will this patient be readmitted within 30 days?*
- **Regression** (in the ML sense): The output is a continuous value. *What will this patient’s HbA1c be in 6 months? What is the expected length of stay?*

Most clinical prediction models — logistic regression, random forests, neural networks for diagnosis — are supervised learning.

13.2.2. Unsupervised Learning

In **unsupervised learning**, there are no labels. The algorithm discovers structure in the data on its own. You have already encountered the two most important families of unsupervised methods in earlier chapters:

- **Dimensionality reduction** (PCA, t-SNE, UMAP): Compress high-dimensional data into fewer dimensions for visualization or preprocessing. You covered these in Chapter 11.
- **Clustering** (K-means, hierarchical, DBSCAN): Group patients into subgroups based on similarity. You covered these in Chapter 12.
- **Anomaly detection**: Identify unusual observations. *Which patients have lab value patterns that are outliers?*

This part of the course focuses on **supervised learning** — the prediction-oriented methods that build on the regression foundations you already know.

13.2.3. Reinforcement Learning

In **reinforcement learning (RL)**, an agent learns by trial and error, receiving rewards or penalties for actions. While RL has exciting potential in medicine (e.g., optimizing treatment policies for sepsis management or dynamic treatment regimes), it requires large amounts of interaction data and is rarely used in standard clinical research. We mention it for completeness but will not cover it further in this course.

i Where Does Traditional Statistics Fit?

Linear regression, logistic regression, and Cox regression are all forms of supervised learning. The distinction between “statistics” and “machine learning” is largely cultural and historical. Statistics emphasizes inference (understanding relationships, testing hypotheses). ML em-

phasizes prediction (making accurate forecasts on new data). The methods overlap substantially.

13.3. The Bias-Variance Tradeoff

This is arguably the single most important concept in machine learning. Every predictive model navigates a tension between two sources of error:

- **Bias:** Error from oversimplifying the model. A model with high bias misses important patterns. It *underfits* the data.
- **Variance:** Error from making the model too flexible. A model with high variance captures noise as if it were signal. It *overfits* the data.

The total prediction error can be decomposed as:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

13.3.1. A Clinical Analogy

Think of a diagnostic criterion:

- **Too specific** (high bias, low variance): A criterion that requires 5 out of 5 symptoms to diagnose a disease. It misses many true cases (underfitting — the rule is too rigid) but rarely triggers a false alarm.
- **Too sensitive** (low bias, high variance): A criterion that diagnoses the disease if *any* 1 of 5 symptoms is present. It catches nearly every true case but also flags many healthy individuals (overfitting to noise).

The optimal diagnostic criterion balances sensitivity and specificity. Similarly, the optimal ML model balances bias and variance.

13.3.2. Visualizing the Tradeoff

💡 You have already seen bias and variance

In Chapter 4, you modelled non-linear relationships. A straight line through a U-shaped relationship has **high bias** — it systematically misses the true pattern. A spline with 20 knots that follows every wiggle in the training data has **high variance** — it captures noise and will perform poorly on new data. The bias-variance tradeoff is the formal name for this tension you have already experienced.

In practice, we cannot directly observe bias and variance separately. We detect the tradeoff by comparing **training error** (how well the model fits the data it learned from) and **test error** (how well it performs on new data). When training error is low but test error is high, the model is overfitting.

13.4. Training, Validation, and Test Sets

To evaluate how well a model will perform on new, unseen patients, we **must** evaluate it on data it has never seen during training. This is the most important methodological principle in ML.

13.4.1. The Three-Way Split

Set	Purpose	Typical Size
Training set	Fit the model (learn parameters)	60–70%
Validation set	Tune hyperparameters, select among models	15–20%

Set	Purpose	Typical Size
Test set	Final, unbiased performance estimate	15–20%

The **test set must never be used for any decision** — not for feature selection, not for hyperparameter tuning, not for choosing between models. It is opened exactly once, at the very end, to report final performance. If you use the test set to make modeling decisions, it becomes a validation set, and your reported performance will be optimistically biased.

The Cardinal Sin of Machine Learning

Using the test set during model development — and then reporting performance on that same test set — is the ML equivalent of p-hacking. It produces overoptimistic results that will not replicate. In clinical ML, this can mean deploying a model that performs worse in practice than expected, with potential patient harm.

13.4.2. When Data Is Limited

In clinical research, we often have hundreds (not millions) of patients. A single 70/15/15 split wastes data and produces unstable estimates. The solution is **cross-validation**.

13.5. Cross-Validation

Cross-validation (CV) is a resampling strategy that uses all the data for both training and validation, by rotating which portion is held out.

13.5.1. k-Fold Cross-Validation

The most common approach:

1. Randomly split the data into k equally sized **folds** (typically $k = 5$ or $k = 10$).
2. For each fold $i = 1, \dots, k$:
 - Train the model on all folds except fold i .
 - Evaluate on fold i .
3. Average the k performance estimates.

This gives a more stable and less biased estimate of performance than a single train/test split.

13.5.2. Variants

- **Stratified k-fold**: Ensures each fold has the same proportion of events/classes. Essential when the outcome is rare (e.g., 5% readmission rate).
- **Repeated k-fold**: Repeat the entire k-fold procedure m times with different random splits, then average. Reduces variance of the estimate. A common choice is 10-fold CV repeated 5 times.
- **Leave-one-out (LOO)**: $k = n$ (each fold is a single observation). Nearly unbiased but high variance and computationally expensive. Useful only for very small datasets.
- **Grouped/clustered CV**: When observations are not independent (e.g., multiple visits per patient), entire groups must be kept together. Never split a patient's data across training and validation sets.

13.5.2.1. R

```

library(tidymodels)

# Simulate clinical data
set.seed(42)
n <- 500
clin_data <- tibble(
  age = rnorm(n, 60, 12),
  bmi = rnorm(n, 28, 5),
  sbp = rnorm(n, 135, 20),
  glucose = rnorm(n, 110, 30),
  smoking = rbinom(n, 1, 0.25),
  readmit = factor(rbinom(n, 1, 0.15), levels = c("0", "1"),
    labels = c("No", "Yes"))
)

# Create 10-fold CV with stratification
folds <- vfold_cv(clin_data, v = 10, strata = readmit, repeats = 5)
cat("Number of resamples:", nrow(folds), "\n")
cat("Training size per fold:", nrow(training(folds$splits[[1]])), "\n")
cat("Validation size per fold:", nrow(testing(folds$splits[[1]])), "\n")

# Fit logistic regression with cross-validation using tidymodels
log_spec <- logistic_reg() %>%
  set_engine("glm")

log_recipe <- recipe(readmit ~ ., data = clin_data)

log_wf <- workflow() %>%
  add_model(log_spec) %>%
  add_recipe(log_recipe)

```

13. What Is Machine Learning? Core Concepts for Clinical Researchers

```
cv_results <- fit_resamples(log_wf, resamples = folds,
                           metrics = metric_set(roc_auc, accuracy))

collect_metrics(cv_results)
```

13.5.2.2. Python

```
import numpy as np
import pandas as pd
from sklearn.model_selection import (StratifiedKFold, RepeatedStratifiedKFold,
                                     cross_val_score)
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

np.random.seed(42)
n = 500

X = pd.DataFrame({
    'age': np.random.normal(60, 12, n),
    'bmi': np.random.normal(28, 5, n),
    'sbp': np.random.normal(135, 20, n),
    'glucose': np.random.normal(110, 30, n),
    'smoking': np.random.binomial(1, 0.25, n)
})
y = np.random.binomial(1, 0.15, n)

# 10-fold stratified CV, repeated 5 times
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=5, random_state=42)

# Pipeline: scale features, then logistic regression
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
```

```
scores = cross_val_score(pipe, X, y, cv=cv, scoring='roc_auc')
print(f"Mean AUC: {scores.mean():.3f} (+/- {scores.std():.3f})")
print(f"Number of CV iterations: {len(scores)}")
```

13.6. Feature Engineering in Clinical Data

Feature engineering is the process of transforming raw variables into features that are more useful for modeling. In clinical ML, this often means incorporating domain knowledge.

13.6.1. Examples of Clinical Feature Engineering

Raw Data	Engineered Feature	Rationale
Date of birth + visit date	Age at visit	More meaningful than raw dates
Systolic BP, Diastolic BP	Mean arterial pressure, pulse pressure	Physiologically meaningful composites
Serum creatinine, age, sex, race	eGFR (CKD-EPI equation)	Standard clinical measure of kidney function
Multiple lab values over time	Rate of change (slope), variability (SD)	Trajectory matters more than single values
ICD-10 codes	Charlson comorbidity index	Summarizes comorbidity burden
Free-text clinical notes	Extracted symptoms via NLP	Unlocks unstructured data

13. What Is Machine Learning? Core Concepts for Clinical Researchers

Raw Data	Engineered Feature	Rationale
Medication list	Binary flags for drug classes	Captures treatment intent

Domain Knowledge Is Your Superpower

Clinical researchers have a massive advantage in ML: you understand the data. A computer scientist might feed raw creatinine into a model; you know to compute eGFR. A data scientist might treat blood pressure as two independent numbers; you know that pulse pressure has specific physiological meaning. Feature engineering is where clinical expertise directly improves model performance.

13.7. Feature Selection

With electronic health record data, you might have hundreds or thousands of candidate features. Including all of them risks overfitting and reduces interpretability. **Feature selection** identifies the most informative subset.

13.7.1. Three Approaches

- 1. Filter methods:** Rank features by some statistical criterion (correlation with outcome, mutual information, chi-squared test) *before* fitting any model. Fast but ignores feature interactions.
- 2. Wrapper methods:** Iteratively fit models with different feature subsets and select the best-performing set. Examples: forward selection, backward elimination, recursive feature elimination (RFE). More accurate but computationally expensive.

3. Embedded methods: Feature selection happens as part of model fitting. Examples: LASSO regression (L1 penalty shrinks some coefficients to zero), random forest variable importance, elastic net. Often the best practical choice.

13.7.1.1. R

```
library(tidymodels)

# LASSO for feature selection
set.seed(42)
n <- 400
df_feat <- tibble(
  age = rnorm(n, 60, 12),
  bmi = rnorm(n, 28, 5),
  sbp = rnorm(n, 135, 20),
  glucose = rnorm(n, 110, 30),
  smoking = rbinom(n, 1, 0.25),
  noise1 = rnorm(n), # irrelevant
  noise2 = rnorm(n), # irrelevant
  noise3 = rnorm(n), # irrelevant
  outcome = factor(rbinom(n, 1, plogis(-3 + 0.02 * rnorm(n, 60, 12) +
    0.05 * rnorm(n, 28, 5))),
    labels = c("No", "Yes"))
)

# LASSO logistic regression
lasso_spec <- logistic_reg(penalty = 0.01, mixture = 1) %>%
  set_engine("glmnet")

lasso_recipe <- recipe(outcome ~ ., data = df_feat) %>%
  step_normalize(all_numeric_predictors())
```

13. What Is Machine Learning? Core Concepts for Clinical Researchers

```
lasso_wf <- workflow() %>%  
  add_model(lasso_spec) %>%  
  add_recipe(lasso_recipe)  
  
lasso_fit <- fit(lasso_wf, data = df_feat)  
  
# Extract coefficients  
tidy(lasso_fit) %>%  
  filter(term != "(Intercept)") %>%  
  arrange(desc(abs(estimate)))
```

13.7.1.2. Python

```
from sklearn.linear_model import LogisticRegression  
from sklearn.feature_selection import RFE  
from sklearn.preprocessing import StandardScaler  
import pandas as pd  
import numpy as np  
  
np.random.seed(42)  
n = 400  
X = pd.DataFrame({  
    'age': np.random.normal(60, 12, n),  
    'bmi': np.random.normal(28, 5, n),  
    'sbp': np.random.normal(135, 20, n),  
    'glucose': np.random.normal(110, 30, n),  
    'smoking': np.random.binomial(1, 0.25, n),  
    'noise1': np.random.normal(0, 1, n),  
    'noise2': np.random.normal(0, 1, n),  
    'noise3': np.random.normal(0, 1, n)  
})
```

13.8. Support Vector Machines

```
y = np.random.binomial(1, 0.15, n)

# LASSO feature selection
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lasso = LogisticRegression(penalty='l1', solver='saga', C=1.0, max_iter=5000)
lasso.fit(X_scaled, y)

coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lasso.coef_[0]
}).sort_values('Coefficient', key=abs, ascending=False)

print("LASSO Coefficients (features with 0 coefficient are excluded):")
print(coef_df.to_string(index=False))

# Recursive Feature Elimination
rfe = RFE(LogisticRegression(max_iter=5000), n_features_to_select=4)
rfe.fit(X_scaled, y)
selected = X.columns[rfe.support_]
print(f"\nRFE selected features: {list(selected)}")
```

13.8. Support Vector Machines

Support vector machines (SVMs) are a supervised learning method that finds the optimal decision boundary between classes.

13.8.1. The Intuition: Maximum Margin

Imagine plotting patients in a 2D space where the x-axis is age and the y-axis is a biomarker level. Some patients have the disease (red dots), others do not (blue dots). Many possible lines could separate the two groups. SVM finds the line (or hyperplane, in higher dimensions) that **maximizes the margin** — the distance between the boundary and the nearest points from each class.

The nearest points are called **support vectors** — they “support” (define) the boundary. Moving any other point does not change the boundary. This makes SVMs robust to outliers that are far from the boundary.

13.8.2. The Kernel Trick

What if the classes are not linearly separable? The **kernel trick** maps the data into a higher-dimensional space where a linear boundary *can* separate the classes. Common kernels include:

- **Linear**: No transformation (works when data is linearly separable).
- **Radial basis function (RBF)**: Maps to infinite-dimensional space. Very flexible, works well in many settings.
- **Polynomial**: Maps to polynomial feature space.

You do not need to understand the mathematics of kernels to use SVMs effectively. The practical implication is: if a linear SVM does not work well, try an RBF kernel.

13.8.3. When SVMs Are Useful

SVMs work well for:

- Binary classification with moderate sample sizes
- High-dimensional data (many features relative to observations)

13.8. Support Vector Machines

- Clear margin of separation between classes

SVMs are less ideal for:

- Very large datasets (training can be slow)
- Multi-class problems (require one-vs-one or one-vs-rest schemes)
- Interpretability (the model is a “black box” — no simple coefficients)

13.8.3.1. R

```
library(tidymodels)
library(kernlab)

set.seed(42)
n <- 300
svm_data <- tibble(
  x1 = rnorm(n),
  x2 = rnorm(n),
  class = factor(ifelse(x1^2 + x2^2 + rnorm(n, 0, 0.3) > 1.5,
                        "Disease", "Healthy"))
)

# Fit SVM with RBF kernel
svm_spec <- svm_rbf(cost = 1, rbf_sigma = 0.5) %>%
  set_engine("kernlab") %>%
  set_mode("classification")

svm_fit <- svm_spec %>% fit(class ~ ., data = svm_data)

# Evaluate with cross-validation
svm_wf <- workflow() %>%
  add_model(svm_spec) %>%
  add_formula(class ~ .)
```

13. What Is Machine Learning? Core Concepts for Clinical Researchers

```
folds <- vfold_cv(svm_data, v = 5, strata = class)
svm_cv <- fit_resamples(svm_wf, resamples = folds,
                        metrics = metric_set(roc_auc, accuracy))
collect_metrics(svm_cv)
```

13.8.3.2. Python

```
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import numpy as np

np.random.seed(42)
n = 300
x1 = np.random.normal(0, 1, n)
x2 = np.random.normal(0, 1, n)
X = np.column_stack([x1, x2])
y = (x1**2 + x2**2 + np.random.normal(0, 0.3, n) > 1.5).astype(int)

# SVM with RBF kernel
pipe = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=1.0, gamma=0.5))

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring='accuracy')
print(f"SVM accuracy: {scores.mean():.3f} (+/- {scores.std():.3f})")

auc_scores = cross_val_score(
    make_pipeline(StandardScaler(), SVC(kernel='rbf', C=1.0, gamma=0.5,
                                         probability=True)),
    X, y, cv=cv, scoring='roc_auc')
```

13.9. ML vs Traditional Statistics: What Is Actually Different?

```
)  
print(f"SVM AUC: {auc_scores.mean():.3f} (+/- {auc_scores.std():.3f})")
```

13.9. ML vs Traditional Statistics: What Is Actually Different?

This is a question clinical researchers often ask, and the honest answer is: less than you think.

Aspect	Traditional Statistics	Machine Learning
Primary goal	Inference (understand relationships)	Prediction (forecast accurately)
Model choice	Guided by theory and assumptions	Guided by cross-validated performance
Feature selection	Guided by domain knowledge, parsimony	Data-driven, automated
Evaluation	p-values, confidence intervals	AUC, accuracy, calibration, cross-validation
Interpretability	Usually high (coefficients)	Varies (logistic regression = high; deep learning = low)
Sample size	Can work with small samples	Often needs more data for complex models
Assumptions	Explicit (linearity, normality, etc.)	Implicit (representative data, stationarity)

The biggest practical difference is in **workflow**. Statistical modeling typically follows a hypothesis-driven process: specify a model based on

13. What Is Machine Learning? Core Concepts for Clinical Researchers

theory, fit it, check assumptions, interpret coefficients. ML follows a more empirical process: try many models, tune them, evaluate on held-out data, pick the best performer.

Neither approach is inherently superior. For a clinical trial with 200 patients and 5 pre-specified predictors, logistic regression is perfectly appropriate and more interpretable than a random forest. For a hospital system with 500,000 patient records, 1,000 candidate features, and the goal of predicting ICU transfer, a gradient-boosted tree with cross-validated tuning may outperform logistic regression.

13.10. Exercises

13.10.1. Exercise 1: Cross-Validation Experiment

Using the simulated clinical dataset below, compare logistic regression and SVM (RBF kernel) using 10-fold stratified cross-validation. Report AUC for both models. Which performs better? Why?

13.10.1.1. R Starter Code

```
set.seed(123)
n <- 600
ex_data <- tibble(
  age = rnorm(n, 65, 10),
  creatinine = rlnorm(n, 0, 0.5),
  hemoglobin = rnorm(n, 12, 2),
  platelets = rnorm(n, 250, 70),
  wbc = rlnorm(n, 2, 0.4),
  icu = factor(rbinom(n, 1, plogis(-4 + 0.03 * rnorm(n, 65, 10) +
    0.5 * rlnorm(n, 0, 0.5))),
```

```

        labels = c("No", "Yes"))
    )

# Your code: set up tidymodels workflows for logistic_reg and svm_rbf
# Use vfold_cv with strata = icu
# Compare using roc_auc

```

13.10.1.2. Python Starter Code

```

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

np.random.seed(123)
n = 600
# Your code: create X and y
# Compare LogisticRegression vs SVC using cross_val_score with scoring='roc_auc'

```

13.10.2. Exercise 2: Feature Engineering Challenge

You have the following raw variables for a diabetes prediction model:

- height_cm, weight_kg
- sbp, dbp (systolic and diastolic blood pressure)
- fasting_glucose, hba1c
- age, sex
- waist_circumference, hip_circumference
- total_cholesterol, hdl, ldl, triglycerides

13. What Is Machine Learning? Core Concepts for Clinical Researchers

1. List at least 5 clinically meaningful engineered features you could create from these variables. For each, explain *why* it is clinically meaningful.
2. Implement the feature engineering in R (using `recipes`) or Python (using `pandas/sklearn`).
3. Discuss which original features might become redundant after engineering.

13.10.3. Exercise 3: The Bias-Variance Tradeoff in Practice

Using polynomial regression on a simulated dataset:

1. Fit polynomials of degree 1, 3, 5, 10, and 20 to a training set.
2. Plot the fitted curves overlaid on the data.
3. Compute training error and test error for each degree.
4. Identify the degree that minimizes test error. Explain this in terms of the bias-variance tradeoff.

13.10.3.1. R Starter Code

```
set.seed(42)
x_train <- sort(runif(50, 0, 10))
y_train <- sin(x_train) + rnorm(50, 0, 0.3)
x_test  <- sort(runif(200, 0, 10))
y_test  <- sin(x_test) + rnorm(200, 0, 0.3)

# Your code: fit poly(x, degree) for degree in c(1, 3, 5, 10, 20)
# Compute RMSE on train and test for each
# Plot the results
```

13.10.3.2. Python Starter Code

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

np.random.seed(42)
x_train = np.sort(np.random.uniform(0, 10, 50))
y_train = np.sin(x_train) + np.random.normal(0, 0.3, 50)
x_test = np.sort(np.random.uniform(0, 10, 200))
y_test = np.sin(x_test) + np.random.normal(0, 0.3, 200)

# Your code: loop over degrees [1, 3, 5, 10, 20]
# Fit PolynomialFeatures + LinearRegression
# Compute train and test RMSE
```

13.11. Summary

This chapter introduced the foundational concepts of machine learning that every clinical researcher should understand before applying ML methods:

- **ML is pattern recognition**, not magic. It requires careful validation and domain knowledge.
- **Supervised learning** (prediction from labeled data) is the most common ML paradigm in clinical research.
- The **bias-variance tradeoff** governs all model selection: too simple = underfitting, too complex = overfitting.
- **Training/validation/test splits** and **cross-validation** are essential for honest performance evaluation.
- **Feature engineering** — where clinical expertise meets data science — is often more important than model choice.

13. What Is Machine Learning? Core Concepts for Clinical Researchers

- **Feature selection** (filter, wrapper, embedded) prevents overfitting in high-dimensional settings.
- **SVMs** find maximum-margin boundaries and can handle non-linear problems via the kernel trick.
- **ML and statistics are complementary**, not competing. The best clinical researchers use both.

13.12. References and Further Reading

For an integrated treatment of machine learning and statistical methods tailored to clinical and public health researchers, see Smits et al. (2026), particularly Chapter 9, which covers the supervised learning concepts and cross-validation approaches discussed here with additional worked medical examples.

Boehmke and Greenwell's *Hands-On Machine Learning with R* (HOML) provides an excellent practical introduction to ML methods in R, using the tidymodels framework. It covers the bias-variance tradeoff, cross-validation, and feature engineering with clear code examples and is freely available online.

Lgatto's *Introduction to Machine Learning* (IntroML) is a concise, well-organized online resource that covers the core concepts without excessive mathematical detail, making it a good supplement for students new to the field.

Christodoulou et al. (2019), "A Systematic Review Shows No Performance Benefit of Machine Learning over Logistic Regression for Clinical Prediction Models," published in the *Journal of Clinical Epidemiology*, provides important context for when ML methods do and do not outperform traditional approaches — a message every clinical researcher should internalize before defaulting to complex methods.

13.12. References and Further Reading

For support vector machines, the original formulation by Vapnik (1995) in *The Nature of Statistical Learning Theory* (Springer) is the theoretical foundation, though Hastie, Tibshirani, and Friedman's *The Elements of Statistical Learning* (Springer, 2nd edition, 2009) provides a more accessible treatment in Chapters 9 and 12. James et al., *An Introduction to Statistical Learning* (ISLR, 2nd edition, 2021), covers SVMs in Chapter 9 at an introductory level with R lab exercises.

For a thoughtful perspective on the relationship between statistics and machine learning, Breiman's (2001) paper "Statistical Modeling: The Two Cultures" in *Statistical Science* remains essential reading.

14. Decision Trees, Random Forests, and Gradient Boosting

14.1. Decision Trees (CART)

A **decision tree** is perhaps the most intuitive machine learning model. It makes predictions by asking a series of yes/no questions about the input features, splitting the data at each step into increasingly homogeneous groups. The result looks like an inverted tree — hence the name.

14.1.1. How Trees Split

The algorithm works top-down, greedily selecting the best split at each node:

1. Consider every feature and every possible split point.
2. For each candidate split, measure how much it improves the “purity” of the resulting child nodes.
3. Choose the split that maximizes purity improvement.
4. Repeat recursively for each child node until a stopping criterion is met (e.g., minimum node size, maximum depth, or no further improvement).

14.1.2. Splitting Criteria

For **classification trees**, the two most common measures of impurity are:

Gini impurity: For a node with K classes and class proportions $p_1, \dots, p_K$:

$$G = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

Gini impurity is 0 when a node is pure (all one class) and maximized when classes are equally mixed. For a binary problem, the maximum Gini is 0.5 (at $p = 0.5$).

Entropy (information gain):

$$H = - \sum_{k=1}^K p_k \log_2(p_k)$$

Entropy is 0 when the node is pure and maximized at $\log_2(K)$ when classes are equally distributed.

In practice, Gini and entropy produce very similar trees. Gini is the default in most implementations because it is slightly faster to compute (no logarithm).

For **regression trees**, the splitting criterion is typically the **reduction in variance** (or equivalently, the reduction in residual sum of squares).

14.1.3. Clinical Example: Hospital Readmission

Let us build a decision tree to predict 30-day hospital readmission — a common quality metric tied to reimbursement penalties in the United States.

14.1.3.1. R

```
# Simulate hospital readmission data
set.seed(42)
n <- 1000

readmit_data <- tibble(
  age = rnorm(n, 68, 12),
  length_of_stay = rpois(n, 5) + 1,
  num_comorbidities = rpois(n, 3),
  prior_admissions = rpois(n, 1),
  discharge_hemoglobin = rnorm(n, 11, 2),
  discharge_creatinine = rlnorm(n, 0.2, 0.5),
  has_diabetes = rbinom(n, 1, 0.35),
  has_chf = rbinom(n, 1, 0.25),
  readmitted = factor(
    rbinom(n, 1, plogis(-3 + 0.02 * (rnorm(n, 68, 12) - 68) +
      0.15 * rpois(n, 1) +
      0.1 * rpois(n, 3) +
      0.3 * rbinom(n, 1, 0.25) -
      0.1 * rnorm(n, 11, 2))),
    labels = c("No", "Yes")
  )
)

cat("Readmission rate:", mean(readmit_data$readmitted == "Yes"), "\n")
cat("N =", nrow(readmit_data), "\n")
```

14. Decision Trees, Random Forests, and Gradient Boosting

```
# Fit a decision tree
tree_model <- rpart(
  readmitted ~ age + length_of_stay + num_comorbidities +
    prior_admissions + discharge_hemoglobin +
    discharge_creatinine + has_diabetes + has_chf,
  data = readmit_data,
  method = "class",
  control = rpart.control(cp = 0.01, maxdepth = 5, minsplit = 20)
)

# Visualize
rpart.plot(tree_model, type = 4, extra = 106, under = TRUE,
  box.palette = "RdYlGn", roundint = FALSE,
  main = "Decision Tree: 30-Day Readmission")
```

14.1.3.2. Python

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import cross_val_score, StratifiedKFold
import matplotlib.pyplot as plt

np.random.seed(42)
n = 1000

df = pd.DataFrame({
  'age': np.random.normal(68, 12, n),
  'length_of_stay': np.random.poisson(5, n) + 1,
  'num_comorbidities': np.random.poisson(3, n),
```

14.1. Decision Trees (CART)

```
'prior_admissions': np.random.poisson(1, n),
'discharge_hemoglobin': np.random.normal(11, 2, n),
'discharge_creatinine': np.random.lognormal(0.2, 0.5, n),
'has_diabetes': np.random.binomial(1, 0.35, n),
'has_chf': np.random.binomial(1, 0.25, n)
})
y = np.random.binomial(1, 0.18, n)

feature_names = list(df.columns)

# Fit decision tree
tree = DecisionTreeClassifier(max_depth=5, min_samples_split=20,
                             min_samples_leaf=10, random_state=42)
tree.fit(df, y)

fig, ax = plt.subplots(figsize=(16, 8))
plot_tree(tree, feature_names=feature_names, class_names=['No', 'Yes'],
          filled=True, rounded=True, ax=ax, fontsize=8, max_depth=3)
ax.set_title("Decision Tree: 30-Day Readmission")
plt.tight_layout()
plt.show()
```

14.1.4. Reading a Decision Tree

Each node in the tree displays:

- The **splitting rule** (e.g., “age ≥ 72 ”)
- The **predicted class** (the majority class in that node)
- The **class probabilities**
- The **percentage of observations** that reach this node

To classify a new patient, start at the root and follow the branches based on the patient’s feature values until you reach a terminal node (leaf). The

14. Decision Trees, Random Forests, and Gradient Boosting

prediction is the majority class of that leaf.

14.1.5. Pruning: Controlling Tree Complexity

An unpruned tree will grow until every leaf is pure (or nearly so), perfectly memorizing the training data — a textbook case of overfitting. **Pruning** cuts back the tree to reduce complexity.

Two strategies:

1. **Pre-pruning** (early stopping): Set constraints *before* growing the tree — maximum depth, minimum samples per leaf, minimum information gain to split. Simple but can stop too early.
2. **Post-pruning** (cost-complexity pruning): Grow a full tree, then prune branches that provide minimal improvement. The **complexity parameter (cp)** in R's `rpart` controls this: higher cp = more pruning. Select cp via cross-validation.

14.1.5.1. R

```
# View the CP table
printcp(tree_model)

# Plot CV error vs cp
plotcp(tree_model)

# Prune to optimal cp
optimal_cp <- tree_model$cptable[which.min(tree_model$cptable[, "xerror"]), "cp"]
pruned_tree <- prune(tree_model, cp = optimal_cp)

cat("Optimal cp:", optimal_cp, "\n")
```


14.2. Why Single Trees Are Unstable

```
cat("Number of terminal nodes (full):", sum(tree_model$frame$var == "<leaf>"), "\n")
cat("Number of terminal nodes (pruned):", sum(pruned_tree$frame$var == "<leaf>"), "\n")
```

14.1.5.2. Python

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

# Compare trees of different depths
depths = [2, 3, 5, 7, 10, 15, None]
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = []
for d in depths:
    t = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_val_score(t, df, y, cv=cv, scoring='roc_auc')
    depth_label = str(d) if d is not None else "None (full)"
    results.append({'max_depth': depth_label,
                    'mean_auc': scores.mean(),
                    'std_auc': scores.std()})
    print(f"max_depth={depth_label:>6s}: AUC = {scores.mean():.3f} (+/- {scores.std():.3f})")

best = max(results, key=lambda x: x['mean_auc'])
print(f"\nBest: max_depth={best['max_depth']} with AUC={best['mean_auc']:.3f}")
```

14.2. Why Single Trees Are Unstable

Decision trees have a critical weakness: **high variance**. Small changes in the training data can produce a completely different tree. This instability

14. Decision Trees, Random Forests, and Gradient Boosting

means that:

- Predictions are unreliable for individual trees.
- Results may not replicate across datasets.
- Performance on new data is often poor.

The solution is to combine many trees into an **ensemble**. Two main strategies exist: **bagging** (reduce variance by averaging) and **boosting** (reduce bias by learning sequentially).

14.3. Bagging: Bootstrap Aggregating

Bagging (Bootstrap AGGregatING), introduced by Leo Breiman in 1996, reduces variance by:

1. Drawing B bootstrap samples (random samples with replacement) from the training data.
2. Fitting a separate decision tree to each bootstrap sample.
3. Averaging predictions (regression) or taking a majority vote (classification) across all B trees.

By averaging many high-variance, low-bias trees, bagging dramatically reduces the overall variance without increasing bias. The key insight: the variance of an average of B identically distributed random variables decreases with B (provided they are not perfectly correlated).

14.4. Random Forests

Random forests extend bagging with one additional trick: at each split, instead of considering all features, the algorithm randomly selects a **subset of features** (typically $\sqrt{p}$ for classification or $p/3$ for regression, where p is the total number of features).

14.4. Random Forests

This feature randomization **decorrelates the trees**. In bagging without feature randomization, if one feature is very strong, most trees will split on it first, making the trees correlated. Random forests force trees to consider different features, producing more diverse trees whose average is more stable.

14.4.1. Key Hyperparameters

Parameter	What It Controls	Typical Default
<code>num.trees</code> / <code>n_estimators</code>	Number of trees in the forest	500–2000
<code>mtry</code> / <code>max_features</code>	Features considered at each split	$\sqrt{p}$ (classification), $p/3$ (regression)
<code>min.node.size</code> / <code>min_samples_leaf</code>	Minimum observations in a leaf	1 (classification), 5 (regression)
<code>max.depth</code>	Maximum tree depth	Unlimited (let trees grow fully)

14.4.2. Out-of-Bag (OOB) Error

Each bootstrap sample includes about 63% of the original observations (due to sampling with replacement). The remaining 37% — the **out-of-bag** observations — were not used to build that particular tree. Random forests use these OOB observations as a built-in validation set: each observation is predicted by the trees that did *not* include it in their bootstrap sample.

The OOB error is a nearly unbiased estimate of the test error — no separate validation set required. This is one of the most elegant features of random forests.

14. Decision Trees, Random Forests, and Gradient Boosting

14.4.3. Fitting a Random Forest

14.4.3.1. R

```
library(ranger)

# Fit random forest
set.seed(42)
rf_model <- ranger(
  readmitted ~ age + length_of_stay + num_comorbidities +
    prior_admissions + discharge_hemoglobin +
    discharge_creatinine + has_diabetes + has_chf,
  data = readmit_data,
  num.trees = 500,
  mtry = 3,
  min.node.size = 10,
  importance = "impurity",
  probability = TRUE, # predict probabilities
  seed = 42
)

cat("OOB prediction error:", rf_model$prediction.error, "\n")
```

```
# Variable importance
importance_df <- tibble(
  Variable = names(rf_model$variable.importance),
  Importance = rf_model$variable.importance
) %>%
  arrange(desc(Importance))

ggplot(importance_df, aes(x = reorder(Variable, Importance), y = Importance))
  geom_col(fill = "#2E86AB") +
  coord_flip() +
```

```
labs(x = NULL, y = "Importance (Gini impurity reduction)",
     title = "Random Forest Variable Importance") +
theme_minimal(base_size = 14)
```

14.4.3.2. Python

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

rf = RandomForestClassifier(
    n_estimators=500,
    max_features='sqrt',
    min_samples_leaf=10,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)
rf.fit(df, y)

print(f"OOB accuracy: {rf.oob_score_:.3f}")

# Cross-validated AUC
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
auc_scores = cross_val_score(rf, df, y, cv=cv, scoring='roc_auc')
print(f"CV AUC: {auc_scores.mean():.3f} (+/- {auc_scores.std():.3f})")
```

14. Decision Trees, Random Forests, and Gradient Boosting

```
importances = pd.Series(rf.feature_importances_, index=feature_names)
importances = importances.sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(8, 5))
importances.plot(kind='barh', ax=ax, color='#2E86AB')
ax.set_xlabel("Importance (Gini impurity reduction)")
ax.set_title("Random Forest Variable Importance")
plt.tight_layout()
plt.show()
```

14.4.4. Variable Importance: Two Measures

1. **Impurity-based importance** (default): Total reduction in Gini impurity (or variance) from splits on that feature, summed across all trees. Fast but can be biased toward high-cardinality features.
2. **Permutation importance**: For each feature, randomly shuffle its values and measure how much the OOB (or test) error increases. More reliable but slower. This is the recommended approach for inference.

Variable Importance Is Not Causal

A feature with high importance in a random forest is *predictive* of the outcome, not necessarily *causally* related to it. A correlated proxy (e.g., number of medications as a proxy for disease severity) may appear important even though intervening on it would have no effect. Do not confuse prediction with causation.

14.5. Gradient Boosting

While bagging reduces variance by averaging independent trees, **gradient boosting** reduces bias by building trees *sequentially*, where each new tree corrects the mistakes of the previous ensemble.

14.5.1. The Intuition: Learning from Mistakes

1. Fit a simple tree to the data.
2. Compute the **residuals** (errors) from this tree.
3. Fit a new tree to predict these residuals.
4. Add this new tree to the ensemble (scaled by a learning rate).
5. Repeat steps 2–4 for B iterations.

Each new tree focuses on the observations the current ensemble gets wrong. Over many iterations, the ensemble becomes increasingly accurate.

14.5.2. The Mathematics (Brief)

Gradient boosting minimizes a loss function $L(y, \hat{y})$ by gradient descent in function space. At iteration m :

$$\hat{f}_m(x) = \hat{f}_{m-1}(x) + \eta \cdot h_m(x)$$

where $h_m(x)$ is a tree fitted to the negative gradient of the loss (the “pseudo-residuals”) and η is the **learning rate** (also called the shrinkage parameter).

14.5.3. Key Hyperparameters

14. Decision Trees, Random Forests, and Gradient Boosting

Parameter	What It Controls	Typical Range
<code>n_estimators</code> / <code>nrounds</code>	Number of boosting iterations	100–5000
<code>learning_rate</code> / <code>eta</code>	Step size for each tree	0.01–0.3
<code>max_depth</code>	Depth of each tree	3–8 (shallow trees!)
<code>subsample</code>	Fraction of data used per tree	0.5–1.0
<code>colsample_bytree</code>	Fraction of features per tree	0.5–1.0
<code>min_child_weight</code>	Minimum sum of instance weights in a leaf	1–10
<code>reg_alpha</code> (L1) / <code>reg_lambda</code> (L2)	Regularization	0–10



The Learning Rate / Number of Trees Tradeoff

A smaller learning rate requires more trees but generally produces better results. A common strategy: set the learning rate to 0.01–0.05, then tune the number of trees via early stopping (stop when validation error stops improving).

14.5.4. XGBoost and LightGBM

XGBoost (eXtreme Gradient Boosting) by Chen and Guestrin (2016) added regularization, efficient computation, and handling of missing values to gradient boosting, making it the dominant algorithm for tabular data competitions and many clinical prediction tasks.

LightGBM by Microsoft further improves efficiency with gradient-based one-side sampling (GOSS) and exclusive feature bundling. It is often faster than XGBoost on large datasets.

14.5.4.1. R

```

library(xgboost)

# Prepare data for xgboost (requires numeric matrix)
X_train <- readmit_data %>%
  select(age, length_of_stay, num_comorbidities, prior_admissions,
         discharge_hemoglobin, discharge_creatinine, has_diabetes, has_chf) %>%
  as.matrix()

y_train <- as.numeric(readmit_data$readmitted == "Yes")

dtrain <- xgb.DMatrix(data = X_train, label = y_train)

# Fit XGBoost with cross-validation to find optimal nrounds
set.seed(42)
xgb_cv <- xgb.cv(
  params = list(
    objective = "binary:logistic",
    eval_metric = "auc",
    eta = 0.05,
    max_depth = 4,
    subsample = 0.8,
    colsample_bytree = 0.8
  ),
  data = dtrain,
  nrounds = 1000,
  nfold = 5,
  early_stopping_rounds = 50,
  verbose = 0
)

cat("Best iteration:", xgb_cv$best_iteration, "\n")

```

14. Decision Trees, Random Forests, and Gradient Boosting

```
cat("Best CV AUC:", max(xgb_cv$evaluation_log$test_auc_mean), "\n")
```

```
# Fit final model with optimal nrounds
xgb_model <- xgb.train(
  params = list(
    objective = "binary:logistic",
    eval_metric = "auc",
    eta = 0.05,
    max_depth = 4,
    subsample = 0.8,
    colsample_bytree = 0.8
  ),
  data = dtrain,
  nrounds = xgb_cv$best_iteration,
  verbose = 0
)

# Variable importance
importance_matrix <- xgb.importance(model = xgb_model)
xgb.plot.importance(importance_matrix, top_n = 8,
  main = "XGBoost Variable Importance")
```

14.5.4.2. Python

```
import xgboost as xgb
from sklearn.model_selection import cross_val_score, StratifiedKFold
import numpy as np

np.random.seed(42)

xgb_model = xgb.XGBClassifier(
```

14.5. Gradient Boosting

```
n_estimators=1000,
learning_rate=0.05,
max_depth=4,
subsample=0.8,
colsample_bytree=0.8,
eval_metric='auc',
early_stopping_rounds=50,
random_state=42,
use_label_encoder=False
)

# Cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
    xgb.XGBClassifier(
        n_estimators=500, learning_rate=0.05, max_depth=4,
        subsample=0.8, colsample_bytree=0.8,
        random_state=42, use_label_encoder=False, eval_metric='logloss'
    ),
    df, y, cv=cv, scoring='roc_auc'
)
print(f"XGBoost CV AUC: {scores.mean():.3f} (+/- {scores.std():.3f})")

# Fit and plot importance
xgb_final = xgb.XGBClassifier(
    n_estimators=500, learning_rate=0.05, max_depth=4,
    subsample=0.8, colsample_bytree=0.8,
    random_state=42, use_label_encoder=False, eval_metric='logloss'
)
xgb_final.fit(df, y)

fig, ax = plt.subplots(figsize=(8, 5))
xgb.plot_importance(xgb_final, ax=ax, importance_type='gain',
```

14. Decision Trees, Random Forests, and Gradient Boosting

```
                                title='XGBoost Feature Importance (Gain)')  
plt.tight_layout()  
plt.show()
```

14.6. Hyperparameter Tuning

All ensemble methods have hyperparameters that must be tuned. Two common strategies:

14.6.1. Grid Search

Exhaustively try every combination of specified parameter values. Reliable but computationally expensive — the number of combinations grows multiplicatively.

14.6.2. Random Search

Randomly sample hyperparameter combinations. Bergstra and Bengio (2012) showed that random search is more efficient than grid search because not all hyperparameters are equally important. Random search explores more values of the important parameters for the same computational budget.

14.6.2.1. R

```
library(tidymodels)  
  
# Define the model with tunable parameters  
rf_spec <- rand_forest(
```

```

    trees = 500,
    mtry = tune(),
    min_n = tune()
) %>%
  set_engine("ranger") %>%
  set_mode("classification")

# Create recipe
rf_recipe <- recipe(readmitted ~ ., data = readmit_data)

# Workflow
rf_wf <- workflow() %>%
  add_model(rf_spec) %>%
  add_recipe(rf_recipe)

# Define tuning grid
rf_grid <- grid_random(
  mtry(range = c(2, 7)),
  min_n(range = c(5, 30)),
  size = 20
)

# Cross-validation
set.seed(42)
folds <- vfold_cv(readmit_data, v = 5, strata = readmitted)

# Tune (this may take a moment)
rf_tune_results <- tune_grid(
  rf_wf,
  resamples = folds,
  grid = rf_grid,
  metrics = metric_set(roc_auc)
)

```

14. Decision Trees, Random Forests, and Gradient Boosting

```
# Best parameters
show_best(rf_tune_results, metric = "roc_auc", n = 5)

# Select best and finalize
best_params <- select_best(rf_tune_results, metric = "roc_auc")
cat("Best mtry:", best_params$mtry, "\n")
cat("Best min_n:", best_params$min_n, "\n")

final_wf <- finalize_workflow(rf_wf, best_params)
final_fit <- fit(final_wf, data = readmit_data)
```

14.6.2.2. Python

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
import numpy as np

np.random.seed(42)

# Define parameter distributions
param_dist = {
    'n_estimators': [100, 200, 500],
    'max_features': ['sqrt', 'log2', 3, 5],
    'min_samples_leaf': [5, 10, 15, 20, 30],
    'max_depth': [5, 10, 15, 20, None]
}

rf = RandomForestClassifier(random_state=42, n_jobs=-1)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Random search
```

14.7. Comparing Methods: When to Use What

```
search = RandomizedSearchCV(
    rf, param_dist, n_iter=20, cv=cv, scoring='roc_auc',
    random_state=42, n_jobs=-1, verbose=0
)
search.fit(df, y)

print(f"Best AUC: {search.best_score_:.3f}")
print(f"Best parameters: {search.best_params_}")
```

14.7. Comparing Methods: When to Use What

Method	Strengths	Weaknesses	Best For
Single tree	Interpretable, handles non-linearity	Unstable, overfits	Exploration, simple rules
Random forest	Robust, low tuning needed, OOB error	Less interpretable than single tree	General-purpose prediction
Gradient boosting	Often highest accuracy, flexible	More tuning, can overfit, slower	Maximizing prediction accuracy
Logistic regression	Interpretable, well-understood inference	Assumes linearity (without transforms)	Inference, small samples, reporting ORs

i Practical Advice for Clinical Prediction Models

Start with logistic regression as your baseline. If you need better prediction and have enough data (typically hundreds of events), try a random forest. If you want to squeeze out the last bit of per-

formance and are willing to tune carefully, use XGBoost. Always compare against the logistic regression baseline — if the improvement is marginal, prefer the simpler model.

14.8. Clinical Example: Readmission Risk Stratification

Let us bring everything together by comparing all three methods on the hospital readmission dataset.

14.8.0.1. R

```
library(tidymodels)

set.seed(42)
folds <- vfold_cv(readmit_data, v = 10, strata = readmitted)

# Logistic regression
lr_spec <- logistic_reg() %>% set_engine("glm")
lr_wf <- workflow() %>%
  add_model(lr_spec) %>%
  add_recipe(recipe(readmitted ~ ., data = readmit_data))
lr_res <- fit_resamples(lr_wf, resamples = folds,
  metrics = metric_set(roc_auc))

# Random forest
rf_spec <- rand_forest(trees = 500, mtry = 3, min_n = 10) %>%
  set_engine("ranger") %>%
  set_mode("classification")
rf_wf <- workflow() %>%
```


14.8. Clinical Example: Readmission Risk Stratification

```
add_model(rf_spec) %>%
  add_recipe(recipe(readmitted ~ ., data = readmit_data))
rf_res <- fit_resamples(rf_wf, resamples = folds,
                        metrics = metric_set(roc_auc))

# Boosted trees (via xgboost engine in tidymodels)
xgb_spec <- boost_tree(trees = 500, tree_depth = 4, learn_rate = 0.05,
                      min_n = 10) %>%
  set_engine("xgboost") %>%
  set_mode("classification")
xgb_wf <- workflow() %>%
  add_model(xgb_spec) %>%
  add_recipe(recipe(readmitted ~ ., data = readmit_data))
xgb_res <- fit_resamples(xgb_wf, resamples = folds,
                        metrics = metric_set(roc_auc))

# Collect and compare results
bind_rows(
  collect_metrics(lr_res) %>% mutate(model = "Logistic Regression"),
  collect_metrics(rf_res) %>% mutate(model = "Random Forest"),
  collect_metrics(xgb_res) %>% mutate(model = "XGBoost")
) %>%
  select(model, .metric, mean, std_err) %>%
  arrange(desc(mean))
```

14.8.0.2. Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

14. Decision Trees, Random Forests, and Gradient Boosting

```
import numpy as np
import pandas as pd

np.random.seed(42)
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

models = {
    'Logistic Regression': make_pipeline(
        StandardScaler(), LogisticRegression(max_iter=1000)
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=500, max_features='sqrt', min_samples_leaf=10,
        random_state=42, n_jobs=-1
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        n_estimators=500, learning_rate=0.05, max_depth=4,
        subsample=0.8, random_state=42
    )
}

results = []
for name, model in models.items():
    scores = cross_val_score(model, df, y, cv=cv, scoring='roc_auc')
    results.append({
        'Model': name,
        'Mean AUC': f"{scores.mean():.3f}",
        'Std': f"{scores.std():.3f}"
    })
    print(f"{name:25s}: AUC = {scores.mean():.3f} (+/- {scores.std():.3f})")

print(pd.DataFrame(results).to_string(index=False))
```

14.9. Exercises

14.9.1. Exercise 1: Build and Prune a Classification Tree

Using the readmission dataset (or a dataset of your choice):

1. Fit a full, unpruned classification tree. How many terminal nodes does it have?
2. Use cross-validation to find the optimal complexity parameter (cp in R) or maximum depth (in Python).
3. Prune the tree and plot it. How does it compare to the full tree?
4. What are the top 3 splitting variables? Do they make clinical sense?

14.9.1.1. R Starter Code

```
# Fit a full tree (set cp very low to avoid early pruning)
full_tree <- rpart(readmitted ~ ., data = readmit_data, method = "class",
                  control = rpart.control(cp = 0.001))

# Your code:
# 1. Count terminal nodes
# 2. Use printcp() and plotcp() to find optimal cp
# 3. Prune and plot
# 4. Examine variable importance: full_tree$variable.importance
```

14.9.1.2. Python Starter Code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Your code:
```

14. Decision Trees, Random Forests, and Gradient Boosting

```
# 1. Fit trees with different max_depth values
# 2. Cross-validate each
# 3. Plot the optimal tree
# 4. Examine feature_importances_
```

14.9.2. Exercise 2: Random Forest vs XGBoost Tuning Challenge

1. Split the readmission data into 80% training and 20% test sets.
2. Using the training set with 5-fold cross-validation:
 - a. Tune a random forest (try different `mtry`/`max_features` and `min_n`/`min_samples_leaf` values).
 - b. Tune an XGBoost model (try different `learning_rate`, `max_depth`, and `n_estimators` values).
3. Select the best hyperparameters for each model.
4. Evaluate both models on the held-out test set. Which performs better?
5. Create variable importance plots for both models. Do they agree on the most important features?

14.9.2.1. R Starter Code

```
# Split data
set.seed(42)
split <- initial_split(readmit_data, prop = 0.8, strata = readmitted)
train <- training(split)
test <- testing(split)

# Your code: define models with tune(), create grids, run tune_grid()
# Finalize models, predict on test set, compare
```

14.9.2.2. Python Starter Code

```
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb

X_train, X_test, y_train, y_test = train_test_split(
    df, y, test_size=0.2, stratify=y, random_state=42
)

# Your code: define param_dist for RF and XGBoost
# Run RandomizedSearchCV for each
# Evaluate on test set with roc_auc_score
```

14.9.3. Exercise 3: Interpreting a Clinical Prediction Model

You have built a gradient-boosted model for 30-day hospital readmission. Your hospital administration asks you to present the results.

1. Write a non-technical summary (3–4 sentences) explaining what the model does and how well it performs, suitable for a hospital quality committee.
2. The model identifies “number of prior admissions” and “discharge creatinine” as the two most important predictors. Explain to a clinical audience what this means and *does not* mean (recall the caution about variable importance and causation).
3. How would you propose to validate this model before deploying it in clinical practice? What could go wrong if you skip external validation?

14.10. Summary

14. Decision Trees, Random Forests, and Gradient Boosting

Method	How It Works	Key Advantage	Key Risk
Decision tree	Sequential yes/no splits	Highly interpretable	Overfits, unstable
Random forest	Average of many decorrelated trees	Robust, minimal tuning	Less interpretable
Gradient boosting	Sequential trees correcting errors	Often best accuracy	Can overfit, many hyper-parameters

The progression from single trees to random forests to gradient boosting follows a clear logic:

- **Single trees** are interpretable but unstable and prone to overfitting.
- **Random forests** reduce variance by averaging many decorrelated trees, with the OOB error providing a built-in validation estimate.
- **Gradient boosting** reduces both bias and variance through sequential learning, often achieving the best predictive accuracy at the cost of more tuning.

For clinical prediction models, always compare against a logistic regression baseline. Use cross-validation for honest evaluation, tune hyperparameters systematically, and remember that the most complex model is not always the best choice.

14.11. References and Further Reading

The foundational paper on random forests is Breiman (2001), “Random Forests,” published in *Machine Learning*. This paper introduced the algorithm, the concept of out-of-bag error estimation, and permutation-based variable importance. It remains one of the most cited papers in machine learning.

14.11. References and Further Reading

For XGBoost, the key reference is Chen and Guestrin (2016), “XGBoost: A Scalable Tree Boosting System,” presented at KDD 2016. This paper describes the algorithmic innovations (regularized objective, approximate split finding, sparsity-aware computation) that made XGBoost the dominant method for tabular data.

Smits et al. (2026) provide an accessible treatment of tree-based methods in the context of clinical research, including practical guidance on when ensemble methods offer meaningful advantages over logistic regression and how to interpret variable importance in clinical terms.

For a thorough treatment of the CART algorithm, see Breiman, Friedman, Olshen, and Stone’s *Classification and Regression Trees* (1984, Wadsworth). Although the original text is somewhat dated, the core ideas remain current.

Hastie, Tibshirani, and Friedman’s *The Elements of Statistical Learning* (Springer, 2nd edition, 2009) covers bagging (Section 8.7), random forests (Chapter 15), and boosting (Chapter 10) with mathematical rigor. James et al.’s *An Introduction to Statistical Learning* (ISLR, 2nd edition, 2021) provides a gentler introduction in Chapters 8 and covers tree-based methods with R lab exercises.

Boehmke and Greenwell’s *Hands-On Machine Learning with R* dedicates several chapters to tree-based methods with practical tidymodels code, and is an excellent companion for the R exercises in this chapter.

For the gradient boosting framework, Friedman (2001), “Greedy Function Approximation: A Gradient Boosting Machine,” in *Annals of Statistics*, is the foundational statistical treatment. The paper introduced the general gradient boosting framework that XGBoost and LightGBM build upon.

Regarding clinical applications and best practices, the TRIPOD statement (Collins et al., 2015) provides guidelines for reporting clinical prediction models, including those based on machine learning. Any readmission model (or similar clinical prediction tool) developed with these methods should

14. Decision Trees, Random Forests, and Gradient Boosting

be reported following TRIPOD guidelines and externally validated before clinical deployment.

15. Introduction to Deep Learning for Health Research

15.1. When Does Deep Learning Make Sense?

In the previous chapter, you learned that gradient-boosted trees are powerful, flexible, and remarkably effective for clinical prediction on structured data. A natural question follows: when should you reach for something more complex?

The short answer: **deep learning excels when your data has spatial, sequential, or linguistic structure that tabular methods cannot exploit.** If your data lives in a spreadsheet — rows of patients, columns of lab values — gradient-boosted trees will usually match or beat a neural network, with less effort and better interpretability. But if your data is a chest X-ray, a clinical note, an ECG tracing, or a pathology slide, deep learning is the tool that unlocked performance previously thought impossible.

15.1.1. Common Misconceptions

1. **“Deep learning is always better than traditional ML.”** False. For tabular clinical data, a 2022 NeurIPS benchmark by Grinsztajn et al. showed that tree-based models consistently outperform neural networks on typical tabular datasets. A 2026 clinical benchmark confirmed that TabPFN — a transformer designed specifically for

15. Introduction to Deep Learning for Health Research

tabular data — exceeded the best traditional ML model in only 17% of clinical prediction tasks. Start with logistic regression or XGBoost; reach for deep learning only when the data type demands it.

2. **“Deep learning requires millions of examples.”** Misleading. Transfer learning — starting from a model pretrained on millions of images and fine-tuning on your hundreds — has made deep learning practical even with small clinical datasets. Many successful medical imaging studies use fewer than 5,000 labelled examples.
3. **“Neural networks are uninterpretable black boxes.”** Partially true, but increasingly addressable. Techniques such as Grad-CAM (which highlights the image regions driving a prediction) and attention visualization (which shows which words or time steps a model focuses on) provide clinically meaningful explanations. They are not as clean as a regression coefficient, but they are far from opaque.
4. **“I need a GPU cluster to do deep learning.”** Not necessarily. Transfer learning with pretrained models can be done on a single consumer GPU or even in the cloud (Google Colab offers free GPU access). Training a model from scratch on large datasets is another matter entirely.

15.2. How Neural Networks Learn

A neural network is, at its core, a series of matrix multiplications followed by non-linear transformations. If you have worked through logistic regression, you already understand the basic idea: take a weighted sum of inputs and pass them through a sigmoid function to produce a probability. A neural network does exactly the same thing — but stacks many such layers on top of each other, allowing it to learn increasingly abstract representations.

15.2.1. The Building Blocks

Neurons (nodes): Each neuron computes a weighted sum of its inputs, adds a bias, and applies a non-linear function (called an **activation function**):

$$a = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

where x_i are the inputs, w_i are the weights, b is the bias, and f is the activation function.

Layers: Neurons are organized into layers:

- **Input layer:** receives the raw data (pixel values, lab values, word embeddings).
- **Hidden layers:** intermediate layers that learn increasingly abstract features. A network with many hidden layers is called “deep” — hence “deep learning.”
- **Output layer:** produces the final prediction (a probability, a class label, a continuous value).

Activation functions: The non-linear functions that give neural networks their power. Without them, stacking layers would be pointless — the composition of linear functions is still linear. Common choices include:

- **ReLU** (Rectified Linear Unit): $f(x) = \max(0, x)$. The default for hidden layers. Simple, fast, and effective.
- **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$. Used in the output layer for binary classification. You already know this from logistic regression.
- **Softmax:** Generalizes sigmoid to multiple classes. Used in the output layer for multi-class classification.

15.2.2. Training: Gradient Descent and Backpropagation

Neural networks learn by adjusting their weights to minimize a **loss function** — a measure of how wrong the predictions are. The process works as follows:

1. **Forward pass:** Feed data through the network to produce a prediction.
2. **Compute loss:** Compare the prediction to the true label using a loss function (cross-entropy for classification, mean squared error for regression).
3. **Backward pass (backpropagation):** Compute the gradient of the loss with respect to every weight in the network, using the chain rule of calculus.
4. **Update weights:** Adjust each weight in the direction that reduces the loss, scaled by a **learning rate** η :

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

This process repeats over many **epochs** (complete passes through the training data). The learning rate is critical: too large, and the model oscillates; too small, and it converges painfully slowly.



You Already Know This Pattern

Gradient descent is not unique to deep learning. It is the same optimization strategy used in logistic regression and many other statistical models. The difference is scale: a logistic regression with 10 predictors has 11 parameters; a deep neural network may have millions.

15.2.3. Regularization: Preventing Overfitting

Deep networks have enormous capacity and will happily memorize the training data if you let them. The strategies for preventing this parallel what you learned in Chapter 7, but with some additions:

- **Dropout:** During training, randomly “turn off” a fraction of neurons in each layer. This prevents co-adaptation and acts as an implicit ensemble. Typical dropout rates range from 0.2 to 0.5.
- **Weight decay (L2 regularization):** Penalize large weights, exactly as in ridge regression.
- **Early stopping:** Monitor performance on a validation set and stop training when performance begins to deteriorate.
- **Data augmentation:** Artificially expand the training set by applying random transformations (rotations, flips, crops, colour jitter). Especially important in medical imaging, where labelled data is scarce.

15.3. Clinical Example: Neural Network for Readmission Prediction

To connect these concepts to what you already know, let us apply a simple neural network to the hospital readmission data from Chapter 14. This is *not* a scenario where deep learning is the right tool — gradient-boosted trees will likely perform as well or better on tabular data — but it illustrates the mechanics.

15.3.0.1. R

15. Introduction to Deep Learning for Health Research

```
library(keras3)

# Simulate readmission data (same structure as Chapter 8)
set.seed(42)
n <- 1000

readmit_data <- tibble(
  age = rnorm(n, 68, 12),
  length_of_stay = rpois(n, 5) + 1,
  num_comorbidities = rpois(n, 3),
  prior_admissions = rpois(n, 1),
  discharge_hgb = rnorm(n, 11, 2),
  discharge_creatinine = rlnorm(n, 0.2, 0.5),
  has_diabetes = rbinom(n, 1, 0.35),
  has_chf = rbinom(n, 1, 0.25)
)

readmit_prob <- plogis(-3 + 0.02 * (readmit_data$age - 68) +
  0.15 * readmit_data$prior_admissions +
  0.1 * readmit_data$num_comorbidities +
  0.3 * readmit_data$has_chf -
  0.1 * readmit_data$discharge_hgb)
readmit_data$readmitted <- rbinom(n, 1, readmit_prob)

# Prepare data: scale features, split into train/test
x <- readmit_data %>% select(-readmitted) %>% as.matrix()
y <- readmit_data$readmitted

# Standardize
x_mean <- apply(x, 2, mean)
x_sd <- apply(x, 2, sd)
x_scaled <- scale(x, center = x_mean, scale = x_sd)
```

15.3. Clinical Example: Neural Network for Readmission Prediction

```
# Train/test split
set.seed(123)
train_idx <- sample(n, 800)
x_train <- x_scaled[train_idx, ]
x_test  <- x_scaled[-train_idx, ]
y_train <- y[train_idx]
y_test  <- y[-train_idx]

# Define a simple feedforward neural network
model <- keras_model_sequential(input_shape = ncol(x_train)) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dropout(rate = 0.3) %>%
  layer_dense(units = 16, activation = "relu") %>%
  layer_dropout(rate = 0.3) %>%
  layer_dense(units = 1, activation = "sigmoid")

model %>% compile(
  optimizer = optimizer_adam(learning_rate = 0.001),
  loss = "binary_crossentropy",
  metrics = "AUC"
)

# Train with early stopping
history <- model %>% fit(
  x_train, y_train,
  epochs = 50,
  batch_size = 32,
  validation_split = 0.2,
  callbacks = list(
    callback_early_stopping(
      patience = 5, restore_best_weights = TRUE
    )
  ),
)
```

15. Introduction to Deep Learning for Health Research

```
    verbose = 0
)

# Evaluate on test set
results <- model %>% evaluate(x_test, y_test, verbose = 0)
cat("Test loss:", round(results[[1]], 3), "\n")
cat("Test AUC:", round(results[[2]], 3), "\n")
```

15.3.0.2. Python

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Simulate readmission data (same structure as Chapter 8)
np.random.seed(42)
n = 1000

age = np.random.normal(68, 12, n)
length_of_stay = np.random.poisson(5, n) + 1
num_comorbidities = np.random.poisson(3, n)
prior_admissions = np.random.poisson(1, n)
discharge_hgb = np.random.normal(11, 2, n)
discharge_creatinine = np.random.lognormal(0.2, 0.5, n)
has_diabetes = np.random.binomial(1, 0.35, n)
has_chf = np.random.binomial(1, 0.25, n)

X = np.column_stack([age, length_of_stay, num_comorbidities,
                     prior_admissions, discharge_hgb,
                     discharge_creatinine, has_diabetes, has_chf])
```


15.3. Clinical Example: Neural Network for Readmission Prediction

```
prob = 1 / (1 + np.exp(-(-3 + 0.02 * (age - 68) +
                        0.15 * prior_admissions +
                        0.1 * num_comorbidities +
                        0.3 * has_chf -
                        0.1 * discharge_hgb)))
y = np.random.binomial(1, prob)

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=123
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define a simple feedforward neural network
model = models.Sequential([
    layers.Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
    layers.Dropout(0.3),
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=[tf.keras.metrics.AUC(name="auc")]
)

# Train with early stopping
history = model.fit(
    X_train, y_train,
```

15. Introduction to Deep Learning for Health Research

```
epochs=50,  
batch_size=32,  
validation_split=0.2,  
callbacks=[  
    callbacks.EarlyStopping(  
        patience=5, restore_best_weights=True  
    )  
],  
verbose=0  
)  
  
loss, auc = model.evaluate(X_test, y_test, verbose=0)  
print(f"Test loss: {loss:.3f}")  
print(f"Test AUC: {auc:.3f}")
```

i Compare This to Chapter 8

Run the XGBoost model from Chapter 14 on the same data and compare the AUC. You will likely find that XGBoost matches or exceeds the neural network — with less code, less tuning, and immediate access to variable importance. This is the norm for tabular clinical data.

15.4. Key Architectures for Clinical Research

You do not need to understand every layer and parameter to use deep learning effectively. But you do need to understand *which architecture suits which data type* and *why*.

15.4.1. Convolutional Neural Networks (CNNs): Learning from Images

CNNs learn spatial hierarchies of features from images. Early layers detect edges and textures; deeper layers combine these into complex patterns (tumour boundaries, retinal vessel structures, skin lesion shapes).

Instead of connecting every neuron to every input, CNNs use **convolutional filters** — small windows that slide across the image, detecting local patterns. A 3×3 filter might learn to detect a horizontal edge; stacking many filters across many layers builds up a rich feature hierarchy.

Landmark clinical applications:

Application	Key Study	Architecture	Performance
Chest X-ray diagnosis	Rajpurkar et al., 2017	DenseNet-121	Radiologist-level pneumonia detection
Diabetic retinopathy	Gulshan et al., <i>JAMA</i> 2016	Inception-v3	AUC 0.991 for referable DR
Skin cancer classification	Esteva et al., <i>Nature</i> 2017	Inception-v3	Dermatologist-level performance
Pathology	Campanella et al., <i>Nature Medicine</i> 2019	ResNet	Weakly supervised cancer detection in whole-slide images

i From CNNs to Vision Transformers

Classic CNN architectures (ResNet, DenseNet) dominated medical imaging from 2015 to 2021. Since then, **Vision Transformers**

(ViTs) have emerged as strong alternatives in research. ViTs split images into patches, treat each patch as a “token” (analogous to a word in NLP), and use self-attention to model relationships between patches. However, there is an “architectural gap” between research and clinical deployment: as of early 2026, nearly all FDA-cleared radiology AI devices still use CNNs, not transformers or foundation models (*Lancet Digital Health*, 2026). In the research literature, CNNs and ViTs achieve comparable performance on many tasks, and hybrid architectures are increasingly common.

15.4.2. Recurrent Networks and Transformers: Learning from Sequences

Recurrent Neural Networks (RNNs) were designed for sequential data — time series, text, and any data where order matters. The network maintains a **hidden state** updated at each time step, allowing it to retain information from earlier in the sequence.

Long Short-Term Memory (LSTM) networks solve the tendency of standard RNNs to forget long-range dependencies. They use gating mechanisms to control what information to retain, forget, and output at each step. LSTMs were the dominant architecture for clinical time series from roughly 2015 to 2022.

Transformers have largely superseded RNNs and LSTMs. Instead of processing sequences one element at a time, transformers use **self-attention** to relate every element to every other element simultaneously. This parallelism makes them faster to train, and the attention mechanism captures long-range dependencies more effectively. Medformer (NeurIPS 2024) is the current state-of-the-art transformer architecture specifically designed for medical time series classification.

Clinical applications of sequential architectures:

15.4. Key Architectures for Clinical Research

Data Type	Example	Current Preferred Architecture
ECG tracings	Arrhythmia detection, digital biomarkers	Transformers
EEG signals	Seizure detection, ICU monitoring	Transformers with attention-based interpretability
ICU vital signs	Mortality prediction, clinical deterioration	Transformers, temporal CNNs
Wearable sensor data	Activity recognition, gait analysis	Temporal CNNs, hybrid models

15.4.3. Large Language Models: Learning from Clinical Text

The transformer architecture is also the foundation of **large language models (LLMs)**. In clinical research, LLMs and their smaller predecessors have been applied to:

- **Named entity recognition:** Identifying diseases, medications, procedures, and lab values in clinical notes. Models like ClinicalBERT and PubMedBERT (~110M parameters) remain workhorses for structured extraction.
- **Information extraction:** Pulling structured data from pathology and radiology reports.
- **Report generation:** Drafting radiology or pathology reports from imaging studies.

15. Introduction to Deep Learning for Health Research

- **Clinical question answering:** Med-PaLM 2 achieved 86.5% on MedQA; Med-Gemini reached 91.1%. The MedHELM benchmark (*Nature Medicine*, 2025) tested 9 frontier LLMs across 121 medical tasks, finding scores of 0.73–0.85 for clinical note generation but only 0.56–0.72 for clinical decision support.

⚠ LLMs Are Not Clinical Decision-Making Tools (Yet)

LLMs can hallucinate — generating plausible-sounding but factually incorrect medical information. They lack the ability to reason causally about individual patients. Current evidence supports their use as *assistants* (drafting notes, extracting structured data, literature search) rather than as autonomous clinical decision-makers. Always verify LLM outputs against primary sources.

15.4.4. Deep Learning for Survival Analysis

If you worked through Chapter 8, you know that time-to-event data requires special handling. Deep learning has extended survival analysis beyond the Cox model:

Model	Approach	Key Feature
DeepSurv (Katzman et al., 2018)	Neural network within Cox PH framework	Learns non-linear risk functions
DeepHit (Lee et al., 2018)	Custom loss; no parametric assumptions	Handles competing risks directly
SurvTRACE (2024)	Transformer-based	Models competing events with attention
DySurv (<i>JAMIA</i> , 2025)	Conditional variational autoencoder	Dynamic risk from longitudinal EHR data

15.5. Transfer Learning and Foundation Models

For most clinical survival analysis, Cox regression and random survival forests remain the appropriate starting point. Deep survival models become relevant with high-dimensional inputs (imaging, genomics) or complex temporal patterns.

15.5. Transfer Learning and Foundation Models

Transfer learning is the single most important practical technique in deep learning for health research. The idea is simple: take a model that has already learned useful features from a large dataset, and adapt it to your specific task with a much smaller dataset.

15.5.1. How Transfer Learning Works

1. **Start with a pretrained model:** A CNN trained on ImageNet (14 million natural images) has learned to detect edges, textures, shapes, and objects. These features transfer surprisingly well to medical images.
2. **Replace the output layer:** Swap the final classification head with one that predicts your clinical outcome.
3. **Fine-tune:** Train the modified model on your clinical dataset. You can freeze the pretrained layers (fast, works with very small datasets) or fine-tune all layers with a small learning rate (better performance, requires more data).

15.5.2. Foundation Models in Medicine

The field is moving beyond ImageNet toward **domain-specific foundation models** pretrained on medical data:

15. Introduction to Deep Learning for Health Research

Model	Domain	Training Data	Published
MedSAM	Image segmentation	1.57M image-mask pairs, 10 modalities	<i>Nature Communications</i> , 2024
Biomed-CLIP	Vision-language	15M biomedical image-text pairs	Microsoft, 2023
RET-Found	Ophthalmology	1.6M retinal images, self-supervised	<i>Nature</i> , 2023
UNI	Pathology	100K+ whole-slide images	<i>Nature Medicine</i> , 2024
Virchow	Pathology	1.5M whole-slide images	Paige/Microsoft, 2024



Practical Advice for Clinical Researchers

If you want to apply deep learning to medical images, do not train from scratch. Use a pretrained foundation model (MedSAM for segmentation, BiomedCLIP for classification) and fine-tune on your labelled data. With even a few hundred annotated images, you can achieve strong performance. This is the practical path for most clinical research groups.

15.5.3. Transfer Learning in Practice

The following example shows the complete workflow for fine-tuning a pretrained ResNet50 for binary image classification. In practice, you would replace the data loading step with your own clinical images.

15.5. Transfer Learning and Foundation Models

15.5.3.1. R

```
library(keras3)

# Load pretrained ResNet50 (trained on ImageNet, without classification head)
base_model <- application_resnet50(
  weights = "imagenet",
  include_top = FALSE,
  input_shape = c(224, 224, 3)
)
freeze_weights(base_model)

# Add new classification head for binary outcome
model <- keras_model_sequential(input_shape = c(224, 224, 3)) %>%
  base_model() %>%
  layer_global_average_pooling_2d() %>%
  layer_dropout(rate = 0.3) %>%
  layer_dense(units = 1, activation = "sigmoid")

model %>% compile(
  optimizer = optimizer_adam(learning_rate = 1e-4),
  loss = "binary_crossentropy",
  metrics = "AUC"
)

# Data augmentation (rotation, flipping, shifting)
train_datagen <- image_data_generator(
  rescale = 1/255,
  rotation_range = 20,
  horizontal_flip = TRUE,
  validation_split = 0.2
)
```

15. Introduction to Deep Learning for Health Research

```
# Point to your image directory organized as: images/class_0/ and images/class_1/
train_gen <- flow_images_from_directory(
  "path/to/images/", train_datagen,
  target_size = c(224, 224), batch_size = 32,
  class_mode = "binary", subset = "training"
)

val_gen <- flow_images_from_directory(
  "path/to/images/", train_datagen,
  target_size = c(224, 224), batch_size = 32,
  class_mode = "binary", subset = "validation"
)

# Fine-tune with early stopping
history <- model %>% fit(
  train_gen, epochs = 20,
  validation_data = val_gen,
  callbacks = list(
    callback_early_stopping(
      patience = 3, restore_best_weights = TRUE
    )
  )
)
```

15.5.3.2. Python

```
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras import layers, models, callbacks
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Load pretrained ResNet50 (trained on ImageNet, without classification head)
```

15.5. Transfer Learning and Foundation Models

```
base_model = ResNet50(weights="imagenet", include_top=False,
                        input_shape=(224, 224, 3))
base_model.trainable = False

# Add new classification head for binary outcome
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
              loss="binary_crossentropy",
              metrics=[tf.keras.metrics.AUC(name="auc")])

# Data augmentation (rotation, flipping, shifting)
train_datagen = ImageDataGenerator(
    rescale=1.0/255, rotation_range=20,
    horizontal_flip=True, validation_split=0.2
)

# Point to your image directory organized as: images/class_0/ and images/class_1/
train_gen = train_datagen.flow_from_directory(
    "path/to/images/", target_size=(224, 224),
    batch_size=32, class_mode="binary", subset="training"
)

val_gen = train_datagen.flow_from_directory(
    "path/to/images/", target_size=(224, 224),
    batch_size=32, class_mode="binary", subset="validation"
)
```

15. Introduction to Deep Learning for Health Research

```
# Fine-tune with early stopping
history = model.fit(
    train_gen, epochs=20, validation_data=val_gen,
    callbacks=[
        callbacks.EarlyStopping(
            patience=3, restore_best_weights=True
        )
    ]
)
```

15.6. Practical Considerations

15.6.1. When to Use (and Not Use) Deep Learning

Scenario	Recommendation
Tabular EHR data, <50 features	Logistic regression or XGBoost
Tabular data combined with images or text	DL for the unstructured component; consider multimodal fusion
Medical images (X-ray, CT, MRI, pathology)	DL with transfer learning
Clinical notes, radiology reports	Transformer-based models
ECG, EEG, ICU time series	DL is appropriate; consider transformers or temporal CNNs
Survival analysis, standard covariates	Cox regression or random survival forests first

15.6.2. Data Requirements

15.6. Practical Considerations

Approach	Typical Data Needed
Training from scratch	Tens of thousands of labelled examples
Transfer learning (fine-tuning)	Hundreds to low thousands
Foundation model adaptation	As few as 50–100 examples for simple tasks
Self-supervised pretraining	Large amounts of <i>unlabelled</i> data

15.6.3. Compute

Approach	Hardware
Fine-tuning a pretrained model	Single GPU; Google Colab (free) works
Training a moderate model from scratch	1–4 GPUs; cloud instance ~\$2–4/hour
Training a foundation model	GPU cluster; not realistic for most research groups
Running inference	CPU is often sufficient

15.6.4. Reporting Deep Learning Studies

If you publish research involving deep learning, several reporting guidelines apply:

- **TRIPOD+AI** (Collins et al., *BMJ* 2024): 27-item checklist for prediction models using regression or ML. Applies to all prediction model studies.

15. Introduction to Deep Learning for Health Research

- **CLAIM** (Mongan, Moy, and Kahn, *Radiology: AI* 2020): Checklist for AI in Medical Imaging. Covers study design, data, model, evaluation, and discussion.
- **CONSORT-AI / SPIRIT-AI** (Liu et al. and Rivera et al., *Nature Medicine* 2020): Extensions for reporting randomised trials and protocols involving AI.
- **MINIMAR** (Hernandez-Boussard et al., *JAMIA* 2020): Minimum information for medical AI reporting.

15.6.5. Regulatory Context

As of early 2026, the FDA has authorized over 1,350 AI-enabled medical devices, with 76% in radiology. Nearly all are Class II devices cleared via the 510(k) pathway. The EU AI Act, which entered into force in August 2024, classifies AI-enabled medical devices as high-risk, with full compliance obligations from August 2026. If you are developing a model intended for clinical deployment, regulatory awareness is essential from the outset.

15.7. Challenges and Limitations

The External Validation Problem

A systematic review of 86 deep learning algorithms in radiology found that **81% exhibited decreased accuracy on external datasets**, with nearly a quarter experiencing a substantial drop of 0.10 or greater in AUC. Domain shift — differences in patient demographics, imaging equipment, acquisition protocols, and disease prevalence between development and deployment sites — remains the greatest obstacle to clinical translation.

Fairness and bias: A 2024 study in *Nature Medicine* demonstrated that even if a model is optimized for fairness at a single site, fairness does not transfer to out-of-distribution datasets. Subgroup analysis across age, sex, and ethnicity is essential.

Interpretability: A systematic review of 67 studies (2019–2024) found that the addition of explainability methods (Grad-CAM, saliency maps) provided no statistically significant improvement in diagnostic accuracy beyond the AI prediction itself. Interpretability tools are valuable for debugging and trust-building, but they are not a substitute for rigorous external validation.

Temporal degradation: Clinical AI models can degrade over time as clinical practice, coding conventions, and patient populations change. Post-deployment monitoring is essential but rarely implemented.

15.8. Exercises

15.8.1. Exercise 1: Neural Network vs XGBoost on Tabular Data

Using the readmission dataset from this chapter, fit both a neural network and an XGBoost model. Compare their performance using 5-fold cross-validated AUC.

15.8.1.1. R Starter Code

```
library(keras3)
library(tidymodels)
library(xgboost)

# Use the readmit_data from above (or re-simulate it)
set.seed(42)
```

15. Introduction to Deep Learning for Health Research

```
n <- 1000

readmit_data <- tibble(
  age = rnorm(n, 68, 12),
  length_of_stay = rpois(n, 5) + 1,
  num_comorbidities = rpois(n, 3),
  prior_admissions = rpois(n, 1),
  discharge_hgb = rnorm(n, 11, 2),
  discharge_creatinine = rlnorm(n, 0.2, 0.5),
  has_diabetes = rbinom(n, 1, 0.35),
  has_chf = rbinom(n, 1, 0.25)
)

readmit_prob <- plogis(-3 + 0.02 * (readmit_data$age - 68) +
  0.15 * readmit_data$prior_admissions +
  0.1 * readmit_data$num_comorbidities +
  0.3 * readmit_data$has_chf -
  0.1 * readmit_data$discharge_hgb)
readmit_data$readmitted <- factor(rbinom(n, 1, readmit_prob),
  labels = c("No", "Yes"))

# Your code:
# 1. Set up a tidymodels XGBoost workflow with 5-fold CV
# 2. Train a Keras neural network with 5-fold CV (use a loop over folds)
# 3. Compare AUC from both approaches
# 4. Which model performs better? Does this surprise you?
```

15.8.1.2. Python Starter Code

```
import numpy as np
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
```



```

from sklearn.pipeline import make_pipeline
import xgboost as xgb
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from sklearn.metrics import roc_auc_score

np.random.seed(42)
n = 1000

# Re-simulate readmission data (same as above)
# Your code:
# 1. Run 5-fold CV with XGBClassifier using cross_val_score
# 2. Run 5-fold CV with a Keras model (manual loop)
# 3. Compare mean AUC across folds
# 4. Which model performs better? Does this surprise you?

```

15.8.2. Exercise 2: Architecture Matching

For each of the following clinical tasks, identify (a) the most appropriate deep learning architecture, (b) whether deep learning is likely to outperform gradient-boosted trees, and (c) the most relevant reporting guideline. Write 2–3 sentences justifying each answer.

1. Predicting 30-day mortality from 15 structured EHR variables (age, labs, vitals, comorbidities).
2. Classifying skin lesions as benign or malignant from dermoscopy images.
3. Detecting atrial fibrillation from 12-lead ECG tracings.
4. Extracting medication names from unstructured discharge summaries.
5. Predicting length of stay from a combination of structured EHR data and a chest X-ray at admission.

15.8.3. Exercise 3: Critical Appraisal of a Deep Learning Study

Find a recent (2024 or later) paper that applies deep learning to a clinical task in your area of interest. Evaluate it against the CLAIM or TRIPOD+AI checklist:

- 1. Was the model externally validated? If so, how did performance compare to internal validation?
- 2. Were subgroup analyses reported (by age, sex, ethnicity)?
- 3. Was the model compared to a simpler baseline (e.g., logistic regression)?
- 4. Were the training data, code, and model weights made available?
- 5. Based on your assessment, how close is this model to clinical deployment? What would you want to see before trusting it with patient care?

15.9. Summary

Concept	Key Takeaway
When to use deep learning	Images, text, time series, and sequences — not tabular data
Neural network basics	Weighted sums + non-linear activations, stacked in layers
CNNs	Learn spatial features from images via convolutional filters
Transformers	Use self-attention to model relationships in sequences; dominant for NLP and increasingly for imaging
LLMs in medicine	Strong for extraction and generation; not reliable for autonomous decision-making
Transfer learning	Start from pretrained weights; fine-tune on your data

15.10. References and Further Reading

Concept	Key Takeaway
Foundation models	Domain-specific pretrained models (MedSAM, RETFound, UNI) dramatically reduce data requirements
External validation	81% of radiology DL models degrade on external data
Reporting	Use TRIPOD+AI, CLAIM, CONSORT-AI, or MINIMAR as appropriate

15.10. References and Further Reading

Goodfellow IJ, Bengio Y, Courville A. *Deep Learning*. MIT Press, 2016. Freely available at deeplearningbook.org. The definitive theoretical reference. Chapters 6 (deep feedforward networks), 9 (CNNs), and 10 (RNNs) are directly relevant.

Zhang A, Lipton ZC, Li M, Smola AJ. *Dive into Deep Learning*. Cambridge University Press, 2023. Freely available at d2l.ai. An interactive textbook with executable code in PyTorch, TensorFlow, and JAX. More practical and accessible than Goodfellow et al. Adopted at over 500 universities.

Howard J, Gugger S. *Deep Learning for Coders with fastai and PyTorch*. O'Reilly, 2020. Companion to the free fast.ai course. Top-down, coding-first approach ideal for researchers who want to apply deep learning without years of theory.

Grinsztajn L, Oyallon E, Varoquaux G. Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS* 2022. The benchmark study demonstrating that XGBoost consistently outperforms neural networks on tabular datasets. Essential reading before choosing deep learning for structured clinical data.

15. Introduction to Deep Learning for Health Research

Ma J, He Y, Li F, et al. Segment anything in medical images. *Nature Communications* 2024;15:654. The MedSAM paper: a foundation model for universal medical image segmentation trained on 1.57 million image-mask pairs across 10 modalities.

Zhou Y, Chia MA, Wagner SK, et al. A foundation model for generalizable disease detection from retinal images. *Nature* 2023. RET-Found: self-supervised pretraining on 1.6 million retinal images, with strong downstream performance from minimal fine-tuning.

Collins GS, et al. TRIPOD+AI statement. *BMJ* 2024;385:e078378. The current reporting standard for prediction models using regression or machine learning.

Bedi S, Cui H, Fuentes M, et al. MedHELM: Holistic Evaluation of Large Language Models for Medical Tasks. *Nature Medicine* 2025. The most comprehensive LLM evaluation for clinical tasks, testing 9 frontier models across 121 tasks.

Wiegrebe S, et al. Deep learning for survival analysis: a review. *Artificial Intelligence Review*, Springer, 2024. Comprehensive taxonomy covering DeepSurv, DeepHit, neural ODEs, and transformer-based approaches.

For hands-on practice, Stanford CS231n (computer vision) and CS224n (NLP) offer free lecture materials. MIT 6.S191 (Introduction to Deep Learning) at introtodeeplearning.com provides accessible video lectures updated annually.

16. Evaluating Models: Beyond Accuracy

16.1. Introduction

You have built a model. It produces predictions. But how do you know if those predictions are any good? The answer is not as simple as asking “how often is it right?” In clinical medicine, the consequences of different types of errors are rarely symmetric, the outcomes we care about are often rare, and the population we apply a model to may look nothing like the population we trained it on. This chapter introduces the core concepts and tools for evaluating the performance of classification models, with a focus on clinical prediction.

We will move beyond the seductive simplicity of “accuracy” and learn to ask better questions: Does the model separate sick from well? Are the predicted probabilities trustworthy? Would using this model actually help patients? And does it help all patients equally?

16.2. The Accuracy Trap

Consider the following scenario. You are developing a model to predict a rare but serious condition — say, pancreatic cancer — in a primary care population. The prevalence is approximately 0.1%. You build a classifier and proudly report 99.9% accuracy. Your colleague, less impressed, points

16. Evaluating Models: Beyond Accuracy

out that a model which simply predicts “no cancer” for every single patient would also achieve 99.9% accuracy. It would miss every true case, which is the entire point of the exercise, but its accuracy would be nearly perfect.

This is the **accuracy paradox**: when outcomes are imbalanced, accuracy becomes meaningless because it is dominated by the majority class. In clinical medicine, outcomes are almost always imbalanced. Most people do not have the disease you are screening for. Most surgical patients do not die within 30 days. Most pregnancies do not result in pre-eclampsia. A metric that ignores the structure of errors is not useful for clinical decision-making.

! The Golden Rule of Model Evaluation

Never report accuracy alone. Always examine how the model performs separately for those with and without the outcome.

16.3. Sensitivity and Specificity

The most fundamental decomposition of model performance separates its behaviour in two groups: those who truly have the condition and those who do not.

Sensitivity (also called recall or the true positive rate) answers: among everyone who truly has the disease, what proportion did the model correctly identify?

$$\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Specificity (the true negative rate) answers: among everyone who is truly disease-free, what proportion did the model correctly identify as negative?

16.3. Sensitivity and Specificity

$$\text{Specificity} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$$

These two metrics are properties of the **test itself** and do not depend on the prevalence of the disease in the population. This makes them useful for describing a test's intrinsic discriminative ability.

16.3.1. Clinical Context: Screening vs Confirmation

The relative importance of sensitivity and specificity depends on the clinical purpose:

- **Screening tests** (e.g., mammography for breast cancer, the PHQ-2 for depression) prioritise **high sensitivity**. The goal is to catch as many true cases as possible. We accept more false positives because the next step is a confirmatory test, not treatment. Missing a case (false negative) is the costly error.
- **Confirmatory tests** (e.g., biopsy for cancer, Western blot for HIV) prioritise **high specificity**. The goal is to be sure that a positive result is real, because a positive typically triggers treatment, which may carry risks. A false positive leading to unnecessary surgery or chemotherapy is the costly error.

Memory Aid

SnNout: a test with high **Sensitivity**, when **Negative**, rules **out** the disease. **SpPin**: a test with high **Specificity**, when **Positive**, rules **in** the disease.

16.4. Predictive Values: When Prevalence Matters

While sensitivity and specificity describe the test, clinicians and patients need to answer a different question: given my test result, what is the probability I actually have the disease?

Positive Predictive Value (PPV): among everyone who tested positive, what proportion truly has the disease?

$$\text{PPV} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Negative Predictive Value (NPV): among everyone who tested negative, what proportion is truly disease-free?

$$\text{NPV} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Negatives}}$$

Critically, PPV and NPV depend on the **prevalence** of the disease in the population being tested. A test with 95% sensitivity and 95% specificity will have very different PPVs depending on context:

Table 16.1.: PPV and NPV for a test with 95% sensitivity and 95% specificity at different prevalence levels.

Prevalence	PPV	NPV
50%	95.0%	95.0%
10%	67.9%	99.4%
1%	16.1%	99.9%
0.1%	1.9%	100%

At 0.1% prevalence, even with an excellent test, fewer than 2% of positive results are true positives. This is why mass screening for rare diseases

16.4. Predictive Values: When Prevalence Matters

generates so many false alarms, and why testing should be targeted to higher-risk populations whenever possible.

16.4.1. Exercise: Bayes' Theorem in Practice

16.4.1.1. R

```
# Calculate PPV using Bayes' theorem
# PPV = (Sensitivity * Prevalence) /
#       (Sensitivity * Prevalence + (1 - Specificity) * (1 - Prevalence))

calculate_ppv <- function(sensitivity, specificity, prevalence) {
  numerator <- sensitivity * prevalence
  denominator <- numerator + (1 - specificity) * (1 - prevalence)
  return(numerator / denominator)
}

# Example: HIV rapid test (sensitivity = 99.7%, specificity = 99.5%)
# In a general population (prevalence ~ 0.4%)
ppv_general <- calculate_ppv(0.997, 0.995, 0.004)
cat("PPV in general population:", round(ppv_general * 100, 1), "%\n")

# In an STI clinic population (prevalence ~ 5%)
ppv_clinic <- calculate_ppv(0.997, 0.995, 0.05)
cat("PPV in STI clinic:", round(ppv_clinic * 100, 1), "%\n")

# In a population with known exposure (prevalence ~ 30%)
ppv_exposed <- calculate_ppv(0.997, 0.995, 0.30)
cat("PPV with known exposure:", round(ppv_exposed * 100, 1), "%\n")

# Plot PPV across a range of prevalences
prevalences <- seq(0.001, 0.5, by = 0.001)
```

16. Evaluating Models: Beyond Accuracy

```
ppvs <- sapply(prevalences, function(p) calculate_ppv(0.997, 0.995, p))

plot(prevalences * 100, ppvs * 100,
     type = "l", lwd = 2, col = "steelblue",
     xlab = "Prevalence (%)", ylab = "Positive Predictive Value (%)",
     main = "PPV depends heavily on prevalence",
     las = 1)
abline(h = 50, lty = 2, col = "grey50")
```

16.4.1.2. Python

```
import numpy as np
import matplotlib.pyplot as plt

def calculate_ppv(sensitivity, specificity, prevalence):
    """Calculate PPV using Bayes' theorem."""
    numerator = sensitivity * prevalence
    denominator = numerator + (1 - specificity) * (1 - prevalence)
    return numerator / denominator

# Example: HIV rapid test (sensitivity = 99.7%, specificity = 99.5%)
# In a general population (prevalence ~ 0.4%)
ppv_general = calculate_ppv(0.997, 0.995, 0.004)
print(f"PPV in general population: {ppv_general*100:.1f}%")

# In an STI clinic population (prevalence ~ 5%)
ppv_clinic = calculate_ppv(0.997, 0.995, 0.05)
print(f"PPV in STI clinic: {ppv_clinic*100:.1f}%")

# In a population with known exposure (prevalence ~ 30%)
ppv_exposed = calculate_ppv(0.997, 0.995, 0.30)
print(f"PPV with known exposure: {ppv_exposed*100:.1f}%")
```

16.5. The Confusion Matrix

```
# Plot PPV across a range of prevalences
prevalences = np.linspace(0.001, 0.5, 500)
ppvs = [calculate_ppv(0.997, 0.995, p) for p in prevalences]

plt.figure(figsize=(8, 5))
plt.plot(prevalences * 100, np.array(ppvs) * 100, lw=2, color="steelblue")
plt.axhline(y=50, linestyle="--", color="grey", alpha=0.7)
plt.xlabel("Prevalence (%)")
plt.ylabel("Positive Predictive Value (%)")
plt.title("PPV depends heavily on prevalence")
plt.tight_layout()
plt.show()
```

16.5. The Confusion Matrix

A **confusion matrix** is the fundamental bookkeeping device for classification evaluation. For a binary outcome, it is a 2x2 table that cross-classifies predictions against truth:

	Condition Positive	Condition Negative
Predicted Positive	True Positive (TP)	False Positive (FP)
Predicted Negative	False Negative (FN)	True Negative (TN)

Every metric discussed in this chapter can be derived from this table. The crucial insight is that a single model can produce **different** confusion matrices depending on the **threshold** you choose for classifying a predicted probability as “positive” or “negative.”

16. Evaluating Models: Beyond Accuracy

16.5.1. Building and Inspecting Confusion Matrices

16.5.1.1. R

```
library(caret)

# Simulate predicted probabilities and true outcomes
set.seed(42)
n <- 1000
true_outcome <- rbinom(n, 1, 0.15) # 15% prevalence
# Simulate a moderately good model
pred_prob <- plogis(rnorm(n, mean = -1 + 2 * true_outcome, sd = 1.2))

# Confusion matrix at threshold = 0.5
pred_class_50 <- ifelse(pred_prob >= 0.5, 1, 0)
cm_50 <- confusionMatrix(factor(pred_class_50), factor(true_outcome), position="right")
print(cm_50)

# Confusion matrix at threshold = 0.2 (more sensitive)
pred_class_20 <- ifelse(pred_prob >= 0.2, 1, 0)
cm_20 <- confusionMatrix(factor(pred_class_20), factor(true_outcome), position="right")
print(cm_20)

cat("\nAt threshold 0.5:\n")
cat("  Sensitivity:", round(cm_50$byClass["Sensitivity"], 3), "\n")
cat("  Specificity:", round(cm_50$byClass["Specificity"], 3), "\n")

cat("\nAt threshold 0.2:\n")
cat("  Sensitivity:", round(cm_20$byClass["Sensitivity"], 3), "\n")
cat("  Specificity:", round(cm_20$byClass["Specificity"], 3), "\n")
```

16.5.1.2. Python

```

import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
from scipy.special import expit

np.random.seed(42)
n = 1000
true_outcome = np.random.binomial(1, 0.15, n) # 15% prevalence
# Simulate a moderately good model
pred_prob = expit(np.random.normal(-1 + 2 * true_outcome, 1.2))

# Confusion matrix at threshold = 0.5
pred_class_50 = (pred_prob >= 0.5).astype(int)
print("=== Threshold = 0.5 ===")
print(confusion_matrix(true_outcome, pred_class_50))
print(classification_report(true_outcome, pred_class_50, target_names=["Negative", "Positive"]))

# Confusion matrix at threshold = 0.2 (more sensitive)
pred_class_20 = (pred_prob >= 0.2).astype(int)
print("=== Threshold = 0.2 ===")
print(confusion_matrix(true_outcome, pred_class_20))
print(classification_report(true_outcome, pred_class_20, target_names=["Negative", "Positive"]))

```

16.6. ROC Curves

The **Receiver Operating Characteristic (ROC) curve** is perhaps the most widely used graphical tool for evaluating classification models. It plots sensitivity (true positive rate) on the y-axis against 1 minus specificity (false positive rate) on the x-axis, tracing out the trade-off as the classification threshold varies from 1 to 0.

16.6.1. How to Read an ROC Curve

- The **diagonal line** from (0,0) to (1,1) represents a model with no discriminative ability — equivalent to flipping a coin.
- A curve that bows toward the **upper-left corner** indicates good discrimination. The model achieves high sensitivity without sacrificing too much specificity.
- The **Area Under the ROC Curve (AUC or C-statistic)** summarises the curve in a single number. It equals the probability that, given one randomly chosen person with the disease and one without, the model assigns a higher predicted risk to the diseased person.

Table 16.2.: Rough guide to AUC interpretation. These benchmarks are context-dependent.

AUC Range	Interpretation
0.90–1.00	Excellent discrimination
0.80–0.90	Good discrimination
0.70–0.80	Acceptable
0.60–0.70	Poor
0.50–0.60	Fail (near chance)

AUC Is Not Enough

A model can have a high AUC but still produce poorly calibrated probabilities. Two models with the same AUC can have very different clinical utility. AUC tells you about ranking (discrimination), not about the accuracy of the predicted probabilities themselves. Always supplement AUC with calibration assessment (see Chapter 18).

16.6.2. Plotting ROC Curves

16.6.2.1. R

```
library(pROC)

# Using the simulated data from above
set.seed(42)
n <- 1000
true_outcome <- rbinom(n, 1, 0.15)
pred_prob <- plogis(rnorm(n, mean = -1 + 2 * true_outcome, sd = 1.2))

roc_obj <- roc(true_outcome, pred_prob)

# Plot the ROC curve
plot(roc_obj,
     main = paste("ROC Curve (AUC =", round(auc(roc_obj), 3), ")"),
     col = "steelblue", lwd = 2,
     print.auc = TRUE, print.auc.y = 0.4,
     legacy.axes = TRUE) # Use 1-Specificity on x-axis

# Add confidence interval for AUC
ci_auc <- ci.auc(roc_obj)
cat("95% CI for AUC:", round(ci_auc[1], 3), "-", round(ci_auc[3], 3), "\n")

# Find the optimal threshold (Youden's J)
coords_best <- coords(roc_obj, "best", ret = c("threshold", "sensitivity", "specificity"))
cat("Optimal threshold (Youden):", round(coords_best$threshold, 3), "\n")
cat("Sensitivity at optimal:", round(coords_best$sensitivity, 3), "\n")
cat("Specificity at optimal:", round(coords_best$specificity, 3), "\n")
```

16. Evaluating Models: Beyond Accuracy

16.6.2.2. Python

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt
import numpy as np
from scipy.special import expit

np.random.seed(42)
n = 1000
true_outcome = np.random.binomial(1, 0.15, n)
pred_prob = expit(np.random.normal(-1 + 2 * true_outcome, 1.2))

fpr, tpr, thresholds = roc_curve(true_outcome, pred_prob)
auc_score = roc_auc_score(true_outcome, pred_prob)

plt.figure(figsize=(7, 7))
plt.plot(fpr, tpr, color="steelblue", lw=2,
         label=f"Model (AUC = {auc_score:.3f})")
plt.plot([0, 1], [0, 1], color="grey", linestyle="--", label="Chance")
plt.xlabel("False Positive Rate (1 - Specificity)")
plt.ylabel("True Positive Rate (Sensitivity)")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()

# Youden's J statistic to find optimal threshold
j_scores = tpr - fpr
best_idx = np.argmax(j_scores)
print(f"Optimal threshold (Youden): {thresholds[best_idx]:.3f}")
print(f"Sensitivity: {tpr[best_idx]:.3f}")
print(f"Specificity: {1 - fpr[best_idx]:.3f}")
```


16.7. Precision-Recall Curves

When outcomes are highly imbalanced — as they often are in clinical prediction — ROC curves can paint an overly optimistic picture. Because specificity is calculated over the large number of true negatives, even a substantial number of false positives barely moves the false positive rate. The **precision-recall (PR) curve** addresses this by focusing entirely on the positive class.

- **Precision** (= PPV): of all predicted positives, how many are truly positive?
- **Recall** (= sensitivity): of all true positives, how many were detected?

The PR curve plots precision on the y-axis against recall on the x-axis. A good model maintains high precision even as recall increases. The baseline for a PR curve is a horizontal line at the prevalence level (not the diagonal, as for ROC).

The **Area Under the Precision-Recall Curve (AUPRC)** is a useful summary, particularly for rare outcomes. While a model may boast an AUC-ROC of 0.95, its AUPRC might reveal that achieving high recall requires accepting very low precision.

16.7.1. Exercise: Comparing ROC and PR Curves

16.7.1.1. R

```
library(PRRROC)

# Simulate an imbalanced dataset (2% prevalence)
set.seed(123)
n <- 5000
true_outcome <- rbinom(n, 1, 0.02)
```

16. Evaluating Models: Beyond Accuracy

```
pred_prob <- plogis(rnorm(n, mean = -2 + 3 * true_outcome, sd = 1.5))

# PR curve
pr <- pr.curve(scores.class0 = pred_prob[true_outcome == 1],
               scores.class1 = pred_prob[true_outcome == 0],
               curve = TRUE)
plot(pr, main = paste("Precision-Recall Curve\nAUPRC =", round(pr$auc.integral, 3)))

# For comparison, the ROC curve
roc <- roc.curve(scores.class0 = pred_prob[true_outcome == 1],
                 scores.class1 = pred_prob[true_outcome == 0],
                 curve = TRUE)
plot(roc, main = paste("ROC Curve\nAUROC =", round(roc$auc, 3)))

cat("Notice how the AUROC looks excellent but the AUPRC reveals\n")
cat("the difficulty of achieving high precision with rare outcomes.\n")
```

16.7.1.2. Python

```
from sklearn.metrics import (precision_recall_curve, average_precision_score,
                             roc_curve, roc_auc_score)
import matplotlib.pyplot as plt
import numpy as np
from scipy.special import expit

np.random.seed(123)
n = 5000
true_outcome = np.random.binomial(1, 0.02, n) # 2% prevalence
pred_prob = expit(np.random.normal(-2 + 3 * true_outcome, 1.5))

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

16.8. Choosing the Threshold: A Clinical Decision

```
# ROC curve
fpr, tpr, _ = roc_curve(true_outcome, pred_prob)
auroc = roc_auc_score(true_outcome, pred_prob)
axes[0].plot(fpr, tpr, color="steelblue", lw=2)
axes[0].plot([0, 1], [0, 1], "--", color="grey")
axes[0].set_title(f"ROC Curve (AUROC = {auroc:.3f})")
axes[0].set_xlabel("False Positive Rate")
axes[0].set_ylabel("True Positive Rate")

# PR curve
precision, recall, _ = precision_recall_curve(true_outcome, pred_prob)
auprc = average_precision_score(true_outcome, pred_prob)
prevalence = true_outcome.mean()
axes[1].plot(recall, precision, color="darkorange", lw=2)
axes[1].axhline(y=prevalence, linestyle="--", color="grey", label=f"Baseline (prevalence={prevalence:.3f})")
axes[1].set_title(f"Precision-Recall Curve (AUPRC = {auprc:.3f})")
axes[1].set_xlabel("Recall (Sensitivity)")
axes[1].set_ylabel("Precision (PPV)")
axes[1].legend()

plt.tight_layout()
plt.show()

print("The AUROC looks impressive, but AUPRC reveals the real challenge.")
```

16.8. Choosing the Threshold: A Clinical Decision

A prediction model typically outputs a probability. To make a binary decision (treat or don't treat, refer or don't refer), you must choose a **threshold**. This is fundamentally a **clinical** decision, not a statistical one.

16. Evaluating Models: Beyond Accuracy

Consider a model predicting 30-day mortality after surgery:

- If the intervention for high-risk patients is simply closer monitoring (low cost, minimal harm), you might choose a **low threshold** (e.g., 5%), accepting many false positives to catch as many at-risk patients as possible.
- If the intervention is a risky re-operation, you might choose a **high threshold** (e.g., 30%), requiring strong evidence before acting.

The optimal threshold depends on the **relative costs** of false positives and false negatives, which are determined by the clinical context, not by the data.

i Youden's Index Is Not Always Optimal

Youden's J statistic (sensitivity + specificity - 1) finds the threshold that maximises the sum of sensitivity and specificity. This implicitly assumes equal costs for false positives and false negatives — an assumption that is rarely true in medicine. Use Youden's index as a starting point, but always adjust based on clinical reasoning.

16.8.1. The Threshold-Performance Trade-off

16.8.1.1. R

```
library(pROC)

set.seed(42)
n <- 1000
true_outcome <- rbinom(n, 1, 0.15)
pred_prob <- plogis(rnorm(n, mean = -1 + 2 * true_outcome, sd = 1.2))

roc_obj <- roc(true_outcome, pred_prob)
```

16.8. Choosing the Threshold: A Clinical Decision

```
# Extract sensitivity and specificity at various thresholds
thresholds <- seq(0.05, 0.95, by = 0.05)
results <- data.frame(
  threshold = thresholds,
  sensitivity = sapply(thresholds, function(t) {
    coords(roc_obj, t, input = "threshold", ret = "sensitivity")
  }),
  specificity = sapply(thresholds, function(t) {
    coords(roc_obj, t, input = "threshold", ret = "specificity")
  })
)

# Plot the trade-off
plot(results$threshold, results$sensitivity, type = "l", lwd = 2, col = "red",
      xlab = "Classification Threshold", ylab = "Metric Value",
      main = "Sensitivity-Specificity Trade-off Across Thresholds",
      ylim = c(0, 1), las = 1)
lines(results$threshold, results$specificity, lwd = 2, col = "blue")
legend("right", legend = c("Sensitivity", "Specificity"),
      col = c("red", "blue"), lwd = 2)
```

16.8.1.2. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve
from scipy.special import expit

np.random.seed(42)
n = 1000
true_outcome = np.random.binomial(1, 0.15, n)
```

16. Evaluating Models: Beyond Accuracy

```
pred_prob = expit(np.random.normal(-1 + 2 * true_outcome, 1.2))

fpr, tpr, thresholds = roc_curve(true_outcome, pred_prob)
specificity = 1 - fpr

plt.figure(figsize=(8, 5))
plt.plot(thresholds, tpr[:-1] if len(tpr) > len(thresholds) else tpr,
         color="red", lw=2, label="Sensitivity")
plt.plot(thresholds, specificity[:-1] if len(specificity) > len(thresholds) else specificity,
         color="blue", lw=2, label="Specificity")
plt.xlabel("Classification Threshold")
plt.ylabel("Metric Value")
plt.title("Sensitivity-Specificity Trade-off Across Thresholds")
plt.legend()
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

16.9. Fairness: Does the Model Work for Everyone?

A model that performs well on average may perform very differently across subgroups defined by age, sex, race, ethnicity, or socioeconomic status. This is not a theoretical concern — it has been documented repeatedly in clinical prediction models.

A well-known example is the pulse oximeter, which systematically overestimates blood oxygen saturation in patients with darker skin pigmentation. Prediction models trained on predominantly White populations may have lower sensitivity for detecting disease in Black or Hispanic patients, potentially widening existing health disparities.

16.9.1. Key Fairness Concepts

- **Equal performance:** Does the model have similar sensitivity and specificity across demographic groups?
- **Calibration fairness:** Are predicted probabilities equally well-calibrated in each group? A model might predict 20% risk for two groups, but if the actual event rate is 20% in one group and 30% in another, the model is miscalibrated for the latter.
- **Demographic parity:** Are positive predictions made at similar rates across groups? (Note: this may conflict with calibration if true prevalence differs.)

16.9.2. Exercise: Evaluating Fairness

16.9.2.1. R

```
library(pROC)

# Simulate data with two demographic groups
set.seed(42)
n <- 2000

group <- sample(c("A", "B"), n, replace = TRUE)
# Group B has slightly different disease prevalence and predictor distribution
true_outcome <- ifelse(group == "A",
                        rbinom(n, 1, 0.10),
                        rbinom(n, 1, 0.15))
# Model performs slightly worse for Group B
pred_prob <- ifelse(group == "A",
                    plogis(rnorm(n, -1.5 + 2.5 * true_outcome, 1.0)),
                    plogis(rnorm(n, -1.5 + 1.8 * true_outcome, 1.2)))
```

16. Evaluating Models: Beyond Accuracy

```
# Calculate AUC by group
for (g in c("A", "B")) {
  idx <- group == g
  roc_g <- roc(true_outcome[idx], pred_prob[idx])
  cat(sprintf("Group %s: AUC = %.3f, Prevalence = %.1f%%\n",
             g, auc(roc_g), mean(true_outcome[idx]) * 100))
}

# Compare sensitivity at a fixed threshold
threshold <- 0.3
for (g in c("A", "B")) {
  idx <- group == g
  pred_class <- ifelse(pred_prob[idx] >= threshold, 1, 0)
  sens <- sum(pred_class == 1 & true_outcome[idx] == 1) / sum(true_outcome[i
  spec <- sum(pred_class == 0 & true_outcome[idx] == 0) / sum(true_outcome[i
  cat(sprintf("Group %s at threshold %.1f: Sensitivity = %.3f, Specificity =
             g, threshold, sens, spec))
}
```

16.9.2.2. Python

```
import numpy as np
from sklearn.metrics import roc_auc_score
from scipy.special import expit

np.random.seed(42)
n = 2000

group = np.random.choice(["A", "B"], n)
true_outcome = np.where(group == "A",
                        np.random.binomial(1, 0.10, n),
                        np.random.binomial(1, 0.15, n))
```


16.10. Putting It All Together: A Complete Evaluation

```
pred_prob = np.where(group == "A",
                      expit(np.random.normal(-1.5 + 2.5 * true_outcome, 1.0)),
                      expit(np.random.normal(-1.5 + 1.8 * true_outcome, 1.2)))

# AUC by group
for g in ["A", "B"]:
    mask = group == g
    auc = roc_auc_score(true_outcome[mask], pred_prob[mask])
    prev = true_outcome[mask].mean()
    print(f"Group {g}: AUC = {auc:.3f}, Prevalence = {prev*100:.1f}%")

# Sensitivity and specificity at a fixed threshold
threshold = 0.3
for g in ["A", "B"]:
    mask = group == g
    pred_class = (pred_prob[mask] >= threshold).astype(int)
    tp = ((pred_class == 1) & (true_outcome[mask] == 1)).sum()
    fn = ((pred_class == 0) & (true_outcome[mask] == 1)).sum()
    tn = ((pred_class == 0) & (true_outcome[mask] == 0)).sum()
    fp = ((pred_class == 1) & (true_outcome[mask] == 0)).sum()
    sens = tp / (tp + fn) if (tp + fn) > 0 else 0
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0
    print(f"Group {g} at threshold {threshold}: Sensitivity = {sens:.3f}, Specificity = {spec:.3f}")
```

16.10. Putting It All Together: A Complete Evaluation

When evaluating a clinical prediction model, no single metric tells the whole story. The following checklist summarises what you should report:

1. **Sample description:** How many patients? How many events?
What is the prevalence?

16. Evaluating Models: Beyond Accuracy

2. **Discrimination:** AUC-ROC with confidence interval. Consider AUPRC for rare outcomes.
3. **Calibration:** Are predicted probabilities accurate? (Covered in detail in Chapter 18.)
4. **Confusion matrix:** At a clinically relevant threshold, not just the default 0.5.
5. **Sensitivity, specificity, PPV, NPV:** At the chosen threshold.
6. **Subgroup performance:** Does the model perform equitably across key demographic groups?
7. **Clinical utility:** Would using this model lead to better decisions than the alternatives? (Covered in Chapter 18.)

! Remember

A model is not good or bad in isolation. It is good or bad **for a specific purpose, in a specific population, at a specific threshold**. Always evaluate with the intended clinical use in mind.

16.11. Summary

- **Accuracy** is misleading for imbalanced outcomes; decompose errors into sensitivity and specificity.
- **Sensitivity** matters most for screening; **specificity** matters most for confirmation.
- **PPV and NPV** depend on prevalence, so a test's clinical usefulness changes with the population.
- **ROC curves** display the sensitivity-specificity trade-off across all thresholds; **AUC** summarises discriminative ability.
- **Precision-recall curves** are more informative than ROC for rare outcomes.
- The **classification threshold** should be chosen based on clinical consequences, not statistical optimality.

- **Fairness** requires checking that model performance is consistent across demographic groups.

16.12. References and Further Reading

The foundational concepts of sensitivity, specificity, and predictive values are covered in every epidemiology textbook, but their application to prediction models requires additional nuance. Van Calster and colleagues provide an excellent overview of performance assessment in their 2025 paper in *The Lancet Digital Health*, which organises evaluation into five domains: discrimination, calibration, overall performance, classification, and clinical utility. This framework is essential reading for anyone developing or evaluating clinical prediction models. The comprehensive textbook by Smits, van Kuijk, and Wynants (2026) devotes several chapters to model evaluation and provides practical guidance on choosing appropriate metrics for different clinical contexts. For the statistical foundations of ROC analysis, the classic text by Pepe (2003), “The Statistical Evaluation of Medical Tests for Classification and Prediction,” remains the definitive reference. Saito and Rehmsmeier (2015) explain why precision-recall curves are preferable to ROC curves for imbalanced datasets in their paper “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets” in *PLOS ONE*. On the topic of fairness in clinical prediction, Obermeyer and colleagues published a landmark 2019 paper in *Science* demonstrating racial bias in a widely used healthcare algorithm, which is required reading for anyone working in this space. Vickers and Elkin (2006) introduced decision curve analysis and net benefit as tools for evaluating clinical utility, which we discuss further in the next chapter.

Part VI.

Clinical Prediction Models

17. Developing Clinical Prediction Models: A Complete Workflow

17.1. Introduction

Building a clinical prediction model is not simply a matter of fitting a regression or running a machine learning algorithm. It is a structured process that begins with a clearly defined clinical question and proceeds through study design, data preparation, model specification, and evaluation. At every step, decisions must be guided by clinical knowledge, not just statistical convenience.

This chapter walks you through the complete workflow of developing a clinical prediction model, following the framework described by Smits, van Kuijk, and Wynants (2026) and the foundational principles laid out by Steyerberg (2019). We will use the Framingham Heart Study data as a running example to illustrate each step, building a model to predict 10-year cardiovascular disease (CVD) risk.

17.2. Step 1: Define the Clinical Question

Before touching any data, you must answer four questions:

1. **Who** is the target population? (e.g., adults aged 40–70 without prior CVD)
2. **What** is the outcome? (e.g., first CVD event within 10 years)

17. Developing Clinical Prediction Models: A Complete Workflow

3. **When** is the prediction made? (e.g., at a routine primary care visit)
4. **Why** is the prediction needed? (e.g., to guide statin therapy decisions)

The intended use determines everything that follows. A model designed to triage emergency department patients needs different predictors, a different time horizon, and different performance requirements than a model for long-term cardiovascular risk stratification.

The PICOTS Framework

You can adapt the familiar PICOTS framework from clinical research: **P**opulation, **I**ntended use (Index model), **C**omparators (existing tools), **O**utcome, **T**iming, **S**etting.

17.2.1. Common Pitfalls in Problem Definition

- **Vague outcomes:** “clinical deterioration” is not a well-defined outcome. Define exactly what events count, how they are ascertained, and over what time window.
- **Leaky predictors:** including information that would not be available at the moment of prediction. If you are predicting ICU admission at the time of hospital arrival, you cannot use ICU vital signs as predictors.
- **Wrong population:** developing a model in hospitalised patients and hoping to apply it in primary care. The case-mix will be entirely different.

17.3. Step 2: Study Design and Sample Size

17.3.1. Study Design

Clinical prediction models are typically developed from **cohort studies** (prospective or retrospective) or **existing databases** (electronic health records, registries). Randomised trials can also be used, though their selected populations may limit generalisability.

Key design requirements:

- **Inception cohort:** all patients should be enrolled at a similar, well-defined point in their clinical trajectory (the “moment of prediction”).
- **Complete follow-up:** outcomes must be ascertained for as many patients as possible. Differential loss to follow-up can bias estimates.
- **Outcome rate:** there must be a sufficient number of events to support the number of candidate predictors.

17.3.2. Sample Size: The Events Per Variable Rule and Beyond

The classical rule of thumb requires at least **10 events per variable (EPV)** in the model. For a logistic regression with 8 predictors, you would need at least 80 events. With 10% prevalence, that means at least 800 patients.

However, this rule is a rough guide. Riley and colleagues (2020) proposed a more rigorous sample size framework based on four criteria:

1. **Small optimism:** the expected shrinkage factor should be close to 1 (e.g., > 0.9).
2. **Small absolute difference** in the apparent and adjusted Nagelkerke R-squared.
3. **Precise estimation** of the overall outcome proportion (intercept).
4. **Precise estimation** of the overall model performance (C-statistic).

17. Developing Clinical Prediction Models: A Complete Workflow

The `pmsampsize` package in R implements these calculations.

17.3.2.1. R

```
library(pmsampsize)

# Sample size for a logistic regression prediction model
# 10 candidate predictors, anticipated outcome prevalence of 15%
# Expected C-statistic of 0.75, Cox-Snell R-squared of 0.1
ss <- pmsampsize(type = "b",          # binary outcome
                  rsquared = 0.10,    # anticipated Cox-Snell R-squared
                  parameters = 10,    # number of candidate predictors
                  prevalence = 0.15)  # anticipated prevalence

print(ss)
cat("\nMinimum sample size:", ss$sample_size, "\n")
cat("Minimum number of events:", ss$events, "\n")
```

17.3.2.2. Python

```
# Riley et al. sample size criteria (simplified implementation)
# For full implementation, see the pmsampsize R package
import numpy as np

def pmsampsize_binary(rsquared, parameters, prevalence, shrinkage=0.9):
    """
    Simplified sample size calculation for binary outcome prediction models.
    Based on Riley et al. (2020) criteria.
    """
    # Criterion 1: Target shrinkage
    #  $n \geq p / ((s - 1) * \ln(1 - R^2_{cs} / s))$ 
```

17.4. Step 3: Handle Missing Data

```
# where s = target shrinkage, p = parameters, R^2_cs = Cox-Snell R^2
n1 = parameters / ((shrinkage - 1) * np.log(1 - rsquared / shrinkage))

# Criterion 2: Small absolute difference in R^2
n2 = parameters / (0.05 * np.log(1 - rsquared)) # delta = 0.05

# Criterion 3: Precise intercept (SE of log odds of prevalence)
n3 = (1.96 / 0.05)**2 * (1 / (prevalence * (1 - prevalence)))

n_min = max(n1, n2, n3)
events = int(np.ceil(n_min * prevalence))

print(f"Criterion 1 (shrinkage): n = {int(np.ceil(n1))}")
print(f"Criterion 2 (R-squared): n = {int(np.ceil(n2))}")
print(f"Criterion 3 (intercept): n = {int(np.ceil(n3))}")
print(f"\nMinimum sample size: {int(np.ceil(n_min))}")
print(f"Minimum events: {events}")

return int(np.ceil(n_min))

pmsampsize_binary(rsquared=0.10, parameters=10, prevalence=0.15)
```

17.4. Step 3: Handle Missing Data

Missing data is the norm in clinical research, not the exception. Laboratory values are missing because they were not ordered. Follow-up data is missing because patients moved. Lifestyle variables are missing because patients did not answer the questionnaire.

17.4.1. Missing Data Mechanisms

Understanding **why** data are missing is essential for choosing the right approach:

- **MCAR (Missing Completely At Random)**: the probability of missingness is unrelated to any observed or unobserved data. Example: a laboratory sample is accidentally dropped. This is rare in practice.
- **MAR (Missing At Random)**: the probability of missingness depends on **observed** data but not on the missing value itself. Example: HbA1c is more likely to be measured in patients already known to have diabetes. Given diabetes status (observed), the missingness is random.
- **MNAR (Missing Not At Random)**: the probability of missingness depends on the **unobserved** value itself. Example: patients with severe depression are less likely to attend follow-up appointments, so their depression scores are missing precisely because they are severe.

17.4.2. Why Complete Case Analysis Fails

Deleting rows with any missing data (complete case analysis) is the default in most software, and it is almost always wrong:

- It **wastes data**: if 10 variables each have 5% missing (independently), only 60% of rows are complete.
- It **biases estimates**: if missingness is related to the outcome or predictors (MAR or MNAR), the complete cases are a non-random subset.
- It **reduces power**: smaller effective sample sizes lead to wider confidence intervals and less stable models.

17.4.3. Multiple Imputation

Multiple imputation (MI) is the recommended approach for handling missing data under the MAR assumption. The procedure has three steps:

1. **Impute:** create m complete datasets by filling in missing values with plausible values drawn from the predictive distribution of the missing data, given the observed data.
2. **Analyse:** fit the prediction model separately in each of the m imputed datasets.
3. **Pool:** combine the m sets of results using Rubin's rules, which account for both within-imputation and between-imputation variability.

Typically, $m = 20$ or more imputations are recommended.

17.4.3.1. R

```
library(mice)
library(dplyr)

# Simulate clinical data with missing values
set.seed(42)
n <- 500
data <- data.frame(
  age = rnorm(n, 55, 10),
  sbp = rnorm(n, 130, 20),
  cholesterol = rnorm(n, 220, 40),
  smoking = rbinom(n, 1, 0.25),
  bmi = rnorm(n, 27, 5)
)
data$cvd <- rbinom(n, 1, plogis(-4 + 0.03*data$age + 0.01*data$sbp +
                                0.005*data$cholesterol + 0.5*data$smoking))
```

17. Developing Clinical Prediction Models: A Complete Workflow

```
# Introduce MAR missingness
# Cholesterol more likely missing in younger patients
data$cholesterol[data$age < 50 & runif(n) < 0.3] <- NA
# BMI more likely missing in non-smokers
data$bmi[data$smoking == 0 & runif(n) < 0.2] <- NA

cat("Missing data pattern:\n")
md.pattern(data, rotate.names = TRUE)

# Perform multiple imputation
imp <- mice(data, m = 20, method = "pmm", seed = 42, printFlag = FALSE)

# Fit model in each imputed dataset and pool
fit_imp <- with(imp, glm(cvd ~ age + sbp + cholesterol + smoking + bmi,
                        family = binomial))
pooled <- pool(fit_imp)
summary(pooled)
```

17.4.3.2. Python

```
import numpy as np
import pandas as pd
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LogisticRegression
from scipy.special import expit

np.random.seed(42)
n = 500

data = pd.DataFrame({
    'age': np.random.normal(55, 10, n),
```

17.4. Step 3: Handle Missing Data

```
'sbp': np.random.normal(130, 20, n),
'cholesterol': np.random.normal(220, 40, n),
'smoking': np.random.binomial(1, 0.25, n),
'bmi': np.random.normal(27, 5, n)
})
lp = -4 + 0.03*data['age'] + 0.01*data['sbp'] + 0.005*data['cholesterol'] + 0.5*data['smokin
data['cvd'] = np.random.binomial(1, expit(lp))

# Introduce MAR missingness
mask_chol = (data['age'] < 50) & (np.random.uniform(size=n) < 0.3)
data.loc[mask_chol, 'cholesterol'] = np.nan
mask_bmi = (data['smoking'] == 0) & (np.random.uniform(size=n) < 0.2)
data.loc[mask_bmi, 'bmi'] = np.nan

print("Missing values per column:")
print(data.isnull().sum())

# Simple multiple imputation using IterativeImputer
# Note: for proper MI with pooling, consider the miceforest package
predictors = ['age', 'sbp', 'cholesterol', 'smoking', 'bmi']
X = data[predictors].values
y = data['cvd'].values

# Create multiple imputed datasets and pool coefficients
m = 20
coefs = []
for i in range(m):
    imputer = IterativeImputer(random_state=i, max_iter=10, sample_posterior=True)
    X_imp = imputer.fit_transform(X)
    model = LogisticRegression(max_iter=1000, penalty=None)
    model.fit(X_imp, y)
    coefs.append(np.concatenate([model.intercept_, model.coef_[0]]))
```

```
# Pool results (simplified Rubin's rules)
coefs = np.array(coefs)
pooled_mean = coefs.mean(axis=0)
within_var = coefs.var(axis=0) # simplified
names = ['intercept'] + predictors
print("\nPooled coefficients:")
for name, coef in zip(names, pooled_mean):
    print(f" {name}: {coef:.4f}")
```

17.5. Step 4: Variable Selection

17.5.1. Choosing Candidate Predictors

The best predictor selection strategy is **expert knowledge**. Decades of cardiovascular research tell us that age, sex, blood pressure, cholesterol, smoking, and diabetes are important predictors of CVD. Starting with established clinical knowledge prevents the discovery of spurious associations and produces more generalisable models.

When expert knowledge is insufficient or the domain is novel, data-driven selection can supplement clinical reasoning. However, the approach matters enormously.

17.5.2. The Dangers of Stepwise Selection

Stepwise selection (forward, backward, or both) is among the most commonly used and most harmful variable selection strategies:

- It inflates the apparent significance of selected variables (multiple testing without correction).
- It produces unstable models: small changes in the data can lead to entirely different variable selections.

17.6. Step 5: Understand and Combat Overfitting

- It biases regression coefficients away from zero (selected variables appear stronger than they are).
- It yields overly optimistic performance estimates.

Avoid Univariable Screening

A particularly damaging practice is pre-selecting variables based on univariable p-values (e.g., keeping only variables with $p < 0.20$). This ignores confounding, suppression, and the fact that weak individual predictors can be strong in combination. If you must reduce the candidate set, use clinical knowledge, not univariable screening.

17.5.3. Better Approaches

- **Full model with all pre-specified predictors:** if sample size permits, include all clinically plausible predictors and apply shrinkage (see below). This is the recommended approach by Steyerberg (2019).
- **Penalised regression:** LASSO (L1 penalty) performs automatic variable selection by shrinking some coefficients to zero. Elastic net combines L1 and L2 penalties. These are covered in detail in Chapter 7.
- **Background knowledge with data-driven fine-tuning:** start with a core set of established predictors and consider a small number of additional candidates.

17.6. Step 5: Understand and Combat Overfitting

17.6.1. What Is Overfitting?

Overfitting occurs when a model captures not only the true underlying patterns in the data but also the **noise** — the random variation specific to

17. *Developing Clinical Prediction Models: A Complete Workflow*

the development sample. An overfitted model performs well on the data it was trained on but poorly on new data.

Clinical data is especially prone to overfitting because:

- **Sample sizes are often small** relative to the number of candidate predictors.
- **Events may be rare**, limiting the information available to estimate parameters.
- **Complex interactions** are tempting to include but expensive in terms of degrees of freedom.

17.6.2. Detecting Overfitting: Optimism

The **optimism** of a model is the difference between its apparent performance (on the training data) and its expected performance on new data. You can estimate optimism using internal validation techniques such as bootstrapping (covered in Chapter 18).

If a model shows high optimism, it is overfitted. The solution is **shrinkage**.

17.6.3. Shrinkage Methods

Uniform shrinkage multiplies all regression coefficients by a factor s (where $0 < s \leq 1$) estimated from the data, typically via bootstrapping. The intercept is then re-estimated to preserve the overall calibration. This pulls extreme predictions toward the average, reducing the impact of noise.

Penalised methods incorporate shrinkage directly into the estimation process:

17.6. Step 5: Understand and Combat Overfitting

- **Ridge regression** (L2 penalty) shrinks all coefficients toward zero but never sets any to exactly zero.
- **LASSO** (L1 penalty) can shrink coefficients to exactly zero, performing variable selection.
- **Elastic net** combines both penalties.

The penalty strength is chosen by cross-validation.

17.6.3.1. R

```
library(glmnet)
library(rms)

# Simulate Framingham-like data
set.seed(42)
n <- 800
age <- rnorm(n, 55, 10)
male <- rbinom(n, 1, 0.5)
sbp <- rnorm(n, 130, 20)
chol <- rnorm(n, 220, 40)
smoking <- rbinom(n, 1, 0.25)
diabetes <- rbinom(n, 1, 0.10)

# True model
lp <- -6 + 0.05*age + 0.3*male + 0.01*sbp + 0.005*chol + 0.4*smoking + 0.5*diabetes
cvd <- rbinom(n, 1, plogis(lp))

df <- data.frame(age, male, sbp, chol, smoking, diabetes, cvd)

# Unpenalised logistic regression
fit_full <- glm(cvd ~ ., data = df, family = binomial)
cat("=== Unpenalised coefficients ===\n")
print(round(coef(fit_full), 4))
```

17. Developing Clinical Prediction Models: A Complete Workflow

```
# Ridge regression (alpha = 0)
X <- model.matrix(cvd ~ ., data = df)[, -1]
y <- df$cvd

fit_ridge <- cv.glmnet(X, y, family = "binomial", alpha = 0, nfolds = 10)
cat("\n=== Ridge coefficients (lambda.min) ===\n")
print(round(as.vector(coef(fit_ridge, s = "lambda.min")), 4))

# LASSO (alpha = 1)
fit_lasso <- cv.glmnet(X, y, family = "binomial", alpha = 1, nfolds = 10)
cat("\n=== LASSO coefficients (lambda.min) ===\n")
print(round(as.vector(coef(fit_lasso, s = "lambda.min")), 4))

# Compare: note how penalised coefficients are shrunk toward zero
par(mfrow = c(1, 2))
plot(fit_ridge, main = "Ridge: CV Error vs log(lambda)")
plot(fit_lasso, main = "LASSO: CV Error vs log(lambda)")
```

17.6.3.2. Python

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from scipy.special import expit
import matplotlib.pyplot as plt

np.random.seed(42)
n = 800
age = np.random.normal(55, 10, n)
male = np.random.binomial(1, 0.5, n)
sbp = np.random.normal(130, 20, n)
```

17.6. Step 5: Understand and Combat Overfitting

```
chol = np.random.normal(220, 40, n)
smoking = np.random.binomial(1, 0.25, n)
diabetes = np.random.binomial(1, 0.10, n)

lp = -6 + 0.05*age + 0.3*male + 0.01*sbp + 0.005*chol + 0.4*smoking + 0.5*diabetes
cvd = np.random.binomial(1, expit(lp))

X = np.column_stack([age, male, sbp, chol, smoking, diabetes])
feature_names = ['age', 'male', 'sbp', 'chol', 'smoking', 'diabetes']

# Unpenalised logistic regression
model_full = LogisticRegression(penalty=None, max_iter=1000)
model_full.fit(X, cvd)
print("=== Unpenalised coefficients ===")
for name, coef in zip(feature_names, model_full.coef_[0]):
    print(f" {name}: {coef:.4f}")

# Ridge regression (L2)
model_ridge = LogisticRegressionCV(penalty='l2', cv=10, max_iter=1000, random_state=42)
model_ridge.fit(X, cvd)
print("\n=== Ridge coefficients ===")
for name, coef in zip(feature_names, model_ridge.coef_[0]):
    print(f" {name}: {coef:.4f}")

# LASSO (L1)
model_lasso = LogisticRegressionCV(penalty='l1', solver='saga', cv=10,
                                   max_iter=5000, random_state=42)
model_lasso.fit(X, cvd)
print("\n=== LASSO coefficients ===")
for name, coef in zip(feature_names, model_lasso.coef_[0]):
    print(f" {name}: {coef:.4f}")
```

17.7. Step 6: Build the Model — A Framingham Example

Let us now walk through a complete model development process using simulated data based on the Framingham Heart Study. We will predict 10-year CVD risk using established risk factors.

17.7.1. Data Preparation

17.7.1.1. R

```
library(rms)

# Simulate Framingham-like cohort
set.seed(2024)
n <- 2000

framingham <- data.frame(
  age = round(runif(n, 30, 74)),
  male = rbinom(n, 1, 0.48),
  sbp = round(rnorm(n, 130, 18)),
  total_chol = round(rnorm(n, 210, 38)),
  hdl_chol = round(rnorm(n, 52, 15)),
  smoking = rbinom(n, 1, 0.22),
  diabetes = rbinom(n, 1, 0.08),
  bp_treatment = rbinom(n, 1, 0.15)
)

# Generate outcome based on known risk factors
lp <- with(framingham,
  -7.5 + 0.06 * age + 0.4 * male + 0.012 * sbp +
  0.005 * total_chol - 0.02 * hdl_chol +
```

17.7. Step 6: Build the Model — A Framingham Example

```
    0.5 * smoking + 0.7 * diabetes + 0.3 * bp_treatment
  )
framingham$cvd_10yr <- rbinom(n, 1, plogis(lp))

cat("Event rate:", round(mean(framingham$cvd_10yr) * 100, 1), "%\n")
cat("Number of events:", sum(framingham$cvd_10yr), "\n")
cat("Events per variable:", round(sum(framingham$cvd_10yr) / 8, 1), "\n")

# Set up the rms environment for enhanced model building
dd <- datadist(framingham)
options(datadist = "dd")

# Fit the model using lrm (logistic regression model from rms)
# Allow nonlinearity for continuous variables using restricted cubic splines
fit <- lrm(cvd_10yr ~ rcs(age, 4) + male + rcs(sbp, 3) +
           total_chol + hdl_chol + smoking + diabetes + bp_treatment,
           data = framingham, x = TRUE, y = TRUE)

print(fit)

# Check for nonlinearity
anova(fit)

# Visualise partial effects
plot(Predict(fit, age), main = "Effect of Age on CVD Risk")
plot(Predict(fit, sbp), main = "Effect of SBP on CVD Risk")
```

17.7.1.2. Python

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
```

17. Developing Clinical Prediction Models: A Complete Workflow

```
from scipy.special import expit

np.random.seed(2024)
n = 2000

framingham = pd.DataFrame({
    'age': np.random.randint(30, 75, n),
    'male': np.random.binomial(1, 0.48, n),
    'sbp': np.round(np.random.normal(130, 18, n)),
    'total_chol': np.round(np.random.normal(210, 38, n)),
    'hdl_chol': np.round(np.random.normal(52, 15, n)),
    'smoking': np.random.binomial(1, 0.22, n),
    'diabetes': np.random.binomial(1, 0.08, n),
    'bp_treatment': np.random.binomial(1, 0.15, n)
})

lp = (-7.5 + 0.06 * framingham['age'] + 0.4 * framingham['male'] +
      0.012 * framingham['sbp'] + 0.005 * framingham['total_chol'] -
      0.02 * framingham['hdl_chol'] + 0.5 * framingham['smoking'] +
      0.7 * framingham['diabetes'] + 0.3 * framingham['bp_treatment'])

framingham['cvd_10yr'] = np.random.binomial(1, expit(lp))

print(f"Event rate: {framingham['cvd_10yr'].mean()*100:.1f}%")
print(f"Number of events: {framingham['cvd_10yr'].sum()}")
print(f"Events per variable: {framingham['cvd_10yr'].sum() / 8:.1f}")

# Fit logistic regression
predictors = ['age', 'male', 'sbp', 'total_chol', 'hdl_chol',
              'smoking', 'diabetes', 'bp_treatment']
X = sm.add_constant(framingham[predictors])
y = framingham['cvd_10yr']
```


17.7. Step 6: Build the Model — A Framingham Example

```
model = sm.Logit(y, X)
result = model.fit(dis=0)
print("\n", result.summary())

# Calculate predicted probabilities
framingham['pred_prob'] = result.predict(X)

# Summary of predicted risk distribution
print("\nPredicted risk distribution:")
print(framingham['pred_prob'].describe())
```

17.7.2. Model Specification Decisions

In the Framingham example above, several important decisions were made:

1. **Pre-specified predictors:** we included the established Framingham risk factors, not variables discovered through data-dredging.
2. **Nonlinearity:** for continuous variables like age and blood pressure, we allowed flexible functional forms using restricted cubic splines (in R's `rms` package). The relationship between age and CVD risk is not linear on the log-odds scale.
3. **No interactions without prior evidence:** we did not fish for interactions. If clinical knowledge suggests an interaction (e.g., the effect of cholesterol might differ by sex), it should be pre-specified.
4. **Sample size check:** with approximately 250 events and 8 predictors (plus a few extra degrees of freedom for splines), we have approximately 25 EPV, which is comfortable.

17.8. Step 7: Assess the Final Model

After fitting, we need to examine the model critically:

17.8.0.1. R

```
library(rms)

# Assuming fit and framingham from above
# 1. Check the discrimination (C-statistic)
cat("C-statistic (AUC):", round(fit$stats["C"], 3), "\n")

# 2. Predicted risk distribution
pred_probs <- predict(fit, type = "fitted")
hist(pred_probs, breaks = 50, col = "steelblue",
     main = "Distribution of Predicted 10-Year CVD Risk",
     xlab = "Predicted Probability")

# 3. Predicted risks in events vs non-events
boxplot(pred_probs ~ framingham$cvd_10yr,
     names = c("No CVD", "CVD"),
     main = "Predicted Risk by Outcome",
     ylab = "Predicted Probability",
     col = c("lightblue", "salmon"))

# 4. Quick calibration check
val <- val.prob(pred_probs, framingham$cvd_10yr)

# 5. Internal validation via bootstrapping (optimism-corrected)
set.seed(123)
val_boot <- validate(fit, B = 200)
print(val_boot)
```

17.8. Step 7: Assess the Final Model

```
cat("\nOptimism-corrected C-statistic:",  
    round(0.5 * (val_boot["Dxy", "index.corrected"] + 1), 3), "\n")
```

17.8.0.2. Python

```
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.metrics import roc_auc_score  
from sklearn.calibration import calibration_curve  
  
# Assuming framingham and result from above  
pred_probs = framingham['pred_prob'].values  
y_true = framingham['cvd_10yr'].values  
  
# 1. Discrimination  
auc = roc_auc_score(y_true, pred_probs)  
print(f"C-statistic (AUC): {auc:.3f}")  
  
# 2. Predicted risk distribution  
fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
  
axes[0].hist(pred_probs, bins=50, color="steelblue", edgecolor="white")  
axes[0].set_xlabel("Predicted Probability")  
axes[0].set_ylabel("Frequency")  
axes[0].set_title("Distribution of Predicted 10-Year CVD Risk")  
  
# 3. Risk by outcome group  
axes[1].boxplot([pred_probs[y_true == 0], pred_probs[y_true == 1]],  
                labels=["No CVD", "CVD"])  
axes[1].set_ylabel("Predicted Probability")  
axes[1].set_title("Predicted Risk by Outcome")
```

17. Developing Clinical Prediction Models: A Complete Workflow

```
plt.tight_layout()
plt.show()

# 4. Calibration plot
prob_true, prob_pred = calibration_curve(y_true, pred_probs, n_bins=10, stratify=y_true)

plt.figure(figsize=(7, 7))
plt.plot(prob_pred, prob_true, 's-', color="steelblue", label="Model")
plt.plot([0, 1], [0, 1], '--', color='grey', label="Perfect calibration")
plt.xlabel("Mean Predicted Probability")
plt.ylabel("Observed Proportion")
plt.title("Calibration Plot")
plt.legend()
plt.tight_layout()
plt.show()
```

17.9. The Complete Workflow: A Summary

[?@fig-workflow-summary](#) outlines the complete prediction model development workflow.

1. **Define** the clinical question (population, outcome, timing, intended use).
2. **Design** the study with adequate sample size (Riley criteria).
3. **Prepare** the data: handle missing values with multiple imputation.
4. **Select** predictors based on clinical knowledge; avoid stepwise procedures.
5. **Specify** the model: consider nonlinear terms for continuous predictors.
6. **Fit** the model, applying shrinkage to combat overfitting.
7. **Evaluate** performance: discrimination, calibration, clinical utility.

17.10. Common Mistakes to Avoid

8. **Validate** internally using bootstrapping or cross-validation.
9. **Report** transparently following TRIPOD+AI guidelines (see Chapter 20).

! The Modelling Pipeline Is Not Linear

In practice, you will iterate. Poor calibration might send you back to model specification. Inadequate discrimination might prompt reconsidering the candidate predictors. Internal validation might reveal severe overfitting, requiring stronger shrinkage. The important thing is that each decision is documented and justified.

17.10. Common Mistakes to Avoid

Table 17.1.: Common mistakes in clinical prediction model development.

Mistake	Why It Is Harmful	Better Approach
Stepwise variable selection	Inflates significance, unstable	Pre-specify predictors; use penalisation
Univariable screening	Ignores confounding and suppression	Include all clinically plausible variables
Complete case analysis	Biases estimates, wastes data	Multiple imputation
Evaluating only on training data	Overestimates performance	Internal validation (bootstrap)
Dichotomising continuous predictors	Loses information, creates arbitrary groups	Model continuous variables continuously

17. Developing Clinical Prediction Models: A Complete Workflow

Mistake	Why It Is Harmful	Better Approach
Ignoring nonlinearity	Poor fit, biased predictions	Use splines or fractional polynomials
Reporting only accuracy or AUC	Incomplete picture	Report calibration, discrimination, clinical utility

17.11. Exercises

1. **Sample size calculation:** You are developing a model to predict pre-eclampsia (expected prevalence 4%) using 12 candidate predictors. Use `pmsampsize` to determine the minimum required sample size. How many pregnancies would you need to observe?
2. **Missing data simulation:** Take the simulated Framingham dataset above and compare the regression coefficients obtained from (a) complete case analysis, (b) single mean imputation, and (c) multiple imputation. Which approach yields coefficients closest to the true values?
3. **Variable selection:** Fit the Framingham model using (a) all pre-specified predictors, (b) forward stepwise selection, and (c) LASSO. Compare the selected variables and the stability of coefficients across 100 bootstrap samples.

17.12. Summary

- A prediction model starts with a **clearly defined clinical question**, not with data analysis.
- **Sample size** should be determined prospectively using the Riley et al. criteria, not just the EPV rule of thumb.

17.13. References and Further Reading

- **Missing data** should be handled with multiple imputation, not deletion.
- **Variable selection** should be driven by clinical knowledge. Stepwise methods are harmful.
- **Overfitting** is the central challenge; combat it with adequate sample size and shrinkage.
- **Penalised regression** (ridge, LASSO, elastic net) embeds shrinkage into the estimation process.
- The final model should be **internally validated** before any claims about performance.

17.13. References and Further Reading

The most comprehensive modern reference for clinical prediction model development is the textbook by Smits, van Kuijk, and Wynants (2026), which provides practical guidance grounded in decades of methodological research. Steyerberg’s “Clinical Prediction Models” (2019, second edition, Springer) remains the foundational text, covering study design, missing data, variable selection, and model evaluation with exceptional rigour. For sample size calculations, Riley and colleagues published a series of papers in *Statistics in Medicine* (2020) establishing formal criteria that go well beyond the traditional events-per-variable rule; the `pmsampsize` R package implements these methods. Van Buuren’s “Flexible Imputation of Missing Data” (2018, second edition, Chapman and Hall) is the definitive resource on multiple imputation, while the `mice` package in R provides the practical tools. Harrell’s “Regression Modeling Strategies” (2015, second edition, Springer) is essential reading on variable selection, nonlinearity, and shrinkage in regression modelling. For penalised regression, Hastie, Tibshirani, and Friedman’s “The Elements of Statistical Learning” (2009, second edition, Springer) provides the theoretical foundations, while the `glmnet` R package by Friedman, Hastie, and Tibshirani is the standard implementation.

18. Performance Assessment and Model Validation

18.1. Introduction

You have developed a clinical prediction model. Its coefficients look reasonable, the predictors make clinical sense, and the apparent performance seems promising. But how good is this model, really? And will it work on patients beyond your development dataset?

This chapter addresses these questions by covering the five domains of model performance — discrimination, calibration, overall performance, classification, and clinical utility — and the methods for internal and external validation. The structure follows the comprehensive framework proposed by Van Calster and colleagues (2025) in *The Lancet Digital Health*, which we strongly recommend as a companion reading.

The core message is this: **no single metric tells the full story.** A model with excellent discrimination (high AUC) can still produce dangerously miscalibrated risk predictions. A well-calibrated model might not discriminate well enough to be useful. Clinical utility must be assessed separately from statistical performance. You need the full picture.

18.2. The Five Performance Domains

Van Calster et al. (2025) organise model performance assessment into five complementary domains:

Domain	Question	Key Metrics
Discrimination	Can the model rank patients by risk?	C-statistic, AUC
Calibration	Are predicted probabilities accurate?	Calibration plot, O:E ratio, calibration slope
Overall performance	How close are predictions to observed outcomes?	Brier score, Nagelkerke R-squared
Classification	How well does the model classify at a threshold?	Sensitivity, specificity, PPV, NPV
Clinical utility	Does using the model improve clinical decisions?	Net benefit, decision curve analysis

We will examine each in detail.

18.3. Domain 1: Discrimination

18.3.1. The C-Statistic / AUC

Discrimination measures how well a model separates patients who experience the outcome from those who do not. The most common measure is the **concordance statistic (C-statistic)**, which for binary outcomes

is equivalent to the area under the ROC curve (AUC) discussed in Chapter 16.

Recall the interpretation: the C-statistic is the probability that a randomly selected patient with the outcome has a higher predicted probability than a randomly selected patient without the outcome.

18.3.2. R

```
library(rms)

# Simulate clinical data: predicting 30-day mortality after stroke
set.seed(2024)
n <- 1500

stroke_data <- data.frame(
  age = round(rnorm(n, 72, 12)),
  nihss = round(pmax(0, rnorm(n, 8, 6))),      # NIH Stroke Scale
  glucose = round(rnorm(n, 140, 50)),
  afib = rbinom(n, 1, 0.25),                  # Atrial fibrillation
  thrombolysis = rbinom(n, 1, 0.30)
)

# True model for 30-day mortality
lp <- -5 + 0.04 * stroke_data$age +
  0.12 * stroke_data$nihss +
  0.003 * stroke_data$glucose +
  0.3 * stroke_data$afib -
  0.5 * stroke_data$thrombolysis

stroke_data$death_30d <- rbinom(n, 1, plogis(lp))
cat("30-day mortality rate:", mean(stroke_data$death_30d), "\n")
```

18. Performance Assessment and Model Validation

```
# Fit prediction model
dd <- datadist(stroke_data)
options(datadist = "dd")

fit <- lrm(death_30d ~ age + nihss + glucose + afib + thrombolysis,
          data = stroke_data, x = TRUE, y = TRUE)

cat("C-statistic (apparent):", fit$stats["C"], "\n")
```

18.3.3. Python

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from scipy.special import expit

np.random.seed(2024)
n = 1500

stroke_data = pd.DataFrame({
    "age": np.round(np.random.normal(72, 12, n)),
    "nihss": np.round(np.maximum(0, np.random.normal(8, 6, n))),
    "glucose": np.round(np.random.normal(140, 50, n)),
    "afib": np.random.binomial(1, 0.25, n),
    "thrombolysis": np.random.binomial(1, 0.30, n)
})

lp = (-5 + 0.04 * stroke_data["age"] +
      0.12 * stroke_data["nihss"] +
      0.003 * stroke_data["glucose"] +
      0.3 * stroke_data["afib"] -
```

```

0.5 * stroke_data["thrombolysis"])

stroke_data["death_30d"] = np.random.binomial(1, expit(lp))
print(f"30-day mortality rate: {stroke_data['death_30d'].mean():.3f}")

predictors = ["age", "nihss", "glucose", "afib", "thrombolysis"]
X = stroke_data[predictors].values
y = stroke_data["death_30d"].values

model = LogisticRegression(max_iter=5000, random_state=42)
model.fit(X, y)

pred_prob = model.predict_proba(X)[:, 1]
auc = roc_auc_score(y, pred_prob)
print(f"C-statistic (apparent): {auc:.3f}")

```

18.3.4. Limitations of the C-Statistic

The C-statistic has important limitations that Van Calster et al. (2025) highlight:

1. **Insensitive to calibration:** A model can perfectly rank patients ($C = 1.0$) but assign completely wrong probabilities.
2. **Context-blind:** It weights all parts of the ROC curve equally, even clinically irrelevant regions.
3. **Hard to improve:** In many clinical domains, the achievable C-statistic is inherently limited. Adding a new biomarker might genuinely improve predictions but barely move the C-statistic.
4. **Misleading comparisons:** A C-statistic of 0.75 in one population is not comparable to 0.75 in another if the case-mix differs.

18.4. Domain 2: Calibration

18.4.1. Why Calibration Matters More Than You Think

Calibration assesses whether predicted probabilities match observed outcomes. If your model says a patient has a 20% risk, do approximately 20 out of 100 such patients actually experience the outcome?

This matters enormously in clinical practice. When a clinician tells a patient “your model-predicted 10-year cardiovascular risk is 15%,” the patient and clinician rely on that number being accurate — not just the patient’s relative ranking. **Risk-based treatment decisions require calibrated predictions.**

18.4.2. Calibration-in-the-Large: The O:E Ratio

The simplest calibration check is the **observed-to-expected (O:E) ratio**:

$$\text{O:E ratio} = \frac{\text{Observed event rate}}{\text{Mean predicted probability}}$$

An O:E ratio of 1 indicates perfect calibration-in-the-large. An O:E ratio of 1.5 means the model underestimates risk by 50%.

18.4.3. Calibration Slope

The **calibration slope** is obtained by regressing the outcome on the linear predictor (log-odds of predictions). A calibration slope of 1 indicates perfect calibration. A slope less than 1 indicates overfitting (predictions are too extreme — high risks are too high, low risks are too low). A slope greater than 1 suggests the opposite.

18.4.4. Calibration Plots

The most informative way to assess calibration is visually, through a **calibration plot**. This plots predicted probabilities (x-axis) against observed proportions (y-axis). A perfectly calibrated model follows the 45-degree diagonal.

Two types are common:

- **Grouped calibration plot:** Patients are divided into groups (e.g., deciles) by predicted probability, and the observed proportion within each group is plotted.
- **Smoothed calibration plot:** A smooth curve (e.g., loess or spline) is fit to the predicted vs. observed data.

18.4.5. R

```
library(rms)

# Use the stroke model from above
pred_prob <- predict(fit, type = "fitted")

# Grouped calibration plot
cal <- val.prob(pred_prob, stroke_data$death_30d,
               m = 100, cex = 0.5,
               main = "Calibration Plot: 30-Day Stroke Mortality Model")

# Calculate calibration metrics manually
obs_rate <- mean(stroke_data$death_30d)
mean_pred <- mean(pred_prob)

cat("Observed event rate:", round(obs_rate, 3), "\n")
cat("Mean predicted probability:", round(mean_pred, 3), "\n")
```

18. Performance Assessment and Model Validation

```
cat("O:E ratio:", round(obs_rate / mean_pred, 3), "\n")

# Calibration slope
cal_model <- glm(stroke_data$death_30d ~ qlogis(pred_prob),
                family = binomial)
cat("Calibration slope:", round(coef(cal_model)[2], 3), "\n")
cat("Calibration intercept:", round(coef(cal_model)[1], 3), "\n")

# Brier score
brier <- mean((pred_prob - stroke_data$death_30d)^2)
cat("Brier score:", round(brier, 4), "\n")
```

18.4.6. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss

# Use the stroke model predictions from above
pred_prob = model.predict_proba(X)[:, 1]
y_true = stroke_data["death_30d"].values

# Grouped calibration curve
prob_true, prob_pred = calibration_curve(y_true, pred_prob,
                                       n_bins=10, strategy="quantile")

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Left panel: calibration plot
axes[0].plot(prob_pred, prob_true, "o-", color="steelblue", lw=2,
            markersize=8, label="Model")
```


18.4. Domain 2: Calibration

```
axes[0].plot([0, 1], [0, 1], "--", color="grey", label="Perfect calibration")
axes[0].set_xlabel("Mean Predicted Probability")
axes[0].set_ylabel("Observed Proportion")
axes[0].set_title("Calibration Plot (Decile Groups)")
axes[0].legend()
axes[0].set_xlim(0, max(prob_pred) * 1.1)
axes[0].set_ylim(0, max(prob_true) * 1.1)

# Right panel: distribution of predicted probabilities
axes[1].hist(pred_prob[y_true == 0], bins=30, alpha=0.5, color="steelblue",
             label="No event", density=True)
axes[1].hist(pred_prob[y_true == 1], bins=30, alpha=0.5, color="coral",
             label="Event", density=True)
axes[1].set_xlabel("Predicted Probability")
axes[1].set_ylabel("Density")
axes[1].set_title("Distribution of Predicted Probabilities")
axes[1].legend()

plt.tight_layout()
plt.show()

# Calibration metrics
obs_rate = y_true.mean()
mean_pred = pred_prob.mean()
oe_ratio = obs_rate / mean_pred
brier = brier_score_loss(y_true, pred_prob)

print(f"\nCalibration metrics:")
print(f"  Observed event rate: {obs_rate:.3f}")
print(f"  Mean predicted probability: {mean_pred:.3f}")
print(f"  O:E ratio: {oe_ratio:.3f}")
print(f"  Brier score: {brier:.4f}")
```

18.4.7. Interpreting Miscalibration

Common patterns and their clinical meaning:

- **O:E > 1 (underestimation):** The model systematically underestimates risk. Patients may not receive treatments they need.
- **O:E < 1 (overestimation):** The model systematically overestimates risk. Patients may receive unnecessary treatments.
- **Calibration slope < 1:** Predictions are too extreme. Patients at high predicted risk actually have lower risk than predicted, and patients at low predicted risk have higher risk. This is the hallmark of overfitting.
- **Calibration slope > 1:** Predictions are too moderate. This is less common and may occur after excessive shrinkage.

18.5. Domain 3: Overall Performance

The **Brier score** combines discrimination and calibration into a single measure:

$$\text{Brier score} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

where p_i is the predicted probability and y_i is the observed outcome (0 or 1). The Brier score ranges from 0 (perfect) to 1 (worst). For a reference, a model that always predicts the overall event rate achieves a Brier score equal to the prevalence times (1 minus the prevalence).

The **scaled Brier score** (or Brier skill score) expresses improvement over a non-informative model:

$$\text{Scaled Brier} = 1 - \frac{\text{Brier score}}{\text{Brier}_{\max}}$$

where $\text{Brier}_{\max} = \bar{y}(1 - \bar{y})$.

18.6. Domain 4: Classification

Classification metrics (sensitivity, specificity, PPV, NPV) were covered in Chapter 16. The key point for model validation is that classification performance depends entirely on the chosen threshold. When reporting classification metrics, **always specify the threshold and justify its choice based on the clinical context.**

18.7. Domain 5: Clinical Utility

18.7.1. Net Benefit and Decision Curve Analysis

A model might have good discrimination and calibration, yet using it may not actually improve clinical decisions. **Decision curve analysis (DCA)**, proposed by Vickers and Elkin (2006), addresses this by quantifying the **net benefit** of using the model compared to the default strategies of treating all patients or treating no patients.

The net benefit at a given threshold probability p_t is:

$$\text{Net benefit} = \frac{TP}{N} - \frac{FP}{N} \times \frac{p_t}{1 - p_t}$$

The term $\frac{p_t}{1 - p_t}$ represents the exchange rate between false positives and true positives, determined by the threshold probability. If the threshold is 10%, you are willing to accept up to 9 false positives for every true positive identified.

18. Performance Assessment and Model Validation

18.7.2. R

```
library(dcurves)

# Add predicted probabilities to the dataset
stroke_data$pred_risk <- predict(fit, type = "fitted")

# Decision curve analysis
dca_result <- dca(death_30d ~ pred_risk,
                  data = stroke_data,
                  thresholds = seq(0, 0.5, by = 0.01))

plot(dca_result,
     smooth = TRUE,
     show_ggplot_code = FALSE) +
  ggplot2::ggtitle("Decision Curve Analysis:\n30-Day Stroke Mortality Model")
```

18.7.3. Python

```
import numpy as np
import matplotlib.pyplot as plt

def net_benefit(y_true, y_pred, threshold):
    """Calculate net benefit at a given threshold."""
    n = len(y_true)
    pred_pos = (y_pred >= threshold).astype(int)
    tp = np.sum((pred_pos == 1) & (y_true == 1))
    fp = np.sum((pred_pos == 1) & (y_true == 0))
    nb = tp / n - fp / n * (threshold / (1 - threshold))
    return nb

thresholds = np.arange(0.01, 0.51, 0.01)
```

```

pred_prob_model = model.predict_proba(X)[: , 1]
y_true = stroke_data["death_30d"].values
n_total = len(y_true)

# Net benefit for model
nb_model = [net_benefit(y_true, pred_prob_model, t) for t in thresholds]

# Net benefit for "treat all" strategy
nb_all = [(y_true.mean() - (1 - y_true.mean()) * t / (1 - t))
          for t in thresholds]

# Net benefit for "treat none" is always 0

plt.figure(figsize=(9, 6))
plt.plot(thresholds, nb_model, color="steelblue", lw=2,
         label="Prediction Model")
plt.plot(thresholds, nb_all, color="coral", lw=2, ls="--",
         label="Treat All")
plt.axhline(y=0, color="black", lw=1, label="Treat None")
plt.xlabel("Threshold Probability")
plt.ylabel("Net Benefit")
plt.title("Decision Curve Analysis: 30-Day Stroke Mortality Model")
plt.legend()
plt.ylim(-0.05, max(max(nb_model), max(nb_all)) * 1.1)
plt.tight_layout()
plt.show()

```

18.7.4. Interpreting the Decision Curve

The decision curve shows net benefit (y-axis) across a range of threshold probabilities (x-axis). Three strategies are compared:

18. Performance Assessment and Model Validation

- **Treat none** (horizontal line at 0): no patients receive the intervention.
- **Treat all** (declining curve): all patients receive the intervention regardless of risk.
- **Model-guided** (the model's curve): only patients above the threshold receive the intervention.

The model is clinically useful at thresholds where its net benefit exceeds both the “treat all” and “treat none” lines. If the model curve is always below one of the default strategies, the model adds no value.

18.8. Internal Validation

18.8.1. Why Internal Validation Is Necessary

As discussed in Chapter 17, apparent performance (measured on the training data) is optimistically biased. Internal validation estimates and corrects for this optimism. It does not replace external validation, but it provides a more honest assessment of likely performance.

18.8.2. Bootstrap Optimism Correction

The recommended approach (Steyerberg 2019, Van Calster et al. 2025) is **bootstrap optimism correction**:

1. Draw a bootstrap sample (with replacement) from the original data.
2. Develop the model in the bootstrap sample (including all modelling decisions).
3. Measure performance in the bootstrap sample (apparent bootstrap performance).
4. Apply the bootstrap model to the original data (test performance).
5. Optimism = apparent bootstrap performance minus test performance.

6. Repeat steps 1–5 many times (e.g., 200+).
7. Average optimism across repetitions.
8. Corrected performance = original apparent performance minus average optimism.

18.8.3. R

```
library(rms)

# Using the stroke model
fit <- lrm(death_30d ~ age + nihss + glucose + afib + thrombolysis,
          data = stroke_data, x = TRUE, y = TRUE)

# Bootstrap validation with 200 resamples
set.seed(42)
val <- validate(fit, B = 200)

cat("Bootstrap Validation Results:\n")
print(val)

# Extract optimism-corrected C-statistic
dxy_corrected <- val["Dxy", "index.corrected"]
c_corrected <- (dxy_corrected + 1) / 2

cat("\nApparent C-statistic:", fit$stats["C"], "\n")
cat("Optimism:", val["Dxy", "optimism"] / 2, "\n")
cat("Optimism-corrected C-statistic:", c_corrected, "\n")

# Calibration slope from bootstrap
cat("\nCalibration slope (optimism-corrected):",
    val["Slope", "index.corrected"], "\n")
```

18.8.4. Python

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, brier_score_loss

np.random.seed(42)

predictors = ["age", "nihss", "glucose", "afib", "thrombolysis"]
X = stroke_data[predictors].values
y = stroke_data["death_30d"].values
n = len(y)

# Apparent performance
full_model = LogisticRegression(max_iter=5000, random_state=42)
full_model.fit(X, y)
pred_full = full_model.predict_proba(X)[:, 1]
apparent_auc = roc_auc_score(y, pred_full)
apparent_brier = brier_score_loss(y, pred_full)

# Bootstrap optimism correction
n_boot = 200
optimism_auc = []
optimism_brier = []

for b in range(n_boot):
    boot_idx = np.random.choice(n, n, replace=True)
    X_boot, y_boot = X[boot_idx], y[boot_idx]

    boot_model = LogisticRegression(max_iter=5000, random_state=42)
    boot_model.fit(X_boot, y_boot)

# Apparent performance on bootstrap sample
```



```

pred_boot = boot_model.predict_proba(X_boot)[:, 1]
auc_boot = roc_auc_score(y_boot, pred_boot)
brier_boot = brier_score_loss(y_boot, pred_boot)

# Test performance on original data
pred_orig = boot_model.predict_proba(X)[:, 1]
auc_orig = roc_auc_score(y, pred_orig)
brier_orig = brier_score_loss(y, pred_orig)

optimism_auc.append(auc_boot - auc_orig)
optimism_brier.append(brier_boot - brier_orig)

mean_opt_auc = np.mean(optimism_auc)
mean_opt_brier = np.mean(optimism_brier)
corrected_auc = apparent_auc - mean_opt_auc
corrected_brier = apparent_brier - mean_opt_brier

print(f"Apparent AUC:                {apparent_auc:.3f}")
print(f"Mean optimism (AUC):         {mean_opt_auc:.4f}")
print(f"Corrected AUC:                  {corrected_auc:.3f}")
print(f"\nApparent Brier:                {apparent_brier:.4f}")
print(f"Mean optimism (Brier):          {mean_opt_brier:.4f}")
print(f"Corrected Brier:                 {corrected_brier:.4f}")

```

18.8.5. Cross-Validation

Cross-validation (typically 10-fold) provides an alternative internal validation approach. The data is split into k folds; each fold serves once as the validation set while the remaining folds are used for model development. The average performance across folds estimates the model's performance on new data.

18. Performance Assessment and Model Validation

Cross-validation is computationally simpler but has limitations for prediction model development: it does not directly yield a single final model or a clear calibration slope correction. Bootstrap optimism correction is generally preferred for clinical prediction models.

18.9. External Validation

18.9.1. Types of External Validation

Internal validation tells you how optimistic your apparent performance is. **External validation** tells you whether the model works on genuinely new data — the real test. Van Calster et al. (2025) and Smits et al. (2026) distinguish:

- **Temporal validation:** Same setting, different time period. Example: model developed on 2015–2019 data, validated on 2020–2022 data. This captures changes in clinical practice and patient populations over time.
- **Geographical validation:** Different hospital or region, same time period. This tests transportability across settings.
- **Domain validation:** Different clinical context entirely (e.g., model developed in a tertiary care centre, validated in primary care). This is the most stringent test.

18.9.2. What to Report in External Validation

Following Van Calster et al. (2025), an external validation study should report at minimum:

1. **C-statistic** with 95% confidence interval
2. **Calibration plot** (both grouped and smoothed)
3. **O:E ratio** with 95% confidence interval

4. **Calibration slope**
5. **Net benefit** (decision curve) at clinically relevant thresholds
6. **Distribution of predicted probabilities** (to understand the range and spread)
7. **Performance by subgroups** (age, sex, comorbidities, etc.)

18.10. Model Updating

18.10.1. When Performance Degrades

External validation often reveals that a model does not perform as well in new settings as it did internally. This does not necessarily mean the model is useless. **Model updating** can adapt an existing model to a new population without developing a completely new model.

18.10.2. Updating Strategies

From least to most complex:

1. **Recalibration-in-the-large:** Adjust only the intercept. Corrects for differences in baseline risk (e.g., higher mortality rate in the new population).
2. **Logistic recalibration:** Adjust the intercept and apply a correction to the calibration slope. This corrects for both level and spread of predictions.
3. **Model revision:** Re-estimate some or all coefficients, or add new predictors. This requires larger sample sizes in the new population.

18.10.3. R

```
# Simulate external validation data (different setting)
set.seed(99)
n_ext <- 800

# New population: older, sicker
ext_data <- data.frame(
  age = round(rnorm(n_ext, 78, 10)), # Older
  nihss = round(pmax(0, rnorm(n_ext, 11, 7))), # Higher severity
  glucose = round(rnorm(n_ext, 155, 55)),
  afib = rbinom(n_ext, 1, 0.35),
  thrombolysis = rbinom(n_ext, 1, 0.20) # Less thrombolysis
)

lp_ext <- -5 + 0.04 * ext_data$age +
  0.12 * ext_data$nihss +
  0.003 * ext_data$glucose +
  0.3 * ext_data$afib -
  0.5 * ext_data$thrombolysis

ext_data$death_30d <- rbinom(n_ext, 1, plogis(lp_ext))

# Apply original model to external data
ext_data$pred_original <- predict(fit, newdata = ext_data, type = "fitted")

cat("External validation (before updating):\n")
cat("  Observed mortality:", mean(ext_data$death_30d), "\n")
cat("  Mean predicted:", mean(ext_data$pred_original), "\n")
cat("  O:E ratio:", round(mean(ext_data$death_30d) / mean(ext_data$pred_origi

# Calibration slope in external data
lp_orig <- qlogis(ext_data$pred_original)
```

18.10. Model Updating

```
cal_ext <- glm(death_30d ~ lp_orig, data = ext_data, family = binomial)
cat(" Calibration slope:", round(coef(cal_ext)[2], 3), "\n")
cat(" Calibration intercept:", round(coef(cal_ext)[1], 3), "\n")

# Recalibration: update intercept and slope
ext_data$pred_recalibrated <- predict(cal_ext, type = "response")

cat("\nAfter logistic recalibration:\n")
cat(" Mean predicted:", round(mean(ext_data$pred_recalibrated), 3), "\n")
cat(" O:E ratio:", round(mean(ext_data$death_30d) / mean(ext_data$pred_recalibrated), 3), "
```

18.10.4. Python

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

np.random.seed(99)
n_ext = 800

ext_data = {
    "age": np.round(np.random.normal(78, 10, n_ext)),
    "nihss": np.round(np.maximum(0, np.random.normal(11, 7, n_ext))),
    "glucose": np.round(np.random.normal(155, 55, n_ext)),
    "afib": np.random.binomial(1, 0.35, n_ext),
    "thrombolysis": np.random.binomial(1, 0.20, n_ext)
}

lp_ext = (-5 + 0.04 * ext_data["age"] +
          0.12 * ext_data["nihss"] +
          0.003 * ext_data["glucose"] +
          0.3 * ext_data["afib"] -
```

18. Performance Assessment and Model Validation

```
0.5 * ext_data["thrombolysis"])

y_ext = np.random.binomial(1, expit(lp_ext))

# Apply original model
X_ext = np.column_stack([ext_data[k] for k in predictors])
pred_original = full_model.predict_proba(X_ext)[:, 1]

obs_rate = y_ext.mean()
mean_pred = pred_original.mean()
print(f"External validation (before updating):")
print(f"  Observed mortality: {obs_rate:.3f}")
print(f"  Mean predicted: {mean_pred:.3f}")
print(f"  O:E ratio: {obs_rate / mean_pred:.3f}")
print(f"  AUC: {roc_auc_score(y_ext, pred_original):.3f}")

# Logistic recalibration: regress outcome on logit(predictions)
from scipy.special import logit
lp_original = logit(np.clip(pred_original, 1e-6, 1 - 1e-6))

recal_model = LogisticRegression(max_iter=5000)
recal_model.fit(lp_original.reshape(-1, 1), y_ext)

pred_recalibrated = recal_model.predict_proba(lp_original.reshape(-1, 1))[:,

print(f"\nAfter logistic recalibration:")
print(f"  Mean predicted: {pred_recalibrated.mean():.3f}")
print(f"  O:E ratio: {y_ext.mean() / pred_recalibrated.mean():.3f}")
print(f"  Calibration slope: {recal_model.coef_[0][0]:.3f}")
print(f"  Calibration intercept: {recal_model.intercept_[0]:.3f}")
```

18.11. Recommended Reporting: The Minimum Set

Based on Van Calster et al. (2025) and Smits et al. (2026), every prediction model evaluation should include at minimum:

1. **AUROC (C-statistic)** with confidence interval — for discrimination
2. **Calibration plot** (smoothed and/or grouped) — for calibration
3. **Decision curve** — for clinical utility
4. **Risk distribution plot** — to show the spread of predicted probabilities

These four visualisations together give a comprehensive picture. A model may look excellent on one plot and problematic on another — which is exactly why you need all four.

18.11.1. R

```
library(pROC)

pred_prob <- predict(fit, type = "fitted")
y_obs <- stroke_data$death_30d

par(mfrow = c(2, 2))

# Panel 1: ROC curve
roc_obj <- roc(y_obs, pred_prob, quiet = TRUE)
plot(roc_obj, col = "steelblue", lwd = 2,
     main = "A. Discrimination (ROC Curve)",
     print.auc = TRUE, print.auc.y = 0.4)

# Panel 2: Calibration plot
val.prob(pred_prob, y_obs, m = 100, cex = 0.5,
         main = "B. Calibration Plot")
```

18. Performance Assessment and Model Validation

```
# Panel 3: Risk distribution
hist(pred_prob[y_obs == 0], breaks = 30, col = rgb(0.4, 0.6, 0.8, 0.5),
     main = "C. Risk Distribution",
     xlab = "Predicted Probability", freq = FALSE, ylim = c(0, 12))
hist(pred_prob[y_obs == 1], breaks = 30, col = rgb(0.9, 0.4, 0.4, 0.5),
     add = TRUE, freq = FALSE)
legend("topright", legend = c("No event", "Event"),
     fill = c(rgb(0.4, 0.6, 0.8, 0.5), rgb(0.9, 0.4, 0.4, 0.5)))

# Panel 4: Simplified decision curve
thresholds <- seq(0.01, 0.5, by = 0.01)
nb_model <- nb_all <- numeric(length(thresholds))
n_total <- length(y_obs)

for (i in seq_along(thresholds)) {
  t <- thresholds[i]
  pred_pos <- as.numeric(pred_prob >= t)
  tp <- sum(pred_pos == 1 & y_obs == 1)
  fp <- sum(pred_pos == 1 & y_obs == 0)
  nb_model[i] <- tp / n_total - fp / n_total * (t / (1 - t))
  nb_all[i] <- mean(y_obs) - (1 - mean(y_obs)) * (t / (1 - t))
}

plot(thresholds, nb_model, type = "l", lwd = 2, col = "steelblue",
     main = "D. Decision Curve Analysis",
     xlab = "Threshold Probability", ylab = "Net Benefit",
     ylim = c(-0.05, max(nb_model, nb_all) * 1.1))
lines(thresholds, nb_all, lwd = 2, col = "coral", lty = 2)
abline(h = 0, col = "black")
legend("topright", legend = c("Model", "Treat All", "Treat None"),
     col = c("steelblue", "coral", "black"),
     lty = c(1, 2, 1), lwd = 2, cex = 0.8)
```



```
par(mfrow = c(1, 1))
```

18.11.2. Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.calibration import calibration_curve

pred_prob = full_model.predict_proba(X)[: , 1]
y_obs = stroke_data["death_30d"].values

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Panel A: ROC curve
fpr, tpr, _ = roc_curve(y_obs, pred_prob)
roc_auc = auc(fpr, tpr)
axes[0, 0].plot(fpr, tpr, color="steelblue", lw=2,
                label=f"AUC = {roc_auc:.3f}")
axes[0, 0].plot([0, 1], [0, 1], "--", color="grey")
axes[0, 0].set_xlabel("1 - Specificity")
axes[0, 0].set_ylabel("Sensitivity")
axes[0, 0].set_title("A. Discrimination (ROC Curve)")
axes[0, 0].legend()

# Panel B: Calibration plot
prob_true, prob_pred = calibration_curve(y_obs, pred_prob, n_bins=10,
                                       strategy="quantile")
axes[0, 1].plot(prob_pred, prob_true, "o-", color="steelblue", lw=2)
axes[0, 1].plot([0, 1], [0, 1], "--", color="grey")
axes[0, 1].set_xlabel("Mean Predicted Probability")
axes[0, 1].set_ylabel("Observed Proportion")
```

18. Performance Assessment and Model Validation

```
axes[0, 1].set_title("B. Calibration Plot")

# Panel C: Risk distribution
axes[1, 0].hist(pred_prob[y_obs == 0], bins=30, alpha=0.5,
                color="steelblue", label="No event", density=True)
axes[1, 0].hist(pred_prob[y_obs == 1], bins=30, alpha=0.5,
                color="coral", label="Event", density=True)
axes[1, 0].set_xlabel("Predicted Probability")
axes[1, 0].set_ylabel("Density")
axes[1, 0].set_title("C. Risk Distribution")
axes[1, 0].legend()

# Panel D: Decision curve
thresholds = np.arange(0.01, 0.51, 0.01)
nb_model = []
nb_all = []
n_total = len(y_obs)

for t in thresholds:
    pred_pos = (pred_prob >= t).astype(int)
    tp = np.sum((pred_pos == 1) & (y_obs == 1))
    fp = np.sum((pred_pos == 1) & (y_obs == 0))
    nb_model.append(tp / n_total - fp / n_total * (t / (1 - t)))
    nb_all.append(y_obs.mean() - (1 - y_obs.mean()) * (t / (1 - t)))

axes[1, 1].plot(thresholds, nb_model, color="steelblue", lw=2,
                label="Model")
axes[1, 1].plot(thresholds, nb_all, color="coral", lw=2, ls="--",
                label="Treat All")
axes[1, 1].axhline(y=0, color="black", lw=1, label="Treat None")
axes[1, 1].set_xlabel("Threshold Probability")
axes[1, 1].set_ylabel("Net Benefit")
axes[1, 1].set_title("D. Decision Curve Analysis")
```

```
axes[1, 1].legend(fontsize=9)

plt.tight_layout()
plt.show()
```

18.12. Exercises

18.12.1. Exercise 1: Calibration Assessment

Using the stroke mortality model developed in this chapter:

- Create a calibration plot using deciles of predicted risk. Is the model well-calibrated?
- Calculate the O:E ratio. What does it tell you?
- Calculate the calibration slope. Is there evidence of overfitting?
- Apply bootstrap optimism correction. How much does the C-statistic decrease? How much does the calibration slope decrease?

18.12.2. Exercise 2: External Validation Simulation

18.12.3. R

```
# Create three external validation populations that differ from development:
# Population A: Same demographics, 3 years later (temporal)
# Population B: Different hospital, younger patients (geographical)
# Population C: Primary care setting with lower severity (domain)

# For each population:
# a. Calculate C-statistic, O:E ratio, calibration slope
# b. Create calibration plots
```

18. Performance Assessment and Model Validation

```
# c. Determine which population shows worst calibration and explain why
# d. Perform logistic recalibration and show the improvement
```

18.12.4. Python

```
# Same exercise as R tab - create three external populations
# and assess model performance in each
```

18.12.5. Exercise 3: Decision Curve Interpretation

Consider a model for predicting preeclampsia in pregnant women. The model has an AUC of 0.82 and good calibration. You plot the decision curve.

- At what range of threshold probabilities is the model useful?
- A colleague argues that the model should not be used because the AUC is “only” 0.82. Using the decision curve, construct a counter-argument.
- How would the decision curve change if the follow-up action (closer monitoring) were very low-cost versus very high-cost?

18.12.6. Exercise 4: Complete Evaluation

Choose a clinical prediction model from the literature (e.g., QRISK3, Wells score, APACHE II). Using published validation data:

- Report the C-statistic with confidence interval.
- Describe the calibration (if a calibration plot is available).
- Is there a decision curve analysis? If not, what threshold range would be clinically relevant?
- Has the model been externally validated? In what populations?

- e. Based on your assessment, would you recommend implementing this model in your setting?

18.13. References and Further Reading

The primary framework for this chapter is drawn from Van Calster, McLerron, van Smeden, Worster,“; Steyerberg, and colleagues (2025) in The Lancet Digital Health, whose guide to assessing the performance of prediction models is the most comprehensive and up-to-date reference available on this topic, covering all five performance domains and providing clear recommendations for what to report. The textbook by Smits, van Kuijk, and Wynants (2026) complements this with a highly accessible treatment of internal and external validation, model updating, and practical guidance for applied researchers. Decision curve analysis was introduced by Vickers and Elkin (2006) in Medical Decision Making and is now considered an essential component of model evaluation; the `dcurves` R package and accompanying tutorials make it straightforward to implement. For calibration assessment specifically, the work of Van Calster, Nieboer, Vergouwe, De Cock, Pencina, and Steyerberg (2016) in the Journal of Clinical Epidemiology provides an excellent treatment of different calibration measures and their interpretation. Bootstrap validation methodology is thoroughly described in Steyerberg’s “Clinical Prediction Models” (2019, Springer) and in Harrell’s “Regression Modeling Strategies” (2015, Springer). The Brier score and its decomposition were formalised by Brier (1950) in Monthly Weather Review, and its application to clinical prediction models is discussed in Steyerberg and colleagues (2010) in Epidemiology.

19. Reporting Standards, TRIPOD+AI, and Clinical Impact

20. Reporting Standards, TRIPOD+AI, and Clinical Impact

20.1. Introduction

Thousands of clinical prediction models are published every year. Yet only a small fraction are ever validated externally, and fewer still make it into clinical practice. One of the major reasons is poor reporting: papers that omit critical methodological details, overstate performance, or fail to describe the model in enough detail for anyone to reproduce or implement it. In 2024, the TRIPOD+AI statement was published to address this problem head-on.

This chapter walks you through the current reporting standards for clinical prediction models, explains why they matter, and gives you the tools to produce work that meets the expectations of top medical journals.

20.2. Why reporting standards matter

Consider trying to use a published prediction model in your own clinical setting. You would need to know:

- What predictors are included, and exactly how they were measured
- What outcome was predicted, and at what time horizon

20. Reporting Standards, TRIPOD+AI, and Clinical Impact

- How missing data were handled
- How the model was validated, and on what population
- The model's calibration, not just its discrimination
- Whether the model would be feasible to use in your context

Remarkably, many published prediction model studies omit several of these details. A systematic review by Collins et al. found that reporting quality was poor across most published prediction model studies, motivating the development of formal reporting guidelines.

20.3. The TRIPOD+AI statement

The **Transparent Reporting of a multivariable prediction model for Individual Prognosis Or Diagnosis** (TRIPOD) guideline was first published in 2015. In 2024, it was updated as **TRIPOD+AI** to address machine learning and AI-based models.

TRIPOD+AI applies to studies that develop, validate, or update a prediction model, whether based on regression, machine learning, or deep learning. The key additions in the +AI version include:

- **Fairness evaluation:** assessing whether the model performs equitably across demographic groups
- **Open science:** sharing code, data (where possible), and the model itself
- **Handling of AI-specific issues:** hyperparameter tuning, computational requirements, software dependencies

20.3.1. Key TRIPOD+AI items

The full checklist contains items across title, abstract, introduction, methods, results, and discussion. Here are some of the most commonly missed items:

Study design and participants:

- Describe the study design (e.g., cohort, case-control, registry)
- Specify the eligibility criteria clearly
- Report dates of recruitment and follow-up

Predictors and outcome:

- List all candidate predictors, how they were defined, and when they were measured
- Define the outcome precisely, including the time horizon for prognostic models

Missing data:

- Report the amount of missing data for each variable
- Describe the method used to handle missing data (complete case analysis, imputation)
- If multiple imputation was used, report the number of imputations and imputation model

Model development:

- Describe the modelling approach (logistic regression, random forest, etc.)
- Report how continuous predictors were handled (linear terms, splines, categorised)
- For ML models: report hyperparameter tuning strategy, software, and random seeds

Model performance:

- Report discrimination (e.g., C-statistic) with confidence intervals
- Report calibration (calibration plot, calibration slope and intercept)
- Report clinical utility (decision curve analysis or net benefit) where relevant

20. Reporting Standards, TRIPOD+AI, and Clinical Impact

Validation:

- Describe the validation approach (internal, external, temporal)
- Report performance in the validation data, not just the development data

Exercise 12.1: TRIPOD+AI audit

Find a recently published clinical prediction model paper (try searching PubMed for “prediction model” in your area of interest). Using the TRIPOD+AI checklist, evaluate how many items the paper reports adequately.

Focus on:

1. Is the outcome clearly defined with a time horizon?
2. Is calibration reported, or only discrimination?
3. How was missing data handled?
4. Were continuous predictors appropriately modelled (splines) or just categorised?
5. Is the model available for others to use (coefficients, code, web calculator)?

Write a brief (one paragraph) critical appraisal.

20.4. From model to bedside: the implementation gap

Even well-developed, well-validated models face an uphill battle to clinical implementation. Smits, van Kuijk & Wynants (2026) dedicate five chapters of their book to this topic, covering the journey from model selection through innovation development, impact evaluation, and implementation.

20.4.1. Key barriers to implementation

1. **The model doesn't address a real clinical need.** Models built for academic interest rather than to solve a genuine decision problem rarely get implemented.
2. **The model isn't embedded in clinical workflow.** A model that requires a clinician to manually enter 15 variables into a spreadsheet will not be used, regardless of how accurate it is.
3. **Performance hasn't been demonstrated in the target population.** External validation in the specific setting where the model will be used is essential.
4. **Clinicians don't trust the model.** Transparency, explainability, and evidence of clinical utility (not just statistical performance) build trust.
5. **There is no plan for maintenance.** Models can degrade over time as populations and practices change. A plan for monitoring and updating is essential.

20.4.2. The prediction model-based innovation (PMBI) framework

Smits et al. (2026) introduce the concept of a PMBI: the complete clinical tool that wraps around a prediction model, including:

- The user interface (how predictions are displayed)
- Decision support (what actions are recommended at different risk levels)
- Integration with electronic health records
- Training materials for end users
- A monitoring and updating plan

i Note

Key insight: Building a good prediction model is necessary but not sufficient for clinical impact. The model must be embedded in a workable clinical tool, validated in the target setting, and demonstrated to improve patient outcomes.

20.5. Risk communication

Presenting predicted probabilities to patients and clinicians is not straightforward. Research in health literacy shows that:

- **Frequencies are easier to understand than probabilities.** “3 out of 100 patients like you” is clearer than “3% probability.”
- **Visual aids help.** Icon arrays (showing 100 faces with 3 highlighted) are effective.
- **Framing matters.** “97% chance of survival” feels different from “3% chance of death” — both are true.
- **Uncertainty should be communicated.** Presenting a point estimate without a confidence interval overstates precision.

20.5.1. Presenting results to different audiences

Audience	Recommended format
Patients	Frequencies, icon arrays, plain language
Clinicians	Risk categories with action thresholds, decision curves
Researchers	Full performance metrics, calibration plots, code
Policymakers	Population-level impact, cost-effectiveness

20.6. Ethical considerations

20.6.1. Algorithmic bias

Prediction models can perpetuate or amplify existing health disparities if:

- Training data underrepresents certain populations
- Predictors serve as proxies for protected characteristics (e.g., postcode as proxy for race/ethnicity)
- Performance is not evaluated across subgroups

TRIPOD+AI specifically requires fairness evaluation: reporting model performance stratified by relevant demographic groups.

20.6.2. Informed consent and transparency

Patients have a right to know when a prediction model is being used in their care, what data it uses, and how its output influences clinical decisions. This is particularly important for models that inform treatment decisions or resource allocation.

20.6.3. The EU AI Act and regulatory considerations

The European Union's AI Act (2024) classifies medical AI applications as “high risk” and imposes requirements for transparency, human oversight, and documentation. Clinical prediction models that qualify as medical devices may need regulatory approval (e.g., CE marking in the EU, FDA clearance in the US).

20.7. Writing a statistical methods section

A well-written methods section for a prediction model study should include:

1. **Study design and setting** (one paragraph)
2. **Participants** — eligibility criteria, dates, sample size (one paragraph)
3. **Predictors** — list, definitions, measurement timing (one paragraph)
4. **Outcome** — definition, ascertainment, time horizon (one paragraph)
5. **Missing data** — amount, mechanism assumed, handling method (one paragraph)
6. **Model development** — method, how continuous variables were handled, variable selection approach (one paragraph)
7. **Model performance** — discrimination, calibration, clinical utility measures (one paragraph)
8. **Validation** — internal and/or external validation approach (one paragraph)
9. **Software** — R/Python version, key packages, random seed (one sentence)



Exercise 12.2: Write a methods section

Using the prediction model you developed in earlier chapters (or a hypothetical one), write a complete statistical methods section following the structure above. Aim for approximately 500 words. Check your methods section against the TRIPOD+AI checklist. Are all key items covered?

Example methods section

Study design and setting. We conducted a retrospective cohort study using data from the Framingham Heart Study (original cohort, examinations 1–10). Participants were

adults aged 30–74 years free of cardiovascular disease at baseline.

Outcome. The primary outcome was incident coronary heart disease (CHD) within 10 years of baseline examination, defined as myocardial infarction, coronary insufficiency, or CHD death.

Predictors. Candidate predictors were age, sex, systolic blood pressure (modelled with restricted cubic splines, 4 knots), total cholesterol (restricted cubic splines, 4 knots), HDL cholesterol, current smoking status, diabetes status, and use of antihypertensive medication.

Missing data. Missing data ranged from 0% (age, sex) to 8% (HDL cholesterol). We used multiple imputation by chained equations (MICE, 20 imputations) assuming data were missing at random.

Model development. We fitted a logistic regression model. Continuous predictors were modelled with restricted cubic splines to allow for non-linear associations. No automated variable selection was performed; all pre-specified predictors were retained. The model was fitted on the full imputed dataset using Rubin's rules.

Model performance. Discrimination was quantified using the C-statistic with 95% confidence intervals. Calibration was assessed using calibration plots (loess smoother) and the calibration slope and intercept. Clinical utility was evaluated using decision curve analysis, reporting net benefit across decision thresholds from 5% to 40%.

Validation. Internal validation was performed using bootstrap resampling (500 samples) to estimate optimism-corrected performance.

20. Reporting Standards, TRIPOD+AI, and Clinical Impact

Software. All analyses were performed in R 4.4.1 using the rms, mice, and dcurves packages.

20.8. The future of clinical prediction

Several trends are shaping the next decade of clinical prediction modelling:

- **Dynamic prediction models** that update as new patient data become available (e.g., during a hospital stay)
- **Federated learning** that trains models across institutions without sharing patient data
- **Foundation models** adapted for clinical tasks (large language models, multimodal models)
- **Continuous model monitoring** with automated detection of performance degradation
- **Patient-facing prediction tools** integrated into health apps and patient portals

Regardless of the technology, the fundamentals covered in this course — proper validation, calibration assessment, clinical utility evaluation, and transparent reporting — will remain essential.

Exercise 12.3: Critical appraisal

Find a prediction model paper from 2024 or 2025 in a journal relevant to your field. Answer the following:

1. What clinical question does the model address?
2. Was the model developed with regression, ML, or both?
3. Was calibration reported? If so, was it assessed using a calibration plot?

4. Was clinical utility (decision curve analysis) reported?
5. Could you implement this model in your own setting with the information provided?
6. Does the paper comply with TRIPOD+AI?

Discuss your findings with a colleague or in the course discussion forum.

20.9. References and further reading

- Collins GS, Moons KGM, Dhiman P, et al. **TRIPOD+AI statement: updated guidance for reporting clinical prediction models that use regression or machine learning methods.** *BMJ* 2024;385:e078378. The definitive reporting guideline for prediction models.
- Smits LJM, van Kuijk SMJ, Wynants L. **Improving Health Care with Clinical Prediction Models: From Idea to Impact.** Maastricht University Press, 2026. Chapters 10–15 cover model selection for impact, innovation development, research question formulation, impact evaluation (decision modelling and empirical), and implementation.
- Van Calster B, Collins GS, Vickers AJ, et al. **Evaluation of performance measures in predictive artificial intelligence models to support medical decisions: overview and guidance.** *Lancet Digital Health* 2025;7:e100916. Comprehensive guide to choosing appropriate performance measures.
- Steyerberg EW. **Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating.** Springer, 2019. Chapter 23 covers implementation.

20. *Reporting Standards, TRIPOD+AI, and Clinical Impact*

- Vickers AJ, van Calster B, Steyerberg EW. **A simple, step-by-step guide to interpreting decision curve analysis.** *Diagnostic and Prognostic Research* 2019;3:18.
- **EU AI Act** (Regulation 2024/1689). Official text available at eur-lex.europa.eu.
- Gigerenzer G, Edwards A. **Simple tools for understanding risks: from innumeracy to insight.** *BMJ* 2003;327:741–744. Classic paper on risk communication.

Part VII.

Advanced Research Toolkit

21. Causal Inference for Observational Health Data

21.1. Introduction

Most clinical research relies on observational data — electronic health records, claims databases, registries, and cohort studies. Yet the questions we care about most are causal: *Does* this drug reduce mortality? *Would* this surgery improve outcomes compared with conservative management? Answering causal questions with observational data requires careful methodology, because the fundamental problem is that patients who receive a treatment are systematically different from those who do not.

This chapter equips you with the conceptual frameworks and practical tools to move from “Drug X is associated with lower mortality” to a defensible causal claim — or, equally important, to recognise when the data cannot support one.

! The stakes are real

Confounding by indication — where sicker patients receive more aggressive treatment — has led to published studies concluding that treatments *cause harm* when they actually help, and vice versa. The Women’s Health Initiative overturned decades of observational evidence on hormone replacement therapy. Causal inference methods

21. Causal Inference for Observational Health Data

do not guarantee correct answers, but they make your assumptions explicit and testable.

21.2. Correlation vs Causation in Medicine

21.2.1. Why simple regression is not enough

Consider a hospital database showing that patients who receive mechanical ventilation have higher mortality than those who do not. A naive analyst might conclude ventilation is harmful. But the confounding is obvious: ventilation is given to the sickest patients.

This is **confounding by indication** — the indication for treatment is itself a risk factor for the outcome. Standard regression can adjust for *measured* confounders, but:

1. You must know which variables to include (and which to exclude).
2. You must model the relationships correctly (linearity, interactions).
3. You cannot adjust for *unmeasured* confounders.

Causal inference methods address problems 1 and 2 more systematically, and provide tools to assess sensitivity to problem 3.

21.2.2. The potential outcomes framework

The modern causal inference framework, formalised by Rubin (1974) and extended by Hernan and Robins, rests on **potential outcomes**:

- $Y^{a=1}$: the outcome a patient *would have* under treatment
- $Y^{a=0}$: the outcome a patient *would have* under control

21.3. Directed Acyclic Graphs (DAGs)

The **individual causal effect** is $Y^{a=1} - Y^{a=0}$, but we can never observe both for the same person. This is the **fundamental problem of causal inference**.

We can, however, estimate the **average treatment effect (ATE)**:

$$\text{ATE} = E[Y^{a=1}] - E[Y^{a=0}]$$

or the **average treatment effect on the treated (ATT)**:

$$\text{ATT} = E[Y^{a=1} - Y^{a=0} \mid A = 1]$$

The key assumption that allows us to estimate these from observational data is **exchangeability** (no unmeasured confounding): conditional on measured covariates L , treatment assignment is independent of potential outcomes.

$$Y^a \perp\!\!\!\perp A \mid L$$

21.3. Directed Acyclic Graphs (DAGs)

21.3.1. What is a DAG?

A **directed acyclic graph** is a visual representation of your causal assumptions. Each node represents a variable. Each arrow represents a direct causal effect. “Acyclic” means no variable can cause itself through a chain of arrows.

Figure 21.1.: A simple DAG showing confounding. Age affects both statin use and cardiovascular death.

21.3.2. Key structures in DAGs

There are three fundamental structures you must recognise:

1. Confounders (common causes)

A variable that causes both treatment and outcome. You **MUST** adjust for confounders to remove bias.

L --> A --> Y
L -----> Y

If L is diabetes severity, A is insulin therapy, and Y is HbA1c, then diabetes severity confounds the insulin-HbA1c relationship.

2. Mediators (intermediate variables)

A variable on the causal pathway from treatment to outcome. You should **NOT** adjust for mediators if you want the total effect.

A --> M --> Y

If A is exercise, M is weight loss, and Y is blood pressure, then weight loss mediates the exercise-BP relationship. Adjusting for weight loss would block part of the effect you want to estimate.

3. Colliders (common effects)

A variable caused by both treatment and outcome (or by variables on each path). You should **NOT** adjust for colliders — doing so *creates* a spurious association (collider bias).

A --> C <-- Y

21.3. Directed Acyclic Graphs (DAGs)

If **A** is obesity, **Y** is genetic predisposition to diabetes, and **C** is being in a diabetes clinic, then conditioning on clinic attendance (a collider) creates a spurious negative association between obesity and genetic risk.

💡 The “d-separation” rule of thumb

A path between treatment and outcome is **blocked** if:

- It passes through a variable you condition on (confounder or mediator), OR
- It passes through a collider you do NOT condition on.

A path is **open** if it is not blocked. Bias exists when there are open non-causal paths (backdoor paths) between treatment and outcome.

21.3.3. Drawing your own DAG

Before any causal analysis, draw a DAG. Steps:

1. List all variables relevant to treatment assignment and the outcome.
2. Use clinical knowledge to draw arrows (not data — DAGs encode assumptions).
3. Identify all backdoor paths from treatment to outcome.
4. Determine the minimal adjustment set that blocks all backdoor paths without opening new ones.

Tools like [DAGitty](#) automate step 4 once you have drawn the DAG.

21.4. Propensity Score Methods

21.4.1. The propensity score

The **propensity score** is the probability of receiving treatment given observed covariates:

$$e(L) = P(A = 1 \mid L)$$

Rosenbaum and Rubin (1983) showed that, if exchangeability holds conditional on L , it also holds conditional on the scalar $e(L)$. This reduces a high-dimensional adjustment problem to a single dimension.

21.4.2. Estimating the propensity score

We typically estimate $e(L)$ using logistic regression (or more flexible methods like gradient boosting):

```
# Simulate clinical data
set.seed(42)
n <- 2000

dat <- tibble(
  age = rnorm(n, 65, 10),
  diabetes = rbinom(n, 1, 0.3),
  egfr = rnorm(n, 60, 15),
  smoking = rbinom(n, 1, 0.25),
  # Treatment assignment depends on covariates (confounding)
  ps_true = plogis(-2 + 0.03 * age + 0.5 * diabetes - 0.02 * egfr + 0.4 * smoking),
  treatment = rbinom(n, 1, ps_true),
  # Outcome depends on treatment AND covariates
  mortality = rbinom(n, 1, plogis(-3 + 0.04 * age + 0.6 * diabetes -
```

21.4. Propensity Score Methods

```
0.03 * egfr + 0.3 * smoking -  
0.5 * treatment))  
  
)  
  
# Estimate propensity score via logistic regression  
ps_model <- glm(treatment ~ age + diabetes + egfr + smoking,  
                data = dat, family = binomial)  
dat$ps <- predict(ps_model, type = "response")  
  
# Visualise overlap  
ggplot(dat, aes(x = ps, fill = factor(treatment))) +  
  geom_density(alpha = 0.5) +  
  labs(x = "Propensity Score", fill = "Treatment",  
       title = "Propensity Score Distribution by Treatment Group") +  
  theme_minimal()
```

Positivity assumption

Every patient must have a non-zero probability of receiving *either* treatment. If some patients have propensity scores near 0 or 1, the overlap assumption is violated and estimates become unstable. Always check the propensity score distributions visually.

21.4.3. Propensity score matching

Matching creates a pseudo-randomised subset by pairing treated and control patients with similar propensity scores.

```
# Using MatchIt for nearest-neighbour matching with caliper  
m_out <- matchit(treatment ~ age + diabetes + egfr + smoking,  
                data = dat,  
                method = "nearest",
```

21. Causal Inference for Observational Health Data

```
distance = "glm",      # logistic regression PS
caliper = 0.2,         # 0.2 SD of logit PS
ratio = 1)            # 1:1 matching

summary(m_out)

# Check covariate balance using cobalt
love.plot(m_out,
  thresholds = c(m = 0.1),    # SMD threshold of 0.1
  binary = "std",
  title = "Covariate Balance: Before and After Matching")

# Extract matched data and estimate treatment effect
m_data <- match.data(m_out)

# Outcome model in matched sample
outcome_model <- glm(mortality ~ treatment,
  data = m_data,
  family = binomial,
  weights = weights)

tidy(outcome_model, conf.int = TRUE, exponentiate = TRUE)
```

Checking balance is the most important step after matching. A standardised mean difference (SMD) below 0.1 for all covariates is the conventional threshold for acceptable balance.

21.4.4. Inverse probability of treatment weighting (IPTW)

Instead of discarding unmatched patients, **IPTW** reweights the entire sample so that the distribution of covariates is balanced between groups:

21.4. Propensity Score Methods

$$w_i = \frac{A_i}{e(L_i)} + \frac{1 - A_i}{1 - e(L_i)}$$

This estimates the ATE. For the ATT, the weights are:

$$w_i = A_i + \frac{(1 - A_i) \cdot e(L_i)}{1 - e(L_i)}$$

```
# Using WeightIt
w_out <- weightit(treatment ~ age + diabetes + egfr + smoking,
                  data = dat,
                  method = "ps",
                  estimand = "ATE")

# Check balance
bal.tab(w_out, thresholds = c(m = 0.1))

# Stabilised weights (recommended to reduce variance)
summary(w_out)

# Estimate treatment effect with IPTW
library(survey)
d_weighted <- svydesign(ids = ~1, weights = w_out$weights, data = dat)
svyglm(mortality ~ treatment, design = d_weighted, family = quasibinomial) |>
  tidy(conf.int = TRUE, exponentiate = TRUE)
```

i Stabilised weights

Extreme weights can inflate variance. **Stabilised weights** replace the numerator 1 with $P(A = 1)$ for treated and $P(A = 0)$ for untreated. The **WeightIt** package does this by default. Always check that weights are not excessively large (e.g., max weight > 20 warrants investigation).

21.4.5. Doubly robust estimation

Doubly robust methods combine a propensity score model and an outcome model. The estimate is consistent if *either* model is correctly specified (but not necessarily both). This provides an extra layer of protection against model misspecification.

The augmented inverse probability weighted (AIPW) estimator is:

$$\hat{\tau}_{DR} = \frac{1}{n} \sum_{i=1}^n \left[\frac{A_i Y_i}{e(L_i)} - \frac{A_i - e(L_i)}{e(L_i)} \hat{\mu}_1(L_i) \right] - \frac{1}{n} \sum_{i=1}^n \left[\frac{(1 - A_i) Y_i}{1 - e(L_i)} + \frac{A_i - e(L_i)}{1 - e(L_i)} \hat{\mu}_0(L_i) \right]$$

where $\hat{\mu}_a(L)$ is the predicted outcome under treatment a .

In practice, packages like AIPW in R or DoWhy in Python handle this computation.

21.5. Target Trial Emulation

21.5.1. The framework

Target trial emulation, developed by Hernan and Robins, asks: “What randomised trial would we *like* to conduct?” and then designs the observational analysis to mimic that trial as closely as possible.

The key components to specify are:

Protocol component	Target trial	Observational emulation
Eligibility criteria	Defined inclusion/exclusion	Same criteria applied to database

21.5. Target Trial Emulation

Protocol component	Target trial	Observational emulation
Treatment strategies	e.g., start drug A vs drug B	Same, defined at “time zero”
Treatment assignment	Random	Adjusted via PS methods or g-estimation
Start of follow-up	Randomisation date	Date eligibility + treatment criteria met
Outcome	e.g., 5-year mortality	Same
Causal contrast	Intention-to-treat or per-protocol	Same (with appropriate methods)
Analysis plan	Intention-to-treat analysis	Clone-censor-weight or similar

21.5.2. Why this reduces bias

Target trial emulation directly addresses several common biases:

Immortal time bias: In a naive observational study comparing drug users to non-users, the time between cohort entry and first prescription is “immortal” — the patient must survive to receive the drug. This artificially inflates survival in the treated group. Target trial emulation avoids this by aligning “time zero” (start of follow-up) with treatment assignment.

Selection bias from prevalent user designs: Studying patients already on a drug (prevalent users) conflates treatment initiation effects with survival conditioning. Target trial emulation uses a **new-user (incident user) design**.

Vague eligibility and time zero: When eligibility criteria and the start of follow-up are not clearly defined, analyses can be plagued by look-ahead bias. The target trial framework forces explicit specification.

21. Causal Inference for Observational Health Data

The TARGET reporting guideline (2025)

The TARGET (TrAnsparent ReportinG of observational studies Emulating a Target trial) guideline provides a structured checklist for reporting target trial emulation studies. It was published in the BMJ in 2025 and should be followed for any observational study making causal claims.

21.5.3. Example: Statin initiation and cardiovascular events

Suppose we want to estimate the effect of initiating statins within 6 months of a type 2 diabetes diagnosis on 5-year cardiovascular events.

Target trial specification:

- **Eligibility:** Adults aged 40–75 with new T2DM diagnosis, no prior CVD, no prior statin use
- **Treatment strategies:** (1) Initiate statin within 6 months, (2) Do not initiate statin within 6 months
- **Time zero:** Date of T2DM diagnosis
- **Outcome:** First major adverse cardiovascular event (MACE) within 5 years
- **Analysis:** Intention-to-treat using IPTW for confounding, clone-censor-weight for treatment strategy adherence

21.6. Regression Discontinuity Design

Regression discontinuity (RD) exploits arbitrary thresholds in clinical decision-making. When treatment is assigned based on whether a continuous variable exceeds a cutoff, patients just above and just below the cutoff are essentially randomly assigned.

Clinical example: Many guidelines recommend antihypertensive therapy when systolic blood pressure exceeds 140 mmHg. Patients with SBP of 141 are very similar to patients with SBP of 139, but the former are much more likely to receive treatment. Comparing outcomes near the cutoff provides a quasi-experimental estimate.

Key requirements for a valid RD design:

1. **Treatment probability changes sharply at the cutoff** (first stage)
2. **Other covariates do not jump at the cutoff** (continuity assumption)
3. **Patients cannot precisely manipulate their score** (no bunching)

RD designs are less common in clinical research than propensity score methods, but when applicable, they provide strong evidence because the assumptions are more testable.

21.7. Implementation in Python

Python's causal inference ecosystem is maturing. Here we demonstrate propensity score matching and IPTW using `dowhy` and `causal inference`.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from scipy import stats
import matplotlib.pyplot as plt

# Simulate clinical data
np.random.seed(42)
n = 2000
```

21. Causal Inference for Observational Health Data

```
age = np.random.normal(65, 10, n)
diabetes = np.random.binomial(1, 0.3, n)
egfr = np.random.normal(60, 15, n)
smoking = np.random.binomial(1, 0.25, n)

# Confounded treatment assignment
ps_true = 1 / (1 + np.exp(-(-2 + 0.03*age + 0.5*diabetes - 0.02*egfr + 0.4*smoking)))
treatment = np.random.binomial(1, ps_true)

# Outcome
mortality_prob = 1 / (1 + np.exp(-(-3 + 0.04*age + 0.6*diabetes - 0.03*egfr + 0.3*smoking - 0.5*treatment)))
mortality = np.random.binomial(1, mortality_prob)

df = pd.DataFrame({
    'age': age, 'diabetes': diabetes, 'egfr': egfr,
    'smoking': smoking, 'treatment': treatment, 'mortality': mortality
})

# --- Propensity Score Estimation ---
covariates = ['age', 'diabetes', 'egfr', 'smoking']
ps_model = LogisticRegression(max_iter=1000)
ps_model.fit(df[covariates], df['treatment'])
df['ps'] = ps_model.predict_proba(df[covariates])[:, 1]

# --- IPTW ---
# Compute ATE weights
df['iptw'] = np.where(
    df['treatment'] == 1,
    1 / df['ps'],
    1 / (1 - df['ps'])
)
```

21.7. Implementation in Python

```
# Stabilised weights
p_treat = df['treatment'].mean()
df['sw'] = np.where(
    df['treatment'] == 1,
    p_treat / df['ps'],
    (1 - p_treat) / (1 - df['ps'])
)

# Check balance: weighted standardised mean differences
def weighted_smd(data, var, treatment_col, weight_col):
    treated = data[data[treatment_col] == 1]
    control = data[data[treatment_col] == 0]
    mean_t = np.average(treated[var], weights=treated[weight_col])
    mean_c = np.average(control[var], weights=control[weight_col])
    sd_pooled = np.sqrt((treated[var].var() + control[var].var()) / 2)
    return (mean_t - mean_c) / sd_pooled

print("Covariate balance (weighted SMD):")
for var in covariates:
    smd = weighted_smd(df, var, 'treatment', 'sw')
    print(f" {var}: {smd:.4f}")

# --- Weighted outcome estimation ---
from statsmodels.api import WLS, add_constant

X = add_constant(df['treatment'])
wls_model = WLS(df['mortality'], X, weights=df['sw']).fit()
print("\nIPTW Treatment Effect Estimate:")
print(f" Coefficient: {wls_model.params[1]:.4f}")
print(f" 95% CI: ({wls_model.conf_int()[1][0]:.4f}, {wls_model.conf_int()[1][1]:.4f})")
```

21. Causal Inference for Observational Health Data

21.7.1. Using DoWhy for causal analysis

```
# DoWhy provides a structured causal inference workflow
# Install: pip install dowhy

import dowhy
from dowhy import CausalModel

# Step 1: Model - encode causal assumptions
model = CausalModel(
    data=df,
    treatment='treatment',
    outcome='mortality',
    common_causes=['age', 'diabetes', 'egfr', 'smoking']
)

# Step 2: Identify - find valid adjustment strategy
identified_estimand = model.identify_effect()
print(identified_estimand)

# Step 3: Estimate - using propensity score weighting
estimate = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_weighting",
    method_params={"weighting_scheme": "ips_stabilized_weight"}
)
print(f"Causal Estimate: {estimate.value:.4f}")

# Step 4: Refute - sensitivity analysis
refutation = model.refute_estimate(
    identified_estimand, estimate,
    method_name="random_common_cause"
```

```
)
print(refutation)
```

21.8. Sensitivity Analysis for Unmeasured Confounding

No observational study can prove the absence of unmeasured confounding. **Sensitivity analysis** quantifies how strong unmeasured confounding would need to be to explain away the observed effect.

The **E-value** (VanderWeele and Ding, 2017) provides a simple summary: it is the minimum strength of association (on the risk ratio scale) that an unmeasured confounder would need to have with both the treatment and the outcome to fully explain away the observed treatment-outcome association.

$$\text{E-value} = RR + \sqrt{RR \times (RR - 1)}$$

where RR is the observed risk ratio. The E-value for the confidence interval bound closest to the null is also reported.

```
# E-value calculation
library(EValue)

# Suppose our IPTW analysis found RR = 0.65 (95% CI: 0.50, 0.85)
# Convert protective RR to above-null: 1/0.65 = 1.538
evaluates.RR(est = 1/0.65, lo = 1/0.85, hi = 1/0.50)
```

21.9. Practical Guidance: Choosing Your Method

Scenario	Recommended approach
Binary treatment, many controls available	Propensity score matching
Binary treatment, need full sample	IPTW
Concern about model misspecification	Doubly robust estimation
Time-varying treatment	Marginal structural models (IPTW over time)
Natural threshold for treatment	Regression discontinuity
Emulating a specific trial design	Target trial emulation framework

21.10. Exercises

21.10.1. Exercise 1: DAG construction (Conceptual)

You are studying the relationship between **ACE inhibitor use** and **acute kidney injury (AKI)** in hospitalised patients.

- List at least five variables that might be relevant to this relationship.
- Draw a DAG (on paper or using DAGitty) encoding your causal assumptions.
- Identify the minimal sufficient adjustment set for estimating the effect of ACE inhibitors on AKI.
- Identify at least one potential collider. What would happen if you adjusted for it?

21.10.2. Exercise 2: Propensity score matching in R

Using the simulated dataset below, perform propensity score matching and estimate the treatment effect.

```
set.seed(123)
n <- 1500
exercise_dat <- tibble(
  age = rnorm(n, 70, 8),
  creatinine = rnorm(n, 1.2, 0.4),
  heart_failure = rbinom(n, 1, 0.35),
  prior_mi = rbinom(n, 1, 0.20),
  # Confounded treatment (beta-blocker use)
  treatment = rbinom(n, 1, plogis(-1 + 0.02*age + 0.3*heart_failure +
                                0.5*prior_mi - 0.8*creatinine)),
  # Outcome: 1-year mortality
  death_1yr = rbinom(n, 1, plogis(-2 + 0.05*age + 0.4*heart_failure +
                                0.6*prior_mi + 1.0*creatinine -
                                0.7*treatment))
)
```

- Estimate propensity scores using logistic regression.
- Perform 1:1 nearest-neighbour matching with a caliper of 0.2 SD.
- Create a Love plot to assess balance before and after matching.
- Estimate the ATT for beta-blocker use on 1-year mortality.
- Calculate the E-value for your estimate. Interpret it.

21.10.3. Exercise 3: IPTW in Python

Using the same data-generating process as Exercise 2 (replicate it in Python):

21. Causal Inference for Observational Health Data

```
import numpy as np
import pandas as pd

np.random.seed(123)
n = 1500
age = np.random.normal(70, 8, n)
creatinine = np.random.normal(1.2, 0.4, n)
heart_failure = np.random.binomial(1, 0.35, n)
prior_mi = np.random.binomial(1, 0.20, n)

ps_true = 1 / (1 + np.exp(-(-1 + 0.02*age + 0.3*heart_failure +
                             0.5*prior_mi - 0.8*creatinine)))
treatment = np.random.binomial(1, ps_true)

mort_prob = 1 / (1 + np.exp(-(-2 + 0.05*age + 0.4*heart_failure +
                             0.6*prior_mi + 1.0*creatinine -
                             0.7*treatment)))
death_1yr = np.random.binomial(1, mort_prob)

df = pd.DataFrame({
    'age': age, 'creatinine': creatinine, 'heart_failure': heart_failure,
    'prior_mi': prior_mi, 'treatment': treatment, 'death_1yr': death_1yr
})
```

- Fit a propensity score model and compute stabilised IPTW weights.
- Assess covariate balance using weighted standardised mean differences.
- Estimate the ATE using weighted regression.
- Perform a sensitivity analysis: add a simulated unmeasured confounder and show how the estimate changes.

21.10.4. Exercise 4: Target trial emulation (Conceptual)

You have access to a large electronic health records database and want to estimate the effect of **early metformin initiation** (within 3 months of type 2 diabetes diagnosis) vs **delayed initiation** (3–12 months after diagnosis) on 5-year cardiovascular events.

- a) Write a complete target trial protocol table (eligibility, treatment strategies, assignment, start of follow-up, outcome, causal contrast, analysis plan).
- b) Identify potential sources of immortal time bias in a naive analysis.
- c) Describe how the new-user, active comparator design addresses confounding by indication.
- d) What unmeasured confounders might remain? How would you conduct a sensitivity analysis?

21.11. Summary

Concept	Key point
Confounding by indication	Sicker patients get more treatment — naive comparisons are biased
DAGs	Encode causal assumptions visually; identify what to adjust for
Propensity score	$P(\text{treatment} \mid \text{covariates})$; reduces dimensionality
Matching	Creates balanced pseudo-randomised groups; check balance
IPTW	Reweights full sample; preserves sample size
Doubly robust	Combines PS + outcome model; consistent if either is correct

21. Causal Inference for Observational Health Data

Concept	Key point
Target trial emulation	Design observational analysis to mimic an ideal RCT
E-value	Quantifies robustness to unmeasured confounding

21.12. References and Further Reading

1. **Hernan MA, Robins JM.** *Causal Inference: What If*. Chapman & Hall/CRC, 2024. Freely available at <https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/>. The definitive modern textbook on causal inference.
2. **Rosenbaum PR.** *Observational Studies*. 2nd ed. Springer, 2002. The classic text on propensity score methods and sensitivity analysis.
3. **Rosenbaum PR, Rubin DB.** The central role of the propensity score in observational studies for causal effects. *Biometrika*. 1983;70(1):41–55.
4. **TARGET Guideline.** TrAnsparent ReportinG of observational studies Emulating a Target trial. *BMJ*. 2025. Reporting checklist for target trial emulation studies.
5. **VanderWeele TJ, Ding P.** Sensitivity analysis in observational research: introducing the E-value. *Annals of Internal Medicine*. 2017;167(4):268–274.
6. **Textor J, van der Zander B, Gilthorpe MS, et al.** Robust causal inference using directed acyclic graphs: the R package ‘dagitty’. *International Journal of Epidemiology*. 2016;45(6):1887–1894.

21.12. References and Further Reading

7. **Ho DE, Imai K, King G, Stuart EA.** MatchIt: nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*. 2011;42(8):1–28.
8. **Greifer N.** WeightIt: weighting for covariate balance in observational studies. R package. <https://ngreifer.github.io/WeightIt/>.
9. **Sharma A, Kiciman E.** DoWhy: An end-to-end library for causal inference. arXiv:2011.04216, 2020.
10. **Lodi S, Phillips A, Lundgren J, et al.** Effect estimates in randomized trials and observational studies: comparing apples with apples. *American Journal of Epidemiology*. 2019;188(8):1569–1577.

22. Meta-Analysis Methods for Evidence Synthesis

22.1. Introduction

A single study, no matter how well designed, provides only one piece of evidence. **Meta-analysis** is the statistical method for combining results from multiple independent studies addressing the same question, producing a more precise and generalisable estimate of an effect.

Meta-analysis sits at the top of the evidence hierarchy in evidence-based medicine. When done well, it can resolve conflicting findings, increase statistical power for rare outcomes, and identify sources of variation across studies. When done poorly, it can amplify biases present in the original studies — “garbage in, garbage out.”

This chapter covers the statistical foundations of meta-analysis, practical implementation in R and Python, and critical appraisal of heterogeneity and publication bias.

i Systematic review vs meta-analysis

A **systematic review** is the process of comprehensively searching for, selecting, and appraising studies. A **meta-analysis** is the *statistical* component — combining the numbers. You can have a systematic review without a meta-analysis (if studies are too heterogeneous to combine), but you should never have a meta-analysis without a

systematic review.

22.2. What Does a Meta-Analysis Combine?

Each study contributes an **effect estimate** and a measure of its **precision** (standard error or confidence interval). The meta-analysis computes a **weighted average**, where studies with greater precision (typically larger studies) receive more weight.

The general formula for a weighted average effect:

$$\hat{\theta}_{\text{pooled}} = \frac{\sum_{i=1}^k w_i \hat{\theta}_i}{\sum_{i=1}^k w_i}$$

where $\hat{\theta}_i$ is the effect estimate from study i and w_i is its weight.

22.3. Effect Measures

The choice of effect measure depends on the type of outcome:

22.3.1. Binary outcomes

Measure	Formula	When to use
Odds ratio (OR)	$\frac{a/c}{b/d} = \frac{ad}{bc}$	Case-control studies; logistic regression
Risk ratio (RR)	$\frac{a/(a+b)}{c/(c+d)}$	Cohort studies; more interpretable than OR

22.3. Effect Measures

Measure	Formula	When to use
Risk difference (RD)	$\frac{a}{a+b} - \frac{c}{c+d}$	When absolute risk is clinically relevant

Where a , b , c , d are the cells of a 2x2 table (treatment events, treatment non-events, control events, control non-events).

ORs and RRs are typically meta-analysed on the **log scale** (because their sampling distributions are approximately normal on that scale) and then back-transformed for presentation.

22.3.2. Continuous outcomes

Measure	Formula	When to use
Mean difference (MD)	$\bar{X}_T - \bar{X}_C$	All studies use the same measurement scale
Standardised mean difference (SMD)	$\frac{\bar{X}_T - \bar{X}_C}{S_{\text{pooled}}}$	Studies use different scales measuring the same construct (e.g., different depression questionnaires)

The SMD is also known as **Hedges' g** (with a small-sample correction) or **Cohen's d**.

22.4. Fixed-Effect vs Random-Effects Models

22.4.1. Fixed-effect model

Assumes all studies estimate the **same true effect** θ . Differences between study results are due solely to sampling variation.

Weights are the inverse of the within-study variance:

$$w_i^{FE} = \frac{1}{\hat{\sigma}_i^2}$$

This model is appropriate when studies are clinically and methodologically very similar (e.g., exact replications).

22.4.2. Random-effects model

Assumes each study estimates its **own true effect** θ_i , drawn from a distribution of true effects: $\theta_i \sim N(\mu, \tau^2)$.

The between-study variance τ^2 is estimated from the data (commonly using the DerSimonian-Laird, REML, or Paule-Mandel methods). Weights incorporate both within-study and between-study variance:

$$w_i^{RE} = \frac{1}{\hat{\sigma}_i^2 + \hat{\tau}^2}$$

The random-effects model is almost always more appropriate in clinical research, because studies differ in populations, interventions, settings, and outcome definitions.

! The random-effects model gives more weight to small studies

Because $\hat{\tau}^2$ is added to every study's variance, the relative weights become more equal. This means small (potentially biased or low-quality) studies have more influence in a random-effects analysis. This is a feature when heterogeneity is real, but a bug when small studies are biased (e.g., publication bias favoring small positive studies).

22.4.3. Prediction intervals

The confidence interval for the pooled effect tells you about the *average* true effect. The **prediction interval** tells you the range within which the true effect of a *future study* is likely to fall:

$$\hat{\mu} \pm t_{k-2, 0.975} \times \sqrt{\text{SE}(\hat{\mu})^2 + \hat{\tau}^2}$$

Prediction intervals are often much wider than confidence intervals and provide a more honest picture of the uncertainty. A pooled effect may be statistically significant while the prediction interval includes the null — meaning that some settings may see no benefit or even harm.

22.5. Heterogeneity

22.5.1. Measuring heterogeneity

Cochran's Q test: Tests H_0 : all studies share the same true effect.

$$Q = \sum_{i=1}^k w_i^{FE} (\hat{\theta}_i - \hat{\theta}_{FE})^2$$

22. Meta-Analysis Methods for Evidence Synthesis

Under H_0 , $Q \sim \chi^2_{k-1}$. The test has low power when k is small.

I^2 statistic: The percentage of total variability due to between-study heterogeneity rather than chance.

$$I^2 = \max\left(0, \frac{Q - (k - 1)}{Q}\right) \times 100\%$$

Rules of thumb (Higgins et al., 2003):

- $I^2 \approx 25\%$: low heterogeneity
- $I^2 \approx 50\%$: moderate heterogeneity
- $I^2 \approx 75\%$: high heterogeneity

τ^2 : The estimated between-study variance on the effect scale. Unlike I^2 , it is not influenced by study precision and is more interpretable for clinical decision-making.

22.5.2. Investigating heterogeneity

When heterogeneity is substantial:

1. **Subgroup analysis:** Split studies by a pre-specified characteristic (e.g., drug dose, patient age group, study quality) and compare pooled effects.
2. **Meta-regression:** Model the effect as a function of study-level covariates.
3. **Sensitivity analysis:** Exclude outlier studies or studies at high risk of bias.



Ecological fallacy in meta-regression

Study-level associations do not imply individual-level associations. If trials with older populations show larger treatment effects, this

does NOT prove that older individuals benefit more — the trials may also differ in other ways. This is the **ecological fallacy**, and it is a fundamental limitation of aggregate data meta-analysis.

22.6. Forest Plots

The **forest plot** is the standard visualisation for meta-analysis. Each study is shown as a square (size proportional to weight) with a horizontal line (95% CI). The pooled estimate is shown as a diamond.

```
# Example data: RCTs of statins for secondary CV prevention
statin_data <- data.frame(
  study = c("4S (1994)", "CARE (1996)", "LIPID (1998)", "HPS (2002)",
            "PROSPER (2002)", "PROVE-IT (2004)", "TNT (2005)",
            "JUPITER (2008)", "HOPE-3 (2016)"),
  events_treat = c(111, 212, 287, 1328, 127, 147, 334, 142, 235),
  n_treat      = c(2221, 2081, 4512, 10269, 2891, 2099, 5006, 8901, 6361),
  events_ctrl  = c(189, 274, 373, 1507, 143, 172, 418, 251, 260),
  n_ctrl       = c(2223, 2078, 4502, 10267, 2913, 2063, 5006, 8901, 6344)
)

# Run random-effects meta-analysis
m1 <- metabin(
  event.e = events_treat, n.e = n_treat,
  event.c = events_ctrl,  n.c = n_ctrl,
  studlab = study,
  data = statin_data,
  sm = "RR",                      # Risk ratio
  method.tau = "REML",           # Restricted maximum likelihood for tau^2
  prediction = TRUE               # Include prediction interval
)
```

22. Meta-Analysis Methods for Evidence Synthesis

```
# Forest plot
forest(m1,
      sortvar = TE,
      label.left = "Favours statins",
      label.right = "Favours control",
      col.diamond = "steelblue",
      col.square = "darkblue",
      print.tau2 = TRUE,
      print.I2 = TRUE,
      print.pval.Q = TRUE)
```

22.6.1. Reading a forest plot

Key elements to examine:

1. **Individual study estimates:** Are they all on the same side of the null?
2. **Confidence intervals:** Do they overlap?
3. **Pooled estimate (diamond):** Where does it fall? Does the CI cross the null?
4. **Prediction interval:** If shown, does it cross the null?
5. **Heterogeneity statistics:** High I^2 ? Significant Q test?
6. **Study weights:** Are results driven by one dominant study?

22.7. Funnel Plots and Publication Bias

22.7.1. The problem

Studies with statistically significant results are more likely to be published. This **publication bias** means the available evidence may overestimate the true effect.

22.7.2. Funnel plots

A **funnel plot** displays each study's effect estimate (x-axis) against its precision, typically the standard error (y-axis, inverted). In the absence of bias, the plot should resemble a symmetric inverted funnel.

```
funnel(m1,  
       xlab = "Risk Ratio (log scale)",  
       studlab = TRUE,  
       col = "steelblue",  
       pch = 16)
```

Asymmetry (typically an excess of small studies with large positive effects) suggests publication bias — but also other causes like genuine heterogeneity, methodological differences in small studies, or chance.

22.7.3. Statistical tests for funnel plot asymmetry

Egger's test: A weighted regression of the effect estimate on its standard error. A significant intercept suggests asymmetry.

```
metabias(m1, method.bias = "Egger")
```

Peters' test: Preferred for binary outcomes (Egger's test can be biased with odds ratios).

22.7.4. Trim-and-fill method

The **trim-and-fill** method estimates the number of “missing” studies, imputes them, and recalculates the pooled effect. It provides a sensitivity analysis rather than a definitive correction.

22. Meta-Analysis Methods for Evidence Synthesis

```
tf <- trimfill(m1)
summary(tf)
funnel(tf)
```

i Publication bias is not the only explanation for asymmetry

Small-study effects can also arise from:

- Genuine heterogeneity (treatment works better in specific populations studied in smaller trials)
- Methodological flaws in smaller studies (less rigorous protocols)
- Chance (especially with fewer than 10 studies)

Egger's test should only be applied when there are at least 10 studies.

22.8. Advanced Meta-Analysis using metafor

The `metafor` package provides the most comprehensive and flexible meta-analysis framework in R.

```
library(metafor)

# Compute log risk ratios and sampling variances
statin_es <- escalc(
  measure = "RR",
  ai = events_treat, n1i = n_treat,
  ci = events_ctrl,  n2i = n_ctrl,
  data = statin_data
)

# Random-effects model with REML
```


22.8. Advanced Meta-Analysis using metafor

```
res <- rma(yi, vi, data = statin_es, method = "REML")
summary(res)

# Prediction interval
predict(res)

# Meta-regression: effect of year of publication
statin_es$year <- c(1994, 1996, 1998, 2002, 2002, 2004, 2005, 2008, 2016)
res_mr <- rma(yi, vi, mods = ~ year, data = statin_es, method = "REML")
summary(res_mr)

# Leave-one-out sensitivity analysis
leavelout(res)

# Influence diagnostics
inf <- influence(res)
plot(inf)
```

22.8.1. Subgroup analysis

```
# Classify as primary vs secondary prevention
statin_es$prevention <- c("secondary", "secondary", "secondary", "secondary",
                          "primary", "secondary", "secondary",
                          "primary", "primary")

# Subgroup analysis
res_sub <- rma(yi, vi, mods = ~ prevention, data = statin_es, method = "REML")
summary(res_sub)

# Forest plot by subgroup using meta package
```

```
update(m1, subgroup = statin_es$prevention, print.subgroup.name = TRUE) |>
  forest(sortvar = TE)
```

22.9. Network Meta-Analysis (Brief Overview)

Standard pairwise meta-analysis can only compare treatments that have been directly compared in head-to-head trials. **Network meta-analysis (NMA)**, also called mixed-treatment comparison, simultaneously compares multiple treatments using both **direct evidence** (from head-to-head trials) and **indirect evidence** (inferred through a common comparator).

22.9.1. When is NMA useful?

Consider three antihypertensive drugs: A, B, and C. If trials have compared A vs B and B vs C, but never A vs C, NMA can provide an indirect estimate for A vs C:

$$\hat{d}_{AC} = \hat{d}_{AB} + \hat{d}_{BC}$$

This relies on the **transitivity assumption**: the studies comparing A vs B and those comparing B vs C are similar enough that indirect comparison is valid.

22.9.2. Key concepts

- **Network geometry**: Nodes represent treatments, edges represent direct comparisons. The network should be well connected.
- **Consistency**: Direct and indirect evidence agree. Inconsistency suggests the transitivity assumption is violated.

22.9. Network Meta-Analysis (Brief Overview)

- **Ranking:** NMA can rank treatments using the surface under the cumulative ranking curve (SUCRA) or P-scores, though rankings should be interpreted cautiously.

```
library(netmeta)

# Example: network meta-analysis of antidepressants
# (using built-in dataset)
data(Senn2013)

# Run NMA
net <- netmeta(TE, seTE, treat1, treat2, studlab,
               data = Senn2013,
               sm = "MD",
               random = TRUE,
               reference.group = "plac")

summary(net)

# Network graph
netgraph(net, plastic = FALSE, thickness = "w.random",
          col = "steelblue")

# Forest plot of all comparisons vs reference
forest(net, reference.group = "plac", sortvar = TE)

# League table
netleague(net)
```

22.10. Individual Participant Data (IPD) Meta-Analysis

In standard meta-analysis, you combine **aggregate data** (summary statistics from each study). In an **IPD meta-analysis**, you obtain the raw, individual-level data from each study and analyse it directly.

22.10.1. Advantages of IPD meta-analysis

1. **Patient-level subgroup analyses** without ecological fallacy
2. **Standardised outcome definitions** and follow-up times
3. **Updated analyses** with extended follow-up
4. **Prediction model development** and validation across multiple settings

22.10.2. Two-stage vs one-stage approaches

- **Two-stage:** Analyse each study separately, then combine study-level estimates using standard meta-analysis. Familiar and transparent.
- **One-stage:** Fit a single model to all individual data, accounting for study clustering. More flexible, especially with sparse data.

```
# Two-stage IPD meta-analysis example
# Suppose we have a stacked dataset with individual data from 5 studies

# Stage 1: Fit model in each study
library(lme4)

# One-stage approach (mixed model with random study effects)
ipd_model <- glmer(
  event ~ treatment + age + sex + (1 + treatment | study_id),
```

22.11. Implementation in Python

```
data = ipd_data,
family = binomial
)

summary(ipd_model)

# The fixed effect for 'treatment' is the pooled effect
# The random slope for 'treatment' captures between-study heterogeneity
```

22.10.3. IPD meta-analysis of prediction models

Riley et al. (2010, 2021) describe frameworks for developing and validating clinical prediction models across multiple studies using IPD:

1. Develop the model using one-stage IPD-MA (internal-external cross-validation)
2. Assess calibration and discrimination in each study
3. Examine heterogeneity in model performance across settings
4. Update model intercept or recalibrate for new settings

22.11. Implementation in Python

Python's meta-analysis ecosystem is less mature than R's, but basic analyses are feasible.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# Study data: log risk ratios and standard errors
```

22. Meta-Analysis Methods for Evidence Synthesis

```
studies = pd.DataFrame({
    'study': ['4S (1994)', 'CARE (1996)', 'LIPID (1998)', 'HPS (2002)',
              'PROSPER (2002)', 'PROVE-IT (2004)', 'TNT (2005)',
              'JUPITER (2008)', 'HOPE-3 (2016)'],
    'events_t': [111, 212, 287, 1328, 127, 147, 334, 142, 235],
    'n_t': [2221, 2081, 4512, 10269, 2891, 2099, 5006, 8901, 6361],
    'events_c': [189, 274, 373, 1507, 143, 172, 418, 251, 260],
    'n_c': [2223, 2078, 4502, 10267, 2913, 2063, 5006, 8901, 6344]
})

# Compute log risk ratios and variances
studies['rr'] = (studies['events_t'] / studies['n_t']) / \
    (studies['events_c'] / studies['n_c'])
studies['log_rr'] = np.log(studies['rr'])

# Variance of log RR
studies['var_log_rr'] = (
    1/studies['events_t'] - 1/studies['n_t'] +
    1/studies['events_c'] - 1/studies['n_c']
)
studies['se_log_rr'] = np.sqrt(studies['var_log_rr'])

# --- Fixed-effect meta-analysis (inverse variance) ---
w_fe = 1 / studies['var_log_rr']
pooled_fe = np.sum(w_fe * studies['log_rr']) / np.sum(w_fe)
se_fe = np.sqrt(1 / np.sum(w_fe))
ci_fe = (pooled_fe - 1.96*se_fe, pooled_fe + 1.96*se_fe)

print(f"Fixed-effect pooled log RR: {pooled_fe:.4f}")
print(f"Fixed-effect pooled RR: {np.exp(pooled_fe):.4f} "
      f"(95% CI: {np.exp(ci_fe[0]):.4f} - {np.exp(ci_fe[1]):.4f})")

# --- Random-effects: DerSimonian-Laird ---
```

22.11. Implementation in Python

```
k = len(studies)
Q = np.sum(w_fe * (studies['log_rr'] - pooled_fe)**2)
C = np.sum(w_fe) - np.sum(w_fe**2) / np.sum(w_fe)
tau2 = max(0, (Q - (k - 1)) / C)

w_re = 1 / (studies['var_log_rr'] + tau2)
pooled_re = np.sum(w_re * studies['log_rr']) / np.sum(w_re)
se_re = np.sqrt(1 / np.sum(w_re))
ci_re = (pooled_re - 1.96*se_re, pooled_re + 1.96*se_re)

I2 = max(0, (Q - (k-1)) / Q) * 100

print(f"\nRandom-effects pooled log RR: {pooled_re:.4f}")
print(f"Random-effects pooled RR: {np.exp(pooled_re):.4f} "
      f"(95% CI: {np.exp(ci_re[0]):.4f} - {np.exp(ci_re[1]):.4f})")
print(f"tau^2: {tau2:.4f}, I^2: {I2:.1f}%")
print(f"Q statistic: {Q:.2f}, p = {1 - stats.chi2.cdf(Q, k-1):.4f}")
```

22.11.1. Forest plot in Python

```
def forest_plot(studies, pooled_est, pooled_ci, weights, title="Forest Plot"):
    """Create a basic forest plot."""
    fig, ax = plt.subplots(figsize=(10, 8))

    k = len(studies)
    y_pos = list(range(k, 0, -1))

    # Individual studies
    for i, (_, row) in enumerate(studies.iterrows()):
        ci_lo = row['log_rr'] - 1.96 * row['se_log_rr']
        ci_hi = row['log_rr'] + 1.96 * row['se_log_rr']
```

22. Meta-Analysis Methods for Evidence Synthesis

```
ax.plot([np.exp(ci_lo), np.exp(ci_hi)], [y_pos[i], y_pos[i]],
        'k-', linewidth=1)
size = weights[i] / weights.max() * 200
ax.scatter(np.exp(row['log_rr']), y_pos[i],
           s=size, c='steelblue', zorder=5, edgecolors='darkblue')

# Pooled estimate (diamond)
ax.axvline(x=np.exp(pooled_est), color='steelblue',
           linestyle='--', alpha=0.5)
ax.axvline(x=1, color='black', linestyle='-', linewidth=0.5)

# Pooled diamond
diamond_y = 0
diamond_x = [np.exp(pooled_ci[0]), np.exp(pooled_est),
             np.exp(pooled_ci[1]), np.exp(pooled_est)]
diamond_yy = [diamond_y, diamond_y + 0.3, diamond_y, diamond_y - 0.3]
ax.fill(diamond_x, diamond_yy, color='steelblue', alpha=0.7)

# Labels
ax.set_yticks(y_pos + [0])
ax.set_yticklabels(list(studies['study']) + ['Pooled'])
ax.set_xlabel('Risk Ratio')
ax.set_title(title)
ax.set_xscale('log')
plt.tight_layout()
plt.show()

forest_plot(studies, pooled_re, ci_re, w_re,
            title="Statins for CV Prevention: Random-Effects Meta-Analysis")
```


22.11.2. Using PythonMeta or PyMeta

```
# For more complete meta-analysis in Python, consider the 'meta-analysis' package
# Install: pip install meta-analysis

# Alternatively, statsmodels has some capabilities
import statsmodels.api as sm

# Egger's test equivalent: weighted regression of effect on SE
X = sm.add_constant(studies['se_log_rr'])
model = sm.WLS(studies['log_rr'], X, weights=w_fe).fit()
print("Egger's test (intercept):")
print(f"    Intercept: {model.params[0]:.4f}, p = {model.pvalues[0]:.4f}")
```

22.12. Reporting a Meta-Analysis: PRISMA 2020

The **PRISMA 2020** statement (Page et al., 2021) provides an updated checklist for reporting systematic reviews and meta-analyses. Key statistical reporting requirements:

1. **Describe the effect measure** and its rationale
2. **Specify the synthesis model** (fixed/random) and estimation method
3. **Present heterogeneity statistics** (τ^2 , I^2 , prediction interval)
4. **Report assessments of bias** (risk of bias, publication bias)
5. **Include a forest plot** for the primary outcome
6. **Describe sensitivity analyses** and their results
7. **Register the protocol** (PROSPERO) and follow it

22.13. Exercises

22.13.1. Exercise 1: Basic meta-analysis in R

The following data come from randomised trials of a hypothetical new anticoagulant vs warfarin for stroke prevention in atrial fibrillation.

```
af_trials <- data.frame(
  study = c("TRAIL-1", "GUARD-AF", "SHIELD", "ORBIT-AF",
            "VENTURE", "COMPASS-AF", "PIONEER-2", "ATLAS-AF"),
  events_new = c(28, 45, 112, 67, 33, 89, 52, 41),
  n_new      = c(1200, 2500, 5400, 3100, 1800, 4200, 2800, 2100),
  events_warf = c(42, 58, 148, 84, 29, 102, 61, 53),
  n_warf      = c(1200, 2500, 5400, 3100, 1800, 4200, 2800, 2100)
)
```

- Compute the risk ratio and 95% CI for each trial by hand (or using `escalc`).
- Perform a random-effects meta-analysis using the `meta` or `metafor` package.
- Create a forest plot. Does the pooled effect favour the new anticoagulant?
- Calculate and interpret I^2 and the prediction interval.
- Create a funnel plot and perform Egger's test. Is there evidence of publication bias?
- Perform a leave-one-out sensitivity analysis. Is the result robust?

22.13.2. Exercise 2: Meta-analysis from scratch in Python

Using the same trial data as Exercise 1:

```
import pandas as pd
import numpy as np

af_trials = pd.DataFrame({
    'study': ['TRAIL-1', 'GUARD-AF', 'SHIELD', 'ORBIT-AF',
             'VENTURE', 'COMPASS-AF', 'PIONEER-2', 'ATLAS-AF'],
    'events_new': [28, 45, 112, 67, 33, 89, 52, 41],
    'n_new': [1200, 2500, 5400, 3100, 1800, 4200, 2800, 2100],
    'events_warf': [42, 58, 148, 84, 29, 102, 61, 53],
    'n_warf': [1200, 2500, 5400, 3100, 1800, 4200, 2800, 2100]
})
```

- Compute log risk ratios and their variances for each study.
- Implement the fixed-effect inverse-variance method.
- Implement the DerSimonian-Laird random-effects method.
- Compute Q , I^2 , and τ^2 .
- Create a forest plot using matplotlib.
- Create a funnel plot and implement Egger's regression test.

22.13.3. Exercise 3: Subgroup analysis and meta-regression in R

Suppose the trials in Exercise 1 were conducted in different settings:

```
af_trials$region <- c("Europe", "North America", "Europe", "Asia",
                    "North America", "Europe", "Asia", "North America")
af_trials$mean_age <- c(72, 68, 74, 65, 70, 71, 63, 69)
af_trials$pct_female <- c(38, 42, 35, 48, 40, 37, 52, 44)
```

- Perform a subgroup analysis by region. Do treatment effects differ by region?
- Perform meta-regression with mean age as a moderator. Is there a relationship between mean age and treatment effect?

22. Meta-Analysis Methods for Evidence Synthesis

- c) Perform meta-regression with percentage female. Interpret the result, noting the ecological fallacy.
- d) Create a bubble plot showing the meta-regression of effect size on mean age.

22.13.4. Exercise 4: Critical appraisal (Conceptual)

You are reviewing a published meta-analysis of 12 trials comparing a new surgical technique to standard care for knee osteoarthritis. The reported results are:

- Pooled standardised mean difference for pain: -0.62 (95% CI: -0.89 to -0.35), $p < 0.001$
 - $I^2 = 78\%$, $\tau^2 = 0.15$, Q test $p < 0.001$
 - Prediction interval: -1.42 to 0.18
 - Egger's test: $p = 0.03$
 - 8 of 12 trials were single-centre with fewer than 100 participants
- a) Interpret the pooled effect and its clinical significance.
 - b) What does the prediction interval tell you that the confidence interval does not?
 - c) What are the implications of $I^2 = 78\%$?
 - d) Given the Egger's test and the predominance of small trials, what concerns do you have?
 - e) What additional analyses would you want to see?
 - f) Would you change clinical practice based on this meta-analysis? Why or why not?

22.14. Summary

22.15. References and Further Reading

Concept	Key point
Fixed-effect model	Assumes one true effect; weights = $1/\sigma_i^2$
Random-effects model	Allows varying true effects; weights = $1/(\sigma_i^2 + \tau^2)$
I^2	Proportion of variability due to heterogeneity (25/50/75% thresholds)
Prediction interval	Range for the true effect in a future setting — often wider than CI
Forest plot	The primary visualisation; always include one
Funnel plot	Checks for small-study effects / publication bias
Network meta-analysis	Combines direct and indirect evidence across multiple treatments
IPD meta-analysis	Uses individual data; avoids ecological fallacy
PRISMA 2020	The reporting guideline for systematic reviews

22.15. References and Further Reading

1. **Higgins JPT, Thomas J, Chandler J, et al.** (eds). *Cochrane Handbook for Systematic Reviews of Interventions*. Version 6.4, 2024. Available at <https://training.cochrane.org/handbook>. The authoritative guide to systematic review methodology.
2. **Riley RD, Debray TPA, Collins GS, et al.** Individual participant data meta-analysis to examine interactions between treatment effect and participant-level covariates. *Statistical Methods in Medical Research*. 2020;29(12):3531–3556.

22. Meta-Analysis Methods for Evidence Synthesis

3. **Riley RD, Moons KGM, Snell KIE, et al.** A guide to systematic review and meta-analysis of prognostic factor studies. *BMJ*. 2019;364:k4597.
4. **Page MJ, McKenzie JE, Bossuyt PM, et al.** The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*. 2021;372:n71.
5. **Viechtbauer W.** Conducting meta-analyses in R with the metafor package. *Journal of Statistical Software*. 2010;36(3):1–48.
6. **Schwarzer G, Carpenter JR, Rucker G.** *Meta-Analysis with R*. Springer, 2015. Comprehensive practical guide using the `meta` and `metafor` packages.
7. **DerSimonian R, Laird N.** Meta-analysis in clinical trials. *Controlled Clinical Trials*. 1986;7(3):177–188.
8. **Higgins JPT, Thompson SG, Deeks JJ, Altman DG.** Measuring inconsistency in meta-analyses. *BMJ*. 2003;327(7414):557–560.
9. **Egger M, Davey Smith G, Schneider M, Minder C.** Bias in meta-analysis detected by a simple, graphical test. *BMJ*. 1997;315(7109):629–634.
10. **Salanti G.** Indirect and mixed-treatment comparison, network, or multiple-treatments meta-analysis: many names, many benefits, many concerns for the next generation evidence synthesis tool. *Research Synthesis Methods*. 2012;3(2):80–97.
11. **IntHout J, Ioannidis JPA, Rovers MM, Goeman JJ.** Plea for routinely presenting prediction intervals in meta-analysis. *BMJ Open*. 2016;6(7):e010247.

23. Producing Journal-Ready Statistical Analyses

23.1. Introduction

You have learned how to fit models, validate predictions, and draw causal inferences. The final — and often most undervalued — skill is **communicating your results** in a form that meets the standards of peer-reviewed medical journals. Reviewers and editors routinely reject papers not because the statistics are wrong, but because the presentation is unclear, tables are poorly formatted, figures are low resolution, or the statistical methods section is incomplete.

This chapter walks you through the complete workflow: from creating publication-quality tables and figures to writing a statistical methods section, meeting reporting guideline requirements, and building a reproducible research pipeline. We conclude with three capstone project descriptions that bring together everything in the course.

23.2. What Do Top Journals Expect?

23.2.1. General standards

Major medical journals (The Lancet, BMJ, JAMA, NEJM, Lancet Digital Health) share common expectations:

23. Producing Journal-Ready Statistical Analyses

1. **Transparent reporting:** Follow the relevant reporting guideline (CONSORT for RCTs, STROBE for observational studies, TRIPOD+AI for prediction models, PRISMA for systematic reviews).
2. **Pre-specified analysis plan:** Distinguish confirmatory from exploratory analyses.
3. **Effect estimates with uncertainty:** Report point estimates, confidence intervals, and (where appropriate) p-values. Never report p-values alone.
4. **Reproducibility:** Many journals now require data availability statements and code sharing.
5. **Figure quality:** Minimum 300 DPI, specific file formats (TIFF, EPS, or high-resolution PDF), accessible colour palettes.

23.2.2. Journal-specific requirements

Journal	Key statistical requirements
Lancet / Lancet Digital Health	TRIPOD+AI for prediction models; structured statistical methods; data sharing encouraged
BMJ	EQUATOR reporting guidelines mandatory; patient involvement statement; open access to data
JAMA	Statistical review by dedicated biostatistician; eTable/eFigure supplements expected
NEJM	Extreme rigour; independent statistical review; full protocol as supplement

! The statistical review

All top medical journals employ dedicated statistical reviewers. Common reasons for rejection on statistical grounds include: inappropriate handling of missing data, failure to account for multiple comparisons, presenting associations as causal claims without proper methodology, and inadequate model validation.

23.3. Table 1: Baseline Characteristics

“Table 1” is the standard first table in any clinical paper, describing the study population by group (treatment arms, exposure groups, or clusters).

23.3.1. What belongs in Table 1

- Demographics: age, sex, race/ethnicity
- Clinical characteristics: comorbidities, disease severity, vital signs
- Laboratory values: relevant biomarkers
- Medications and prior treatments
- Summary statistics: mean (SD) or median (IQR) for continuous variables; n (%) for categorical variables

23.3.2. Creating Table 1 in R with gtsummary

```
library(gtsummary)

# Using built-in trial dataset (modify for your data)
trial |>
```

23. Producing Journal-Ready Statistical Analyses

```
select(trt, age, grade, stage, marker, response) |>
tbl_summary(
  by = trt,
  statistic = list(
    all_continuous() ~ "{mean} ({sd})",
    all_categorical() ~ "{n} ({p}%)"
  ),
  digits = list(
    all_continuous() ~ 1,
    all_categorical() ~ c(0, 1)
  ),
  label = list(
    age ~ "Age, years",
    grade ~ "Tumour grade",
    stage ~ "Cancer stage",
    marker ~ "Biomarker level, ng/mL",
    response ~ "Tumour response"
  ),
  missing_text = "Missing"
) |>
add_overall() |>
add_p(
  test = list(
    all_continuous() ~ "t.test",
    all_categorical() ~ "chisq.test"
  )
) |>
add_n() |>
modify_header(label = "**Characteristic**") |>
modify_spanning_header(c("stat_1", "stat_2") ~ "**Treatment Group**") |>
bold_labels() |>
as_gt() |>
gt::tab_header(
```

23.3. Table 1: Baseline Characteristics

```
title = "Table 1. Baseline Characteristics of the Study Population"
)
```

💡 Should Table 1 include p-values?

In a randomised trial, baseline differences are due to chance, so p-values for baseline comparisons are widely considered inappropriate (they test a hypothesis you know to be true). In observational studies, standardised mean differences (SMDs) are preferred over p-values because SMDs are not influenced by sample size.

23.3.3. Creating Table 1 in Python with tableone

```
import pandas as pd
from tableone import TableOne
import numpy as np

# Simulate clinical trial data
np.random.seed(42)
n = 500

data = pd.DataFrame({
    'Treatment': np.random.choice(['Drug A', 'Drug B'], n),
    'Age': np.random.normal(65, 10, n).round(1),
    'Sex': np.random.choice(['Male', 'Female'], n, p=[0.55, 0.45]),
    'BMI': np.random.normal(28, 5, n).round(1),
    'Diabetes': np.random.choice(['Yes', 'No'], n, p=[0.3, 0.7]),
    'eGFR': np.random.normal(62, 18, n).round(0),
    'Systolic_BP': np.random.normal(140, 20, n).round(0),
    'Smoking': np.random.choice(['Never', 'Former', 'Current'], n, p=[0.4, 0.35, 0.25])
})
```

23. Producing Journal-Ready Statistical Analyses

```
# Define variable types
columns = ['Age', 'Sex', 'BMI', 'Diabetes', 'eGFR', 'Systolic_BP', 'Smoking']
categorical = ['Sex', 'Diabetes', 'Smoking']
nonnormal = ['eGFR'] # Report as median [IQR]

# Create Table 1
table1 = TableOne(
    data,
    columns=columns,
    categorical=categorical,
    groupby='Treatment',
    nonnormal=nonnormal,
    pval=True,
    smd=True # Standardised mean differences
)

print(table1.tabulate(tablefmt="github"))

# Export to various formats
# table1.to_excel('table1.xlsx')
# table1.to_latex('table1.tex')
# table1.to_html('table1.html')
```

23.4. Regression Tables

23.4.1. Presenting regression results

A well-formatted regression table should include:

- Variable names (not raw column names)
- Effect estimates (OR, HR, RR, or coefficients) with 95% CIs

- P-values (to appropriate decimal places)
- Sample size and number of events
- Model fit statistics (R-squared, C-statistic, AIC)

23.4.2. Regression tables in R

```
library(gtsummary)
library(survival)

# Fit a Cox model
cox_model <- coxph(Surv(time, status) ~ age + sex + ph.ecog + ph.karno + wt.loss,
                  data = lung)

# Publication-quality table
tbl_regression(
  cox_model,
  exponentiate = TRUE,
  label = list(
    age ~ "Age, per year",
    sex ~ "Sex (female vs male)",
    ph.ecog ~ "ECOG performance status",
    ph.karno ~ "Karnofsky performance score",
    wt.loss ~ "Weight loss, kg"
  ),
  pvalue_fun = ~ style_pvalue(.x, digits = 3)
) |>
  add_global_p() |>
  bold_p(t = 0.05) |>
  modify_header(label = "**Variable**") |>
  modify_footnote(
    estimate ~ "HR = hazard ratio; CI = confidence interval",
  ) |>
```

23. Producing Journal-Ready Statistical Analyses

```
as_gt() |>
gt::tab_header(
  title = "Table 2. Multivariable Cox Regression for Overall Survival"
)
```

23.4.3. Regression tables in Python

```
import pandas as pd
import numpy as np
from lifelines import CoxPHFitter
from lifelines.datasets import load_lung

# Load and prepare data
lung = load_lung()
lung = lung[['T', 'status', 'age', 'sex', 'ph.ecog', 'ph.karno', 'wt.loss']]
lung['status'] = lung['status'] - 1 # Convert to 0/1

# Fit Cox model
cph = CoxPHFitter()
cph.fit(lung, duration_col='T', event_col='status')

# Extract results as a formatted table
summary_df = cph.summary[['exp(coef)', 'exp(coef) lower 95%',
                          'exp(coef) upper 95%', 'p']].copy()
summary_df.columns = ['HR', 'Lower 95% CI', 'Upper 95% CI', 'p-value']
summary_df = summary_df.round({'HR': 2, 'Lower 95% CI': 2,
                              'Upper 95% CI': 2, 'p-value': 4})

# Create formatted CI column
summary_df['HR (95% CI)'] = summary_df.apply(
    lambda r: f"{r['HR']:.2f} ({r['Lower 95% CI']:.2f}-{r['Upper 95% CI']:.2f})",
    axis=1)
```

```

        axis=1
    )

    # Rename index for publication
    name_map = {
        'age': 'Age, per year',
        'sex': 'Sex (female vs male)',
        'ph.ecog': 'ECOG performance status',
        'ph.karno': 'Karnofsky score, per 10 points',
        'wt.loss': 'Weight loss, per kg'
    }
    summary_df.index = summary_df.index.map(lambda x: name_map.get(x, x))

    print(summary_df[['HR (95% CI)', 'p-value']].to_string())

```

23.5. Publication-Quality Figures

23.5.1. Journal specifications

Most journals require:

- **Resolution:** minimum 300 DPI (600 DPI for line art)
- **Format:** TIFF, EPS, or high-resolution PDF
- **Size:** single column (89 mm) or double column (183 mm)
- **Fonts:** Arial, Helvetica, or Times New Roman, minimum 8pt
- **Colours:** Accessible to colour-blind readers (avoid red-green combinations)

23.5.2. Creating journal-quality figures in R

```
library(ggplot2)
library(survival)
library(survminer)
library(RColorBrewer)

# Define a publication theme
theme_publication <- function(base_size = 11) {
  theme_minimal(base_size = base_size) %+replace%
  theme(
    text = element_text(family = "Arial"),
    plot.title = element_text(size = base_size + 2, face = "bold",
                              hjust = 0, margin = margin(b = 10)),
    axis.title = element_text(size = base_size, face = "bold"),
    axis.text = element_text(size = base_size - 1, colour = "black"),
    legend.title = element_text(size = base_size, face = "bold"),
    legend.text = element_text(size = base_size - 1),
    legend.position = "bottom",
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(colour = "grey90"),
    strip.text = element_text(size = base_size, face = "bold"),
    plot.margin = margin(10, 10, 10, 10)
  )
}

# Colour-blind friendly palette
cb_palette <- c("#0072B2", "#D55E00", "#009E73", "#CC79A7",
               "#F0E442", "#56B4E9", "#E69F00", "#000000")

# --- Kaplan-Meier survival curve ---
fit <- survfit(Surv(time, status) ~ sex, data = lung)
```


23.5. Publication-Quality Figures

```
p_surv <- ggsurvplot(  
  fit,  
  data = lung,  
  palette = cb_palette[1:2],  
  risk.table = TRUE,  
  risk.table.col = "strata",  
  pval = TRUE,  
  conf.int = TRUE,  
  xlab = "Time (days)",  
  ylab = "Survival probability",  
  legend.labs = c("Male", "Female"),  
  legend.title = "Sex",  
  font.main = c(14, "bold"),  
  font.x = c(12, "bold"),  
  font.y = c(12, "bold"),  
  font.tickslab = 11,  
  risk.table.fontsize = 3.5,  
  ggtheme = theme_publication()  
)  
  
print(p_surv)  
  
# --- Saving at journal quality ---  
# For TIFF (most journals)  
ggsave("figure1_survival.tiff",  
  plot = p_surv$plot,  
  width = 183, height = 120,  
  units = "mm", dpi = 300,  
  compression = "lzw")  
  
# For PDF (vector format, scalable)  
ggsave("figure1_survival.pdf",  
  plot = p_surv$plot,
```

23. Producing Journal-Ready Statistical Analyses

```
width = 183, height = 120,  
units = "mm", device = cairo_pdf)
```

23.5.3. Publication figures in Python

```
import matplotlib.pyplot as plt  
import matplotlib as mpl  
import numpy as np  
from lifelines import KaplanMeierFitter  
from lifelines.datasets import load_lung  
  
# Set publication defaults  
plt.rcParams.update({  
    'font.family': 'Arial',  
    'font.size': 11,  
    'axes.labelsize': 12,  
    'axes.titlesize': 13,  
    'axes.labelweight': 'bold',  
    'figure.dpi': 300,  
    'savefig.dpi': 300,  
    'figure.figsize': (7.2, 4.7), # ~183mm x 120mm  
    'axes.spines.top': False,  
    'axes.spines.right': False,  
})  
  
# Colour-blind friendly palette  
cb_palette = ['#0072B2', '#D55E00', '#009E73', '#CC79A7']  
  
# Prepare data  
lung = load_lung()  
lung['status'] = lung['status'] - 1
```

23.5. Publication-Quality Figures

```
fig, ax = plt.subplots()

kmf = KaplanMeierFitter()

for i, (sex_val, label) in enumerate([(1, 'Male'), (2, 'Female')]):
    mask = lung['sex'] == sex_val
    kmf.fit(lung.loc[mask, 'T'], lung.loc[mask, 'status'], label=label)
    kmf.plot_survival_function(ax=ax, color=cb_palette[i], ci_show=True)

ax.set_xlabel('Time (days)')
ax.set_ylabel('Survival Probability')
ax.set_title('Kaplan-Meier Survival by Sex')
ax.legend(loc='lower left', frameon=False)
ax.set_ylim(0, 1.05)

plt.tight_layout()
plt.savefig('figure1_survival.tiff', dpi=300, bbox_inches='tight')
plt.savefig('figure1_survival.pdf', bbox_inches='tight')
plt.show()
```

23.5.4. Multi-panel figures

Journals often prefer combined multi-panel figures (Figure 1A, 1B, etc.) rather than separate images.

```
library(patchwork)

p1 <- ggplot(lung, aes(x = factor(sex), y = age)) +
  geom_boxplot(fill = cb_palette[1:2], alpha = 0.7) +
  labs(x = "Sex", y = "Age (years)", title = "A") +
  scale_x_discrete(labels = c("Male", "Female")) +
  theme_publication()
```

23. Producing Journal-Ready Statistical Analyses

```
p2 <- ggplot(lung, aes(x = age, y = ph.karno)) +  
  geom_point(alpha = 0.5, colour = cb_palette[1]) +  
  geom_smooth(method = "lm", colour = cb_palette[2], se = TRUE) +  
  labs(x = "Age (years)", y = "Karnofsky Score", title = "B") +  
  theme_publication()  
  
combined <- p1 + p2 +  
  plot_layout(ncol = 2, widths = c(1, 1))  
  
ggsave("figure2_panels.tiff",  
  plot = combined,  
  width = 183, height = 90,  
  units = "mm", dpi = 300)
```

23.6. Writing a Statistical Methods Section

23.6.1. Structure

A complete statistical methods section should address:

1. **Study design and population:** Brief recap (detailed in Methods)
2. **Primary and secondary outcomes:** How each was defined and measured
3. **Sample size / power calculation:** How you determined the required sample size
4. **Descriptive statistics:** How continuous and categorical variables are summarised
5. **Primary analysis:** The statistical model, its assumptions, and how they were checked
6. **Missing data:** How much was missing, the assumed mechanism (MCAR/MAR/MNAR), and the handling method

23.6. *Writing a Statistical Methods Section*

7. **Sensitivity analyses:** Pre-specified alternative approaches
8. **Multiple comparisons:** Whether and how adjusted (Bonferroni, FDR, etc.)
9. **Software:** Packages, versions, and random seeds

23.6.2. **Example statistical methods paragraph**

Baseline characteristics were summarised as means (SD) for normally distributed continuous variables, medians (IQR) for skewed continuous variables, and frequencies (percentages) for categorical variables. The primary outcome (time to first major adverse cardiovascular event) was analysed using multivariable Cox proportional hazards regression, adjusting for age, sex, diabetes, eGFR, systolic blood pressure, and smoking status. The proportional hazards assumption was tested using Schoenfeld residuals. Missing covariate data (3.2% overall, assumed missing at random) were handled using multiple imputation by chained equations (50 imputed datasets, 20 iterations). Rubin's rules were used to pool effect estimates. Model discrimination was assessed using Harrell's C-statistic with 95% confidence intervals estimated via 500 bootstrap resamples. Calibration was assessed graphically and using the Greenwood-Nam-D'Agostino test. Internal-external cross-validation was performed across the three recruitment centres. All analyses were pre-specified and conducted using R version 4.4.1 (R Foundation for Statistical Computing) with the survival (v3.7), mice (v3.16), and rms (v6.8) packages. Two-sided p-values < 0.05 were considered statistically significant.

 Common mistakes in statistical methods sections

1. Not specifying the handling of missing data
2. Claiming “data were normally distributed” without stating how this was assessed
3. Using stepwise variable selection without acknowledging its limitations
4. Reporting only p-values without effect estimates and confidence intervals
5. Not specifying the software and package versions
6. Not describing how the proportional hazards or linearity assumptions were checked
7. Claiming causation from an observational study without using causal inference methods

23.7. TRIPOD+AI Compliance

The **TRIPOD+AI** statement (Collins et al., 2024) extends the original TRIPOD guideline for prediction models to include AI-specific items. If your study develops, validates, or updates a clinical prediction model — whether based on logistic regression or deep learning — you should follow TRIPOD+AI.

23.7.1. Key TRIPOD+AI items

Item	Requirement	How to address
Title	State it is a prediction model study and the type (development, validation, both)	“Development and external validation of a machine learning model for...”

23.7. TRIPOD+AI Compliance

Item	Requirement	How to address
Source of data	Describe the data source, setting, dates	EHR data from Hospital X, 2015-2023
Participants	Eligibility criteria, sampling, flow diagram	Include a CONSORT-style flow diagram
Outcome	Clear definition, timing, blinding	“30-day all-cause mortality ascertained from hospital records”
Sample size	Justify based on Riley et al. criteria	Use <code>pmsampsize</code> package
Missing data	Report extent and handling	Multiple imputation; report complete case sensitivity analysis
Model development	Full specification including hyperparameters	Report all hyperparameters; include code supplement
Model performance	Discrimination (C-statistic, AUROC) AND calibration (calibration plot, slope, intercept)	Never report AUROC alone — calibration is equally important
Model fairness	Assess performance across demographic subgroups	Report C-statistic and calibration by sex, age group, ethnicity
Code and data	Availability statement	Share code via GitHub; data via controlled access repository

23. Producing Journal-Ready Statistical Analyses

23.7.2. Step-by-step TRIPOD+AI walkthrough

```
# Step 1: Sample size calculation for a prediction model
library(pmsampsize)

# Binary outcome model
# Assume: 15 candidate predictors, outcome prevalence 10%,
# target C-statistic 0.80, max shrinkage 10%
ss <- pmsampsize(type = "b",
                 rsquared = 0.15, # Cox-Snell R-squared
                 parameters = 15,
                 prevalence = 0.10)

ss

# Step 2: Develop model with full pre-specification
library(rms)

# Set up data distribution summaries
dd <- datadist(your_data)
options(datadist = "dd")

# Fit model
full_model <- lrm(outcome ~ age + sex + rcs(sbp, 4) + diabetes + egfr +
                  smoking + rcs(bmi, 3),
                  data = your_data, x = TRUE, y = TRUE)

# Step 3: Internal validation with bootstrap
val <- validate(full_model, B = 200)
print(val)

# Optimism-corrected C-statistic
c_stat_apparent <- full_model$stats["C"]
c_stat_corrected <- c_stat_apparent - (val["Dxy", "index.orig"] -
```


23.8. Reproducible Research Workflow

```
val["Dxy", "index.corrected"])/ 2

# Step 4: Calibration plot
cal <- calibrate(full_model, B = 200)
plot(cal, xlab = "Predicted probability", ylab = "Observed proportion",
      main = "Calibration Plot (Bootstrap-Corrected)")

# Step 5: Fairness assessment
# Evaluate performance in subgroups
for (subgroup in c("Male", "Female")) {
  sub_data <- your_data |> filter(sex == subgroup)
  pred <- predict(full_model, newdata = sub_data, type = "fitted")
  auc <- pROC::auc(sub_data$outcome, pred)
  cat(subgroup, ": C-statistic =", round(auc, 3), "\n")
}
```

23.8. Reproducible Research Workflow

23.8.1. Project structure

A reproducible research project should follow a consistent directory structure:

```
project/
|-- README.md           # Project overview
|-- renv.lock           # R package versions (or environment.yml for conda)
|-- _targets.R          # Pipeline definition (or Makefile)
|-- data/
|   |-- raw/            # Never modify raw data
|   |-- processed/      # Cleaned datasets
|   |-- metadata/       # Data dictionaries
```

23. Producing Journal-Ready Statistical Analyses

```
|-- R/                                # R functions
|-- python/                          # Python scripts
|-- analysis/
|   |-- 01_data_cleaning.qmd
|   |-- 02_descriptive.qmd
|   |-- 03_primary_analysis.qmd
|   |-- 04_sensitivity.qmd
|-- output/
|   |-- figures/
|   |-- tables/
|-- manuscript/
|   |-- manuscript.qmd               # The paper itself
|   |-- references.bib
|-- Dockerfile                      # For full computational reproducibility
```

23.8.2. Package management

R (renv):

```
# Initialise renv in your project
renv::init()

# After installing packages, take a snapshot
renv::snapshot()

# Collaborator restores exact versions
renv::restore()
```

Python (conda/pip):

```
# Create environment
# conda create -n my_project python=3.11
# conda activate my_project
```

23.8. Reproducible Research Workflow

```
# pip install pandas numpy scikit-learn lifelines tableone

# Export environment
# pip freeze > requirements.txt
# OR
# conda env export > environment.yml

# Collaborator recreates environment
# pip install -r requirements.txt
# OR
# conda env create -f environment.yml
```

23.8.3. Version control with Git

Essential git practices for research:

1. **Commit analysis scripts, not data** (unless data are small and non-sensitive)
2. **Use .gitignore** to exclude data files, outputs, and sensitive information
3. **Write meaningful commit messages:** “Add multiple imputation for missing eGFR values” not “update script”
4. **Tag milestones:** `git tag -a v1.0-submission -m "Initial journal submission"`
5. **Use branches** for exploratory analyses that may not make it into the paper

23.8.4. Docker for full reproducibility (brief)

A Dockerfile captures the complete computational environment:

23. Producing Journal-Ready Statistical Analyses

```
FROM rocker/verse:4.4.1

# Install system dependencies
RUN apt-get update && apt-get install -y libglpk-dev

# Install R packages
RUN R -e "install.packages(c('survival', 'rms', 'mice', 'gtsummary', 'pmsamp

# Copy project files
COPY . /home/rstudio/project

WORKDIR /home/rstudio/project
```

This guarantees that anyone, anywhere, at any time can reproduce your exact results.

23.9. Case Study: Replicating a Lancet Digital Health Analytical Approach

To illustrate the complete workflow, we replicate the analytical structure of a 2025 Lancet Digital Health prediction model study.

Study design: Development and external validation of a machine learning model to predict 30-day hospital readmission in heart failure patients.

```
# --- STEP 1: Data preparation ---
library(tidyverse)
library(mice)

# Load data (using a public dataset as proxy)
# In practice, this would be your EHR extract
```

23.9. Case Study: Replicating a Lancet Digital Health Analytical Approach

```
data("heart", package = "survival") # Stanford heart transplant data

# Document missingness
missing_summary <- data.frame(
  variable = names(heart),
  n_missing = sapply(heart, function(x) sum(is.na(x))),
  pct_missing = sapply(heart, function(x) mean(is.na(x)) * 100)
)
print(missing_summary)

# Multiple imputation
imp <- mice(heart, m = 50, maxit = 20, method = "pmm", seed = 42,
           printFlag = FALSE)

# --- STEP 2: Table 1 ---
library(gtsummary)

heart |>
  tbl_summary(
    by = transplant,
    statistic = list(
      all_continuous() ~ "{mean} ({sd})",
      all_categorical() ~ "{n} ({p}%)"
    )
  ) |>
  add_overall() |>
  add_p()

# --- STEP 3: Model development (in imputed data) ---
library(rms)
library(pROC)

# Pool results across imputations using Rubin's rules
```

23. Producing Journal-Ready Statistical Analyses

```
# (Simplified example with complete cases for illustration)
dd <- datadist(heart)
options(datadist = "dd")

# Pre-specify model based on clinical knowledge
model <- lrm(event ~ age + year + surgery + rcs(futime, 4),
             data = heart, x = TRUE, y = TRUE)

print(model)

# --- STEP 4: Internal validation ---
set.seed(42)
val <- validate(model, B = 500)
print(val)

# Optimism-corrected C-statistic
cat("Apparent C-statistic:", model$stats["C"], "\n")
cat("Optimism:", (val["Dxy", "index.orig"] - val["Dxy", "index.corrected"])/2)

# --- STEP 5: Calibration plot ---
cal <- calibrate(model, B = 200)
plot(cal,
     xlab = "Predicted Probability",
     ylab = "Observed Proportion",
     main = "Calibration Plot (Internal Bootstrap Validation)")
abline(0, 1, lty = 2, col = "grey50")

# --- STEP 6: Decision curve analysis ---
library(dcurves)

dca_result <- dca(event ~ predicted_prob,
                  data = heart_with_preds,
                  thresholds = seq(0, 0.5, by = 0.01))
```

```

plot(dca_result)

# --- STEP 7: Export all results ---
# Tables to Word/HTML
tbl_summary_result |> as_gt() |> gt::gtsave("output/tables/table1.docx")

# Figures at publication quality
ggsave("output/figures/figure1_calibration.tiff",
       width = 89, height = 89, units = "mm", dpi = 300)

```

23.10. Capstone Projects

Choose one of the following three capstone projects. Each requires you to produce a complete, journal-ready analysis with Table 1, regression/model results, publication-quality figures, and a statistical methods section.

23.10.1. Capstone 1: Cardiovascular Risk Prediction Model

Objective: Develop a cardiovascular risk prediction model using Framingham Heart Study methodology and validate it in NHANES data.

Dataset: Framingham Heart Study (public teaching dataset from `riskCommunicator` R package) for development; NHANES 2017-2020 (via `nhanesA` package) for external validation.

Requirements:

1. **Table 1:** Baseline characteristics of the development cohort, stratified by 10-year CVD event status.
2. **Model development:** Logistic regression predicting 10-year CVD risk using age, sex, total cholesterol, HDL cholesterol, systolic blood pressure, blood pressure treatment, smoking, and diabetes.

23. *Producing Journal-Ready Statistical Analyses*

3. **Internal validation:** Bootstrap-corrected C-statistic and calibration slope.
4. **External validation in NHANES:** Apply the model, report discrimination and calibration in the external population.
5. **Fairness assessment:** Report model performance stratified by sex and race/ethnicity.
6. **Figures:** (A) Calibration plot (development, bootstrap-corrected), (B) Calibration plot (NHANES external validation), (C) Decision curve analysis.
7. **Statistical methods section:** Following TRIPOD+AI structure.
8. **Code:** Fully reproducible R or Python scripts with renv/conda environment.

23.10.2. Capstone 2: Metabolic Phenotype Clustering

Objective: Identify metabolic phenotypes in the US adult population using unsupervised learning and assess their association with mortality.

Dataset: NHANES 2017-2020 (demographics, laboratory, examination, and mortality linkage files).

Requirements:

1. **Data preparation:** Select metabolic variables (glucose, HbA1c, insulin, triglycerides, HDL, LDL, waist circumference, BMI, blood pressure, CRP). Handle missing data with multiple imputation.
2. **Dimensionality reduction:** PCA to identify major axes of metabolic variation. Present scree plot and loadings table.
3. **Clustering:** Apply k-means and Gaussian mixture models to PC scores. Use the gap statistic, silhouette width, and BIC to select the number of clusters. Visualise clusters in PC space.
4. **Table 1:** Baseline characteristics stratified by cluster assignment.
5. **Cluster characterisation:** Describe the clinical profile of each cluster (the “metabolic phenotypes”).

6. **Survival analysis:** Kaplan-Meier curves and Cox regression for all-cause mortality by cluster, adjusting for age, sex, race/ethnicity, smoking, and education.
7. **Figures:** (A) Scree plot, (B) Cluster visualisation in PC1-PC2 space, (C) Kaplan-Meier curves by cluster, (D) Forest plot of hazard ratios.
8. **Statistical methods section:** Including justification for unsupervised approach and cluster validation.

23.10.3. Capstone 3: Bayesian Survival Analysis

Objective: Fit a Bayesian hierarchical survival model to the primary biliary cholangitis (PBC) dataset, comparing Bayesian and frequentist approaches.

Dataset: `pbc` dataset from the `survival` R package (Mayo Clinic PBC trial, 418 patients).

Requirements:

1. **Table 1:** Baseline characteristics stratified by treatment arm (D-penicillamine vs placebo).
2. **Frequentist Cox model:** Standard Cox PH regression with selected predictors. Check proportional hazards assumption.
3. **Bayesian Cox model:** Fit using `brms` (R) or `PyMC` (Python) with:
 - Weakly informative priors on log-hazard ratios: $\beta \sim N(0, 1)$
 - Compare with informative priors based on prior literature
 - Report posterior medians, 95% credible intervals, and posterior probability of benefit
4. **Hierarchical extension:** If the data include centre/site information, fit a model with random intercepts. Otherwise, use histological stage as a grouping variable.
5. **Model comparison:** Compare DIC/WAIC/LOO-CV between Bayesian models.

23. Producing Journal-Ready Statistical Analyses

6. **Posterior predictive checks:** Simulate survival curves from the posterior and overlay on observed Kaplan-Meier.
7. **Figures:** (A) Prior vs posterior distributions for key parameters, (B) Posterior predictive survival curves, (C) Forest plot comparing Bayesian vs frequentist estimates.
8. **Statistical methods section:** Justify the Bayesian approach, describe priors and sensitivity to prior choice.



Capstone assessment criteria

Each capstone will be assessed on:

- **Statistical correctness** (40%): Appropriate methods, valid assumptions, correct interpretation
- **Reproducibility** (20%): Code runs from start to finish; environment documented
- **Presentation quality** (20%): Journal-ready tables, figures, and methods section
- **Critical thinking** (20%): Discussion of limitations, sensitivity analyses, clinical implications

23.11. Exercises

23.11.1. Exercise 1: Table 1 in both R and Python

Using the `lung` dataset (R) or an equivalent simulated dataset (Python):

- a) Create a Table 1 stratified by sex, with appropriate summary statistics for age, ECOG score, Karnofsky score, meal calories, and weight loss.
- b) Include standardised mean differences instead of p-values.
- c) Export the table to a format suitable for journal submission (Word or LaTeX).

R:

```
data(lung, package = "survival")
# Your code here using gtsummary
```

Python:

```
from lifelines.datasets import load_lung
lung = load_lung()
# Your code here using tableone
```

23.11.2. Exercise 2: Publication-quality figure

Create a multi-panel figure (at least 3 panels) summarising a survival analysis of the `lung` dataset:

- Panel A: Kaplan-Meier curve by sex
- Panel B: Forest plot of hazard ratios from a multivariable Cox model
- Panel C: Calibration plot for a 1-year survival prediction

Requirements: colour-blind accessible palette, Arial font, 300 DPI, single figure file suitable for a double-column journal layout (183 mm wide).

23.11.3. Exercise 3: Write a statistical methods section

Write a complete statistical methods section (250-400 words) for the following study:

A retrospective cohort study of 5,000 patients with type 2 diabetes from a hospital EHR database (2015-2023). The primary outcome is time to first major adverse cardiovascular event (MACE). The exposure is SGLT2 inhibitor use (vs DPP-4

23. Producing Journal-Ready Statistical Analyses

inhibitor use). Covariates include age, sex, BMI, HbA1c, eGFR, prior CVD, hypertension, and smoking. Missing data range from 2% (age) to 15% (BMI). The analysis uses propensity score matching followed by Cox regression in the matched cohort.

Your methods section should cover all items listed in the “Writing a statistical methods section” subsection above.

23.11.4. Exercise 4: TRIPOD+AI checklist

Download the TRIPOD+AI checklist (from the EQUATOR Network website). For one of the three capstone projects above, complete the checklist, indicating:

- The page/section number where each item is addressed
- Items that are not applicable and why
- Items that you cannot fully address and what you would need

23.12. Summary

Component	Tool/Package	Key considerations
Table 1	gtsummary (R), tableone (Python)	SMDs over p-values in observational studies
Regression tables	gtsummary, broom, gt (R)	Always include CIs; exponentiate where appropriate
Figures	ggplot2 + patchwork (R), matplotlib (Python)	300+ DPI, colour-blind safe, journal size specs

23.13. References and Further Reading

Component	Tool/Package	Key considerations
Methods section	-	Missing data, assumptions, software versions
Reporting guideline	TRIPOD+AI, STROBE, CONSORT	Choose based on study design
Reproducibility	renv (R), conda (Python), Git, Docker	Capture full computational environment
Calibration	rms, probably (R), scikit-learn (Python)	Never report discrimination without calibration

23.13. References and Further Reading

1. **Collins GS, Moons KGM, Dhiman P, et al.** TRIPOD+AI statement: updated guidance for reporting clinical prediction models that use regression or machine learning methods. *BMJ*. 2024;385:e078378.
2. **von Elm E, Altman DG, Egger M, et al.** The Strengthening the Reporting of Observational Studies in Epidemiology (STROBE) statement. *Lancet*. 2007;370(9596):1453–1457.
3. **Sjoberg DD, Whiting K, Curry M, Lavery JA, Larmarange J.** Reproducible summary tables with the gtsummary package. *The R Journal*. 2021;13(1):570–580.
4. **Riley RD, Ensor J, Snell KIE, et al.** Calculating the sample size required for developing a clinical prediction model. *BMJ*. 2020;368:m441.
5. **Steyerberg EW.** *Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating*. 2nd ed. Springer, 2019.

23. *Producing Journal-Ready Statistical Analyses*

6. **Van Calster B, McLernon DJ, van Smeden M, et al.** Calibration: the Achilles heel of predictive analytics. *BMC Medicine*. 2019;17:230.
7. **Vickers AJ, Elkin EB.** Decision curve analysis: a novel method for evaluating prediction models. *Medical Decision Making*. 2006;26(6):565–574.
8. **Hernan MA, Robins JM.** *Causal Inference: What If*. Chapman & Hall/CRC, 2024.
9. **Harrell FE.** *Regression Modeling Strategies*. 2nd ed. Springer, 2015.
10. **The Lancet Digital Health.** Author guidelines. Available at <https://www.thelancet.com/digital-health>.
11. **EQUATOR Network.** Reporting guidelines for health research. Available at <https://www.equator-network.org>.
12. **Wickham H.** *ggplot2: Elegant Graphics for Data Analysis*. 3rd ed. Springer, 2024.
13. **Wilson G, Bryan J, Cranston K, et al.** Good enough practices in scientific computing. *PLoS Computational Biology*. 2017;13(6):e1005510.

Part VIII.

Appendices

24. Dataset Codebook

Dataset Codebook

This appendix describes all datasets used throughout the course. Each dataset is stored as a CSV file in the `data/` directory.

24.1. Framingham Heart Study (Teaching Dataset)

File: `data/framingham.csv` **Source:** NHLBI Biologic Specimen and Data Repository (BioLINCC) teaching dataset. Also available via the R package `riskCommunicator`. **Description:** Prospective cohort study data from Framingham, Massachusetts. This is the classic cardiovascular epidemiology dataset used to develop the Framingham Risk Score.

Variable	Type	Description
<code>age</code>	Continuous	Age at baseline examination (years)
<code>sex</code>	Binary	0 = Female, 1 = Male
<code>bmi</code>	Continuous	Body mass index (kg/m^2)
<code>sbp</code>	Continuous	Systolic blood pressure (mmHg)
<code>dbp</code>	Continuous	Diastolic blood pressure (mmHg)
<code>totchol</code>	Continuous	Total cholesterol (mg/dL)
<code>hdl</code>	Continuous	HDL cholesterol (mg/dL)
<code>glucose</code>	Continuous	Fasting blood glucose (mg/dL)
<code>smoking</code>	Binary	0 = Non-smoker, 1 = Current smoker
<code>diabetes</code>	Binary	0 = No, 1 = Yes
<code>bp_meds</code>	Binary	0 = Not on BP meds, 1 = On BP meds
<code>chd_10yr</code>	Binary	10-year incident CHD (0 = No, 1 = Yes)

Variable	Type	Description
<code>time_chd</code>	Continuous	Time to CHD event or censoring (days)

Used in: Chapters 3, 4, 5, 10, 11, 19 (Capstone 1)

24.2. Wisconsin Diagnostic Breast Cancer (WDBC)

File: `data/wdbc.csv` **Source:** UCI Machine Learning Repository. Also available via `sklearn.datasets.load_breast_cancer()` in Python and `mlbench::BreastCancer` in R. **Description:** Features computed from digitised images of fine needle aspirates (FNA) of breast masses. Each row represents one tumour sample.

Variable	Type	Description
<code>id</code>	Integer	Patient ID
<code>diagnosis</code>	Binary	M = Malignant, B = Benign
<code>radius_mean</code>	Continuous	Mean of distances from centre to perimeter
<code>texture_mean</code>	Continuous	Standard deviation of grey-scale values
<code>perimeter_mean</code>	Continuous	Mean tumour perimeter
<code>area_mean</code>	Continuous	Mean tumour area
<code>smoothness_mean</code>	Continuous	Local variation in radius lengths
<code>compactness_mean</code>	Continuous	$\text{Perimeter}^2 / \text{area} - 1.0$
<code>concavity_mean</code>	Continuous	Severity of concave portions
<code>concave_points_mean</code>	Continuous	Number of concave portions
<code>symmetry_mean</code>	Continuous	Tumour symmetry

24.3. Primary Biliary Cholangitis (PBC)

Variable	Type	Description
<code>fractal_dimension_mean</code> <i>Plus <code>_se</code> and <code>_worst</code> variants of each</i>	Continuous	“Coastline approximation” - 1 Standard error and worst (largest) value

Total variables: 32 (ID + diagnosis + 30 features) **Observations:** 569
(357 benign, 212 malignant) **Used in:** Chapters 5, 8, 9, 15

24.3. Primary Biliary Cholangitis (PBC)

File: `data/psc.csv` **Source:** Mayo Clinic trial in primary biliary cholangitis (formerly primary biliary cirrhosis). Available via `survival::psc` in R. **Description:** Data from 418 patients enrolled in a randomised trial of D-penicillamine vs placebo for PBC, a chronic liver disease.

Variable	Type	Description
<code>id</code>	Integer	Patient ID
<code>time</code>	Continuous	Days from registration to death, transplant, or censoring
<code>status</code>	Integer	0 = Censored, 1 = Transplant, 2 = Dead
<code>trt</code>	Binary	1 = D-penicillamine, 2 = Placebo
<code>age</code>	Continuous	Age (years)
<code>sex</code>	Binary	0 = Male, 1 = Female
<code>ascites</code>	Binary	Presence of ascites
<code>hepato</code>	Binary	Presence of hepatomegaly
<code>spiders</code>	Binary	Presence of spider angiomas

Dataset Codebook

Variable	Type	Description
edema	Ordinal	0 = None, 0.5 = Untreated/resolved, 1 = Despite treatment
bili	Continuous	Serum bilirubin (mg/dL)
chol	Continuous	Serum cholesterol (mg/dL)
albumin	Continuous	Serum albumin (g/dL)
copper	Continuous	Urine copper (µg/day)
alk_phos	Continuous	Alkaline phosphatase (U/L)
ast	Continuous	Aspartate aminotransferase (U/L)
trig	Continuous	Triglycerides (mg/dL)
platelet	Continuous	Platelet count
protime	Continuous	Prothrombin time (seconds)
stage	Ordinal	Histologic stage (1–4)

Used in: Chapters 4, 6, 14 (Capstone 3)

24.4. NHANES Subset

File: `data/nhanes_subset.csv` **Source:** National Health and Nutrition Examination Survey (CDC). Extracted using the `nhanesA` R package from NHANES 2017–2020 cycle. **Description:** A curated subset of NHANES data with demographics, lab results, and health indicators for dimensionality reduction and clustering exercises.

Variable	Type	Description
seqn	Integer	Respondent sequence number

24.4. NHANES Subset

Variable	Type	Description
age	Continuous	Age (years)
sex	Binary	1 = Male, 2 = Female
race_ethnicity	Categorical	Race/ethnicity category
education	Ordinal	Education level
bmi	Continuous	Body mass index (kg/m ²)
sbp	Continuous	Systolic blood pressure (mmHg)
dbp	Continuous	Diastolic blood pressure (mmHg)
hba1c	Continuous	Glycated haemoglobin (%)
totchol	Continuous	Total cholesterol (mg/dL)
hdl	Continuous	HDL cholesterol (mg/dL)
ldl	Continuous	LDL cholesterol (mg/dL)
triglycerides	Continuous	Triglycerides (mg/dL)
creatinine	Continuous	Serum creatinine (mg/dL)
egfr	Continuous	Estimated GFR (mL/min/1.73m ²)
albumin	Continuous	Serum albumin (g/dL)
diabetes	Binary	Self-reported diabetes
hypertension	Binary	Hypertension (SBP ≥ 140 or DBP ≥ 90 or on meds)
smoking	Categorical	Never / Former / Current
physical_activity	Continuous	Minutes of moderate-vigorous activity per week

Used in: Chapters 2, 10, 15, 16, 19 (Capstone 2)

24.5. Simulated Meningitis Data

File: `data/meningitis_sim.csv` **Source:** Simulated dataset inspired by the case study in Lopez-Ayala et al. (*BMJ* 2025). Variable distributions are based on published summary statistics from the Duke University Medical Center meningitis cohort. **Description:** Simulated data on acute meningitis patients for demonstrating spline modelling of non-linear associations.

Variable	Type	Description
<code>id</code>	Integer	Patient ID
<code>age</code>	Continuous	Age (years)
<code>sex</code>	Binary	0 = Female, 1 = Male
<code>csf_glucose</code>	Continuous	CSF glucose (mg/dL)
<code>csf_leuk</code>	Continuous	CSF leucocyte count (cells/mm ³)
<code>csf_protein</code>	Continuous	CSF protein (mg/dL)
<code>blood_glucose</code>	Continuous	Blood glucose (mg/dL)
<code>bacterial</code>	Binary	1 = Acute bacterial meningitis, 0 = Viral

Observations: 501 **Used in:** Chapter 4 (primary), Chapter 3

24.6. Downloading the data

All datasets are included in the course repository. If you have cloned the repo, they are in the `data/` directory.

To load them:

24.7. R

```
library(readr)
framingham <- read_csv("data/framingham.csv")
wdbc <- read_csv("data/wdbc.csv")
pbc <- read_csv("data/pbc.csv")
nhanes <- read_csv("data/nhanes_subset.csv")
meningitis <- read_csv("data/meningitis_sim.csv")
```

24.8. Python

```
import pandas as pd
framingham = pd.read_csv("data/framingham.csv")
wdbc = pd.read_csv("data/wdbc.csv")
pbc = pd.read_csv("data/pbc.csv")
nhanes = pd.read_csv("data/nhanes_subset.csv")
meningitis = pd.read_csv("data/meningitis_sim.csv")
```


25. Mathematical Notation Reference

Mathematical Notation Reference

This appendix provides a quick reference for mathematical notation used throughout the course. You do not need to memorise these — use this page as a lookup when you encounter unfamiliar symbols.

25.1. Basic notation

Symbol	Meaning	Example
x	A variable (lowercase)	Blood pressure measurement
X	A random variable (uppercase)	Blood pressure in the population
$\bar{x}$	Sample mean	Mean blood pressure in your study
μ	Population mean	True mean blood pressure
s or $\hat{\sigma}$	Sample standard deviation	SD of blood pressures in your data
σ	Population standard deviation	True SD of blood pressures
n	Sample size	Number of patients
p	Probability or proportion	Prevalence of a disease
$\hat{p}$	Estimated proportion	Observed prevalence in your sample

25.2. Probability

Symbol	Meaning
$P(A)$	Probability of event A
$P(A B)$	Probability of A given B (conditional probability)
$P(A \cap B)$	Probability of both A and B
$P(A \cup B)$	Probability of A or B (or both)

25.3. Distributions

Notation	Distribution	Parameters
$X \sim N(\mu, \sigma^2)$	Normal	Mean μ , variance σ^2
$X \sim \text{Bin}(n, p)$	Binomial	Trials n , success probability p
$X \sim \text{Pois}(\lambda)$	Poisson	Rate λ
$X \sim \text{Beta}(\alpha, \beta)$	Beta	Shape parameters α, β

25.4. Regression

Symbol	Meaning
y_i	Outcome for patient i
x_i	Predictor value for patient i
β_0	Intercept
$\beta_1, \beta_2, \dots$	Regression coefficients
$\hat{y}_i$	Predicted value for patient i
ϵ_i	Residual (error) for patient i
$\hat{\beta}$	Estimated coefficient

25.5. Penalised regression

Symbol	Meaning
--------	---------

Linear regression: $y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \dots + \epsilon_i$

Logistic regression: $\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \dots$

25.5. Penalised regression

Symbol	Meaning
λ	Penalty strength (regularisation parameter)
α	Elastic net mixing parameter (0 = ridge, 1 = LASSO)
$\ \beta\ _1 = \sum \beta_j $	L1 norm (sum of absolute values)
$\ \beta\ _2^2 = \sum \beta_j^2$	L2 norm squared (sum of squares)

25.6. Survival analysis

Symbol	Meaning
T	Survival time (random variable)
t	A specific time point
$S(t) = P(T > t)$	Survival function
$h(t)$	Hazard function (instantaneous risk)
$H(t)$	Cumulative hazard
HR	Hazard ratio (from Cox model)

Cox model: $h(t | \mathbf{x}) = h_0(t) \exp(\beta_1 x_1 + \beta_2 x_2 + \dots)$

25.7. Bayesian statistics

Symbol	Meaning
θ	Parameter of interest
$P(\theta)$ or $\pi(\theta)$	Prior distribution
$P(D \theta)$ or $L(\theta D)$	Likelihood
$P(\theta D)$	Posterior distribution
$P(D)$	Marginal likelihood (evidence)

Bayes' theorem: $P(\theta | D) = \frac{P(D|\theta)P(\theta)}{P(D)}$

Or in words: Posterior $\propto$ Likelihood $\times$ Prior

25.8. Model performance

Symbol	Meaning
TP, FP, TN, FN	True/false positives/negatives
$Se = TP / (TP + FN)$	Sensitivity (recall)
$Sp = TN / (TN + FP)$	Specificity
$PPV = TP / (TP + FP)$	Positive predictive value (precision)
$NPV = TN / (TN + FN)$	Negative predictive value
AUC	Area under the ROC curve
NB	Net benefit (from decision curve analysis)

25.9. Subscripts and superscripts

25.10. Greek letters commonly used in statistics

Notation	Meaning
x_i	Value for individual i
x_{ij}	Value of variable j for individual i
$\hat{\theta}$	Estimated value of θ
θ^*	Bootstrap replicate of θ
x^2	x squared
$\sqrt{x}$	Square root of x

25.10. Greek letters commonly used in statistics

Letter	Name	Common use
α	Alpha	Significance level, elastic net mixing
β	Beta	Regression coefficients, Type II error rate
γ	Gamma	Effect modifier, shape parameter
δ	Delta	Difference, effect size
ϵ	Epsilon	Error term, small quantity
λ	Lambda	Rate parameter, penalty parameter
μ	Mu	Population mean
π	Pi	Proportion, prior
σ	Sigma	Standard deviation
τ	Tau	Between-study variance (meta-analysis)
χ^2	Chi-squared	Chi-squared test statistic

26. Further Reading and Resources

Further Reading and Resources

This annotated bibliography organises key references by topic. It emphasises recent (2024–2026) publications from top medical journals, along with foundational textbooks. Resources are grouped by course part for easy navigation.

26.1. Textbooks

- **Smits LJM, van Kuijk SMJ, Wynants L.** *Improving Health Care with Clinical Prediction Models: From Idea to Impact*. Maastricht University Press, 2026. Open access (CC BY 4.0). Covers the complete prediction model journey from development to implementation. Ideal companion to this course.
- **Steyerberg EW.** *Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating*. 2nd ed. Springer, 2019. The definitive reference on clinical prediction model methodology. Supplementary materials with R code at clinicalpredictionmodels.org.
- **Harrell FE.** *Regression Modeling Strategies*. 2nd ed. Springer, 2015. Comprehensive treatment of regression modelling with emphasis on correct handling of continuous variables, splines, and validation. Free course materials at hbiostat.org.
- **McElreath R.** *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*. 2nd ed. CRC Press, 2020. The most

Further Reading and Resources

accessible introduction to Bayesian statistics. Lecture videos freely available online.

- **Gelman A, Carlin JB, Stern HS, et al.** *Bayesian Data Analysis*. 3rd ed. CRC Press, 2013. The comprehensive Bayesian reference. PDF freely available from the authors.
- **Hernán MA, Robins JM.** *Causal Inference: What If*. Chapman & Hall/CRC, 2024. The standard text on causal inference from observational data. Freely available at miguellhernan.org.
- **Boehmke B, Greenwell B.** *Hands-On Machine Learning with R*. CRC Press, 2019. Practical ML in R. Free online at bradley-boehmke.github.io/HOML.
- **Vittinghoff E, Glidden DV, Shiboski SC, McCulloch CE.** *Regression Methods in Biostatistics*. 2nd ed. Springer, 2012. Clear, practical regression modelling for biomedical researchers.

26.2. Reporting Guidelines

- **Collins GS, et al.** TRIPOD+AI statement: updated guidance for reporting clinical prediction models that use regression or machine learning methods. *BMJ* 2024;385:e078378. The current standard for reporting prediction models.
- **CONSORT 2025 statement.** Updated guideline for reporting randomised trials. *BMJ* 2025. The latest CONSORT update.
- **TARGET reporting guideline.** For target trial emulation studies. *JAMA* 2025. 21-item checklist analogous to CONSORT.

26.3. Statistical Methods in Medicine (Key Papers 2024–2026)

26.3.1. Splines and Continuous Variables

- **Lopez-Ayala P, Riley RD, Collins GS, Zimmermann T.** Dealing with continuous variables and modelling non-linear associations in healthcare data: practical guide. *BMJ* 2025;390:e082440. Essential reading on splines and fractional polynomials.
- **Austin PC.** Graphical methods to illustrate the nature of the relation between a continuous variable and the outcome when using restricted cubic splines with a Cox proportional hazards model. *Statistical Methods in Medical Research* 2025.

26.3.2. Prediction Model Performance

- **Van Calster B, et al.** Evaluation of performance measures in predictive AI models to support medical decisions: overview and guidance. *Lancet Digital Health* 2025;7:e100916. Reviews 32 performance measures across 5 domains. Essential for understanding discrimination, calibration, and clinical utility.
- **Riley RD, et al.** Evaluation of clinical prediction models (parts 1–3). *BMJ* 2024. Three-part series on development, external validation, and sample size.

26.3.3. Bayesian Methods

- **FDA.** Use of Bayesian Methodology in Clinical Trials of Drug and Biological Products: Draft Guidance for Industry. January 2026. Signals regulatory acceptance of Bayesian methods.

Further Reading and Resources

- **Bürkner PC.** brms: An R Package for Bayesian Multilevel Models Using Stan. *Journal of Statistical Software* 2017;80(1). The definitive reference for the brms package.
- **Gelman A, Vehtari A, McElreath R.** Statistical Workflow. December 2025.

26.3.4. Missing Data

- **Wijesuriya R, et al.** Multiple Imputation for Longitudinal Data: A Tutorial. *Statistics in Medicine* 2025. Practical MI tutorial with R and Stata code.

26.3.5. Causal Inference

- **Hernán MA.** Target trial emulation: methods and applications. *NEJM* 2024 (Perspective). Key framework paper.
- **TARGET Reporting Guideline.** *JAMA* 2025. 21-item checklist for target trial emulation.
- **The Causal Roadmap.** Improving the rigor and transparency of causal analyses. *Epidemiology* 2024.

26.3.6. Survival Analysis

- **Austin PC.** Restricted cubic splines with Cox proportional hazards models. *Statistical Methods in Medical Research* 2025. Graphical methods for non-linear survival associations.

26.3.7. Decision Curve Analysis

- **Vickers AJ, et al.** A simple, step-by-step guide to interpreting decision curve analysis. *Diagnostic and Prognostic Research* 2019. Practical tutorial on clinical utility assessment.
- **Vickers AJ, Elkin EB.** Decision curve analysis: a novel method for evaluating prediction models. *Medical Decision Making* 2006;26(6):565–574.

26.3.8. Meta-Analysis

- **Cochrane Handbook for Systematic Reviews of Interventions.** Version 6.5, 2024. The standard reference. Available at training.cochrane.org/handbook.
- **Riley RD, et al.** Individual participant data meta-analysis of prediction model studies. Various publications covering development, validation, and updating.

26.3.9. Machine Learning in Medicine

- **Boehmke B, Greenwell B.** *Hands-On Machine Learning with R*. Free online. Practical R-based ML with good coverage of tree-based methods.
- **Lgatto.** Introduction to Machine Learning with R. Free online at lgatto.github.io/IntroMachineLearningWithR. Concise, exercise-driven introduction.

26.3.10. Deep Learning

- **Goodfellow IJ, Bengio Y, Courville A.** *Deep Learning*. MIT Press, 2016. Freely available at deeplearningbook.org. The definitive theoretical reference for neural networks, CNNs, and RNNs.

Further Reading and Resources

- **Zhang A, Lipton ZC, Li M, Smola AJ.** *Dive into Deep Learning*. Cambridge University Press, 2023. Freely available at d2l.ai. Interactive textbook with executable code. Adopted at 500+ universities.
- **Howard J, Gugger S.** *Deep Learning for Coders with fastai and PyTorch*. O'Reilly, 2020. Companion to fast.ai. Coding-first approach ideal for applied researchers.
- **Grinsztajn L, Oyallon E, Varoquaux G.** Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS* 2022. Benchmark demonstrating that XGBoost beats neural networks on tabular data.
- **Ma J, et al.** Segment anything in medical images. *Nature Communications* 2024;15:654. MedSAM foundation model for universal medical image segmentation.
- **Bedi S, et al.** MedHELM: Holistic Evaluation of Large Language Models for Medical Tasks. *Nature Medicine* 2025. Most comprehensive evaluation of LLMs for clinical tasks.
- **Wiegrefe S, et al.** Deep learning for survival analysis: a review. *Artificial Intelligence Review*, Springer, 2024. Comprehensive taxonomy of deep survival methods.

26.4. R Packages (Key Packages, Recently Updated)

Package	Purpose	Last Updated
<code>tidyverse</code>	Data wrangling and visualisation	2024
<code>rms</code>	Regression modelling strategies (Harrell)	2024
<code>glmnet</code>	Penalised regression (LASSO, ridge, elastic net)	2024
<code>survival</code>	Survival analysis	2024
<code>survminer</code> / <code>ggsurvfit</code>	Survival plot visualisation	2024
<code>tidymodels</code>	Unified ML framework	2024
<code>ranger</code>	Fast random forests	2024
<code>xgboost</code>	Gradient boosting	2025

26.5. Python Packages (Key Packages, Recently Updated)

Package	Purpose	Last Updated
<code>brms</code>	Bayesian regression with Stan	2024
<code>rstanarm</code>	Bayesian regression (simpler interface)	2024
<code>mice</code>	Multiple imputation by chained equations	2024
<code>dcurves</code>	Decision curve analysis	2024
<code>CalibrationCurves</code>	Calibration plots	2024
<code>pROC</code>	ROC curves and AUC	2023
<code>gtsummary</code>	Publication-ready summary tables	2024
<code>gt</code>	Publication-quality tables	2024
<code>MatchIt</code>	Propensity score matching	2024
<code>WeightIt</code>	Propensity score weighting	2024
<code>cobalt</code>	Covariate balance assessment	2024
<code>meta / metafor</code>	Meta-analysis	2024
<code>uwot</code>	UMAP in R	2024
<code>Rtsne</code>	t-SNE in R	2023
<code>cluster</code>	Clustering algorithms	2023
<code>factoextra</code>	Clustering visualisation	2023
<code>keras3</code>	Deep learning (TensorFlow/JAX backend)	2025
<code>torch</code>	Deep learning (PyTorch backend)	2025
<code>naniar</code>	Missing data visualisation	2024
<code>finalfit</code>	Regression modelling and reporting	2024
<code>marginaleffects</code>	Marginal effects, predictions, contrasts	2025
<code>performance</code>	Model performance and diagnostics	2025
<code>parameters</code>	Model parameter extraction	2025
<code>easystats</code>	Meta-package for modern statistics	2025

26.5. Python Packages (Key Packages, Recently Updated)

Further Reading and Resources

Package	Purpose	Last Updated
pandas	Data manipulation	2025
numpy	Numerical computing	2025
scikit-learn	Machine learning	2025
statsmodels	Statistical modelling	2024
lifelines	Survival analysis	2024
xgboost	Gradient boosting	2025
lightgbm	Gradient boosting (fast)	2025
pymc	Bayesian modelling	2025
bambi	Bayesian regression (brms-like)	2025
arviz	Bayesian visualisation and diagnostics	2025
umap-learn	UMAP	2024
matplotlib	Visualisation	2025
seaborn	Statistical visualisation	2024
plotnine	ggplot2 for Python	2024
miceforest	Multiple imputation (random forest)	2024
tableone	Baseline characteristics tables	2024
great_tables	Publication-quality tables	2025
shap	ML model interpretation	2024
dowhy	Causal inference	2024
tensorflow / keras	Deep learning	2025
torch	Deep learning (PyTorch)	2025
pycox	Deep learning survival analysis	2024
torchsurv	Deep survival analysis (Novartis/FDA)	2024
plotly	Interactive visualisation	2025
patsy	Design matrices (spline bases)	2023
pygam	Generalised additive models	2024

26.6. JAMA Guide to Statistics and Methods

The JAMA Guide to Statistics and Methods is an ongoing essay series explaining statistical techniques in plain language for clinicians. Available at jamanetwork.com. Key recent topics include:

- Bayesian hierarchical models (2024)
- Instrumental variables and heterogeneous treatment effects (2025)
- Target trial emulation (2025)
- Nonparametric statistical analysis (2025)

26.7. BMJ Statistics Notes

The BMJ Statistics Notes series, edited by Doug Altman and Martin Bland, provides short, accessible guides on statistical topics for medical researchers. The series has been running since 1994. Full listing at bmj.com/specialties/statistics-notes.

26.8. Online Courses and Tutorials

- **Frank Harrell's Biostatistics for Biomedical Research** — hbiostat.org/bbr. Free, comprehensive course with R code. Excellent coverage of regression, prediction, and Bayesian methods.
- **Harrell's Regression Modeling Strategies course** — hbiostat.org/course. 4-day intensive course materials.
- **Statistics in Medicine tutorials** — The journal publishes tutorial papers on a wide range of topics. Accessible from onlinelibrary.wiley.com.

A. References

